

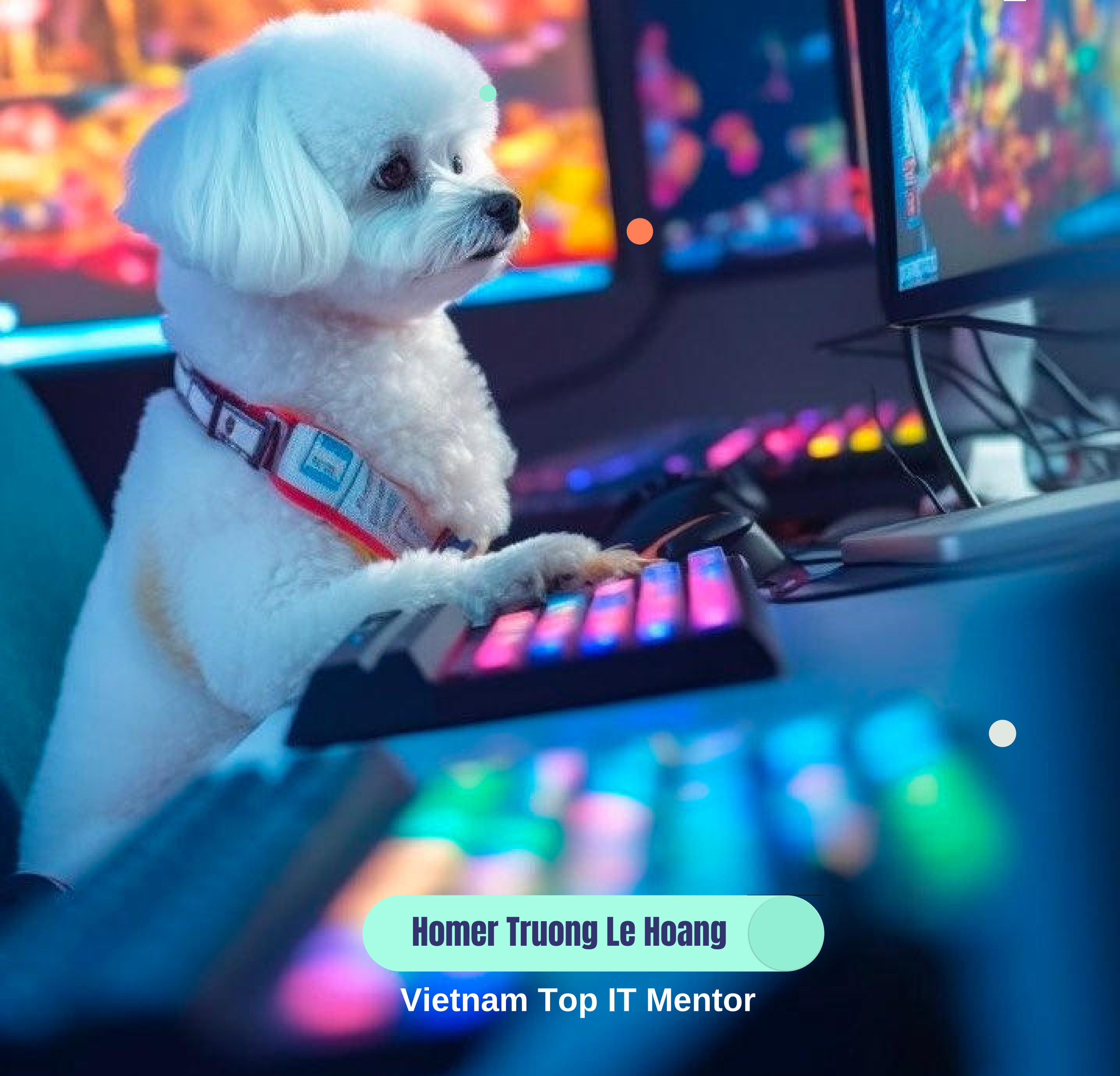


HOA

HOMER Academy

Hiếu đúng - Học đúng - Hành đúng

KHÁM PHÁ THẾ GIỚI CÔNG NGHỆ



Homer Truong Le Hoang

Vietnam Top IT Mentor

TABLE OF _____ CONTENTS

01

**Nền Tảng
Công Nghệ Thông Tin**

02

**Hướng Nghiệp
Công Nghệ Thông Tin**

03

**Trải Nghiệm Công Nghệ
Thông Tin 30 Năm**

LỜI MỞ ĐẦU	12
LỜI CẢM ƠN	13
VỀ BỘ CẨM NANG	14
VỀ TÁC GIẢ	22
VỀ BẢN QUYỀN	28
VÀI GHI CHÚ	29
Danh Mục Hình	30
Phần 1: Nền Tảng Công Nghệ Thông Tin	33
I. Welcome to IT World (Nhập Môn IT)	34
1) Ngành IT	35
2) Lợi Ích Học Nền Móng IT	38
3) Tư Duy Logic (Sự Hợp Lý)	38
4) Phương Pháp Học Hiệu Quả	39
5) Practice (Thực Hành)	41
II. IT English (Tiếng Anh IT)	42
1) Nature of Word (Bản Chất Của Từ)	45
a. Article (Mạo Từ)	45
b. Adjective – Adj (Tính Từ)	45
c. Adverb – Adv (Trạng Từ)	46
d. Noun (Danh Từ)	47
e. Pronoun (Đại Từ)	47
f. Verb (Động Từ)	48
g. Preposition (Giới Từ)	48
h. Conjunction (Liên Từ)	49
i. Practice (Thực Hành)	49
2) Function of Word (Chức Năng Của Từ)	50
a. Subject (Chủ Ngữ)	50
b. Verb (Động Từ)	50
c. Subject Complement (Bổ Nghĩa Cho Chủ Ngữ)	50
d. Direct Object (Đối Tượng Trực Tiếp)	51
e. Indirect Object (Đối Tượng Gián Tiếp)	51
f. Conjunction (Liên Từ)	51
g. Practice	51
3) Sentence Synthesis (Tổng Hợp Câu)	52
a. Simple Sentence (Câu Đơn)	52
b. Compound Sentence (Câu Ghép)	52
c. Complex Sentence (Câu Phức)	53
d. Practice	54
4) Tense Synthesis (Tổng Hợp Thì)	55
a. Past Tenses (Nhóm Các Thì Quá Khứ)	55
b. Present Tenses (Nhóm Các Thì Hiện Tại)	56
c. Future Tenses (Nhóm Các Thì Tương Lai)	57
d. Practice	57

5) Reading Skill (Kỹ Năng Đọc)	58
a. Word Family (Gia Đình Từ)	58
b. Context (Ngữ Cảnh)	58
c. Compound Noun (Danh Từ Ghép)	58
d. Practice	59
6) Popular Misunderstanding (Hiểu Nhầm Phổ Biến)	60
a. Article Misunderstanding (Hiểu Nhầm Mạo Từ)	60
b. Adjective Misunderstanding (Hiểu Nhầm Tính Từ)	60
c. Verb Misunderstanding (Hiểu Nhầm Động Từ)	60
d. Meaning Misunderstanding (Hiểu Nhầm Nghĩa)	61
e. Practice	61
III. IT Foundation (Nền Tảng IT)	62
1) Computer Structure (Cấu Trúc Máy Tính)	62
a. Input Devices (Các Thiết Bị Nhập)	64
b. Processing Devices (Các Thiết Bị Xử Lý)	66
c. Output Devices (Các Thiết Bị Xuất)	67
d. Storage Devices (Các Thiết Bị Lưu Trữ)	68
e. Practice	71
2) Operating System – OS (Hệ Điều Hành)	72
a. OS History (Lịch Sử OS)	73
b. OS Type (Loại OS)	74
c. User Account (Tài Khoản Người Dùng)	74
d. Processor Management (Quản Lý CPU)	75
e. Memory Management (Quản Lý RAM)	76
f. Disk Management (Quản Lý Đĩa)	77
g. Application Management (Quản Lý Ứng Dụng)	77
h. Process Management (Quản Lý Tiến Trình)	78
i. Others (Những Mục Khác)	79
j. Practice	79
3) Software (Phần Mềm)	80
a. Software Type (Loại Phần Mềm)	82
b. Packaged Software (Phần Mềm Đóng Gói)	83
c. Software as a Service – SaaS (Phần Mềm Như Là Một Dịch Vụ)	85
d. Google Word	86
e. Google Excel	86
f. Practice	87
4) Programming (Lập Trình)	88
a. Overview (Tổng Quan)	88
b. Programming Concepts (Các Khái Niệm Trong Lập trình)	91
c. Data Structure (Cấu Trúc Dữ Liệu)	95
d. Input/ Output – I/O (Nhập/ Xuất)	99
e. SubProgram (Chương Trình Con)	99

f. Procedural Programming – PP (Lập Trình Thủ Tục)	102
g. Object-Oriented Programming – OOP (Lập Trình Hướng Đối Tượng)	103
h. SOLID Principles (Bộ nguyên tắc SOLID)	111
i. Design Pattern (Mẫu Thiết Kế)	112
j. Aspect-Oriented Programming – AOP (Lập Trình Hướng Khía Cạnh)	113
k. Functional Programming – FP (Lập Trình Hàm)	115
l. Code Quality (Chất Lượng Mã)	116
m. Practice	119
5) Database (Cơ Sở Dữ Liệu)	122
a. Relational Database (Cơ Sở Dữ Liệu Quan Hệ)	123
b. MySQL	127
c. NoSQL Database (Cơ Sở Dữ Liệu NoSQL)	128
d. MongoDB	129
e. Database Selection (Chọn Lựa Cơ Sở Dữ Liệu)	131
f. Practice	131
6) Computer Network (Mạng Máy Tính)	132
a. Network Models (Các Mô Hình Mạng)	133
b. Network Services (Các Dịch Vụ Trên Mạng)	138
c. Networking Application (Ứng Dụng Có Nối Mạng)	138
d. Web Application (Ứng Dụng Web)	139
e. Practice	142
7) Software Engineering (Công Nghệ Phần Mềm)	143
a. Analysis (Phân Tích)	144
b. Design (Thiết Kế)	145
c. Implementation (Hiện Thực)	146
d. Testing (Kiểm Thử)	146
e. Deployment (Triển Khai)	147
f. Maintenance (Bảo Trì)	148
g. Practice	148
8) Artificial Intelligence – AI (Trí Tuệ Nhân Tạo)	149
a. AI Application (Ứng Dụng AI)	149
b. Cờ Tướng Project (Dự Án Cờ Tướng)	150
c. Practice	158
9) Data Science (Khoa Học Dữ Liệu)	159
a. Data Analytics (Phân Tích Dữ Liệu)	159
b. Data Mining (Khai Thác Dữ Liệu)	160
c. Data Warehouse (Kho Dữ Liệu)	162
d. Practice	164
IV. Java Technology Foundation (Nền Tảng Công Nghệ Java)	165
1) Java Core Concepts (Các Khái Niệm Cốt Lõi Java)	167
a. Java Virtual Machine – JVM (Máy Ảo Java)	167
b. Java Runtime Environment (Môi Trường Thực Thi Java)	167
c. Java Development Kit (Bộ Công Cụ Phát Triển Java)	167

d. Java Standard Edition – JavaSE (Phiên Bản Nền Tảng Java)	168
e. Java Enterprise Edition – JavaEE (Phiên Bản Doanh Nghiệp Java)	168
f. Java Data Structure (Cấu Trúc Dữ Liệu Java)	169
g. Java I/O (Nhập/Xuất Java)	170
h. Java Swing (Thư Viện UI Java)	171
i. Practice	171
2) Database Access (Truy Cập Database)	173
a. Java Database Connectivity – JDBC	173
b. Database Row Mapper	173
c. Object-Relational Mapping – ORM	174
e. Practice	175
3) Networking Application (Ứng Dụng Kết Nối Mạng)	175
a. ServerSocket	176
b. Socket	176
c. Practice	176
4) Web Application (Ứng Dụng Web)	177
a. Servlet/JSP	178
b. Multiple Page App – MPA (Ứng Dụng Đa Trang)	178
c. MVC Pattern	178
d. Single Page App – SPA (Ứng Dụng Đơn Trang)	179
e. Practice	180
5) JavaSE Overview (Tổng Quan JavaSE)	180
a. JavaSE History (Lịch Sử JavaSE)	181
b. Base Libraries (Các Thư Viện Nền Tảng)	181
c. Integration Libraries (Các Thư Viện Tích Hợp)	181
d. User Interface Toolkits (Các Bộ Công Cụ UI)	182
e. Language Tools (Các Công Cụ Của Ngôn Ngữ)	182
f. Generics	182
g. Lambda Expression	182
h. Stream API	183
i. Thread	185
j. Thread Pool	186
k. Java Concurrency	190
l. Practice	191
6) JavaEE Overview (Tổng Quan JavaEE)	193
a. JavaEE History (Lịch Sử JavaEE)	194
b. Servlet	194
c. Java Server Page – JSP	194
d. Servlet-JSP Combination	194
e. Context and Dependency Injection – CDI	194
f. Common Annotations	195

g. JAX-RS	195
h. Interceptor	195
i. Enterprise Java Bean – EJB	196
j. Java Persistence API – JPA	196
k. Java Transaction API – JTA	196
l. JavaEE Utilities	196
m. Practice	196
7) Spring Framework Overview (Tổng Quan Spring Framework)	198
a. Spring Framework History (Lịch Sử Spring Framework)	198
b. Spring Bean and DI	198
c. Spring Web	199
d. Spring Security	199
e. Spring Transaction	199
f. Spring JDBC/Hibernate	199
g. Spring AOP	199
h. Spring Messaging	199
i. Spring Boot	200
j. Practice	200
8) Java World Systhesis (Tổng Hợp Thế Giới Java)	201
a. Spring Application	201
b. JEE Application	201
c. My Java Journey 2002 – 2023 (Hành Trình Java Của Tôi)	202
d. Practice	203
V. Tech Product (Sản Phẩm Công Nghệ)	204
1) Introduction (Giới Thiệu)	204
2) Development Process (Quy Trình Phát Triển)	205
Phần 2: Hướng Nghiệp Công Nghệ Thông Tin	206
I. ITer Characteristics (Đặc Điểm Của Dân IT)	207
1) PASSION (ĐAM MÊ)	207
2) Attitude (Tư Duy)	208
a. Can-do (Có Thể Làm)	208
b. Logic (Sự Hợp Lý)	208
c. Eager-to-learn Spirit (Tinh Thần Ham Học Hỏi)	208
d. Entrepreneur Spirit (Tinh Thần Làm Chủ)	208
e. Goal (Mục Tiêu)	209
f. Environment (Môi Trường)	209
g. Mentor	209
3) Background (Nền Tảng)	210
4) 3S Method (Phương Pháp 3S)	210
5) 3WH Formula (Công Thức 3WH)	211
6) CSI Formula (Công Thức CSI)	211

II. IT Jobs (Các Nghề IT)	215
1) IT System (Hệ Thống IT)	215
a. Introduction (Giới Thiệu)	215
b. Components in IT System (Các Thành Phần Trong IT System)	215
c. Practice	217
2) IT Company (Công Ty IT)	218
a. Introduction (Giới Thiệu)	218
b. SaaS Company (Công Ty SaaS)	219
c. ITOS Company (Công Ty Dịch Vụ Gia Công IT)	219
d. Practice	220
3) Software Development Process (Quy Trình Phát Triển Phần Mềm)	221
a. Introduction (Giới Thiệu)	221
b. Role-Employee Relationship (Quan Hệ Giữa Role & Employee)	225
c. Practice	225
4) Scrum Process (Quy Trình Scrum)	227
a. Waterfall Process (Quy Trình Thác Nước)	227
b. Agile Software Development (Phát Triển Phần Mềm Nhanh)	227
c. Scrum Process (Quy Trình Scrum)	227
d. Pros & Cons of Scrum Process (Ưu/Nhược Điểm Của Quy Trình Scrum)	229
e. Practice	230
5) Software Development Life Cycle (Vòng Phát Triển Phần Mềm)	231
a. Introduction (Giới Thiệu)	231
b. Practice	232
6) Role vs Job/ Position/ Title (Vai Trò vs Nghề/ Vị Trí/ Chức Danh)	233
a. Introduction (Giới Thiệu)	233
b. Practice	233
7) Graphic Designer – GD (Nhà Thiết Kế Đồ Họa)	234
a. Introduction (Giới Thiệu)	234
b. Real Project (Dự Án Thực Tế)	236
c. Challenge (Thử Thách)	236
d. Qualification (Năng Lực)	236
e. Career Path (Con Đường Sự Nghiệp)	236
f. Practice	237
8) Business Analyst – BA (Nhà Phân Tích Nghiệp Vụ)	238
a. Introduction (Giới Thiệu)	238
b. Real Project (Dự Án Thực Tế)	240
c. Challenge (Thử Thách)	240
d. Qualification (Năng Lực)	241
e. Career Path (Con Đường Sự Nghiệp)	241
f. Practice	242
9) Backend Developer – BE Dev (Nhà Phát Triển Backend)	243
a. Introduction (Giới Thiệu)	243
b. Real Project (Dự Án Thực Tế)	244

c. Challenge (Thử Thách)	245
d. Qualification (Năng Lực)	245
e. Career Path (Con Đường Sự Nghiệp)	246
f. Practice	247
10) Frontend Developer – FE Dev (Nhà Phát Triển Frontend)	248
a. Introduction (Giới Thiệu)	248
b. Real Project (Dự Án Thực Tế)	248
c. Challenge (Thử Thách)	249
d. Qualification (Năng Lực)	249
f. Career Path (Con Đường Sự Nghiệp)	250
g. Practice	250
11) Software Architect – SA (Kiến Trúc Sư Phần Mềm)	251
a. Introduction (Giới Thiệu)	251
b. Real Project (Dự Án Thực Tế)	251
c. Challenge (Thử Thách)	252
d. Qualification (Năng Lực)	253
e. Career Path (Con Đường Sự Nghiệp)	253
f. Practice	253
12) DevOps – DO (Nhà Vận Hành Phát Triển)	254
a. Introduction (Giới Thiệu)	254
b. Real Project (Dự Án Thực Tế)	254
c. Challenge (Thử Thách)	255
d. Qualification (Năng Lực)	255
e. Career Path (Con Đường Sự Nghiệp)	255
f. Practice	256
13) Quality Control – QC (Nhà Quản Lý Chất Lượng)	257
a. Introduction (Giới Thiệu)	257
b. Real Project (Dự Án Thực Tế)	258
c. Challenge (Thử Thách)	259
d. Qualification (Năng Lực)	260
e. Career Path (Con Đường Sự Nghiệp)	260
f. Practice	261
14) Database Administrator – DBA (Nhà Quản Trị Database)	262
a. Introduction (Giới Thiệu)	262
b. Real Project (Dự Án Thực Tế)	264
c. Challenge (Thử Thách)	264
d. Qualification (Năng Lực)	264
e. Career Path (Con Đường Sự Nghiệp)	264
f. Practice	265
15) Project Manager – PM (Nhà Quản Lý Dự Án)	266
a. Introduction (Giới Thiệu)	266
b. Challenge (Thử Thách)	267
c. Qualification (Năng Lực)	267

d. Career Path (Con Đường Sự Nghiệp)	267
e. Practice	268
16) Engineering Manager – EM (Nhà Quản Lý Công Nghệ)	269
a. Introduction (Giới Thiệu)	269
b. Challenge (Thử Thách)	269
c. Qualification (Năng Lực)	270
d. Career Path (Con Đường Sự Nghiệp)	270
e. Practice	270
17) Product Manager – PdM (Nhà Quản Lý Sản Phẩm)	271
a. Introduction (Giới Thiệu)	271
b. Challenge (Thử Thách)	271
c. Qualification (Năng Lực)	271
d. Career Path (Con Đường Sự Nghiệp)	272
e. Practice	272
18) Quality Assurance – QA (Nhà Quản Lý Chất Lượng Quy Trình)	273
a. Introduction (Giới Thiệu)	273
b. Challenge (Thử Thách)	274
c. Qualification (Năng Lực)	274
d. Career Path (Con Đường Sự Nghiệp)	274
e. Practice	275
19) Data Scientist – DS (Nhà Khoa Học Dữ Liệu)	276
a. Introduction (Giới Thiệu)	276
b. Challenge (Thử Thách)	276
c. Qualification (Năng Lực)	276
d. Career Path (Con Đường Sự Nghiệp)	277
e. Practice	277
20) Security Professional – SP (Chuyên Gia Bảo Mật)	278
a. Introduction (Giới Thiệu)	278
b. Challenge (Thử Thách)	279
c. Qualification (Năng Lực)	279
d. Career Path (Con Đường Sự Nghiệp)	279
e. Practice	279
21) IT Support (Chuyên Viên Hỗ Trợ Kỹ Thuật)	280
a. Introduction (Giới Thiệu)	280
b. Challenge (Thử Thách)	280
c. Career Path (Con Đường Sự Nghiệp)	280
d. Qualification (Năng Lực)	281
e. Practice	281
III. Soft Skills (Các Kỹ Năng Mềm)	282
1) Professional English (Tiếng Anh Làm Việc)	282
a. Reading Skill (Kỹ Năng Đọc)	282
b. Writing Skill (Kỹ Năng Viết)	283
c. Listening Skill (Kỹ Năng Nghe)	283

d. Speaking Skill (Kỹ Năng Nói)	284
e. Practice	285
2) Logical Thinking (Suy Nghĩ Luận Lý)	286
a. Introduction (Giới Thiệu)	287
b. Practice	287
3) Problem Solving (Giải Quyết Vấn Đề)	288
a. Introduction (Giới Thiệu)	288
b. Practice	289
4) Time Management (Quản Lý Thời Gian)	290
a. Introduction (Giới Thiệu)	290
b. Practice	291
5) Teamwork (Làm Việc Nhóm)	292
a. Introduction (Giới Thiệu)	292
b. Practice	293
6) Presentation & Communication (Thuyết Trình và Giao Tiếp)	294
a. Introduction (Giới Thiệu)	294
b. Practice	295
IV. IT Career Orientation (Hướng Nghiệp Công Nghệ)	296
1) 5P Formula (Công Thức 5P)	296
a. Perspective (Tầm Nhìn)	296
b. Present (Trạng Thái Hiện Tại)	296
c. Plan (Kế Hoạch)	296
d. Process (Quy Trình)	296
e. Perform (Thực Hiện)	296
2) Apply 5P Formula (Áp Dụng Công Thức 5P)	297
a. Perspective (Tầm Nhìn)	297
b. Present (Trạng Thái Hiện Tại)	297
c. Plan (Kế Hoạch)	298
d. Process (Quy Trình)	298
e. Perform (Thực Hiện)	298
3) Study Oversea (Du Học)	299
a. Tìm Trường & Học Bổng	299
b. Nộp Hồ Sơ Xin Giấy Nhập Học Của Trường (Study Admission)	299
c. Nộp Hồ Sơ Xin Giấy Phép Cho Học (Study Permit)	299
d. Nộp Hồ Sơ Xin Thị Thực (Visa)	300
e. Lên Đường	300
4) Job Seeking (Tìm Việc Làm)	301
a. Xác Định Nghề Phù Hợp	301
b. Tìm Các Việc Làm Đang Mở	301
c. Ứng Tuyển	301
5) Job Interview (Phỏng Vấn Việc Làm)	302
a. Trước Khi Vô Phỏng Vấn	302
b. Trong Khi Phỏng Vấn	302
c. Sau Khi Phỏng Vấn	302

6) Experience Sharing (Chia Sẻ Kinh Nghiệm)	303
Phần 3: TRẢI NGHIỆM Công Nghệ Thông Tin 30 Năm	306
I. Học IT ở Việt Nam	307
1) Học nghề IT để thi tốt nghiệp PTCS	307
2) Học lập trình Pascal ở cấp 3	310
3) Học Đại Học	313
II. Học IT ở Canada	317
1) Nhận học bổng và lo hồ sơ nhập học	317
2) Học Master (Thạc Sĩ)	323
III. Tham gia phát triển các Tech Product	331
1) Dự án XiangqiClub (Vietnam)	331
2) Dự án DMA (Vietnam)	334
3) Dự án SER5500 (Canada)	338
4) Dự án USPS (Canada)	344
5) Dự án LMA (Canada)	354
6) Dự án iVA (Canada)	357
7) Dự án MPA (Canada)	360
8) Dự án GPM (Canada)	363
9) Dự án YCMS (Vietnam)	365
10) Dự án YCRM (Vietnam)	367
11) Dự án SmartPush (Australia)	368
12) Dự án SmartIntegrate (Australia)	371
13) Dự án Awesm (USA)	375
14) Dự án UP (USA)	383
15) Dự án P4SVVN (Vietnam)	385
16) Dự án RMS (Vietnam)	389
17) Dự án SUPA (Taiwan)	391
18) Dự án CPT (Australia)	393
19) Dự án MPT (Australia)	394
20) Dự án DDS (Canada)	395
PHẦN THAM KHẢO	396
I. Software Architecture	396
II. Backend Programming	397
III. Frontend Programming	398
IV. Technology Infrastructure	399
V. Non-Functional Requirement	401
VI. Reverse Engineering	401
VII. Tech Product Development Toolkit	402
VIII. Java Application Development Toolkit	403
SELF-EXPERIENCE	404
LỜI KẾT	405

LỜI MỞ ĐẦU

Trong thời đại kỹ thuật số ngày nay, con người được tiếp cận nhiều máy móc thiết bị hiện đại giúp cho cuộc sống chúng ta thoải mái hơn ngày xưa rất nhiều. Một vài thứ thật tiện lợi mà chúng ta sử dụng hằng ngày có thể kể dưới đây như:

- **Smart Tivi** (Tivi Thông Minh) là Tivi có kết nối **Internet** (Mạng Máy Vi Tính Toàn Cầu) giúp chúng ta vừa xem Tivi ở các kênh truyền hình truyền thống vừa có thể nghe nhạc hoặc xem phim **Online** (Trực Tuyến) với các nền tảng hiện tại như Youtube, Netflix... theo sở thích riêng bất cứ lúc nào mà không phụ thuộc vào nội dung và lịch chiếu của các kênh truyền thống như trước.
- **Smart Phone** (Điện Thoại Thông Minh) giúp chúng ta nói chuyện với người thân, bạn bè, đồng nghiệp, đối tác... và thấy hình ảnh của họ đang ở xa nửa vòng trái đất y như đang ngồi nói chuyện ngay bên cạnh nhau. Smart Phone còn giúp chúng ta cập nhật tin tức thông qua báo mạng cực nhanh so với báo giấy thời trước (tuy nhiên báo giấy vẫn có những ưu điểm giúp nó còn tồn tại song song với báo mạng).
- **eCommerce Platform** (Nền Tảng Thương Mại Điện Tử) và **Logistics Technology Platform** (Nền Tảng Công Nghệ Vận Chuyển) như Grab, Gojek... giúp chúng ta mua đồ ăn/đi chợ Online chỉ với vài thao tác và hàng được giao tận nơi, đặc biệt trong đợt dịch Covid thì các nền tảng này càng đóng vai trò cực kỳ quan trọng trong phòng chống dịch.

Sở dĩ chúng ta có được mấy tiện ích này mà từ thời xa xưa đến gần cuối thế kỉ 20 chưa từng có là do sự phát triển vượt bậc của các ngành công nghệ hiện đại, trong đó ngành công nghệ quan trọng bậc nhất và phát triển cũng nhanh nhất trong mấy thập niên gần đây là **Information Technology** (Công Nghệ Thông Tin), viết tắt là **IT**.

Đến đây chắc bạn sẽ thắc mắc ngành IT là gì? Tại sao nó ra đời? và nó hoạt động thế nào mà có sức mạnh vượt bậc thay đổi hoàn toàn cuộc sống của nhân loại như thế, đúng không nào?

Bộ Cẩm Nang **“Khám Phá Thế Giới Công Nghệ”** mà bạn đang đọc đây sẽ dẫn dắt bạn khám phá thế giới IT từng bước một qua việc cung cấp những kiến thức IT mang tính nền tảng được hệ thống hóa giúp bạn nắm bắt dễ dàng hơn và sẽ giúp bạn hưởng nghiệp tốt hơn nếu bạn chọn theo đuổi đam mê ngành IT.

LỜI CẢM ƠN

Những kiến thức và kinh nghiệm IT của tôi được chia sẻ trong Bộ Cẩm Nang này ngoài sự **ĐAM MÊ** và nỗ lực hết sức của bản thân tích lũy được thì không thể không kể đến sự giúp đỡ của rất nhiều người trong quá trình học và làm IT của tôi trong 30+ năm qua.

Tôi muốn gửi lời cảm ơn đến Ba/Má tôi, những thầy/cô thời phổ thông, những Thầy/Cô đầu tiên dạy tôi những môn Tin Học cơ bản MS-DOS, VNI, Vietres, Lotus... ở các lớp dạy nghề dành cho học sinh cấp 2, các Giảng Viên ở trường Đại Học Bách Khoa TP HCM, Vị Giáo Sư Pierre Dumouchel ở Đại Học ÉTS, Montreal, Canada, các sếp và đồng nghiệp của tôi.

Ngoài ra, tôi cũng gửi lời cảm ơn đến các bạn bè tôi và những độc giả đã gửi phản hồi rất hữu ích giúp cho Bộ Cẩm Nang này ngày càng hoàn thiện hơn.

VỀ BỘ CẨM NANG

AI NÊN ĐỌC BỘ CẨM NANG

Trong vài thập kỉ gần đây, ngành IT đã và đang phát triển cực kỳ mạnh mẽ và sẽ tiếp tục phát triển nhanh chóng trong rất nhiều thập kỉ tới, đem lại thu nhập tốt cho dân Việt Nam, cũng như giúp nước Việt Nam rút ngắn khoảng cách với các nước tiên tiến trên thế giới.

Mục tiêu của Bộ Cẩm Nang này là truyền lửa **ĐAM MÊ** công nghệ và cung cấp kiến thức IT nền móng được hệ thống hóa cho các bạn trẻ có niềm yêu thích khám phá thế giới công nghệ nhằm phát triển hơn nữa cộng đồng IT của Việt Nam, góp phần vào việc phát triển ngành IT của Việt Nam hiện đang rất cần thiết ở hầu hết ngành khác nhau trong nền kinh tế Việt Nam.

Đối tượng chính của Bộ Cẩm Nang này là những ai muốn có một nền tảng IT vững để theo đuổi đam mê ngành IT hoặc phục vụ cho việc học/làm của mình trong các ngành khác.

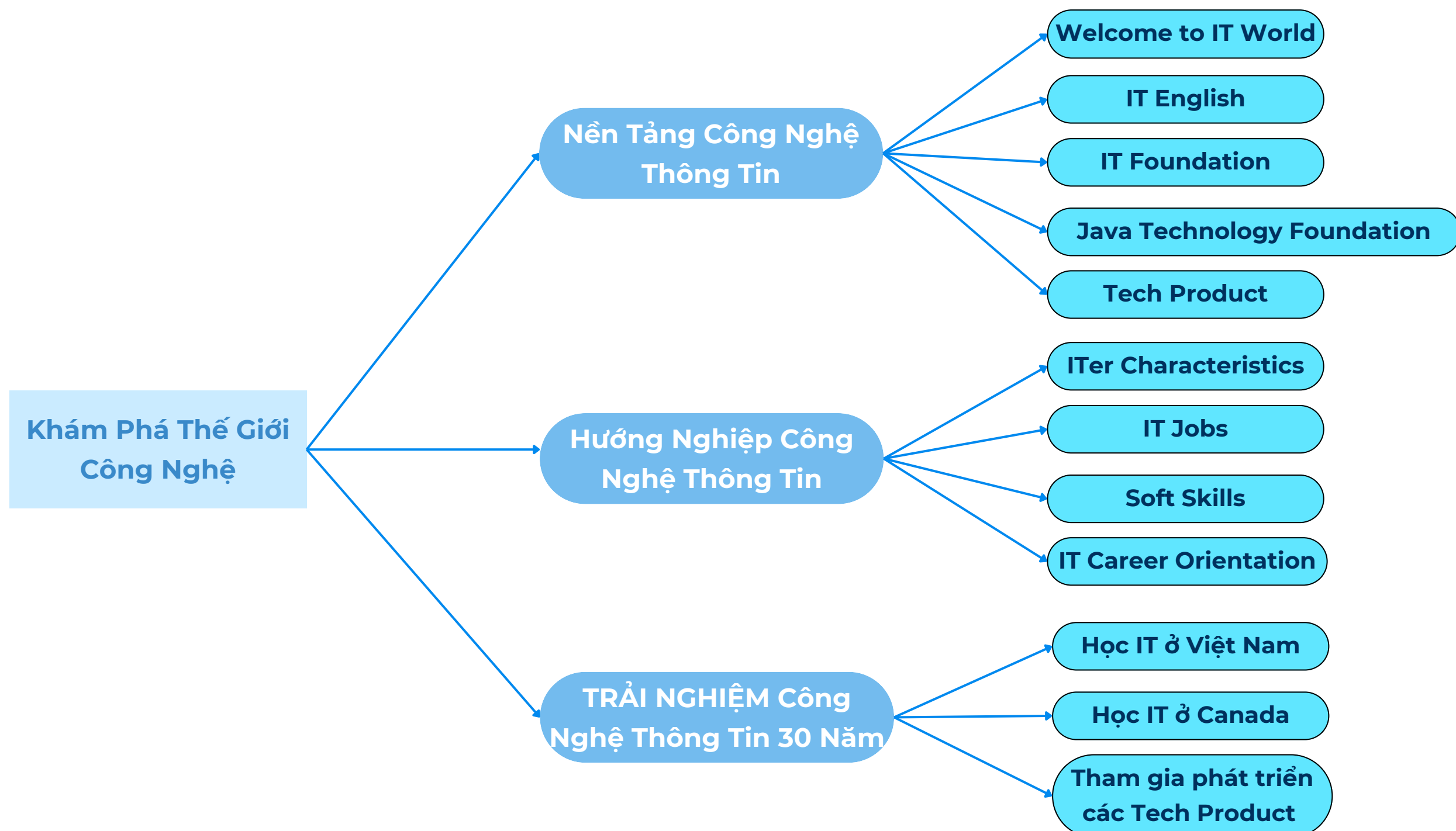
Độc giả không yêu cầu cần có kiến thức IT trước khi đọc Bộ Cẩm Nang này.

Các GIÁ TRỊ mà Bộ Cẩm Nang sẽ đem đến cho độc giả bao gồm:

- Có được cái nhìn tổng quan về ngành IT
- Biết rõ phương pháp học IT hiệu quả
- Nắm vững nền tảng **English** (Tiếng Anh) cần thiết để theo ngành IT
- Hiểu được những khái niệm cốt lõi được hệ thống hóa của ngành IT
- Nắm vững nền tảng Công Nghệ Java, một trong những công nghệ phổ biến trong ngành IT
- Hiểu được những tính chất nào của con người sẽ phù hợp với ngành IT
- Biết được các nghề chính yếu trong ngành IT
- Nắm được các kỹ năng mềm cần thiết để làm việc tốt trong ngành IT
- Biết áp dụng phương pháp 5P để hưởng nghiệp IT hiệu quả
- Hành trình 30+ năm TRẢI NGHIỆM ngành IT của tôi có thể tạo động lực và sự tự tin cho bạn theo đuổi ĐAM MÊ IT của mình

VỀ BỘ CẨM NANG

TỔ CHỨC CỦA BỘ CẨM NANG



Bộ Cẩm Nang được chia thành 3 Phần với 3 Mục Tiêu rõ ràng:

- Phần 1 **“Nền Tảng Công Nghệ Thông Tin”** giúp bạn xây dựng kiến thức nền tảng IT vững chắc cho mình, gồm các Chương sau:
 - a.Chương **“Welcome to IT World”** cung cấp kiến thức tổng quan về ngành IT, những lợi ích của việc học IT từ gốc, tư duy logic và phương pháp học hiệu quả.
 - b.Chương **“IT English”** chia sẻ những kiến thức tổng hợp về English cần thiết nhất để tìm hiểu ngành IT dễ dàng hơn.
 - c.Chương **“IT Foundation”** cung cấp kiến thức IT nền móng được hệ thống hóa về các mảng trong ngành IT cấu tạo nên nhiều hệ thống IT phục vụ con người khắp mọi nơi.
 - d.Chương **“Java Technology Foundation”** giới thiệu tổng quan về một công nghệ rất phổ biến trong thế giới IT được sử dụng trên hàng tỉ thiết bị trong cuộc sống hàng ngày.
 - e.Chương **“Tech Product”** chia sẻ một số kiến thức sơ bộ về một loại sản phẩm vô hình cực kỳ phổ biến ngày nay đem lại doanh thu rất cao cho rất nhiều công ty xây dựng các sản phẩm loại này cũng như vô vàn tiện ích cho người dùng toàn cầu.
- Phần 2 **“Hướng Nghiệp Công Nghệ Thông Tin”** giúp bạn thấy được tổng quan các nghề trong ngành IT cùng với những yêu cầu về nghề và phương pháp 5P hỗ trợ bạn hướng nghiệp cho mình theo đuổi ngành IT tốt nhất, gồm các Chương sau:

VỀ BỘ CẨM NANG

TỔ CHỨC CỦA BỘ CẨM NANG

- a. Chương **“ITer Characteristics”** cho biết các yếu tố về con người để học và làm tốt trong ngành IT.
 - b. Chương **“IT Jobs”** giới thiệu tổng quan về hệ thống IT, các loại công ty IT trên thị trường, các vai trò tham gia trong việc phát triển một IT System, quy trình phát triển phần mềm phổ biến nhất hiện nay và chi tiết về các việc làm trong ngành IT.
 - c. Chương **“Soft Skills”** cung cấp kiến thức về các kỹ năng mềm mà người làm IT cần có để làm việc hiệu quả nhất.
 - d. Chương **“IT Career Orientation”** hướng dẫn phương pháp 5P cho các bạn ĐAM MÊ IT định hướng nghề nghiệp phù hợp cho mình.
- Phần 3 **“TRẢI NGHIỆM Công Nghệ Thông Tin 30 Năm”** chia sẻ quá trình học và làm IT của tôi trong và ngoài nước trong suốt 30 năm qua nhằm truyền động lực cho bạn theo đuổi ngành IT cũng như giúp bạn tham khảo những khó khăn mà tôi đã gặp phải khi theo đuổi ngành này.

Sau cùng là Phần Phụ Lục tổng hợp những khái niệm quan trọng mà dân IT cần phải biết và hiểu rõ để làm việc hiệu quả nhất được chia thành các danh mục sau:

- **“Software Architecture”** liệt kê những khái niệm quan trọng trong kiến trúc phần mềm.
- **“BE Programming”** tổng hợp các khái niệm thiết yếu trong lập trình Backend.
- **“FE Programming”** tổng hợp các khái niệm cốt lõi trong lập trình Frontend.
- **“Technology Infrastructure”** giới thiệu vài khái niệm phổ biến trong hạ tầng công nghệ.
- **“Non-Functional Requirement”** tổng hợp các khái niệm cần xử lý cho các yêu cầu phi chức năng.
- **“Reverse Engineering”** là phương pháp tìm hiểu hệ thống IT đang hoạt động một cách nhanh chóng.
- **“Tech Product Development Toolkit”** tổng hợp bộ công cụ phát triển sản phẩm công nghệ ở từng bước.
- **“Java Application Development Toolkit”** tổng hợp bộ công cụ phát triển ứng dụng trên nền tảng công nghệ Java.

VỀ BỘ CẨM NANG

PHẢN HỒI VỀ BỘ CẨM NANG

Sau khi đọc Bộ Cẩm Nang này mà bạn có những câu hỏi hoặc phản hồi thì vui lòng liên hệ với tôi qua các kênh dưới đây.

- **Facebook**

- Name: Homer Truong Le Hoang
- Link: <https://www.facebook.com/homertruong66>

- **LinkedIn**

- Name: Homer Truong Le Hoang
- Link: <https://www.linkedin.com/in/truunglehoang>

- **Youtube**

- Name: Homer Truong Le Hoang – Vietnam Top IT Mentor
- Link: <https://www.youtube.com/c/HomerTruong66>



Homer Truong Le Hoang

Vietnam Top IT Mentor

VỀ BỘ CẨM NANG

ĐÁNH GIÁ CỦA CÁC ĐỘC GIẢ

“Cuốn sách này giúp em hiểu biết thêm về ngành tin học và phương pháp học tin học hiệu quả.” – **Dũng (14 tuổi, Học Sinh)**

“Sách có giá trị hướng nghiệp chung về ngành IT cho học sinh. Sách được trình bày ngắn gọn và giải thích các khái niệm về ngành IT. Nội dung trình bày dễ hiểu và trực quan. Đây là cuốn sách tham khảo tổng lược để đọc và cung cấp các thông tin cơ sở nhất kích khởi sự hứng thú và hướng nghiệp đối với ngành IT cho đối tượng học sinh. Cảm ơn tác giả đã chia sẻ các trải nghiệm và tri thức của mình qua cuốn sách này.” – **Đức (15 tuổi, Học Sinh)**

“Cuốn sách cung cấp những khái niệm cốt lõi về IT, khái niệm dễ hiểu có demo, bài tập, giúp người đọc có nền móng về IT, ngoài ra còn chia sẻ phương pháp học hiệu quả sẽ giúp tiết kiệm rất nhiều thời gian. Đặc biệt là sách có phần Tiếng Anh cho IT rất cần thiết cho những bạn yếu về mặt ngoại ngữ.

Cuốn sách mô tả rất chi tiết về quy trình phát triển phần mềm, các vị trí công việc giúp em định hướng được công việc của mình trong quy trình tạo ra 1 sản phẩm phần mềm. Em học được rất nhiều thuật ngữ chuyên ngành IT mới mẻ, ví dụ là DevOps, PO, DBA. Phần cuối sách có nói về kỹ năng mềm rất cần thiết để giúp em có được sự chuẩn bị tốt để tìm việc khi ra trường. Nếu các bạn học sinh, sinh viên hoặc là những người đã đi làm trái ngành mà có đam mê với IT thì có thể đọc quyển sách này vì sách cung cấp các kiến thức căn bản và tổng hợp được những vị trí cụ thể chứ không mơ hồ chung chung như các trang web trên Internet.”

– **Hùng (21 tuổi, Sinh Viên Năm 3)**

“Quyển sách là một lựa chọn tuyệt vời cho các bạn trẻ đang theo học IT và những bạn đang có định hướng học ngành IT giúp các bạn có cái nhìn rõ ràng hơn, hệ thống hoá lại kiến thức bằng cách tiếp cận từ gốc và đi sâu vào các khái niệm chuyên môn, đặc biệt sách được anh Hoàng viết kết hợp với các từ ngữ chuyên môn thuần Tiếng Anh để các bạn tiếp cận dần với các thuật ngữ. Thủy ấn tượng về những kiến thức để xây dựng một nền tảng Tiếng Anh vững cũng như kỹ năng mềm dành cho các bạn đang học và làm trong lĩnh vực IT.

VỀ BỘ CẨM NANG

ĐÁNH GIÁ CỦA CÁC ĐỘC GIẢ

Là một sinh viên năm 4, Thủy tâm đắc nhất về nội quyền sách giúp Thủy rất nhiều trong quá trình chuẩn bị nội dung phỏng vấn của mình, đặc biệt về Java. Cảm ơn anh Hoàng vì đã tâm huyết tạo ra quyển sách này bằng tất cả những kinh nghiệm của mình.

Quyển sách giúp em hệ thống rõ ràng về các kiến thức về quy trình, các mô hình làm việc, các vị trí công việc và giúp em học thêm những Term mới mẻ từ một số vị trí em chưa từng biết trên ghế nhà trường, ví dụ như SA hay DevOps, hoặc những kỹ năng mềm giúp em soạn được một số câu hỏi trả lời phỏng vấn từ kinh nghiệm của người đi trước, sẽ thực tiễn và độ tin tưởng cao hơn. Quyển sách này rất phù hợp với đối tượng là học sinh đang có niềm quan tâm đặc biệt với ngành IT vì ngày xưa cá nhân em không có được một trang web hay tìm thấy được một quyển sách nào có thể tổng hợp được những vị trí cụ thể như thế này.”

– **Thủy (22 tuổi, Sinh Viên Năm Cuối)**

“Cuốn sách rất chi tiết và rất cần cho những bạn muốn học IT sau này, còn đối với các bạn đang học IT thì giúp củng cố nhanh về kiến thức cơ sở ngành của mình. Em thấy sách hay và rất phù hợp nhất và có ý nghĩa tác động lớn với các bạn đang có định hướng học IT từ cấp 3 hoặc là đang theo học IT năm 1 năm 2.” – **Trà (22 tuổi, Dân IT Mới Đi Làm)**

“Sách dùng từ ngữ khá đơn giản, dễ hiểu, cách viết thân thiện, không quá khô cứng, có lúc cảm thấy như người anh đang chia sẻ kinh nghiệm. Các thuật ngữ Tiếng Anh được sử dụng xuyên suốt trong sách và lặp lại nhiều lần, làm người đọc có cảm giác quen thuộc, nhớ từ và nghĩa lúc nào cũng không hay. Ấn tượng phần “Phương pháp học hiệu quả” được hệ thống và hướng dẫn một cách tỉ mỉ, có mô hình để người đọc dễ dàng theo dõi. Phần “English for IT” mình nghĩ sẽ không có trong những quyển sách khác, với kiến thức tổng quát quá đầy đủ. Ngoài ra có lồng ghép nhưng không quá nhiều những ngữ cảnh, từ vựng đơn giản về IT khiến mình cảm thấy khá thú vị. Các Knowledge Flow Diagram – KFD là những phần mình rất thích vì bao quát và giúp người đọc dễ hình dung.” – **Nhi (28 tuổi, Ngành Logistics)**

VỀ BỘ CẨM NANG

ĐÁNH GIÁ CỦA CÁC ĐỘC GIẢ

“Sách của anh giúp người ngoài lĩnh vực IT như em hiểu hơn về kiến thức chuyên môn và phần nào áp dụng vào công việc hiện tại. Ngoài ra thì nó cũng là một sự tổng hợp về nghề IT đương thời, sẽ giúp cho người đọc không bị tụt hậu về công nghệ.” – **Thiện (36 tuổi, CEO)**

“Sách có giá trị phổ cập kiến thức tổng quan cho những người đi làm muốn hiểu tổng quát về ngành IT. Nhìn tổng thể sách được trình bày ngắn gọn, lồng ghép nhiều chia sẻ thực tế về trải nghiệm và tri thức của tác giả. Mặt khác, các nội dung được trình bày cô đọng và trực quan dưới hình thức các Knowledge Flow Diagram – KFD giúp độc giả không chuyên dễ dàng hệ thống hóa các thông tin được trình bày trong cuốn sách này. Đây là cuốn sách tham khảo nhỏ và tổng lược dễ đọc và cung cấp vừa đủ các thông tin cơ sở nhất cho các đối tượng người đi làm không chuyên về IT tìm hiểu về ngành này. Cảm ơn tác giả đã chia sẻ các trải nghiệm và tri thức của mình qua cuốn sách này.” – **Lâm (42 tuổi, Ngành Nha Khoa)**

“Bản thân thì thấy nội dung rất là giá trị, đặc biệt cho các bạn hay các bé nào quan tâm yêu thích về IT.” – **Quế (44 tuổi, Ngành Bảo Hiểm)**

“Về hướng nghiệp anh thấy khá phù hợp, có thể nghiêng về nhiều cho sinh viên IT sắp ra trường hơn, giúp các bạn hiểu các role ở các công ty IT, có thể nghiêng về software nhiều hơn. Phần 5P và các kỹ năng cũng rất thú vị. Nói chung anh thấy rất ok, sách xuất bản anh sẽ giới thiệu sinh viên để mua.” – **Quân (44 tuổi, Giảng Viên Đại Học ngành IT)**

“Sách trình bày khoa học, rõ ràng, rất dễ đọc & giá trị.” – **Châu (46 tuổi, Ngành Thiết Kế)**

VỀ BỘ CẨM NANG

CÁC TÀI LIỆU THAM KHẢO KHÁC

- Các phương pháp học:
<https://www.youtube.com/watch?v=1pB66XBzMqo>
- Tự điển Anh – Anh – Việt: <https://www.thefreedictionary.com>
- Programming:
https://www.tutorialspoint.com/computer_programming_tutorials.htm
- Java: <https://www.baeldung.com>
- Maven: <https://books.sonatype.com/mvnref-book/reference/index.html>
- SQL: <https://www.w3schools.com/mysql/default.asp>
- Web: <https://www.w3schools.com/html>
- Git: <https://www.atlassian.com/git/tutorials>
- LinkedIn: <https://www.linkedin.com/> : Đây là mạng xã hội nghề nghiệp lớn nhất thế giới, bạn nên kết nối với những người trong nghề IT để cập nhật tin tức công nghệ và thị trường việc làm.
- Mạng lưới bạn bè của bạn: Hãy luôn mở rộng mạng lưới bạn bè của bạn, bao gồm cả các sếp cũ của bạn nếu bạn là người đã từng làm cho vài công ty trước đó.
- **Head Hunting Service** (Dịch Vụ Tuyển Dụng) làm trung gian giữa những người tìm việc và các công ty. Bạn hãy kết nối với các **IT Recruiter** (Người Tuyển Dụng IT) của các Head Hunting Company. Một ưu điểm của việc tìm việc qua dịch vụ tuyển dụng là họ sẽ giúp bạn tìm các công việc bạn đang mong muốn nhanh hơn và hỗ trợ bạn trong qui trình phỏng vấn, theo dõi sau phỏng vấn...

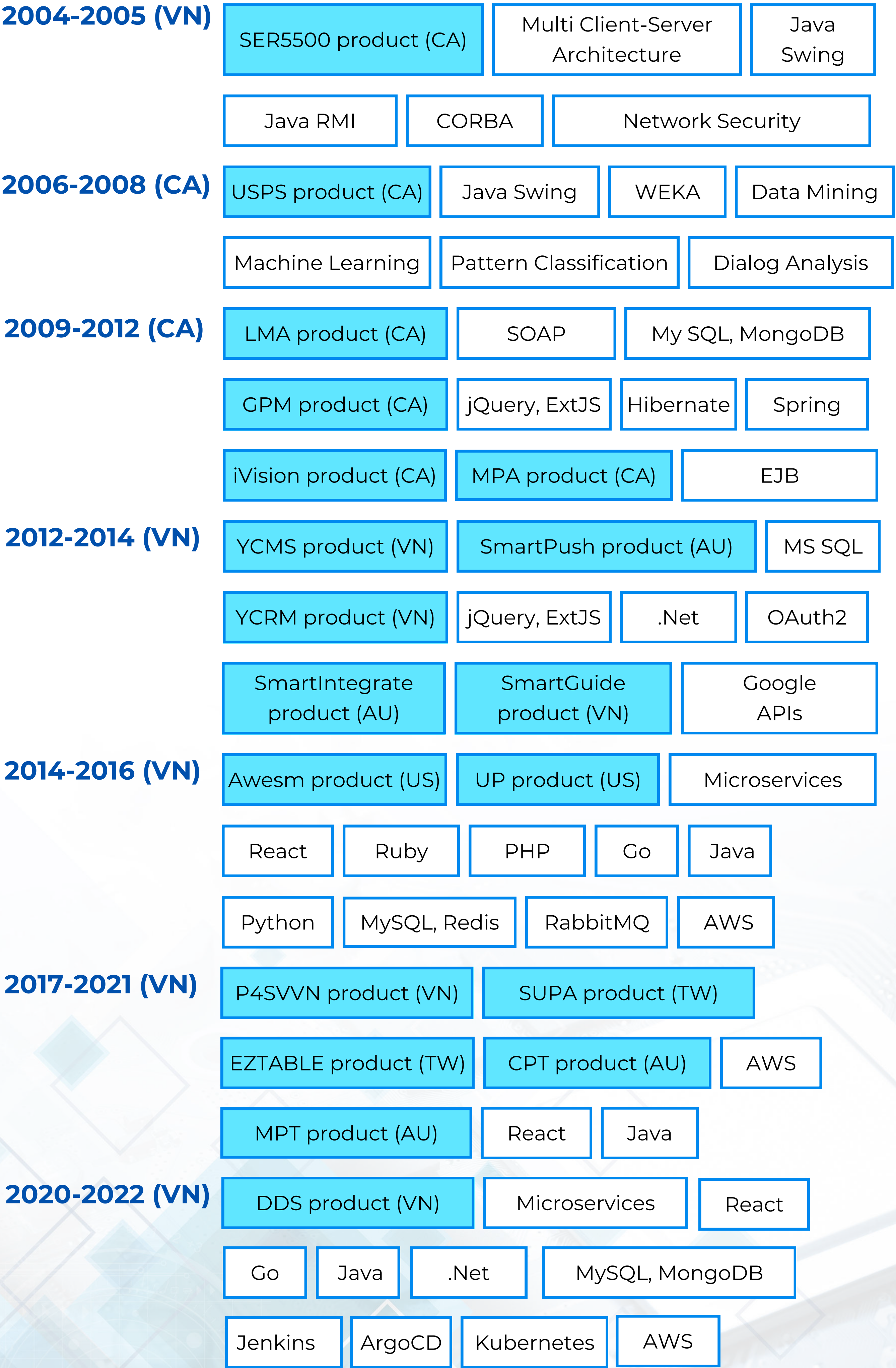
VỀ TÁC GIẢ

HOMER TRUONG LE HOANG



Trương Lê Hoàng (tên Tiếng Anh: Homer) là Chuyên Gia IT với **30 năm ĐAM MÊ** học và làm IT được thể hiện tổng quan trong hành trình sau đây.

1992-1997 (VN)	MS-DOS	Norton	VNI	Vietres
	Lotus 123	Pascal	Word	Excel
1998-2003 (VN)	Computer Structure	Operating System	Programming Language	Data Structure
	Database	Computer Network	Software Engineering	Artificial Intelligence
	XiangqiClub product (VN)	J2SE	Java Swing	Alpha-Beta Algorithm
	HTML/CSS/JS	Servlet	JSP	J2EE
				MS Access
2003-2004 (VN)	DMA product (VN)	.Net	Layered Architecture	
	Desktop Programming	Stored Procedure	C#	



Hình 1: 30 Năm ĐAM MÊ IT

Sinh ra và trưởng thành tại Thành Phố Hồ Chí Minh – Việt Nam, anh ấy đã bắt đầu đến với thế giới IT từ năm 1992 lúc 12 tuổi.

Anh ấy đã lấy bằng Kỹ Sư IT ở trường Đại Học Bách Khoa TPHCM khóa 1998-2003, bằng Master of IT tại trường Đại Học ÉTS, Montreal, Canada khóa 2006-2008.

Anh ấy đã từng làm việc tại Việt Nam, Canada và Mỹ, tham gia phát triển hơn 20 dự án xây dựng các sản phẩm công nghệ của những công ty Việt Nam, Canada, Úc, Đài Loan và Mỹ trải dài trên nhiều lĩnh vực như **Library** (Thư Viện), **Healthcare** (Chăm Sóc Sức Khỏe), **Call Center** (Trung Tâm Điện Thoại), **Global Trade** (Thương Mại Thế Giới), **eCommerce** (Thương Mại Điện Tử), **Fintech** (Công Nghệ Tài Chính), **Cyber Security** (An Ninh Mạng)... phục vụ cho thị trường toàn cầu.

Ngoài ra, anh ấy còn là Kiện Tướng Cờ Tướng Quốc Gia, đạt được năm 19 tuổi (1999).

Anh đã giành được chức vô địch Giải Cờ Tướng Đông Nam Á lứa tuổi U18 năm 1998.



Anh cũng đã có 30 năm ĐAM MÊ học và thi đấu Cờ Tướng được thể hiện tổng quan trong hành trình dưới đây.

1986-1990 (VN)	Trung Cuộc, Tàn Cuộc từ Ba	Trung Cuộc, Tàn Cuộc từ Hàng Xóm	Trung Cuộc, Tàn Cuộc từ Giang hồ
1991-1998 (VN)	Các Giải Học Sinh TPHCM	Quán Quân một số Giải HS TPHCM	Á Quân một số Giải HS TPHCM
1994-1994 (VN)	Giải Học Sinh Toàn Quốc ở Hà Nội	Quý Quân Giải HS TQ ở Hà Nội	Trung Cuộc, Tàn Cuộc từ Giang hồ
1995-1995 (VN)	Giải Học Sinh Toàn Quốc ở Quy Nhơn	Quán Quân Giải HS TQ ở Quy Nhơn	Khai Cuộc từ sư phụ Diệp Khai Nguyên
1996-1996 (VN)	Giải Học Sinh Toàn Quốc ở Đà Nẵng	Á Quân Giải HS TQ ở Đà Nẵng	Giải A1 Toàn Quốc ở TPHCM
1997-1997 (VN)	Giải Học Sinh Toàn Quốc ở Gia Lai	Á Quân Giải HS TQ ở Gia Lai	Khai Cuộc từ các thầy khác ở trường Năng Khiếu TDTT TPHCM
1998-1998 (VN)	Giải HS TQ ở Vịnh Hạ Long	Á Quân Giải HS TQ ở Vịnh Hạ Long	
	Giải U18 ĐNÁ	Quán Quân Giải U18 Đông Nam Á	
1999-1999 (VN)	Giải A1 TQ ở Vũng Tàu	Giải A2 TQ ở Từ Sơn, Bắc Ninh	Kiên Tướng Quốc Gia Việt Nam
1999-2005 (VN)	Các Giải A1/A2 TPHCM	Quý Quân Giải A1 TPHCM - 2000	
2000-2005 (VN)	Các Giải A1 TQ ở TPHCM, Bình Định, Hà Nội, Đà Nẵng, TPHCM, Vịnh Hạ Long		
	Các Giải A2 TQ ở Đà Nẵng, TPHCM, Vũng Tàu, Cần Thơ,		
2005-2012 (CA)	Giao hữu Online		
2012-2012 (VN)	Giải A2 TQ ở Nha Trang		
2012-2016 (VN)	Các Giải phong trào		

Hình 2: 30 Năm ĐAM MÊ Cờ Tướng



Từ năm 2016, ngoài công việc chính là xây dựng các sản phẩm công nghệ, anh ấy đã bắt đầu công việc ngoài giờ theo sở thích cá nhân bao gồm:

- Chia sẻ kiến thức, kinh nghiệm của mình đến hàng ngàn sinh viên IT cả **Offline** (Ngoại Tuyến) ở Thành Phố Hồ Chí Minh, Nha Trang, Phú Yên và **Online** (Trực Tuyến).
- Tư vấn cho một số công ty ở Việt Nam về phương pháp xây dựng các sản phẩm công nghệ và cách thức quản lý đội ngũ phần mềm một cách hiệu quả nhất.

Tại thời điểm hoàn thiện Bộ Cẩm Nang này, anh ấy đang đảm nhận vị trí có tên gọi là **Engineering Manager – EM** (Nhà Quản Lý Công Nghệ) cho một công ty Canada có văn phòng tại Thành Phố Hồ Chí Minh, phát triển sản phẩm công nghệ thuộc mảng Cyber Security giúp cho người dùng toàn cầu theo dõi và quản lý các máy vi tính của họ, có khả năng khóa máy hoặc xóa dữ liệu quan trọng trên máy từ xa khi phát hiện ra máy bị bọn trộm lấy mất nhằm đảm bảo những dữ liệu cần bảo mật không bị tiết lộ.

Dưới đây là những điều mà đồng nghiệp/đối tác đã nói về anh ấy.



Khanh Le  · 1st

Vice President of Engineering | Building Inspiring Team Cultures | Software Engineering | Cybersecurity | Agile Practitioner | AI Enthusiast

May 19, 2024, Khanh managed Homer directly

 All LinkedIn members

On 

I had the pleasure of working with Homer, who served as an exceptional Engineering Manager reporting to me. Homer is not only a great manager but also an inspiring leader for his team. He consistently leads his team to deliver outstanding outcomes while simultaneously supporting and mentoring his team members to foster their career growth.

Moreover, Homer's commitment extends beyond his job. For the past seven years, he has been giving back to the IT community through both online and offline mentoring. His dedication to nurturing talent and contributing to the broader community is truly commendable.



Shawn Douglass  · 1st

CEO and Co-Founder at Amberdata

December 6, 2016, Shawn was Homer's client

 All LinkedIn members

On 

Hoang, was a technical lead for us on an acquired technology stack. He dug deep into the code and infrastructure, operationalizing things for us while reducing our costs deprecating unused functions and infrastructure. Hoang, is an A-player and a pleasure to work with.

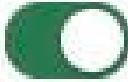


Vien Nguyen · 1st

Managing Director at VTL Invest Corp.

July 18, 2009, Vien worked with Homer on the same team

 All LinkedIn members

On 

Hoang Truong co-founded of YGS Corp. with me and other guys but I knew him from high school. He's very intelligent and hard working. Always with a smile on his face, Hoang is highly respected in our company for his willingness to help co-workers and customers. He is well organized, diligent in his software development, and always completes projects prior to due date. It is very delightful to work with him. For that, I believe our company will be very successful.



Loan Pham · 1st

Chief Technology Officer at NashTech Limited

July 15, 2009, Loan was senior to Homer but didn't manage Homer directly

 All LinkedIn members

On 

I'd worked with Hoang during 2 years in TMA Solutions. Hoang is very dedicated and responsible at work. He always finishes his works with the good quality in the time maner. Besides, he is willing to learn the new things. He always is open and honest with his colleagues.



Pierre Dumouchel · 1st

Professeur titulaire (ÉTS) et Directeur transfert technologique (IVADO)

July 15, 2009, Pierre managed Homer directly

 All LinkedIn members

On 

I did supervise Hoang for his master thesis.

Not only, Hoang had to learn French, he also did his research in two years. The title of his master thesis was Detection of problematic dialogs and prediction of user satisfaction in Human-Computer Dialog System.

Hoang was a brilliant and hard worker student. He is now a friend.

VỀ BẢN QUYỀN

Bộ Cẩm Nang này thuộc bản quyền của tác giả Homer Truong Le Hoang.

Không một phần nào của Bộ Cẩm Nang có thể được sao chép, lưu trữ trong hệ thống truy xuất, hoặc truyền đi, trong bất kỳ hình thức hoặc phương tiện điện tử, photocopy, hoặc bằng các cách khác, mà không có sự cho phép trước của tác giả.

©2022 by Homer Truong Le Hoang, All rights reserved.

VÀI GHI CHÚ

Ngành IT được khai sinh và phát triển mạnh nhất ở nước Mỹ, nơi sử dụng **English** là ngôn ngữ chính, và cũng là ngôn ngữ quốc tế, do đó khi bạn học IT thì tốt nhất là học trực tiếp bằng English, nếu mà English của bạn chưa đủ để học hoàn toàn bằng English thì ít nhất cũng phải sử dụng các **IT Terminology – Term** (Thuật Ngữ IT) bằng English.

Vì vậy các Term trong cuốn sách này được viết rất nhiều bằng English, có nhiều Term sẽ lặp đi lặp lại nhiều lần một cách cố tình nhằm giúp bạn làm quen và nhớ mặt chữ. Việc xen kẽ tiếng Việt và Tiếng Anh có thể sẽ làm bạn khó đọc lúc đầu, nhưng từ từ bạn sẽ thấy quen và nhớ những Term này rất nhanh.

Khi gặp 1 Term “X” mới nào đó, để hiểu nghĩa của X thì bạn không những tra từ điển xem từ tiếng Việt tương ứng của X là gì nhằm mục đích chủ yếu là để hình dung, mà còn phải tra thêm **Term Definition** (Định Nghĩa Thuật Ngữ) của X (bắt đầu bằng cụm từ “**X is** bla bla bla”) để hiểu chính xác ý nghĩa của X, sau đó ghi nhớ Definition này là chính (nếu nhớ thêm nghĩa tiếng Việt được nữa thì càng tốt).

Ví dụ: Term “Information Technology” khi tra từ điển tiếng Việt sẽ là Công Nghệ Thông Tin, rõ ràng là chúng ta cũng chưa hiểu lắm nghĩa của nó, mà cần tra thêm Definition của nó như sau: “**Information Technology is** the use of computers to create, process, store, retrieve, and exchange all kinds of electronic data and information.”

-> tạm dịch: “**Công nghệ thông tin là** việc sử dụng các máy vi tính để tạo, xử lý, lưu trữ, truy xuất và trao đổi tất cả các loại dữ liệu và thông tin dưới dạng điện tử.”

Knowledge Flow Diagram – KFD (Lược Đồ Dòng Chảy Kiến Thức) được sử dụng nhằm giúp bạn hệ thống hóa kiến thức tốt hơn và kết hợp với **Text** (Chữ) truyền đạt nội dung hiệu quả hơn là chỉ sử dụng mỗi Text.

Có nhiều loại **Operating System – OS** (Hệ Điều Hành) trên thị trường, tuy nhiên Microsoft Windows là loại OS được sử dụng phổ biến nhất và cũng dễ sử dụng nhất cho **User** (Người Dùng), cho nên mặc định là User sử dụng Windows OS xuyên suốt Bộ Cẩm Nang này.

Phần chia sẻ thêm cho Bộ Cẩm Nang nằm ở kênh:
<https://youtube.com/c/HomerTruong66>.

DANH MỤC HÌNH

Hình 1: 30 Năm ĐAM MÊ IT
Hình 2: 30 Năm ĐAM MÊ Cờ Tướng
Hình 3: Nhập Môn IT
Hình 4: The First Computer - Xerox Alto
Hình 5: Logical Thinking Process
Hình 6: Effective Study Method
Hình 7: English for IT
Hình 8: English Tense Synthesis
Hình 9: Computer Structure
Hình 10: Keyboard (Source: Internet)
Hình 11: Hard Disk Physical Structure (Source: Internet)
Hình 12: Hard Disk Partitions
Hình 13: Operating System
Hình 14: CPU Management
Hình 15: Memory Management
Hình 16: Disk Management
Hình 17: Application Management
Hình 18: Software
Hình 19: Programming
Hình 20: Git Flow
Hình 21: Data Structure Synthesis
Hình 22: Linear Data Structure Examples
Hình 23: Non-Linear Data Structure Examples
Hình 24: Function Parameters
Hình 25: Data Scope
Hình 26: Activity Table
Hình 27: Call Stack
Hình 28: Procedural Programming
Hình 29: Object-Oriented Programming
Hình 30: Procedural Programming vs OOP
Hình 31: OOP Inheritance
Hình 32: OOP Characteristics
Hình 33: Aspect-Oriented Programming
Hình 34: Boiler-plate Code
Hình 35: Redundant Code
Hình 36: Hide Redundant Code
Hình 37: Bloated Code
Hình 38: Spaghetti Code
Hình 39: Database

DANH MỤC HÌNH

Hình 40: Relational Database Examples
Hình 41: ERD Examples
Hình 42: SQL Commands
Hình 43: HeidiSQL
Hình 44: MongoDB Compass
Hình 45: Computer Network
Hình 46: OSI Model
Hình 47: TCP/IP Model
Hình 48: TCP/IP vs OSI
Hình 49: Protocol
Hình 50: Client-Server Model
Hình 51: Web Application
Hình 52: HTML Page
Hình 53: Web Session
Hình 54: Software Engineering
Hình 55: Software Development Life Cycle – SDLC
Hình 56: Artificial Intelligence
Hình 57: Xiangqi Board
Hình 58: Minimax Algorithm
Hình 59: Alpha-Beta Algorithm
Hình 60: Node Traversal in Alpha-Beta Algorithm
Hình 61: Data Science Areas
Hình 62: Data Analytics
Hình 63: Data Mining
Hình 64: Top-down Data Warehouse
Hình 65: Bottom-up Data Warehouse
Hình 66: WEKA Tool
Hình 67: Java Technology
Hình 68: JDK vs JRE vs JVM
Hình 69: Database Row Mapper
Hình 70: Networking Application
Hình 71: Threading Program
Hình 72: Thread State
Hình 73: Thread Pool
Hình 74: Java World
Hình 75: Tech Product Development Process
Hình 76: Mentor
Hình 77: CSI Formula
Hình 78: CSI Formula Practice

DANH MỤC HÌNH

Hình 79: IT System Components

Hình 80: Software Development Process

Hình 81: Software Development Role -> Scrum Process Role

Hình 82: Scrum Process Pros & Cons

Hình 83: Software Development Life Cycle

Hình 84: UI Transition of eVP

Hình 85: Voucher Redemption Flow Chart

Hình 86: Software Architecture

Hình 87: Merchant Create/Update UI

Hình 88: Eisenhower Matrix

Hình 89: 5P Process

Hình 90: Career Orientation Plan

Hình 91: Bachelor Courses

Hình 92: Master Courses

Hình 93: Xiangqi Club

Hình 94: DMA

Hình 95: SER5500

Hình 96: USPA

Hình 97: Decision Tree

Hình 98: LMA

Hình 99: iVA

Hình 100: MPA

Hình 101: GPM

Hình 102: YCMS

Hình 103: YCRM

Hình 104: SmartPush

Hình 105: SmartIntegrate

Hình 106: Awesm

Hình 107: P4SVVN

Hình 108: RMS

Hình 109: SUPA

Hình 110: Microservices Synthesis

Hình 111: Docker Synthesis

Hình 112: Kubernetes Synthesis

Hình 113: AWS Synthesis

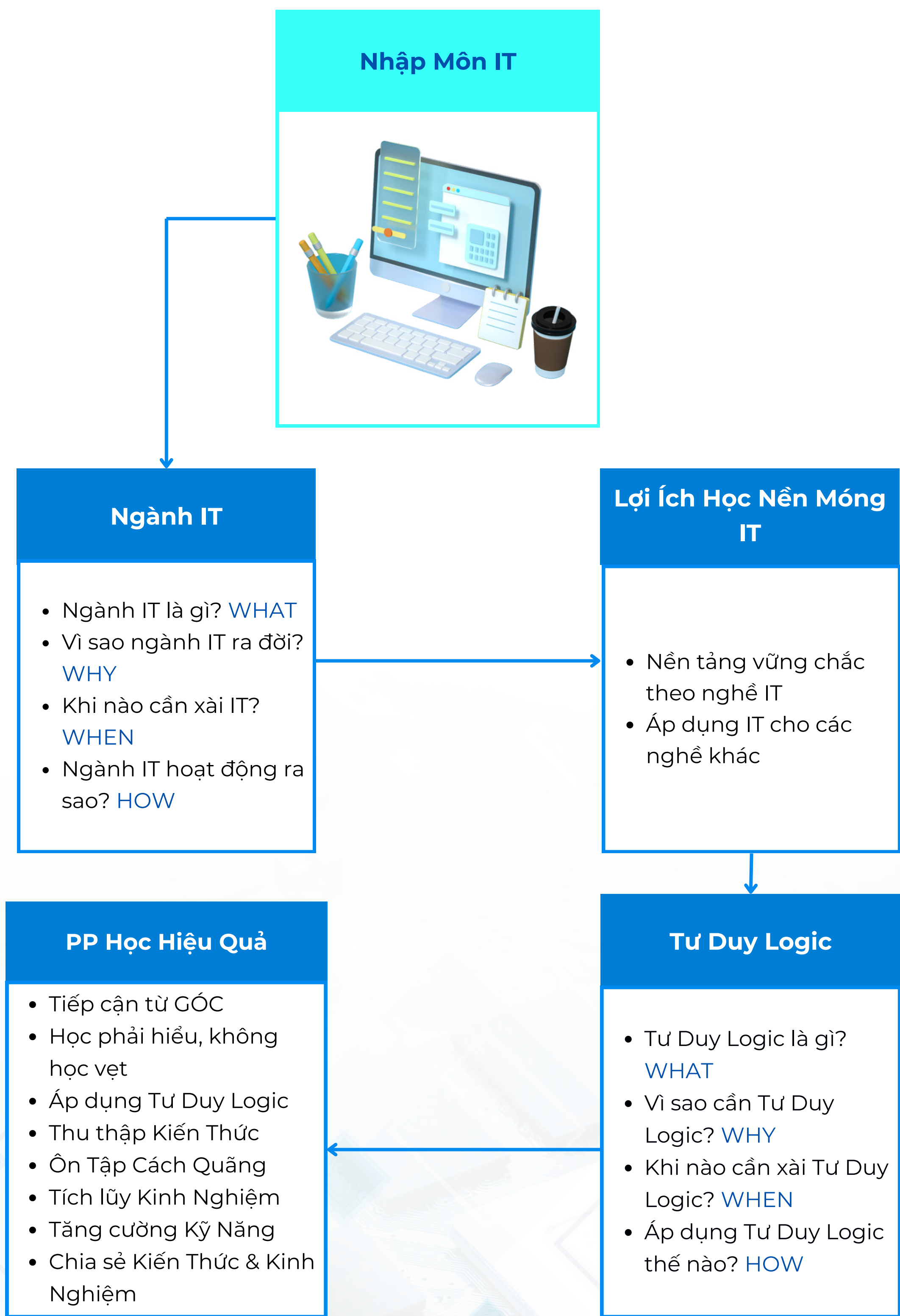
Hình 114: Reverse Engineering



PHẦN 1: NỀN TẢNG CÔNG NGHỆ THÔNG TIN

I. WELCOME TO IT WORLD (NHẬP MÔN IT)

KFD cho phần Nhập Môn IT được thể hiện trong hình dưới đây.



Hình 3: Nhập Môn IT

1. Ngành IT

Information Technology – IT (Công Nghệ Thông Tin) là ngành sử dụng **Computer** (Máy Vi Tính) để lưu trữ, truy cập, truyền tải và xử lý **Information** (Thông Tin) đem lại lợi ích cho **User** (Người Dùng), mở rộng ra là cho Enterprise (Doanh nghiệp), **Society** (Xã Hội), **Country** (Quốc Gia) và **Global** (Toàn Cầu).

Câu hỏi về việc **Computer đầu tiên** ra đời khi nào đến nay vẫn chưa có câu trả lời thống nhất trong thế giới IT. Theo tôi tìm hiểu thì Computer đầu tiên, giống Computer ngày nay nhất, tên là **Xerox Alto** ra đời năm 1974 có kích thước khá là to bao gồm:

- **Monitor** (Màn Hình)
- **User Interface – UI** (Giao Diện Người Dùng)
- **Keyboard** (Bàn Phím)
- **Mouse** (Chuột)
- **Case** (Thùng Máy) bên dưới rất bự như hình sau.



Hình 4: The First Computer - Xerox Alto
Source: https://en.wikipedia.org/wiki/Xerox_Alto

Ngày nay, những loại Computer mà User hay sử dụng bao gồm:

- **Desktop** (Computer Để Bàn): loại Computer này có các thành phần được đặt cố định ở nhà hoặc công ty.



- **Laptop** (Computer Xách Tay): loại Computer này có các thành phần gọn nhẹ nên được sử dụng khi User có nhu cầu di chuyển nhiều.



- **Smart Phone** (Computer Nằm Trong Điện Thoại): đây là chiếc điện thoại truyền thống dùng nghe/gọi có thêm tính năng của 1 Computer.



Sự ra đời của ngành IT đã đem đến rất nhiều hữu ích cho nhân loại toàn cầu, thay đổi mọi mặt đời sống xã hội loài người:

- Chuyển đổi việc quản lý **Paperwork** (Hành Chính), **Project** (Dự Án) một cách thủ công bằng ghi chép giấy tờ qua quản lý **Digitalization** (Số Hóa) bằng **Software** (Phần Mềm) vừa nhanh hơn và vừa chính xác hơn rất nhiều.
- Chuyển phần lớn giao tiếp của con người từ ngoài đời lên **Social Network** (Mạng Xã Hội), và từ **Telephone** (Gọi qua điện thoại) qua **Video Call** (Gọi có hình ảnh qua Internet) giúp giao tiếp nhanh hơn và thuận tiện hơn.
- Chuyển khá nhiều việc kinh doanh/giáo dục từ cửa hàng/lớp học ngoài đời lên **Online Business** (Kinh Doanh Trực Tuyến), **Online Education** (Giáo Dục Trực Tuyến) đạt hiệu quả cao hơn.
- Chuyển rất nhiều việc mua sắm từ chợ hay siêu thị ngoài đời lên **Online Shopping** (Mua Hàng Trực Tuyến) cực kỳ nhanh chóng và tiện lợi.
- Xuất hiện nhiều dịch vụ giao hàng dùng công nghệ như Grab, Gojek rất tiện dụng.
- Những dịch vụ **Online Game** (Trò Chơi Trực Tuyến) cũng nở rộ.
- ...

Với sức mạnh của ngành IT như vậy thì khi cần giải quyết 1 **Problem** (Vấn Đề) nào đó, bạn nên nghĩ ngay đến việc áp dụng IT. Tùy theo Problem nhỏ hay lớn mà bạn sẽ sử dụng các loại công cụ IT cho phù hợp, ví dụ:

- Cần quản lý việc vắng mặt của học sinh 1 lớp thì có thể sử dụng Software như Word/Excel của công ty Microsoft để in ra danh sách học sinh rồi đánh dấu trên đó.
- Cần quản lý học sinh của nhiều lớp trong 1 trường thì phải sử dụng Software về quản lý giáo dục cung cấp nhiều chức năng hơn như quản lý học sinh, môn học, thi cử, báo cáo...

Đầu ra của ngành IT là **Technology Product** – viết gọn hơn là **Tech Product** (Sản Phẩm Công Nghệ). Một vài vd các Tech Product đình đám hiện nay là:

- Tìm kiếm thông tin với Google Search: <https://www.google.com>
- Xem video trực tuyến với Youtube: <https://www.youtube.com>
- Mua hàng trực tuyến với Amazon eCommerce: <https://www.amazon.com>
- Quản lý khách hàng với Salesforce CRM: <https://www.salesforce.com>
- Giao tiếp mạng xã hội với Facebook: <https://www.facebook.com>
- Giao tiếp công việc với LinkedIn: <https://www.linkedin.com>
- Thuê khách sạn/nhà dạng chia sẻ để ở khi đi du lịch với Airbnb: <https://www.airbnb.com>

Để xây dựng được Tech Product thì cần sự phối hợp của nhiều mảng trong ngành IT, các mảng chính bao gồm:

- **Computer Structure** (Cấu Trúc Máy Tính)
- **Operating System** (Hệ Điều Hành)
- **Programming** (Lập Trình)
- **Database** (Cơ Sở Dữ Liệu)
- **Computer Network** (Mạng Máy Tính)
- **Software Engineering** (Công Nghệ Phần Mềm)

Ngoài ra còn các mảng phụ như **Artificial Intelligence** (Trí Tuệ Nhân Tạo) và **Data Science** (Khoa Học Dữ Liệu) cung cấp khả năng phân tích, xử lý dữ liệu, thống kê, dự đoán, chẩn đoán... vượt trội cho Tech Product.

Bạn sẽ lần lượt khám phá những mảng này trong phần IT Foundation sau này.

Tiếp theo, bạn sẽ tìm hiểu lợi ích học IT từ gốc, tư duy logic và phương pháp học hiệu quả trong các phần ngay sau đây.

2. Lợi Ích Học Nền Móng IT

Việc nắm được kiến thức IT từ **GỐC** sẽ giúp bạn:

- Xây dựng được một nền tảng IT vững chắc để theo đuổi chuyên sâu ngành IT nhanh hơn và hiệu quả hơn.
- Nếu bạn không theo ngành IT thì bạn vẫn có thể sử dụng kiến thức IT ứng dụng vào các ngành khác giúp bạn học/làm việc hiệu quả hơn bởi ngày nay hầu như ngành nào cũng áp dụng IT không nhiều thì ít nhằm nâng cao năng suất công việc của con người.

3. Tư Duy Logic (Sự Hợp Lý)

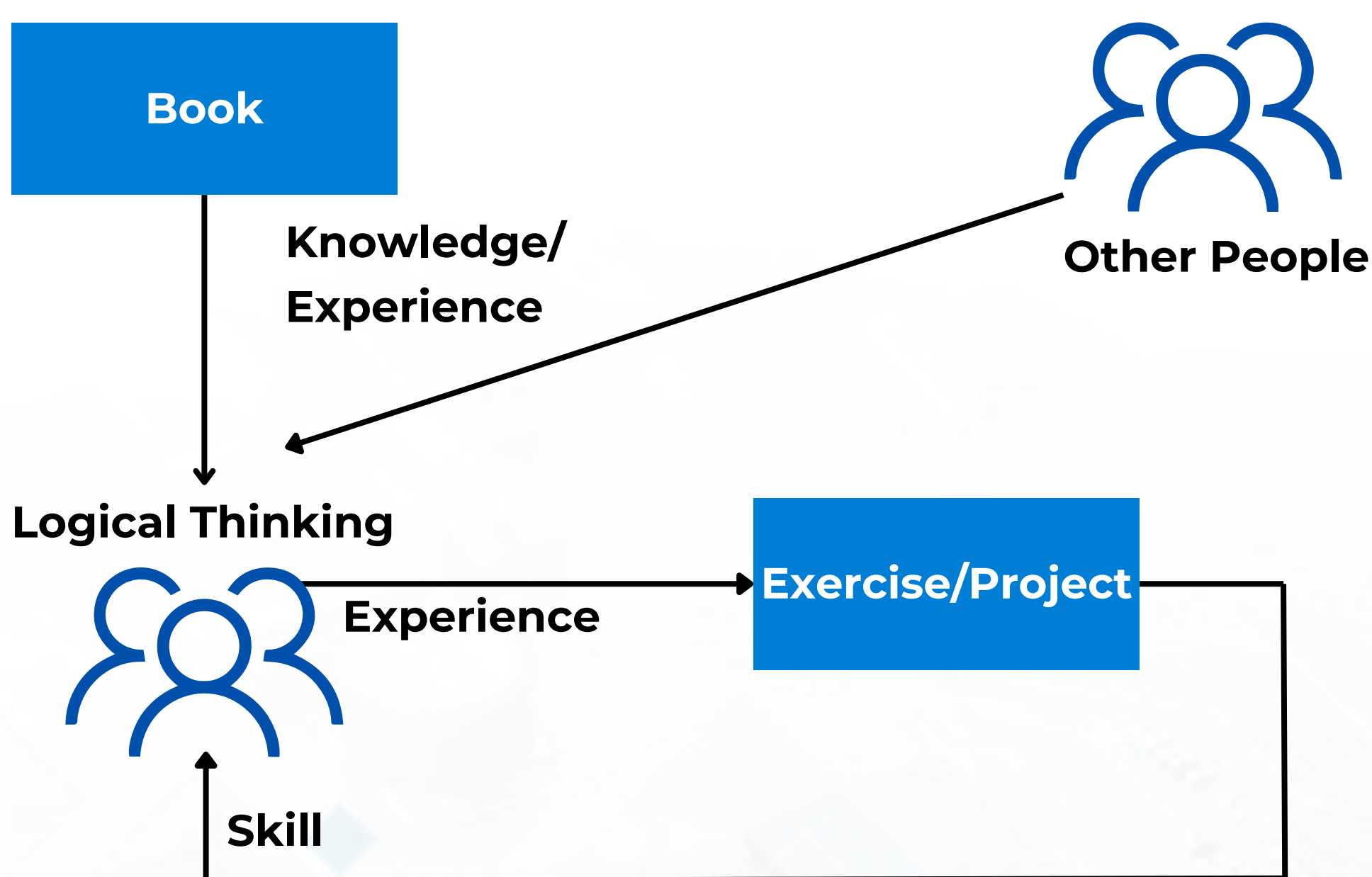
Tư Duy Logic (Sự Hợp Lý) là hoạt động của 1 người để phân tích các **Fact** (Dữ Kiện), **Event** (Sự Kiện) dựa trên **Knowledge** (Kiến Thức), **Experience** (Kinh Nghiệm) và **Reasoning** (Suy Luận) của họ nhằm đưa ra **Conclusion** (Kết Luận) hoặc **Solution** (Giải Pháp) cho 1 **Problem** (Vấn Đề) mà họ đang muốn tìm hiểu hoặc giải quyết. Ví dụ: Bạn cần tư duy Logic để hiểu xem cách User mua hàng trực tuyến thế nào, thanh toán ra sao là hợp lý... để thiết kế ra 1 Tech Product phục vụ kinh doanh trực tuyến cho bạn.

Tư Duy Logic ra đời giúp bạn đánh giá các **Situation** (Tình Huống) dựa trên Fact, Event nhằm hỗ trợ đưa ra các **Decision** (Quyết Định) hay giải quyết các Problem khác nhau trong cuộc sống. Bạn hãy luôn nhớ sử dụng Tư Duy Logic khi học một **Topic** (Chủ Đề) mới hoặc khi giải quyết những Problem mà bạn chưa gặp bao giờ hoặc bạn đã gặp mà chúng lại có những biến thể mới.

-> Phải tìm hiểu trước các Concept nền tảng như **Computer Program** (Chương Trình Máy Vi Tính), **Programming** (Lập Trình), **Translating Program** (Chương Trình Dịch), **Compiler** (Trình Biên Dịch), **Translator** (Trình Thông Dịch), **Debugging** (Tìm Lỗi)...

- **Học phải hiểu, không học vẹt:** tránh học như một con vẹt, tức là học thuộc lòng những kiến thức mà sách/vở hoặc thầy cô truyền đạt mà không hiểu rõ kiến thức đó thế nào, khi được hỏi thì đọc ra như con vẹt mà không hiểu.
- **Thu thập và hệ thống hóa Knowledge**
- **Áp dụng Tư Duy Logic**
- **Tích lũy Experience**
- **Tăng cường Skill**
- **Ôn tập cách quăng**
- **Chia sẻ Knowledge và Experience vừa thu được với người khác**

Phương pháp học hiệu quả được thể hiện qua mô hình sau đây:



Hình 6: Effective Study Method

- Bước đầu tiên là bạn thu thập Knowledge/Experience của Topic từ **Book** (Sách Vở) hay **Other People** (Người Khác: Thầy/Cô/Sếp/Đồng Nghiệp/Trainer/Mentor), có nghĩa là bạn có thể mua sách tự đọc hoặc đi học ở trường hay các trung tâm có thầy/cô cung cấp Knowledge/Experience của họ cho bạn. Sau đó bạn cần hệ thống hóa lại lượng kiến thức khổng lồ mà bạn đã tiếp nhận.
- Tiếp theo bạn áp dụng Tư Duy Logic để hiểu Topic.
- Sau khi nắm được Topic thì bạn bắt đầu **Practice** (Thực Hành) với **Exercise** (Bài Tập) hoặc **Project** (Dự Án) nhằm tích lũy Experience.

- Practice càng nhiều thì bạn càng thuần thục sẽ giúp tăng cường **Skill** (Kỹ Năng) để đến 1 lúc nào đó thì bạn làm vừa nhanh vừa hiệu quả mà không cần suy nghĩ, sau đó bạn có không làm 1 thời gian thì khi quay lại bạn vẫn làm được, Ví dụ: Skill bơi, lái xe, lập trình...
- Sau đó bạn cần thực hiện **Spaced Repetition** (Ôn Tập Cách Quãng) tức là ôn luyện Knowledge lặp đi lặp lại cách nhau các khoảng thời gian như vài tuần/tháng chẳng hạn.
- Cuối cùng là bạn nên tìm cơ hội để chia sẻ Knowledge và Experience mà bạn vừa thu được với người khác để giúp bạn hiểu rõ hơn và nhớ lâu hơn nữa.

5. Practice (Thực Hành)

Bạn hãy trả lời các câu hỏi dưới đây bằng cách vẽ/viết ra, sau đó tìm bạn bè để chia sẻ lại với họ dưới dạng thuyết trình những gì bạn đã hiểu.

- Ngành IT là gì?
- Vì sao IT ra đời?
- Khi nào cần sử dụng IT?
- Lợi ích của việc học IT từ GỐC ?
- Tư Duy Logic là gì?
- Vì sao cần Tư Duy Logic?
- Khi nào cần sử dụng Tư Duy Logic?
- Áp dụng Tư Duy Logic như thế nào?
- Phương Pháp Học Hiệu Quả mà bạn đang sử dụng ra sao?
- Phương Pháp Học Hiệu Quả đã trình bày ở trên như thế nào?
- So sánh sự giống và khác nhau giữa 2 Phương Pháp Học Hiệu Quả ở câu i. & j.
- Hãy trích xuất những từ English trong chương này ra 1 bảng, ứng với mỗi từ hãy đặt 1 câu bằng English có chứa từ đó.

Đến đây bạn đã có cái nhìn tổng quan về ngành IT. Bạn đã sẵn sàng để khám phá thế giới công nghệ chưa nào? Tôi đoán chắc là bạn đã sẵn sàng 😊

Tuy nhiên, hãy dừng lại một tí..., để tìm hiểu ngành IT tốt nhất thì một thứ cực kỳ quan trọng mà bạn bắt buộc phải trang bị hoặc củng cố, đó là... **IT English** sẽ được trình bày trong chương tiếp theo.

II. IT ENGLISH (TIẾNG ANH IT)

English là ngôn ngữ được dùng chính thức ở nước **United States of America – USA** (Mỹ), nơi ngành IT được khai sinh và phát triển mạnh mẽ lan ra khắp thế giới, cũng như được sử dụng phổ biến trong giao tiếp toàn cầu giữa những nước nói ngôn ngữ khác English.

Bạn phải chuẩn bị vốn English thật tốt để học IT vì mọi thuật ngữ trong ngành IT đều là English và các tài liệu về ngành IT đều được viết bằng English. Hiển nhiên có nhiều tài liệu được dịch ra các ngôn ngữ khác nhau phục vụ người dân của những nước đó, như sách IT Tiếng Việt, nhưng bạn nên đọc tài liệu GỐC thì sẽ hiểu rõ và chính xác ý của tác giả hơn.

English cũng như bao ngôn ngữ khác rất là rộng lớn, bạn không thể học hết mọi ngóc ngách của nó và cũng không cần thiết để học như vậy trừ khi bạn muốn đi sâu vào ngành nghiên cứu ngôn ngữ học chuyên ngành English. Thế thì học English chỉ để phục vụ cho việc học IT thôi thì bạn cần học những gì?

Trước hết thì bạn chỉ cần học **English Foundation** (Nền Tảng English) và **Reading Skill** (Kỹ Năng Đọc Hiểu). Khi nào bạn đi làm thì sẽ phải học thêm 3 thứ khác nữa là **Writing Skill** (Kỹ Năng Viết), **Listening Skill** (Kỹ Năng Nghe) và **Speaking Skill** (Kỹ Năng Nói).

Cách học English Foundation và Reading Skill được đặc tả chi tiết trong KFD dưới đây:

1. **Article → Noun, Pronoun:** a Monitor, the Other

2. **Adjective → Noun, Pronoun:** big Data, small Ones

3. **Adverb → Adjective:** extremely critical, really needed

4. **Adverb → Adjective → Noun:** really needed Requirement

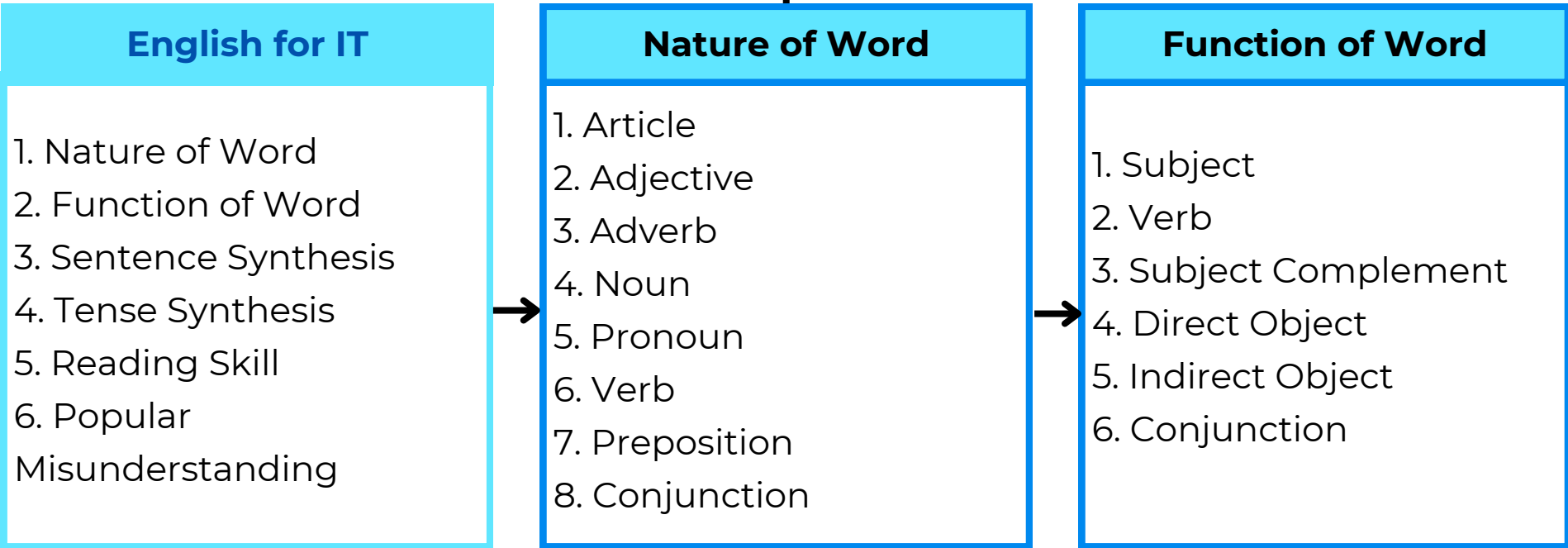
5. **Adverb → Verb:** work charmingly

- **Adjective:** qualifying, numeral, demonstrative, possessive, interrogative, indefinite

• **Adverb:** manner, time, place, quantity, frequency, negation

• **Pronoun:** personal, relative, demonstrative, possessive, interrogative, indefinite

• **Conjunction**
+ **Coordinating:** and, but, so, both...and, either...or, neither...nor, not only...but also
+ **Subordinating:** who/whom, which, that, whose, when, where | that, if/whether | before/after, when, until | because, so that, in order to | although | instead, instead of, except, without



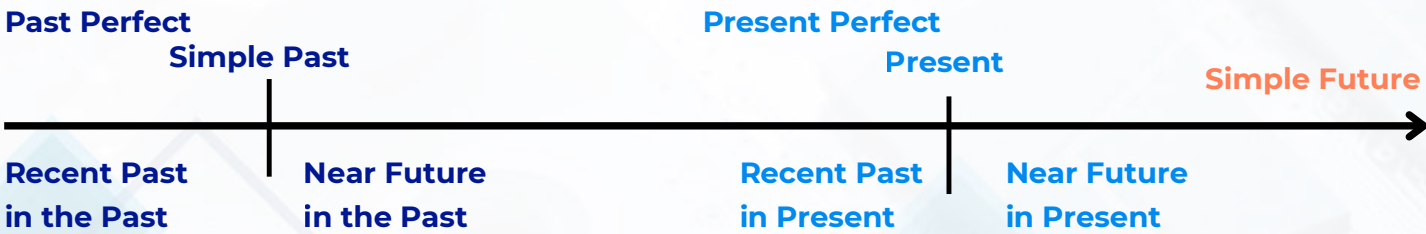
01. Subject + Verb + Subject Complement.
02. Subject + Verb + Direct Object.
03. Subject + Verb + Direct Object + Bare Infinitive.
04. Subject + Verb + Direct Object + To Infinitive.
05. Subject + Verb + To Infinitive.
06. Subject + Verb + Gerund.
07. Subject + Verb + Direct Object + to Indirect Object.
08. Subject + Verb + Indirect Object + Direct Object.
09. It's Adj [for Object] to Infinitive. (Expletive)
10. Clause 1+ and/but/so + Clause 2.
11. Complex Sentence

- Tense Synthesis
- 5 Past Tenses

• 5 Present Tenses

• 2 Future Tenses

- Sentence Synthesis
- Simple Sentence: 01-09 (above)
Compound Sentence: 10 (above)
Complex Sentence:
• **Adjective Clause:**
+ Subject (people/animal/thing)
+ Direct Object
+ Indirect Object
+ Noun Complement
+ Adverbial Complement
• **Adverb Clause:**
+ Time Clause
+ Condition Clause
+ Cause Clause
+ Purpose Clause
+ Contrast Clause
• **Noun Clause**



- **Past Tenses**

1. Simple Past: V2 (Past)

2. Past Continuous: Was/Were + V-ing

3. Near Future in the Past: Was/Were going to + V1

4. Recent Past in the Past: Had just +V3

5. Past Perfect: Had + V3

• **Present Tenses**

1. Present: V1

2. Present Continuous: Am/Is/Are + V-ing

3. Near Future at Present: Am/Is/Are going to + V1

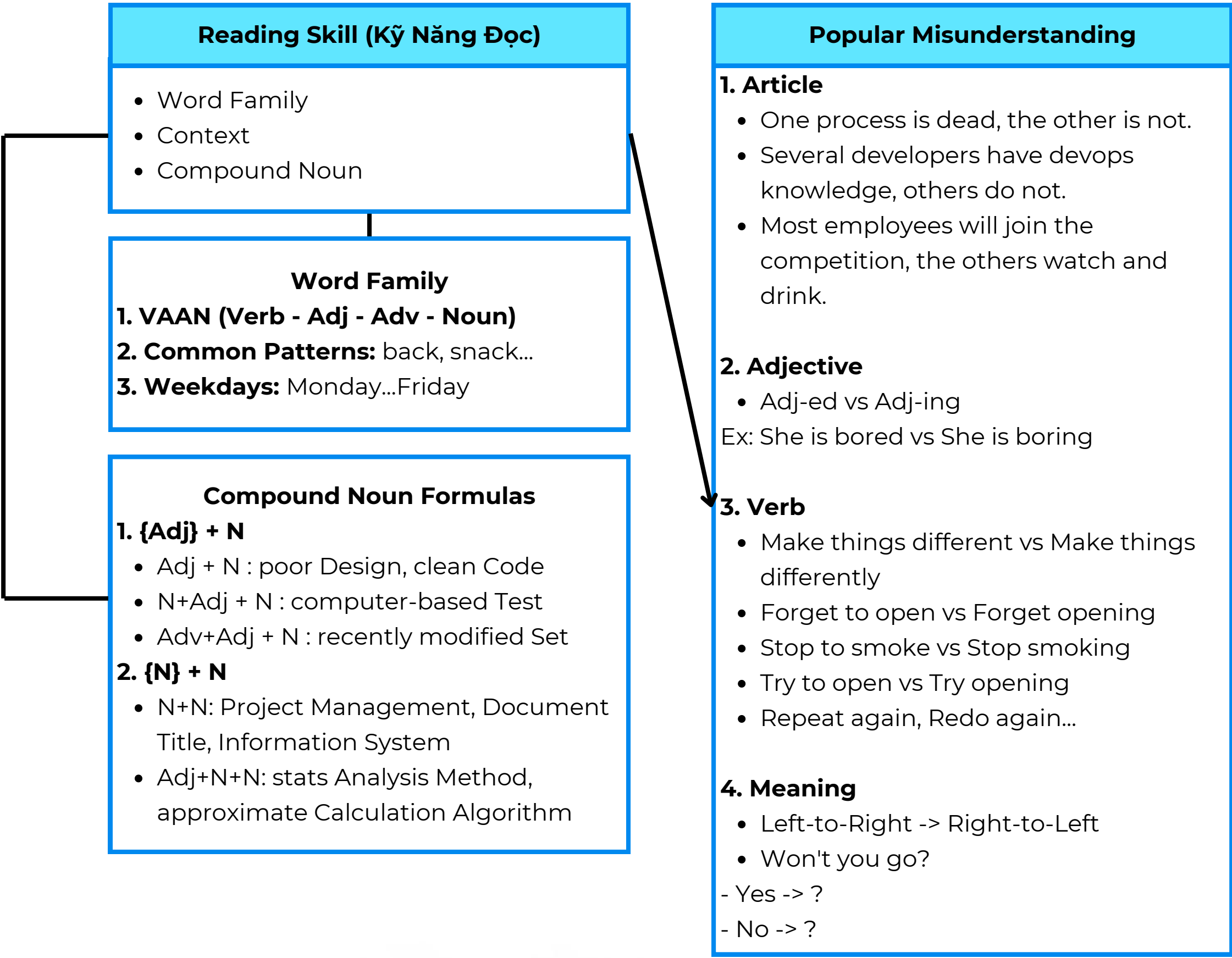
4. Recent Past at Present: Have/Has just + V3 (Past .articipple)

5. Present Perfect: Have + V3

• **Future Tenses**

1. Simple Future: Will/Shall + V1

2. Future Continuous: Will/Shall + be + V-ing



Hình 7: English For IT

1. Đầu tiên bạn phải nắm rõ **Nature of Word** (Bản Chất Của Từ), tức là hiểu được bản chất của mỗi từ loại (ví dụ: Tính Từ, Danh Từ, Động Từ...) trong English có ý nghĩa gì, quan hệ giữa chúng ra sao, mỗi từ loại được chia nhóm thế nào và khi nào sử dụng.
2. Tiếp theo bạn phải hiểu được **Function of Word** (Chức Năng Của Từ), có nghĩa là khi một từ English đứng trong câu nó có nhiệm vụ gì.
3. Sau đó bạn phải biết được các loại **Sentence** (Câu) trong English.
4. Tiếp tục phải nắm được các loại **Tense** (Thì) trong English để hiểu đúng các hành động trong 1 câu xảy ra ở thời điểm: **Past** (Quá Khứ), **Present** (Hiện Tại), **Future** (Tương Lai).
5. Sau khi bạn nắm vững những phần nền tảng trên thì sẽ tìm hiểu những yếu tố giúp cho việc đọc hiểu English tốt: **Word Family** (Gia Đình Từ), **Compound Noun** (Danh Từ Ghép) và **Context** (Ngữ Cảnh).
6. Cuối cùng, bạn nên xem qua một số **Popular Misunderstanding** (Hiểu Nhầm Phổ Biến) của người học English để mà tránh mắc sai sót.

1. Nature of Word (Bản Chất Của Từ)

Mỗi từ English sẽ có bản chất là một trong các từ loại sau:

1. **Article** (Mạo Từ) : xác định Noun

2. **Adjective** (Tính Từ) : đặc tả tính chất của Noun/Pronoun

3. **Adverb** (Trạng Từ) : đặc tả Time (Thời Gian), Place (Vị Trí), Manner (Thế Cách)...

4. **Noun** (Danh Từ) : thể hiện con người, loài vật, đồ vật...

5. **Pronoun** (Đại Từ) : thay thế cho Noun giúp cho câu ngắn gọn hơn

6. **Verb** (Động Từ) : thể hiện hành động

7. **Preposition** (Giới Từ) : một loại từ đặc trưng của English có nhiều chức năng

8. **Conjunction** (Liên Từ) : kết nối các từ hoặc các thành phần trong câu

Một số loại từ này có quan hệ bổ nghĩa cho nhau, ví dụ:

- **Article** bổ nghĩa cho **Noun/Pronoun**, ví dụ: a Monitor, the Other
 - **Adjective** bổ nghĩa cho **Noun/Pronoun**, ví dụ: big Data, small Ones
 - **Adverb** bổ nghĩa cho **Adjective/Verb**, ví dụ: extremely critical, work charmingly
- Hiểu được sự bổ nghĩa cho nhau của các loại từ giúp chúng ta đọc hiểu đúng ý câu. Ngay sau đây là phần tổng hợp kiến thức của mỗi từ loại.

a. Article (Mạo Từ)

1. **the** (definite) : **The engineer** constructed **the bridge** last year.

2. **a / an** (indefinite) : That's **a good idea**. It plays **an important role** in that field.

3. **<none>** : Nurses look after **patients** in hospital.

b. Adjective – Adj (Tính Từ)

1. **Qualifying** (So Sánh):
 - **as** Adj **as** : **as** slow **as**, **as** intelligent **as**
 - Adj + **er, less/more** Adj **than** : slower, **less/more** important **than**
 - **the** Adj + **est, the least/most** Adj : **the** slow**est**, **the least/most** important

2. **Cardinal/Ordinal** (Số Đếm, Số Thứ Tự):
 - **one** (1), **two** (2), **three** (3), **four** (4), **five** (5), **six** (6)...
 - **first** (1st), **second** (2nd), **third** (3rd), **fourth** (4th), **fifth** (5th), **sixth** (6th)...

3. **Demonstrative** (Thể Hiện):
 - **this** : Did you watch **this really boring movie**.
 - **that** : I used to cross **that fairly big jungle** which has a lot of valuable trees.
 - **these** : **These nice boys and girls** look happy.
 - **those** : **Those important things** are already done.

4. Possessive (Sở Hữu):

- **my** : **My father** is a pharmacist.
- **your** : I saw **your mother** last night in the shopping center.
- **his, her, its** : **His house, her house, its house** have been all decorated for X'mas.
- **our** : I think **our project** will be done in time.
- **your** : Do you usually use **your blender** to make orange juice?
- **their** : **Their company** is gradually growing in this market.

5. Interrogative (Hỏi Đáp):

- **what** : **What time** is it?
- **which** : **Which book** have you bought? **Which homework** do you do?
Which lessons do you recite?

6. Indefinite (Không Xác Định):

- **some** : **Some pupils** are satisfied with the results, others are not.
- **many** : I see that **many students** in this class are qualified for the tournament.
- **other** : **Other people** will not follow that trend.
- **certain** : We need to solve **certain problems** before getting it done.
- **the same** : **The same procedure** will be applied for this case.
- **each** : I usually travel 2 or 3 times **each month**.
- **every** : There's always an exciting party at that beautiful place **every year**.
- **all** : It took me **all day** to finish my task.

c. Adverb – Adv (Trạng Từ)

1. Manner (Thể Cách):

- Adj + **ly** : **quickly, rapidly, slowly, carefully, gradually, fluently...**
- Irregular (Bất qui tắc) : **fast, together, in this way, like this, well, bad, on purpose, as...**

2. Time (Thời Gian): **before/after, as soon as, as soon as possible (asap), at once, at the moment, following, immediately, during the summer, last year, next year, now, since, soon, Sunday morning, the day before yesterday, yesterday, today, tomorrow, the day after tomorrow, two years ago, when, whenever, at any time, while...**

3. Place (Vị Trí): **above, below, top, bottom, where, in front of, behind...**

4. Quantity (Số Lượng): **many/much, more, very, so, too, enough, as well as, almost, quite, entirely**

5. Frequency (Tần Suất): **always, almost, mostly, nearly, usually, often, sometimes, occasionally, rarely, hardly, never**

6. Negation (Phủ Định): **no, not, never, no longer, only**

Ngoài Adv chỉ thể cách có số lượng rất nhiều, những Adv còn lại có số lượng hữu hạn được lập đi lập lại nhiều nên bạn hãy cố gắng thuộc hết mặt chữ để đọc cho nhanh.

d. Noun (Danh Từ)

1. Proper (Riêng): Vietnam, France, Canada, United States

2. Common (Chung): people, animal, botany

3. Concrete (Cụ Thể): table, chair, desk, bed

4. Abstract (Trừu Tượng): happiness, sadness, anxiety

5. Collective (Tập Hợp): group, team

6. Compound (Ghép): beautiful Girl, computer-based Test, recently used Book, document Title, information System, statistical analysis Method

e. Pronoun (Đại Từ)

1. Personal (Ngôi Thứ):
 - **I/me** : I love you. Do you love **me**?
 - **you/you** : **You** are a very good student. Your parents are proud of **you**.
 - **he/him** : **He** is an IT expert. We'll need **him** to solve our problems.
 - **she/her** : **She** is a great singer. Everyone wants to meet **her**.
 - **it/it** : **It**'s an interesting movie. You should plan to watch **it**.
 - **we/us** : **We** are on top of the world. Anyone has to look at **us**.
 - **you/you** : **You** are a super team. Here is the award for **you**.
 - **they/them** : **They** made his day. He thanked **them** a lot.

2. Relative (Quan Hệ):
 - **who** : It took me a while to get used to people **who** eat popcorn during the movie.
 - **which** : The science fair, **which** lasted all day, ended with an awards ceremony.
 - **that** : The house **that** Jack built is large.
 - **whom** : The professor, **whom** I respect, recently received tenure.
 - **whose** : The book **whose** author won a Pulitzer has become a bestseller.
 - **when** : That was the miserable time **when** I started my Master study.
 - **where** : This is the house **where** I lived when I first came to Canada.

3. Demonstrative (Thể Hiện):
 - **this/that, these/those** : **That**'s a good idea.
 - **the one/ones** : This pretty lady is **the one** I love.
 - **this/that one, these/those ones** : I got 3 cars: **this one** is white, and **those ones** are black.

4. Possessive (Sở Hữu):
 - **mine**: This book is **mine**.
 - **yours**: My girlfriend is very nice, how about **yours**?
 - **his**: I love my mother, he loves **his**.
 - **hers**: He has a really wise dog. **Hers** is smart too.
 - **ours**: These expensive cars are **ours**.
 - **yours**: It looks like **yours**.
 - **theirs**: We take care our old parents, they take care **theirs**.

5. Interrogative (Hỏi Đáp):

- **who** : **Who** are you?
- **what** : **What** is more beautiful than the sun?
- **when** : **When** will you be free?
- **where** : **Where** do you come from?
- **why** : **Why** does he want to do that?
- **how** : **How** are you?
- **how much** : **How much** money do you earn a month?
- **how many** : **How many** bottles are there in the refrigerator?

6. Indefinite (Không Xác Định):

- **someone/somebody** : **Someone** wants to meet you.
- **something** : Is there **something** wrong here?
- **anyone/anybody** : **Anyone** who is rich has to help the poor.
- **anything** : You can do **anything** you want.
- **everyone/everybody** : Hello everybody! **Everyone** has arrived.
- **everything** : **Everything** will fade away.
- **no one/nobody** : **Nobody** has arrived.
- **nothing** : There's **nothing** special today.

f. Verb (Động Từ)

1. **Bare/To Infinity – V1** (Nguyên Mẫu [có “to”]): [to] be, [to] work, [to] give

2. **Present Participle – V2**: was/were, worked, gave

3. **Past Participle – V3**: been, worked, given

g. Preposition (Giới Từ)

1. **Time** (Thời Gian): before (8 o'clock), after (8 o'clock), during (my study), since (some years), at (the same time), from (now), until (8 o'clock)...

2. **Place** (Địa Điểm/Vị Trí): on (the table), in (Montreal), towards (the bridge), in (France), in (Vietnam), at (my house), between (North and South), in (the laboratory), among (the nations), across (the street), through (the wall), out of (the house), under (the table), on top of (the sea), far from, beyond (my reach), inside, outside...

3. **Cause** (Nguyên Nhân): because of (the rain), due to (the nervousness)...

4. **Purpose** (Mục Đích): to, in order to, so as to (win the lottery) ...

5. **Means/Manner** (Phương Tiện/Thế Cách): on (foot), by (car), in (a systematic manner), by means of, with (a machine), by (this way), according to (him), following (the law) ...

6. **Consequence** (Kết Quả) as a result of

7. **Accompaniment** (Đồng Hành): with (me)

8. **Agent** (Chủ Thể): by (Mr A)

9. **Possession** (Sở Hữu): of (someone)

10. **Attribution** (Dành Cho): for (my father)

11. Opposition (Đối Lập): against

12. Condition (Điều Kiện): in condition of

13. Exception/Exclusion (Ngoại Lệ/Loại Trừ): without, except, except for, exclusif...

Example:

- **at** + time -> **at** 6 o'clock, noon, midnight
- **on** + date -> **on** Friday January 13th, 2004
- **on** + day -> **on** Monday, Tuesday...
- **on** + holiday -> **on** my holiday
- **in** + month -> **in** January
- **in** + year -> **in** 1980
- **in** + moment -> **in** the morning, afternoon, evening
- **in** + season -> **in** spring, summer, autumn, winter

h. Conjunction (Liên Từ)

1. Coordinating (Hợp Tác):

- and, but, so
- both...and, either...or, neither...nor, not only...but also

2. Subordinating (Bổ Trợ):

- *In Adjective Clause (Trong Mệnh Đề Tính Từ):*

+ who, which, that, whom, whose, when, where

- *In Adverb Clause (Trong Mệnh Đề Trạng Từ):*

+ Time: after, before, when (while), until, for, since

+ Condition: if (3 cases), on the condition that

+ Cause: because, because of, due to, due to the fact that, for, too...to, so...that

+ Purpose: to, in order to, so as to, so that

+ Weak contrast: although, in spite of, despite, even though, even if, still

+ Contrast: instead, instead of, except, except for, without, while, in contrast, on the contrary, on the other hand

- *In Noun Clause (Trong Mệnh Đề Danh Từ):* for, that, if/whether, wh_

i. Practice (Thực Hành)

Bạn hãy trả lời các câu hỏi dưới đây bằng cách vẽ/viết ra, sau đó tìm bạn bè để chia sẻ lại với họ dưới dạng thuyết trình những gì bạn đã hiểu.

1. English có những loại từ gì?

2. Quan hệ giữa những loại từ ra sao? Cho ví dụ cho mỗi mối quan hệ và đặt 1 câu tương ứng.

3. Mỗi loại từ được phân chia thành những nhóm nhỏ nào? Ứng với mỗi nhóm đặt 1 câu có chứa loại từ đó.

4. Đọc 1 bài English với chủ đề mà bạn thích, sau đó:

- a. Xác định từng từ trong bài đọc thuộc loại từ gì? và thuộc nhóm nào trong loại từ đó?
- b. Xác định các quan hệ giữa những loại từ trong bài đọc

Mỗi từ trong English có một bản chất tự nhiên, vậy thì khi nó đứng trong câu sẽ có chức năng gì để đem đến ý nghĩa cho câu?

Phần tiếp theo sẽ giúp bạn biết được các chức năng của từ trong câu English.

2. Function of Word (Chức Năng Của Từ)

Giống như tiếng Việt, cấu trúc câu của English cũng bao gồm 2 phần:

Subject (Chủ Ngữ) + **Predicate** (Vị Ngữ).

Mỗi từ English có 1 bản chất duy nhất, nhưng khi đứng trong 1 câu thì nó có thể có nhiều chức năng. Việc hiểu rõ chức năng của từ sẽ giúp bạn hiểu đúng nghĩa của câu khi đọc.

a. Subject (Chủ Ngữ)

Từ có chức năng Subject đóng vai trò nhân tố chính của câu. Các loại từ có thể làm Subject bao gồm Noun và Pronoun, ví dụ:

- **Information Technology** is a very interesting domain.
- **It's** also super challenging.

Đôi lúc Verb cũng có thể làm Subject, ví dụ: **To live** is to fight.

b. Verb (Động Từ)

Từ có chức năng Verb đóng vai trò thể hiện hành động của Subject hoặc diễn tả về Subject khi đi với Verb "to be", ví dụ:

- Pupils **go** to school and students **go** to university everyday.
- Mai **is** a student. She **is** really kind.

c. Subject Complement (Bổ Nghĩa Cho Chủ Ngữ)

Từ có chức năng Subject Complement đóng vai trò bổ nghĩa cho Subject. Các loại từ có thể làm Subject Complement là Noun và Adjective, ví dụ:

- I am **an IT expert**.
- Software Engineering is **quite complex**.

d. Direct Object (Đối Tượng Trực Tiếp)

Từ có chức năng Direct Object đóng vai trò nhận tác động trực tiếp từ Subject. Các loại từ có thể làm Direct Object bao gồm Noun và Pronoun, ví dụ:

- In the match yesterday, the football player **shot the ball** quickly and scored.
- Business Analyst **helps development team** clarify software requirements.
- A Software Developer **asks the Business Analyst to** explain a user story.

e. Indirect Object (Đối Tượng Gián Tiếp)

Từ có chức năng Indirect Object đóng vai trò nhận tác động gián tiếp từ Subject. Các loại từ có thể làm Indirect Object bao gồm Noun và Pronoun, ví dụ: I will **send** a surprising gift **to her** on her birthday.

f. Conjunction (Liên Từ)

Từ có chức năng Conjunction đóng vai trò kết nối các từ hoặc các **Clause** (Mệnh Đề) trong câu, ví dụ: You should have a good career orientation **before** choosing a profession.

Ghi Chú:

- Một loại từ có thể đảm nhiệm nhiều chức năng khác nhau, ví dụ: Noun có thể làm chức năng **Subject** hoặc **Object**, ví dụ:
- **Water melon** is very delicious. (Subject)
- I like to eat **water melon** a lot. (Object)
- Một trường hợp đặc biệt gọi là **Expletive** (Hư Từ): “It's Adjective [for Object] to...”, nghĩa là cụm từ “It's Adjective” xuất hiện trong câu nhưng hư danh, không đóng góp vào nghĩa của câu mà chỉ giúp cho câu trọn vẹn về mặt cú pháp, ví dụ: **It's quite difficult for developers to troubleshoot** issues on production servers -> Có nghĩa là “**Thật là khó cho các lập trình viên để tìm sự cố** trên môi trường sản xuất”.

g. Practice

Bạn hãy trả lời các câu hỏi dưới đây bằng cách vẽ/viết ra, sau đó tìm bạn bè để chia sẻ lại với họ dưới dạng thuyết trình những gì bạn đã hiểu

1. Từ trong English có những Function gì? Cho ví dụ cho mỗi Function của từ.
2. Đọc 1 bài English với chủ đề mà bạn thích.
3. Ứng với mỗi câu trong bài đọc trên, xác định các Function của từng từ trong câu.

Đến đây bạn đã hiểu rõ mỗi loại từ sẽ có bản chất là gì và có chức năng như thế nào khi xuất hiện trong câu. Vậy thì English có bao nhiêu loại câu?

Bạn sẽ nắm vững từng loại câu trong English ngay trong phần sau.

3. Sentence Synthesis (Tổng Hợp Câu)

Tương tự Tiếng Việt, English cũng có 3 loại câu như sau:

- 1. **Simple Sentence** (Câu Đơn)
- 2. **Compound Sentence** (Câu Ghép)
- 3. **Complex Sentence** (Câu Phức)

Chúng ta lần lượt tìm hiểu mỗi dạng câu ngay dưới đây.

a. Simple Sentence (Câu Đơn)

Câu đơn chỉ có 1 **Clause** (Mệnh Đề) gồm 1 Subject & 1 Predicate diễn tả 1 ý nghĩa, ví dụ: Pupils go to school. Workers go to work.

Các dạng câu đơn được tổng hợp trong bảng sau đây.

1. Subject + Verb + S.Complement : I am an engineer .
2. Subject + Verb + DO : I miss you very much.
3. Subject + Verb + DO + Bare_Inf : I helped her finish the difficult exercises.
4. Subject + Verb + DO + To_Inf : I want you to be straightforward and faithful.
5. Subject + Verb + To_Inf : I want to be a president.
6. Subject + Verb + V-ing : I have delayed leaving for school.
7. Subject + Verb + DO + to IO : I will send a surprising gift to her on her birthday.
8. Subject + Verb + IO + DO : I will send you a letter sealed with a kiss.
It's Adj for Object To_Inf : It's difficult for me to make a decision.

b. Compound Sentence (Câu Ghép)

Câu ghép do nhiều Clause độc lập ghép lại. Các Clause độc lập này được kết nối với nhau bởi các Conjunction thông dụng là “**and**”, “**but**”, “**so**”, ví dụ:

- Developers implement features, **and** QCs prepare test cases.
- There were a lot of issues during the sprint, **but** we have just finished the sprint well.
- There are some misunderstandings among teams, **so** we need a meeting to clarify

Ghi Chú: Nếu bỏ đi 1 trong các Clause thì phần còn lại vẫn còn có nghĩa bởi các Clause trong câu ghép độc lập với nhau.

c. Complex Sentence (Câu Phức)

Câu phức được tạo thành từ 1 **Main Clause** (Mệnh Đề Chính) và có thể có 1 hoặc nhiều **Subordinate Clause** (Mệnh Đề Phụ) bổ nghĩa cho Main Clause.

Subordinate Clause có 3 loại: **Adjective Clause**, **Adverb Clause** và **Noun Clause**.

- **Adjective Clause** (Mệnh Đề Tính Từ)

1. Subject (people): who/ that/ present participle

- I have just met a PM **who** has great management skill. (*defined*)
- James Gosling, **who** is the father of Java, has left Oracle. (*non-defined*)
- People **that** have design ideas like yours are really rare.
- User **having** read privilege can access that folder.

2. Subject (animals/things): which/ that/ past participle/ -

- It is the subject **which/that** interests me.
- The directory **used** to store source code is nearly full.
- Flan, **a** kind of cake, is tasty.
- Note: I have read several articles, **most of which** were useful.

3. Direct Object (people/animals/things): whom/ which/ that/ -

- The man [**whom**] you met yesterday is the BA of our team.
- The Java book [**which/that**] I bought last week was very expensive.
- The monitoring tool we have used so far is extremely helpful. (- : *omitted*)

4. Indirect Object (people/animals/things): preposition + whom/which

- That is the client representative **to whom** you have talked.
- It is an interesting project **on which** I will work.
- She has suggested a topic [**which**] we have been discussing **about**.

5. Noun Complement: whose

- The serializable object is the one **whose** properties are all serializable.

6. Adverbial Complement: when, where

- The season **when** we organize technical events is the summer.
- That is the directory **where** we store all configuration and log files.

- **Adverb Clause** (Mệnh Đề Trạng Từ)

1. Time Clause:

- **Before** coming Canada, I didn't know French.
- I started learning French **when** I came to Canada in 2006.
- I have been learning French **since** 2006.
- I took 2 French courses 2 years **ago**.
- I have been learning French **for** 3 years.

2. Condition Clause:

- If I **had known**, I **wouldn't have waited** for her.
- If I were a millionaire, I **would buy** that de lux car.
- If I **can register** for that course, we **will study** together.
- I **will go** out **on the condition that** the weather **is** good.

3. Cause Clause:

- The code is **so** messy **that** we can not maintain.
- The code structure is **too** spaghetti **to** refactor.
- **Thanks to** QC and development teams, we can release on time.
- We can not connect to the server **because of** down time

4. Purpose Clause:

- I usually practice designing software architecture **so that** I can become a TA.
- We should refactor the source code **in order to/to** maintain better.

5. Contrast Clause:

- **Although** we have tried the hot fix, the system is still buggy.
- Memcached does not support set operation, let's use Redis **instead**.
- We can deploy the code change **without** having to restart the server.

• Noun Clause (Mệnh Đề Danh Từ)

- I was happy **that** I had won the lottery.
- I am happy **that** I have won the lottery.
- I was excited **to go** to the cinema last night.
- I don't know **if** her office is on this floor.
- Please tell me **where** we should push our commits to.
- I don't know **what** I have to do to fix that issue.

d. Practice

Bạn hãy trả lời các câu hỏi dưới đây bằng cách vẽ/viết ra, sau đó tìm bạn bè để chia sẻ lại với họ dưới dạng thuyết trình những gì bạn đã hiểu.

1. English có những loại câu gì?

a.Simple Sentence: Cho 9 ví dụ cho mỗi loại

b.Compound Sentence: Cho 3 ví dụ ứng với 3 Conjunction “and”, “but”, “so”

c.Complex Sentence: Với mỗi loại Clause cho 1 ví dụ

2. Đọc 1 bài English với chủ đề mà bạn thích.

3. Ứng với mỗi câu trong bài đọc trên, xác định loại câu và phân tích các thành phần cấu tạo nên câu.

Với việc nắm vững các loại câu, bạn sẽ đọc hiểu và viết chuẩn xác hơn. Ngoài ra, để diễn đạt những ý đã xảy ra trong quá khứ, đang xảy ra trong hiện tại và sẽ xảy ra trong tương lai, cũng như quan hệ giữa các mệnh đề của một câu hoặc các câu, bạn cần nắm thêm Tense (Thì).

4. Tense Synthesis (Tổng Hợp Thì)

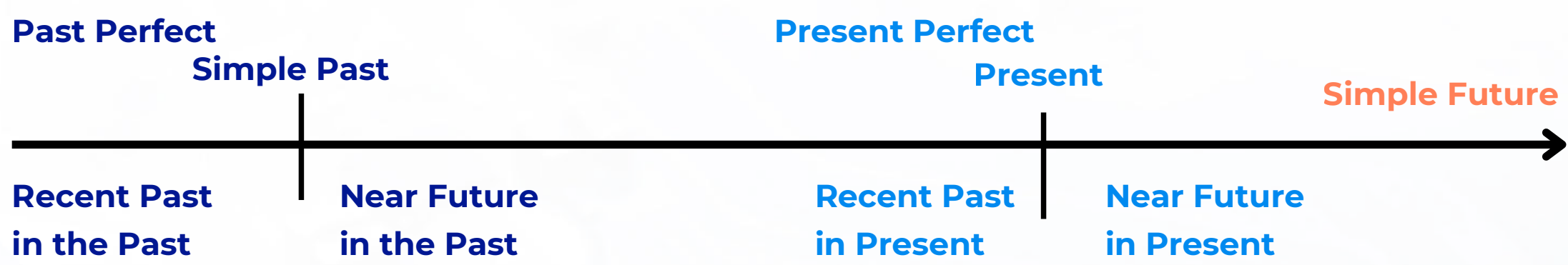
Tiếng Việt chỉ có 3 Tense và cách thành lập cũng rất đơn giản:

- Quá khứ: thêm từ “**đã**” trước động từ, ví dụ: Chúng ta **đã đi** đến đó tuần trước.
- Hiện tại: động từ bình thường, ví dụ: Chúng ta **đi** đến đó nào.
- Tương lai: thêm từ “**sẽ**” trước động từ, ví dụ: Chúng ta **sẽ đi** đến đó ngày mai.

Đôi khi ta cũng không cần thêm từ “đã” hoặc “sẽ” mà dựa vào Adv để ngầm hiểu Tense, ví dụ: “**Hôm bữa** tôi chả biết sao mà chương trình của tôi không chạy được.” -> đây là 1 câu trong thì quá khứ mà không có từ “đã”, thay vào đó Adv “Hôm bữa” cho biết nghĩa quá khứ.

Nói đến Tense trong English thì chắc ai từng học qua cũng phải nhũn não vì có quá nhiều loại Tense và cách sử dụng cũng quá phức tạp, đôi khi cách dùng lại trùng lặp lên nhau nên có những tình huống người học English chả biết dùng Tense nào là hợp lý nhất.

Tôi đã tổng hợp các Tense được sử dụng phổ biến nhất một cách đơn giản hơn với 3 nhóm Tense được thể hiện qua lược đồ sau.



Sơ Đồ Tổng Hợp Các English Tense

a. Past Tenses (Nhóm Các Thì Quá Khứ)

1. Simple Past (Quá Khứ Đơn)	: V2
2. Past Continuous (Quá Khứ Tiếp Diễn)	: Was/Were + V-ing
3. Near Future in the Past (Tương Lai gần trong Quá Khứ)	: Was/Were going to + V1
4. Recent Past in the Past (Quá Khứ gần trong Quá Khứ)	: Had just + V3
5. Past Perfect (Quá Khứ Hoàn Thành)	: Had + V3

- **Simple Past** diễn tả hành động xảy ra tại 1 THỜI ĐIỂM XÁC ĐỊNH TRONG QUÁ KHỨ: T1. Ta lấy T1 làm **mốc thời gian cho các Past Tenses khác**.
 - I **studied** IT **yesterday afternoon (T1)**.

- Ngay mốc T1 có hành động **đang xảy ra** ta có thì **Past Continuous**.
 - I **was walking yesterday afternoon (T1)**.
- Nhích tới bên phải T1 một tí ta có thì **Near Future in the Past** diễn tả hành động **sắp xảy ra** sau thời điểm T1.
 - I **was going to have dinner yesterday afternoon (T1)**.
- Lùi về bên trái T1 một tí ta có thì **Recent Past in the Past** diễn tả hành động **vừa xảy ra** trước thời điểm T1.
 - I **had just gone shopping yesterday afternoon (T1)**.
- **Past Perfect** diễn tả hành động **xảy ra** trước T1 và tiếp tục **kéo dài** đến T1. Ngoài ra Past Perfect còn diễn tả 1 hành động xảy ra trước 1 hành động khác X trong quá khứ mà hành động X này có thể có hoặc không có thời gian xác định.
 - The washing machine **had run** until **yesterday afternoon (T1)**.
 - The washing machine **had finished** working before I studied IT.

Ví dụ tổng hợp: I studied IT yesterday afternoon. I was studying in a library. I was going to have dinner with my mentor yesterday evening, so I had bought a present for him before studying.

b. Present Tenses (Nhóm Các Thì Hiện Tại)

1. **Simple Present** (Hiện Tại Đơn) : V1
2. **Present Continuous** (Hiện Tại Tiếp Diễn) : Am/Is/Are + V-ing
3. **Near Future at Present** (Tương Lai gần ở Hiện Tại) : Am/Is/Are going to + V1
4. **Recent Past at Present** (Quá Khứ gần ở Hiện Tại) : Have/Has just + V3
5. **Present Perfect** (Hiện Tại Hoàn Thành) : Have + V3

- Tiếp theo ta cầm 1 cục các Past Tense tịnh tiến qua bên phải thì ta có tương tự cho các Present Tense. Bắt đầu với **Simple Present** diễn tả hành động xảy ra ở thời điểm nói: T2. Ngoài ra Simple Present còn diễn tả hành động có tính chất chung chung, hay chân lý.
 - Let's **go** to school. (T2: thời điểm nói)
 - Nurses **look after** patients. (chung chung)
 - The sun **rises** in morning. (chân lý)
- Ngay mốc T2 có hành động **đang xảy ra** ta có thì **Present Continuous**.
 - We **are programming** in a Hackathon event.
- Nhích tới bên phải T2 một tí ta có thì **Near Future at Present** diễn tả hành động **sắp xảy ra** sau thời điểm T2, ví dụ: I **am going to play** football.
- Lùi về bên trái T2 một tí ta có thì **Recent Past at Present** diễn tả hành động **vừa xảy ra** trước thời điểm T2, ví dụ: I **have just won** the game .

- **Present Perfect** diễn tả hành động **xảy ra** trước T2 và tiếp tục **kéo dài** đến T2. Ngoài ra Present Perfect còn diễn tả hành động xảy ra ở thời điểm không xác định trong quá khứ.
 - I have worked on the project for 3 years.
 - She has been the good girl this year, so she deserves a Christmas gift. -> “Cô ấy đã là cô gái tốt suốt năm nay nên cô ấy xứng đáng một món quà Giáng sinh.” (giờ cô bé vẫn tốt nhưng không biết cô bé bắt đầu tốt khi nào, có thể trước đó không tốt).

c. Future Tenses (Nhóm Các Thì Tương Lai)

- 1. Simple Future : Will/Shall + V1
 - 2. Future Continuous : Will/Shall + be + V-ing

- Nếu hành động có thể xảy ra trong tương lai nhưng chưa chắc, chúng ta cứ sử dụng **Simple Future** cho đơn giản, các thì tương lai khác rất ít khi được sử dụng.
 - I **will become** an IT expert in 5-6 years.
- Trong trường hợp chúng ta cần diễn tả một hành động sẽ đang diễn ra trong tương lai liên quan đến hành động khác thì sử dụng Future Continuous.
 - Let visit my house the day after tomorrow, I'll **be playing** tennis at that time, just phone me when you arrive.

d. Practice

Bạn hãy trả lời các câu hỏi dưới đây bằng cách vẽ/viết ra, sau đó tìm bạn bè để chia sẻ lại với họ dưới dạng thuyết trình những gì bạn đã hiểu.

- Viết/vẽ ra những gì bạn hiểu về các loại Tense
- Ứng với mỗi loại Tense, cho 1 ví dụ.

Trên đây chúng ta đã tìm hiểu xong những phần nền tảng nhất của English để giúp chúng ta tiếp tục đi vào phần quan trọng nhất cho việc học IT, đó là đọc hiểu English.

5. Reading Skill (Kỹ Năng Đọc)

Kỹ năng đọc English đóng vai trò quan trọng nhất trong việc học IT. Các yếu tố ảnh hưởng đến kỹ năng đọc bao gồm: Word Family, Context, và Compound Noun.

a. Word Family (Gia Đình Từ)

Để đọc hiểu tốt English trong ngành IT thì điều đầu tiên bạn phải làm là liên tục nâng cao **Vocabulary** (Vốn Từ Vựng) của mình trong mảng IT. Sau đó mở rộng ra các mảng khác.

Học từ vựng từng từ một không phải là cách hay, mà ta **nên học theo Word Family**, hiểu nôm na là học từng nhóm từ có chung gốc từ hoặc liên quan với nhau theo tiêu chí nào đó sẽ giúp ta dễ nhớ nhiều từ hơn, từ đó nâng cao nhanh hơn vốn từ vựng của mình, ví dụ:

- **VAAN** (Verb, Adjective, Adverb, Noun): nhóm các từ có Nature khác nhau nhưng có chung gốc từ, ví dụ: **attend** (V), **attentive** (Adj), **attentively** (Adv), **attention** (Noun). Tham khảo thêm nhiều ví dụ khác: <https://utnenglish.files.wordpress.com/2013/09/nous-verbs-adjectives-adverbs.pdf>
- Common Pattern (Cùng mẫu chung): nhóm các từ chung mẫu, ví dụ: back, black, crack, hack, snack, track... Tham khảo thêm: <https://tutoring.charlotte.edu/sites/tutoring.charlotte.edu/files/media/Word%20Families.pdf>
- Weekdays (Ngày trong tuần): Monday, Tuesday, Wednesday, Thursday, Friday
- Computer Hardware (Phần Cứng Máy Vi Tính): Case, Mainboard, Keyboard, Mouse...

b. Context (Ngữ Cảnh)

Một từ English cũng như các ngôn ngữ khác luôn có nhiều nghĩa khi bạn tra từ điển (ví dụ: <https://www.thefreedictionary.com/play>), do đó để hiểu được **nghĩa của 1 từ trong 1 câu** thì phải nắm được Context cụ thể của câu đó, ví dụ: cùng là từ “**play**” nhưng khi đặt vào 2 Context khác nhau sẽ có nghĩa khác nhau:

- We **play football** every week. -> Chúng tôi **chơi bóng đá** mỗi tuần.
- IT **plays a very important role** nowadays. -> Ngành IT **đóng 1 vai trò** rất quan trọng trong thời đại ngày nay.

Điều này có nghĩa là khi chúng ta đọc thì trước khi xem xét nghĩa của các từ trong câu thì phải xác định rõ Context của câu trước.

c. Compound Noun (Danh Từ Ghép)

Một yếu tố đóng vai trò cực kỳ quan trọng trong việc đọc hiểu English đó là xác định được **Compound Word** (Từ Ghép) và hiểu đúng nghĩa, trong đó **Compound Noun** là chính yếu.

Các Compound Noun trong English được **viết từ trái qua phải** nhưng khi **hiểu nghĩa** thì lại là **từ phải qua trái** (điều này dễ gây ra sự hiểu nhầm nghĩa cho người Việt Nam do tiếng Việt được viết và hiểu nghĩa đều từ trái qua phải).

Ví dụ: **“Management Information System”** vs **“System Information Management”** -> 2 từ này hiểu từ phải qua trái sẽ là **“Hệ thống thông tin quản lý”** vs **“Quản lý thông tin hệ thống”**, từ 1 nói đến “Hệ thống” còn từ 2 nói về “Việc quản lý” rõ ràng là khác nhau.

Các Compound Noun trong English rất đa dạng, tựu trung có 2 nhóm công thức sau đây:

1. Nhóm 1: Noun được bổ nghĩa bởi một **Adj** có thể được thành lập bởi 3 dạng:

- **Adj đơn** : poor Design, clean Code, high Cohesion, loose Coupling
- **N + Adj** : computer-based Test, tree like Structure, time-consuming Task
- **Adv + Adj** : recently modified Set, extremely critical Issue, quite nice Feature

2. Nhóm 2: Noun được bổ nghĩa bởi một **Noun** có thể được thành lập bởi 2 dạng:

- **N đơn** : Project Management, Document Title, Information System
- **Adj + N** : statistical Analysis Method, approximate Calculation Algorithm

d. Practice

Bạn hãy trả lời các câu hỏi dưới đây bằng cách vẽ/viết ra, sau đó tìm bạn bè để chia sẻ lại với họ dưới dạng thuyết trình những gì bạn đã hiểu.

1. Những yếu tố để nâng cao Reading Skill là gì?
2. Phân tích đoạn văn bản <https://en.wikipedia.org/wiki/Computer> thành danh sách các câu
3. Với mỗi câu
 - Xác định Context, các Compound Noun và nghĩa của chúng
 - Phân tách câu để xác định các Clause
 - Dịch toàn bộ câu sang tiếng Việt

Trong phần cuối của chương IT English, bạn sẽ được chia sẻ một số hiểu nhầm về mặt ngữ nghĩa của từ/câu trong English do hiểu chưa đúng cách dùng các từ loại English.

6. Popular Misunderstanding (Hiểu Nhầm Phổ Biến)

a. Article Misunderstanding (Hiểu Nhầm Mạo Từ)

- One process is dead, **the other** is not. (Một tiến trình thì chết, **cái còn lại** thì không).
- Several software developers have devops knowledge, **others** do not. (Một vài nhà phát triển phần mềm có kiến thức về devops, **những người khác** thì không).
- Most employees will join the competition, **the others** watch and drink. (Hầu hết nhân viên sẽ tham gia cuộc thi, **những người còn lại** xem và uống).

b. Adjective Misunderstanding (Hiểu Nhầm Tính Từ)

Một lưu ý với các Adjective có đuôi là “ing” hoặc “ed”.

- **Adj** có **-ed** mang nghĩa thụ động, ví dụ: She is bored (Cô ấy trông buồn chán, chắc cô ấy đang có gì đó không vui).
- **Adj** có **-ing** mang nghĩa chủ động, ví dụ: She is boring (Cô ấy làm người khác cảm thấy chán, ý là cô ấy không có gì thú vị để tiếp xúc hay hẹn hò).

c. Verb Misunderstanding (Hiểu Nhầm Động Từ)

Đặc biệt chú ý đến các từ/cụm từ nhìn gần giống nhau nhưng ý nghĩa khác nhau dẫn đến hiểu nghĩa của câu, đôi khi gây hậu quả nghiêm trọng.

- **“Make things different.”** vs **“Make things differently.”** -> Từ 1: chữ “different” là Adj bổ nghĩa cho chữ “things” là Noun nên câu này nghĩa là “Hãy làm những điều khác biệt”, tức là làm những điều gì đó mới mẻ >< Từ 2: chữ “differently” là Adv bổ nghĩa cho chữ “Make” là Verb nên câu này nghĩa là “Hãy làm những điều đó bằng các cách khác nhau”.
- **Forget to shut down** the server vs **Forget shutting down** the server: Quên tắt server (server chưa bị tắt -> server vẫn còn chạy) vs Quên cái việc đã tắt server (server đã bị tắt nhưng người tắt quên bém việc tắt này -> server không còn chạy).
- **Stop to smoke** vs **Stop smoking**: Dừng lại để hút thuốc vs Dừng việc hút thuốc (bỏ hút thuốc vĩnh viễn).
- **Try to open** the door vs **Try opening** the door: Cố gắng mở cánh cửa (chắc do khó mở hay bị khóa) vs Thử mở cánh cửa (chắc đang tìm gì đó).
- **Repeat again, Redo again...** : từ có **Prefix** (Tiếp Đầu Ngữ) “Re” đã có nghĩa lặp lại thì không cần chữ “again” phía sau nữa.

d. Meaning Misunderstanding (Hiểu Nhầm Nghĩa)

Văn phong khác nhau giữa English & tiếng Việt gây ra hiểu nhầm về nghĩa, ví dụ:

- Won't you do that task? -> Yes, I do (*Tôi làm*) vs No, I don't (*Tôi không làm*).
- Bạn không làm nhiệm vụ đó hả? -> Ừ (*Tôi không làm*) vs Không (*Tôi làm*).

=> **Cách trả lời của English thì không quan tâm câu hỏi (phủ định hay khẳng định) trong khi tiếng Việt thì cần quan tâm câu hỏi, để hiểu lầm với câu hỏi phủ định.**

e. Practice

Bạn hãy trả lời các câu hỏi dưới đây bằng cách vẽ/viết ra, sau đó tìm bạn bè để chia sẻ lại với họ dưới dạng thuyết trình những gì bạn đã hiểu.

1. Các Popular Misunderstanding của người học English là gì?
2. Cho câu sau: "*We usually analyze recently modified linked list like data structure highly used in management information system in designing information management system.*"
 - Xác định Nature của mỗi từ trong câu trên
 - Xác định Function của mỗi từ/cụm từ trong câu trên
 - Xác định loại Sentence của câu trên
 - Xác định loại Tense được sử dụng trong câu trên
 - Xác định Context, các Compound Noun và nghĩa của chúng
 - Phân tách câu để xác định các Clause
 - Dịch toàn bộ câu sang tiếng Việt

Đến đây thì bạn đã được trang bị English đủ để tìm hiểu ngành IT. Chúng ta bắt đầu đi vào khám phá những mảng nền tảng khác nhau trong thế giới công nghệ.

III. IT FOUNDATION (NỀN TẢNG IT)

Nắm trọn nền móng IT trong tay

Bản PRO giúp bạn hiểu toàn cảnh các mảng cốt lõi trong ngành IT và cách chúng kết nối để tạo nên mọi sản phẩm công nghệ.

👉 [Nâng cấp lên PRO để học ngay]

IV. JAVA TECHNOLOGY FOUNDATION (NỀN TẢNG CÔNG NGHỆ JAVA)

Ngôn ngữ quyền lực chạy trên hàng tỷ thiết bị

Bản PRO giúp bạn nắm nền tảng của công nghệ Java để phát triển các sản phẩm công nghệ dựa trên Java nhanh và hiệu quả.

👉 [Nâng cấp lên PRO để khám phá]

V. TECH PRODUCT (SẢN PHẨM CÔNG NGHỆ)

1. Introduction (Giới Thiệu)

Trong phần giới thiệu của Bộ Cẩm Nang này thì tôi đã điểm sơ một vài Tech Product đình đám hiện nay như Google Search, Youtube, Amazon eCommerce, Facebook, LinkedIn, Grab... là những Tech Product đã thành công vượt bậc sau 1 thời gian dài được đầu tư xây dựng, bảo trì và phát triển liên tục từ những ý tưởng ban đầu của các **Product Owner** (Chủ Sản Phẩm).

Hàng triệu Tech Product khác vẫn đang tiếp tục được xây dựng bởi hàng triệu **Tech Startup** (Công Ty Khởi Nghiệp Công Nghệ) hoặc **Small and Medium Enterprise – SME** (Doanh Nghiệp Vừa và Nhỏ) trên toàn cầu nhằm đáp ứng các nhu cầu ngày càng tăng của nhân loại trong thời đại công nghệ thông tin.

Nếu bạn có ý tưởng gì đó mà cảm thấy có khả năng dùng công nghệ để tạo ra Tech Product đem đến lợi ích cho các doanh nghiệp, xã hội, quốc gia hoặc thế giới thì bạn có thể xây dựng Tech Product cho riêng mình.

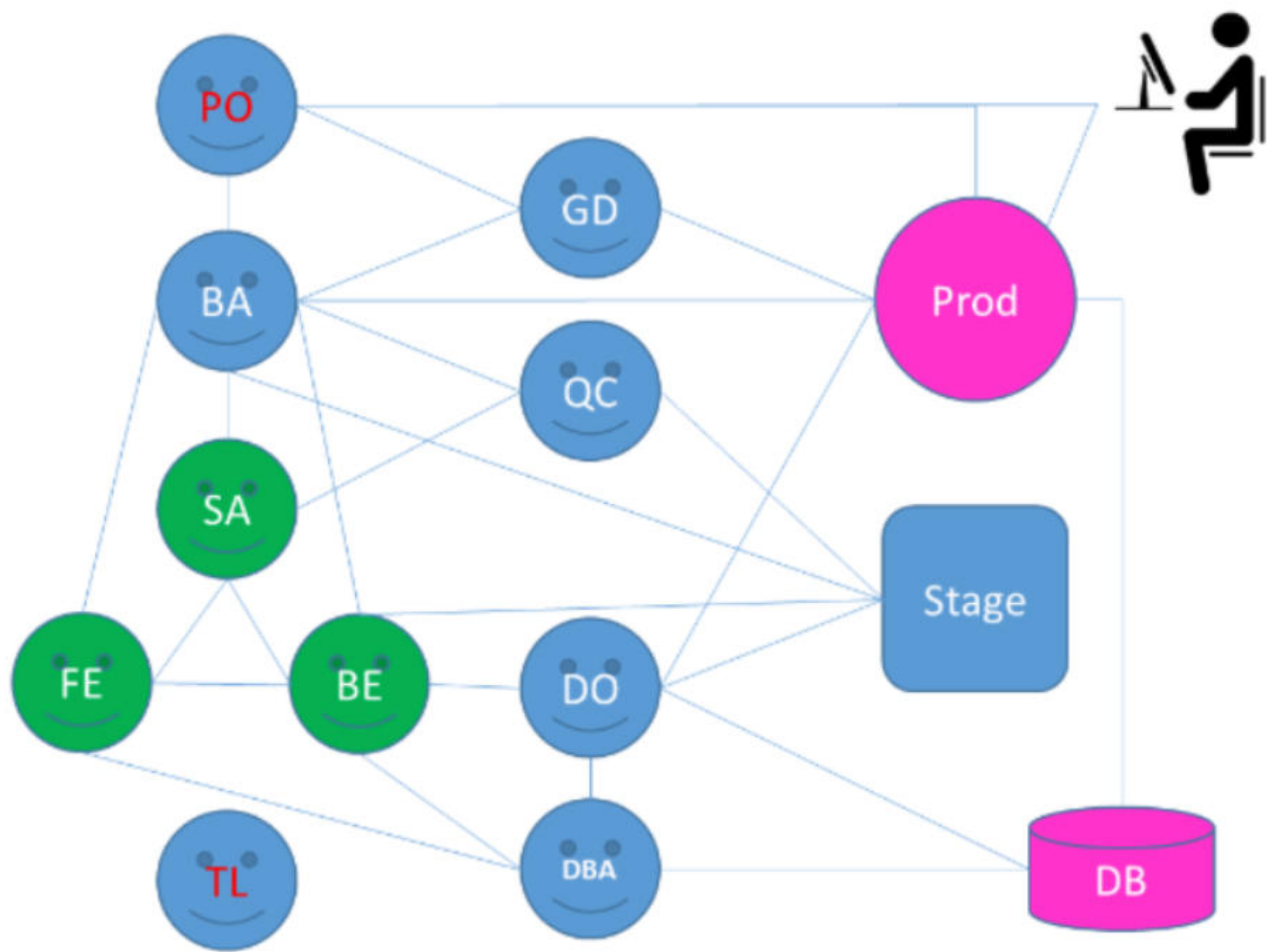
Việc xây dựng Tech Product không hề đơn giản, đòi hỏi bạn phải có kiến thức tốt và kinh nghiệm nhiều về lĩnh vực bạn đang muốn nhảy vào (ví dụ: eCommerce, Sức Khỏe, Thư Viện...) cùng đội ngũ IT giỏi. Tuy nhiên, để Tech Product thành công (thương mại hóa được thị trường chấp nhận, đem lại doanh thu cao để phát triển lên thành công ty công nghệ mạnh) thì việc xây dựng Tech Product chỉ chiếm 50% của cuộc chơi, phần còn lại đòi hỏi bạn phải có nhiều thứ khác ví dụ như:

- **Business Model** (Mô Hình Kinh Doanh)
- **Product Strategy** (Chiến Lược Sản Phẩm)
- **Business Operation** (Vận Hành Doanh Nghiệp)
- **Product Marketing/Sales** (Tiếp Thị & Bán Sản Phẩm)
- **Product Operation** (Vận Hành Sản Phẩm)
- **Customer Support** (Hỗ Trợ Khách Hàng)
- **Product Research & Development** (Nghiên Cứu và Phát Triển Sản Phẩm)...

Khi nói đến khái niệm Tech Product là chúng ta đang đứng từ góc nhìn của khách hàng và đội ngũ vận hành sản phẩm, còn từ góc nhìn của nhóm phát triển thì Tech Product chính là Software được xây dựng bằng Software Engineering đã được trình bày ở trên.

2. Development Process (Quy Trình Phát Triển)

Quy trình xây dựng Tech Product được thể hiện trong hình dưới đây.



Hình 75: Tech Product Development Process

Các **Role** (Vai Trò) tham gia trong quy trình bao gồm:

1. PO (Product Owner)
2. GD (Graphic Designer)
3. BA (Business Analyst)
4. SA (Software Architect)
5. BE (Backend)
6. FE (Frontend)
7. QC (Quality Control)
8. DO (DevOps)
9. DBA (Database Administrator)
10. TL (Team Lead)

Hoạt động của quy trình này đi liền với các Role được đặc tả chi tiết trong phần “Hướng Nghiệp Công Nghệ Thông Tin” dưới đây.



PHẦN 2: HƯỚNG NGHIỆP CÔNG NGHỆ THÔNG TIN

Phần 1 đã trang bị cho bạn những kiến thức nền tảng công nghệ thông tin vững chắc.

Nếu bạn có hứng thú với ngành công nghệ thông tin và muốn theo đuổi thì phần 2 này sẽ hỗ trợ bạn rất nhiều.

Để có thể học và làm tốt trong ngành IT thì bạn cần tìm hiểu trước những tính chất nào cần thiết ở con người phù hợp với ngành IT.

I. ITER CHARACTERISTICS (ĐẶC ĐIỂM CỦA DÂN IT)

1. PASSION (ĐAM MÊ)

Oprah Winfrey đã từng bày tỏ rằng: "Passion is energy. Feel the power that comes from focusing on what excites you."

Đề ý rằng khi ta có đam mê với một công việc, năng lượng tích cực sẽ được tạo ra, lan tỏa khắp cơ thể, giúp chúng ta tràn đầy cảm hứng và tập trung hết mình vào công việc mình đang thực hiện. Đôi khi, chúng ta sẽ quên luôn cả thời gian và những sự kiện xung quanh, và kết quả đạt được thường rất tích cực.

Ngược lại, nếu chúng ta làm việc mà không có đam mê, chỉ vì bị ép buộc hoặc vì bất kỳ lý do nào khác, tâm trí chúng ta sẽ không hỗ trợ hoàn thành công việc đó, và kết quả cuối cùng thường không như mong đợi.

Do đó, đam mê đóng vai trò quan trọng trong việc thành công ở mọi lĩnh vực, đặc biệt là trong ngành công nghệ thông tin, một lĩnh vực đầy thách thức và áp lực nhưng không kém phần hấp dẫn.

Vì vậy, hãy dành thời gian suy ngẫm xem liệu bạn thực sự theo đuổi ngành IT vì đam mê hay vì lý do khác:

- Nếu câu trả lời là vì đam mê, xin chúc mừng bạn!
- Nếu bạn chưa chắc chắn, hãy thử sức với ngành IT.
- Nếu sau một thời gian làm việc hoặc học tập trong ngành IT mà bạn vẫn không cảm thấy đam mê, hãy tìm kiếm đam mê của mình trong các cơ hội nghề nghiệp khác.

2. Attitude (Tư Duy)

a. Can-do (Có Thể Làm)

Chúng ta đang dự định làm một việc gì đó mà lúc nào cũng nghĩ việc này khó quá, sợ làm không được thì cơ thể sẽ sinh ra một lực vô hình chống lại hành động của chúng ta và khả năng cao là việc này sẽ thất bại.

Ngược lại, dù việc rất khó (khả thi chứ không viễn vông) nhưng lúc nào chúng ta cũng tin tưởng là sẽ làm được và luôn cố gắng hết sức cũng như tâm trí để thực hiện thì cơ thể sẽ sản sinh ra một nguồn năng lượng tích cực (**Positive Energy**) giúp hoàn thành công việc hiệu quả một cách đáng kinh ngạc.

Cho nên, tư duy **“Có thể làm” (“Can-do”)** là cực kỳ quan trọng, đóng góp vào sự thành công của chúng ta ngay từ khi bắt đầu làm một việc gì đó.

b. Logic (Sự Hợp Lý)

Trong cuộc sống, mọi vấn đề trong mọi lĩnh vực luôn vận hành theo những sự hợp lý (**Logic**) nào đó. Hệ thống IT giúp giải quyết những vấn đề đó một cách tự động hóa thì cũng phải tuân theo những logic đó, vì thế những người xây dựng hệ thống IT cũng phải luôn suy nghĩ logic (**Logical Thinking**) thì mới tạo ra được những hệ thống IT tốt, hạn chế những lỗi sai logic.

c. Eager-to-learn Spirit (Tinh Thần Ham Học Hỏi)

Ngành IT luôn phát triển rất nhanh, cho nên tinh thần ham học hỏi (**Eager-to-learn Spirit**) và tính thích ứng cao (**High Adaptability**) để nắm bắt công nghệ mới đóng vai trò then chốt trong việc bắt kịp thời đại và làm ra những sản phẩm tốt nhất.

d. Entrepreneur Spirit (Tinh Thần Làm Chủ)

Khi xây dựng 1 sản phẩm IT, chúng ta nên có tinh thần làm chủ (**Entrepreneur Spirit**) của sản phẩm dù có đang làm thuê. Chỉ khi có tinh thần làm chủ thì chúng ta sẽ nhìn sản phẩm một cách bao quát hơn, tìm hiểu chi tiết hơn, luôn luôn tìm cách tốt nhất để xây dựng sản phẩm. Rồi khi sản phẩm ngày càng tốt sẽ giúp công ty phát triển tốt lên thì chúng ta cũng được hưởng lợi cả về tinh thần và vật chất.

Ngược lại nếu chúng ta nghĩ làm công ăn lương thì chỉ làm cho xong những việc được giao rồi về nhà cho khỏe, khỏi suy nghĩ nhiều chi cho mệt óc thì sẽ khó tiến lên cao trên con đường sự nghiệp và sản phẩm công ty cũng không được phát triển hết tiềm năng của nó.

e. Goal (Mục Tiêu)

Ở mỗi giai đoạn phát triển sự nghiệp, chúng ta nên đặt ra mục tiêu dài hạn (**Long-term Goal**), trong đó sẽ có nhiều mục tiêu ngắn hạn (**Short-term Goal**), bởi vì mục tiêu giúp chúng ta có động lực và năng lực để phấn đấu tiến tới.

f. Environment (Môi Trường)

Mỗi lần chúng ta bắt đầu tìm hiểu một lĩnh vực mới, ngoài việc học qua trường lớp, sách vở, và tự học thì chúng ta cũng nên tìm những môi trường nơi mà những người khác đã thành công trong lĩnh vực này như các hội/nhóm để có cơ hội học hỏi kiến thức/kinh nghiệm từ họ để ngày nào đó chúng ta sẽ thành công như họ.

g. Mentor

Tốt hơn nữa là chúng ta có được các **Mentor** hỗ trợ.



MENTOR follows and helps you a long the way.

Hình 76: Mentor

Khái niệm Mentor không dịch được qua tiếng Việt vì không có từ tương ứng bên tiếng Việt, có thể hiểu nôm na Mentor là người đã trải qua những gì chúng ta đang/sẽ gặp, họ chỉ ra hướng đi phù hợp đồng thời đi theo sửa sai khi chúng ta học/làm việc thực tế. Mentor có thể là những người bạn trong mạng lưới bạn bè của chúng ta hay những người mà chúng ta gặp trong các sự kiện hoặc những trưởng nhóm/đồng nghiệp trong môi trường làm việc thực tế.

3. Background (Nền Tảng)

Một ngôi nhà vững chãi thì chắc chắn cái móng của nó phải vững vàng. Một cái cây khỏe thì chắc chắn gốc rễ của nó rất tốt.

Tương tự, khi làm bất cứ việc gì thì chúng ta phải quan tâm xây dựng nền tảng vững chắc trước, từ đó mới phát triển tiếp lên thì sẽ bền vững hơn. Ví dụ thay vì chúng ta nhảy vô học ngay một ngôn ngữ lập trình (**Programming Language – PL**) cụ thể nào đó (như Java, C#, PHP, Python, Ruby...) thì chúng ta nên tìm hiểu trước căn bản của PL, rồi thực hành bằng một PL cụ thể để hiểu rõ các đặc điểm, tính chất cốt lõi của PL. Sau này chúng ta có nhu cầu phải chuyển qua học các PL khác sẽ nhanh hơn nhiều so với chúng ta học từng PL riêng rẽ.

4. 3S Method (Phương Pháp 3S)

Tục ngữ có câu “Học đi đôi với hành”, nghĩa là chúng ta học mà không thực hành để sử dụng những kiến thức chúng ta đã tiếp thu được thì chúng ta sẽ mau quên.

Thật đúng là như vậy, với những gì tôi đã trải qua giờ nhìn lại tôi thấy là những kiến thức tôi đã học mà không có dịp thực hành thì tôi đều đã quên gần hết. Tôi cũng nhận thấy thêm một điều nữa là dù tôi có thực hành những kiến thức tôi đã học, nhưng thời gian và tần suất thực hành không đủ dài và nhiều để biến thành kỹ năng cho tôi thì sau một thời gian dài tôi cũng hầu như không còn dùng được những kiến thức đó.

Với những trải nghiệm trên, tôi đã tổng hợp ra phương pháp 3S (3S Method) giúp cho việc học hiệu quả hơn, đó là **Study -> Self-Experience -> Skill**:

- **Study** (Học): khi học một công nghệ A nào đó thì chúng ta nên học theo thứ tự **WHAT** (nó là cái gì, nó có những tính năng gì...) -> **WHY** (tại sao nó ra đời/tồn tại) -> **WHEN** (khi nào nó nên được sử dụng) -> **HOW** (nó hoạt động thế nào, sử dụng nó ra sao...).
- **Self-Experience** (Tự trải nghiệm): học xong công nghệ A thì phải tự trải nghiệm rồi ngẫm nghĩ cách mình đã thực hành có phải là cách phù hợp nhất chưa, nếu chưa thì tìm cách làm tốt hơn rồi trải nghiệm tiếp... lặp lại quá trình này cho đến khi tìm được cách làm phù hợp nhất.
- **Skill** (Kỹ năng): trải nghiệm đủ lâu và nhiều để tạo ra kỹ năng cho chúng ta trên công nghệ A, gọi là kỹ năng thì khi chúng ta sử dụng công nghệ A thì chúng ta không cần phải suy nghĩ, làm một cách vô thức mà kết quả luôn tốt đẹp.

Từ khi tổng hợp được 3S Method, tôi đã áp dụng thử thì thấy việc học của tôi hiệu quả hơn trước khá nhiều. Tôi tin là 3S Method đóng vai trò nền tảng trong việc học của dân IT.
Bạn nên áp dụng thử 3S Method xem hiệu quả thế nào nhé.

5. 3WH Formula (Công Thức 3WH)

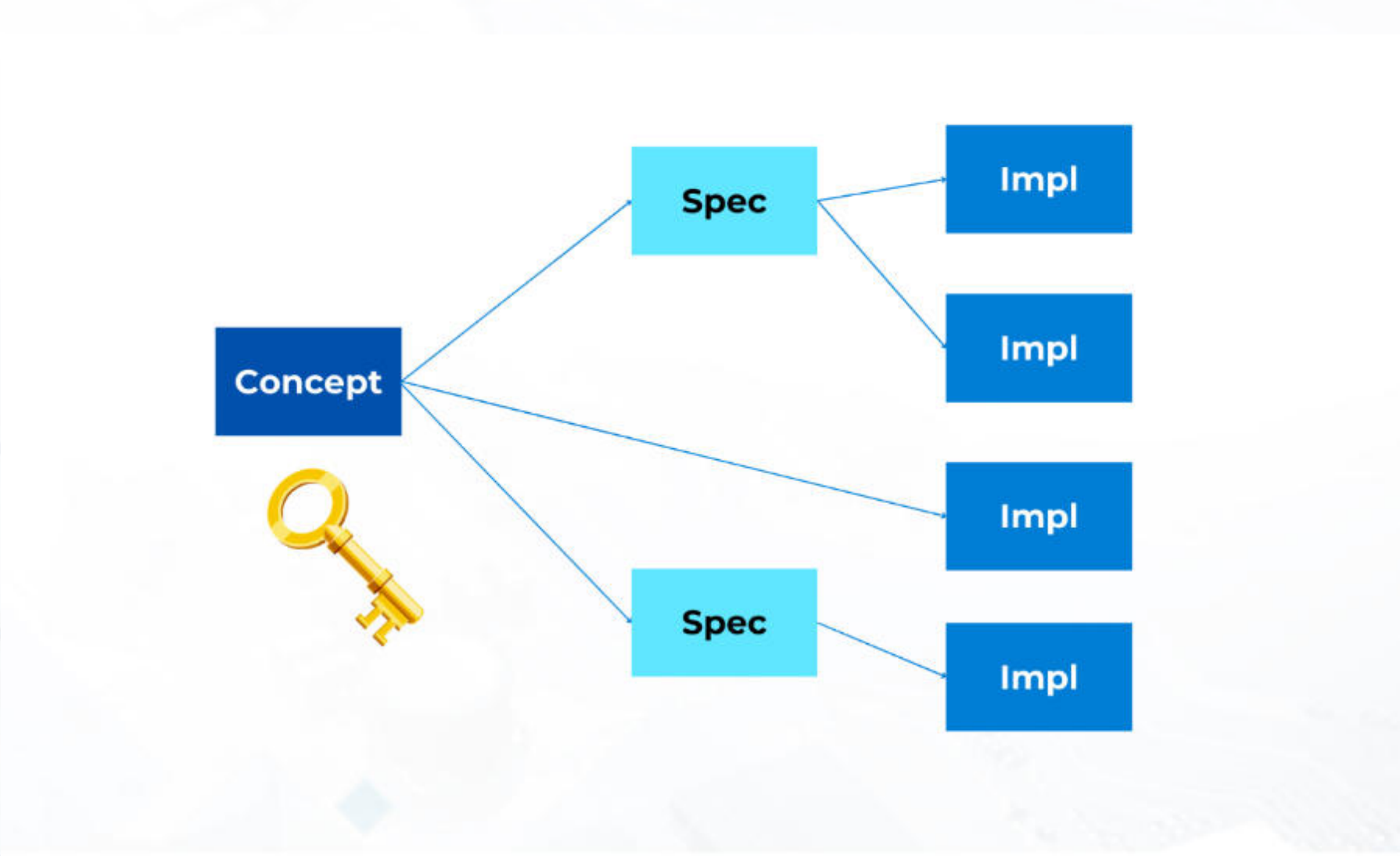
Trong chương về Tư Duy Logic ở trên tôi đã giới thiệu về công thức 3WH. Mục này chỉ để nhắc lại và nhấn mạnh với bạn tầm quan trọng của công thức này để bạn ghi nhớ và áp dụng khi học và làm trong ngành IT.

6. CSI Formula (Công Thức CSI)

Thế giới IT là cực kỳ rộng lớn, kiến thức công nghệ bao la như biển và thay đổi rất nhanh trong khi thời gian của mỗi chúng ta là hữu hạn. Vì vậy, có được một phương pháp phù hợp tiếp cận với các công nghệ sẽ giúp chúng ta ít bị lạc hậu.

Sau một quá trình khá dài trải nghiệm nhiều công nghệ trong ngành IT, tôi nhận thấy hầu hết công nghệ đều có một công thức chung mà tôi đặt tên là công thức CSI: xuất phát điểm là khái niệm (**Concept**) -> từ đó mới phát triển lên thành các đặc tả (**Specification – Spec**) -> rồi cuối cùng là các hiện thực (**Implementation – Impl**) khác nhau.

Mô hình diễn tả mối quan hệ giữa Concept – Spec – Impl được biểu diễn qua hình sau:



Hình 77: CSI Formula

Như vậy, phần tử cốt lõi đầu tiên khi chúng ta tìm hiểu một công nghệ chính là **Concept**. Đây là chìa khóa để chúng ta rút ngắn thời gian khám phá các công nghệ.

Một Concept là kết quả của quá trình chuyển đổi từ các ý tưởng sang một mục tiêu cụ thể, ví dụ một số Concept trong Software Engineering sau:

- Client-side scripting
- AJAX
- Dynamic Web Page
- Web Service
- Business Transaction
- Object Relational Mapping (ORM)

Một Spec là một đặc tả chi tiết cho một Concept, ví dụ:

- ECMAScript là 1 Spec của Concept “Client-side Scripting”.
- JSP là 1 Spec của Concept “Dynamic Web Page”.
- JPA là 1 Spec của Concept “ORM”.

Mỗi Concept có thể có nhiều Spec, ví dụ:

- JSP là 1 Spec của Concept “Dynamic Web Page”.
- ASP.Net là 1 Spec của Concept “Dynamic Web Page”.
- PHP là 1 Spec của Concept “Dynamic Web Page”.

Một Spec có thể được hiện thực bởi nhiều công ty/nhóm/cá nhân tạo ra nhiều Implementation, ví dụ:

- Javascript of Mozilla là một Implementation của Spec “ECMAScript”.
- Jscript of Microsoft là một Implementation của Spec “ECMAScript”.
- Glassfish of Oracle là một Implementation của Spec “JSP” và “JPA”.
- WebSphere of IBM là một Implementation của Spec “JSP” và “JPA”.
- Hibernate JPA là một Implementation của Spec “JPA”.

Ví dụ:

- Object-Relational Mapping (ORM) là 1 Concept
- Java Persistence API (JPA) là 1 Spec cho Concept ORM
- OpenJPA, Hibernate JPA, EclipseLink là những Implementation cho Spec JPA

Từ ví dụ trên chúng ta thấy là thay vì nhảy vô học các Implementation cụ thể như OpenJPA, Hibernate JPA, EclipseLink ... sẽ mất rất nhiều thời gian, chúng ta nên tìm hiểu Concept ORM trước, sau đó xem qua Spec JPA, khi đã nắm được Concept & Spec thì chỉ cần học sử dụng 1 Implementation như Hibernate JPA, sau này có việc phải chuyển qua dùng những Implementation khác như OpenJPA, EclipseLink sẽ tốn ít thời gian vì chúng tương tự nhau, nếu có khác thì chỉ khác nhau chút đỉnh.

Concept đến Implementation), đôi khi một Spec của một Concept không có tên hoặc không được biết đến, khi đó Implementation được sử dụng thay cho Spec, ví dụ:

- Hibernate (native) là một Implementation của Concept “ORM”.
- Entity Framework là một Implementation của Concept “ORM”.

Luyện Tập

Cho một danh sách các thuật ngữ ngẫu nhiên sau:

- Eclipse
- MS Visual Studio
- Hibernate
- EclipseLink
- JPA
- JSP
- Spring
- MSSQL
- MySQL
- Oracle DB
- IoC
- JDBC
- Spring Transaction
- JDBC Template
- MongoDB
- Redis
- ASP.Net
- Castle
- Entity Framework
- Memcached
- Riak

- 1.Hãy sắp xếp chúng vào 1 bảng có 3 cột: Concept-Spec-Impl.
- 2.Sau đó điền vào những chỗ còn trống cho đầy bảng, nghĩa là mỗi dòng sẽ có đủ 3 thành phần Concept-Spec-Impl.

(Đáp án nằm ở trang kế)

Đáp án câu 1:

Concept	Specification	Implementation
		Eclipse, Netbeans
	JPA	Hibernate, EclipseLink
IoC		Spring, Castle
		MSSQL, MySQL, Oracle DB
		MongoDB, Memcached, Riak, Redis
	JDBC	
	Spring Transaction	
	JDBC Template	
	ASP.Net	
		Entity Framework

Đáp án câu 2:

Concept	Specification	Implementation
IDE		Eclipse, MS Visual Studio
ORM	JPA	Hibernate, EclipseLink
	Entity Framework	MS .Net Framework
IoC	Dependency Injection	Spring, Castle
DBMS	RDBMS	MSSQL, MySQL, Oracle DB
	NoSQL	MongoDB, Memcached, Riak, Redis
Database Connectivity	JDBC	Java SE
Business Transaction	Spring Transaction	Spring Framework
Data Access Template	JDBC Template	Spring Framework
Dynamic Web Page	JSP	Tomcat, Glassfish
	ASP.Net	IIS

Hình 78: CSI Formula Practice

II. IT JOBS (CÁC NGHỀ IT)

Bức tranh 360° về nghề IT

Từ công ty sản phẩm đến gia công, từ BA đến Developer – Bản PRO sẽ mở rộng toàn cảnh nghề nghiệp công nghệ cho bạn.

👉 [Nâng cấp lên PRO để mở rộng lựa chọn]

III. SOFT SKILLS (CÁC KỸ NĂNG MỀM)

Bộ kỹ năng mềm thiết yếu – thăng tiến sự nghiệp

Bản PRO hướng dẫn các kỹ năng mềm để làm việc hiệu quả và thăng tiến trong sự nghiệp.

👉 [Nâng cấp lên PRO để nâng cao bộ kỹ năng mềm]

IV. IT CAREER ORIENTATION (HƯỚNG NGHIỆP CÔNG NGHỆ)

Tìm đúng nghề – đi nhanh hơn

Bản PRO hướng dẫn phương pháp 5P để chọn nghề IT phù hợp và lập kế hoạch phát triển.

👉 [Nâng cấp lên PRO để định hướng nghề]



PHẦN 3: TRẢI NGHIỆM CÔNG NGHỆ THÔNG TIN 30 NĂM

Hai phần 1,2 đã cung cấp nền tảng IT cũng như hướng nghiệp ngành IT cho bạn.

Phần 3 này sẽ chia sẻ quá trình học và làm IT của tôi trong và ngoài nước trong suốt 30 năm qua nhằm truyền động lực cho bạn theo đuổi ngành IT cũng như giúp bạn tham khảo những thử thách mà tôi đã gặp phải khi theo đuổi ngành này.

I. HỌC IT Ở VIỆT NAM

1. Học nghề IT để thi tốt nghiệp PTCS

Ngày xưa ngày xưa cách đây cỡ 30 năm, Sài Gòn rất vắng vẻ và bình yên. Mỗi buổi sáng thanh bình tôi nghe tiếng gà gáy rất hay.

Buổi trưa thảnh thơi tôi nghe tiếng leng keng của những ông bán cà lem cây.

Buổi tối yên tĩnh lâu lâu được khuấy động bởi tiếng rao thật lớn và vang thật xa của một bà cụ đội cái giỏ tròn vo trên đầu vừa đi vừa rao trong xóm: “Ai hột vịt lộn, gà giũa, vịt ung không?”

Gia đình tôi sống trong một xóm lao động lam lũ ở Quận Tân Bình với phần lớn là dân từ miền Trung vào Sài Gòn lập nghiệp. Ba Má tôi cũng là “Du Học Sinh” từ Bình Định. Thời đó nhà ai cũng khó khăn nhưng cuộc sống xóm làng rất vui, láng giềng rất gần gũi và quan tâm nhau.

Những trò chơi cho trẻ con trong xóm thì vô cùng phong phú: nhảy lò cò, nhảy dây, chơi keo, ô quan, banh đũa, ném banh lố, đá banh mũ, bắn bi, tạt lon, tạt hình, tạt vé số, rượt bắt cứu bồ, gồng gánh lên mây, bịt mắt bắt dê, bida, điện tử, cờ tướng, cờ vua, cờ cá ngựa, đánh bài tiến lên, binh xập xám, bài cào, bài xì-lát, đô-mi-nô...

Trong những món này thì tôi, khi ấy 9 tuổi, thích chơi cờ tướng nhất, kế đến là điện tử, chỉ khi nào không chơi được hai món này thì tôi mới chơi những món khác. Thế rồi tôi đã quánh cờ tướng & chơi điện tử đến lúc ghiền hồi nào không hay luôn đó mà!

Với cờ tướng thì tôi thích việc bày binh bố trận, khám phá các thế trận biến hóa khôn lường, tính toán trước nhiều nước các chiến lược để làm sao tiêu diệt quân hoặc công thủng phòng tuyến và tiêu diệt con tướng của đối phương để giành chiến thắng.

Với điện tử thì tôi thích các trò chơi mang tính khám phá như Super Mario đi hái nấm cứu công chúa, Contra đi bắn đạn giết trùm cuối, Phi thuyền ngoài không gian, Hai chị em... bởi tôi là người có tính hiếu kỳ khá là cao.

Vào năm 1993, tôi đang học lớp 8 ở trường chuyên Nguyễn Gia Thiều – Quận Tân Bình.

Tôi không nhớ là Sở GD&ĐT TPHCM hay cơ quan quản lý giáo dục nào đã tổ chức các chương trình dạy nghề với nhiều ngành nghề khác nhau (IT, Điện, Điện Tử, May, Thêu...) cho học sinh lớp 8,9 tiếp xúc với các loại nghề nghiệp trong xã hội, rồi sẽ thi lấy chứng chỉ nghề cơ bản. Học sinh lớp 9 nào đạt được điểm tốt thì sẽ được cộng thêm điểm (1-2 điểm) cho kỳ thi tốt nghiệp PTCS (Phổ Thông Cơ Sở).

Dù thích chơi điện tử nhưng tôi đã không học thử nghề Điện Tử mà cùng với vài đứa bạn đã đăng ký học IT vì nghe tụi nó nói ngành IT mới mẻ có vẻ hay ho lắm!

Môn IT đầu tiên tôi được học là **MS-DOS** – Version 3.3 với các khái niệm cơ bản về File, Directory, Boot với các System File như *config.sys*, *command.com*, *autoexec.bat*, các Command cốt lõi của MS-DOS...

Khi xuống phòng thực hành thì tôi thấy có một số Computer với Monitor đen trắng hoặc mono (xanh lá cây), có vài cái Keyboard, chưa có Mouse.

Các Computer được khởi động bằng Floppy Disk 1.2MB (hình vuông, mỏng dính như lá lúa và dài) hoặc 1.44MB (hình vuông dày hơn và ngắn hơn) chứa MS-DOS.

Đặc điểm của loại Floppy Disk này là rất dễ hư và cũng mau hư. Tôi nhớ có lần đi học về để một số Floppy Disk 1.2MB ở cái giỏ trước xe đạp rồi xe bị sụp ổ gà nhẹ thôi mà cũng hư hết vài cái. Máy cái Floppy Disk 1.44MB để vài tháng không sử dụng thì khi dùng lại hay bị lỗi.

Ở mỗi bữa thực hành, sau khi khởi động Computer xong thì bọn tôi thực hành gõ các Command thao tác các File trên đĩa như **copy** (sao chép), **move** (di chuyển), **edit** (sửa đổi) File.

Cuối buổi, bọn tôi hay chạy một Software huyền thoại bọn tôi gọi là “Bắn Trứng” như sau:

- Từ trên đỉnh Monitor các quả trứng (1 chấm trắng) rơi từ từ xuống với tốc độ rơi tăng dần theo từng mức độ khó.
- Phía dưới cùng Monitor là hình cái Keyboard thể hiện vị trí các ký tự trên Keyboard.
- Trứng rơi theo cột của ký tự nào thì khi gõ ký tự đó sẽ có 1 tia đạn màu trắng bắn bùm chéo lên nát cái trứng thì ghi được 1 điểm.

Software này dùng để cho học viên luyện gõ Keyboard dùng cả 10 ngón tay cho nhanh và giúp nhớ các ký tự trên Keyboard.

Hè năm đó tôi được dịp tham gia Giải Cờ Tướng Học Sinh Toàn Quốc lần thứ 1 được tổ chức tại Thủ đô Hà Nội thật là vui và thú vị.



Sang năm 1994 khi tôi lên lớp 9, tôi tiếp tục học IT trong chương trình nghề đó.

Lúc này tôi học về **VNI, VietRes** là các Software soạn thảo văn bản trên nền MS-DOS (tựa như MS Word bây giờ trên nền Windows).

Tiếp đó là món **Lotus 123** để thao tác trên các **SpreadSheet** (Bảng Tính) cũng dựa trên nền MS-DOS (tựa MS Excel bây giờ trên nền Windows).

Điều thú vị của năm này là các Computer trong phòng thực hành đã có nhiều Software và **Game** (Trò Chơi) hơn để chúng tôi vọc:

- **NC (Norton Commander)**: Software giúp thao tác trên File với GUI dạng khung dễ dàng hơn so với gõ lệnh trong Command Line đen trắng. Hiện nay Software “Windows Commander” rất thịnh với User sử dụng hệ điều hành Windows.
- **Snake**: Trò chơi con rắn chạy trong các ô tường có hiện các số, càng “ăn” số theo thứ tự tăng dần thì con rắn càng dài thêm càng dễ húc đầu vô tường.
- **Billiards**: Trò chơi bida lỗ.
- **Tam Quốc Chí**: Trò chơi đánh nhau giữa 2 người sử dụng các nhân vật trong truyện Tam Quốc Diễn Nghĩa (hình thức tương tự như trò “Street Fighter” trong trò chơi điện tử với các nhân vật: Cô gái, Thăng Nhật, Thăng Mỹ, Thăng Mập, Trùm cuối là Thăng Công An).

2. Học lập trình Pascal ở cấp 3

Lên cấp 3 tôi học ở trường chuyên Lê Hồng Phong, Quận 5, TPHCM. Trường này dạy một môn IT về lập trình với ngôn ngữ Pascal trên nền MS-DOS cho học sinh lớp 10 và 11.

Tôi quyết định thi vào trường Lê Hồng Phong là do một sự tình cờ.

Số là sau kỳ thi tốt nghiệp PTCS, tôi đủ điểm để được tuyển thẳng vào trường Nguyễn Thượng Hiền, Quận Tân Bình và tôi đã nộp hồ sơ xong, chờ ngày nhập học.

Tình cờ vào một ngày đẹp trời, Ba tôi chở đi đâu đó ngang qua trường Lê Hồng Phong. Tôi thấy ngôi trường này khá to và đẹp nhưng không nghĩ đến chuyện thi vô vì đã nộp xong hồ sơ bên kia rồi.

Thế rồi đám bạn cấp 2 rủ tôi đi luyện thi vào trường Lê Hồng Phong. Lúc ấy tôi đang rảnh và cũng thắc mắc là đề thi vào trường đó thế nào nên mới đi học thử vài tháng rồi thi cho biết.

Tôi thi Ban A với 3 môn Toán/Văn/English. Đề thi công nhận khá khó so với trình độ tôi khi đó, dù đã học luyện thi trước. Tôi đã làm hết khả năng của mình nhưng không có môn nào xong hết nên tôi nghĩ chắc chắn là toi rồi.

Ai dè với kết quả tôi đạt được khá khiêm tốn là 6đ Toán, 5.5đ Văn & 3.5đ English (quả thật là thê thảm so với điểm cấp 2), nhưng lại vừa đủ điểm đậu vô Ban A (Toán nhân 3).

Nói đến thi tuyển thì yếu tố may mắn cũng đóng vai trò khá quan trọng ngoài việc đã chuẩn bị chuyên môn. Nhiều khi mình làm bài tốt cũng chưa chắc đậu, và làm bài không được tốt cũng chưa chắc rớt, phụ thuộc vào các thí sinh khác cùng thi với mình trong kỳ thi đó nữa.

Ba Má tôi thấy tôi đậu vô trường Lê Hồng Phong thì đã khuyên tôi chuyển trường Nguyễn Thượng Hiền qua với giải thích là trường Lê Hồng Phong tốt hơn.

Thật sự lúc đó tôi không quan tâm lắm đến trường nào tốt hơn trường nào mà tôi quan tâm nhiều hơn đến việc luyện tập/ thi đấu các giải Cờ Tướng học sinh (bởi tôi đang là thành viên đội Cờ Tướng Năng Khiếu Dự Bị Tập Trung của trường nghiệp vụ TPHCM).

Vì thế tôi cân nhắc giữa 2 trường này ở chỗ trường Nguyễn Thượng Hiền gần nhà tôi hơn (cách ~1km so với ~6km) còn trường Lê Hồng Phong to và đẹp hơn.

Cuối cùng tôi đã chọn trường Lê Hồng Phong vì nó to đẹp và vài đứa bạn cũng đã chọn nó nên lúc đi học xa mà đi chung cả đám cũng vui.

Vào mùa thu năm 1995, tôi đã bước vào năm lớp 10 ở trường Lê Hồng Phong với quá nhiều ngỡ ngàng do năm đầu cấp 3 và bị cú sốc khá nặng với cái trường này: Thi kiểm tra đầu năm với 3 môn Toán, Lý, Hóa tôi đều đạt điểm dưới trung bình ☹. Đây là điều chưa từng xảy ra với tôi trong 9 năm học trước đó.

Dù sốc nhưng đây là điều hợp lý vì tôi vô học trường này mà không có chuẩn bị kỹ lưỡng từ trước so với các bạn trong lớp và học cấp 3 cũng khác và khó hơn nhiều so với cấp 2.

Tôi đã rất vất vả và đăm đuối để đu theo chương trình lớp 10 của trường nhằm “trụ hạng” thành công. Bởi thế tôi đã không có nhiều thời gian cho môn Pascal vốn chỉ là môn phụ, mà tôi học môn ấy chủ yếu lấy điểm cho xong chứ không có tâm trí để tìm tòi học hỏi thêm ở ngoài.

Qua năm lớp 11, việc học của tôi đỡ hơn tí nên tôi có nhiều thời gian hơn dành cho môn Pascal. Năm này học sinh được thực hành trên Computer nhiều hơn năm trước nên tôi vọc được Pascal nhiều hơn, có thể tự viết được vài Program đơn giản (ví dụ như giải phương trình bậc 1, bậc 2, tính đệ qui giai thừa, dãy số Fibonacci, in các hình tam giác, hình ngôi sao, hình kim tử tháp...) chủ yếu sử dụng các Data Type cơ bản và các Control Flow cốt lõi.

Trong lớp tôi có một số bạn không biết học Pascal từ bao giờ mà lập trình gõ bàn phím nhanh như đánh đàn Piano, viết mấy chương trình hay ho với số lượng Code khá nhiều và nhìn rất ư là phức tạp. Điều này cũng gây cho tôi sự tò mò và thích thú với môn Programming nói riêng và ngành IT nói chung.

Cuối năm lớp 12 lúc chọn trường đại học để thi, một số bạn trong lớp tôi đã có kế hoạch đi du học từ trước, phần lớn còn lại, trong đó có tôi, thì cũng chưa biết theo ngành nghề gì vì việc định hướng nghề nghiệp giai đoạn đó không có nhiều và rõ ràng.

Trong lớp mọi người trao đổi sôi nổi để điền phiếu đăng ký thi thì có một số ngành được số đông lựa chọn:

- IT & Điện/Điện Tử & Xây Dựng (thi đợt 1)
- Ngoại Thương & Kinh Tế (thi đợt 2)
- Luật (thi đợt 3)

Kỳ thi đại học năm 1998 cho phép mỗi học sinh được thi 3 trường, được chọn thi 1 trường/đợt.

Tôi nghĩ tính tình tôi sẽ phù hợp với ngành kỹ thuật và thích IT nên đã chọn thi ngành IT trong đợt 1, bên cạnh đó tôi đã chọn thi ngành Kinh Tế đợt 2 & thi ngành Luật đợt 3. Tuy chọn 3 ngành nhưng trọng tâm của tôi vẫn là ngành IT, tôi tính trước nếu lỡ bị rớt ngành IT thì dự là học thi lại vào năm sau.

Kết quả thi là tôi đã đủ điểm đậu vào Khoa IT của trường Đại Học Bách Khoa (ĐH BK) TPHCM. ...Phù... hura hura hura.. một gánh nặng ngàn cân vừa được trút bỏ khỏi vai tôi bởi vì áp lực phải đậu đại học của học sinh trong giai đoạn đó cực lớn chứ không như thời nay việc vào đại học không phải là con đường duy nhất cho học sinh vào đời.

Cuối năm 1998 tôi đã tham gia Giải Cờ Tướng Vô Địch Đông Nam Á ở hạng mục U18 và tôi đã giành được Huy Chương Vàng. Đúng là một năm song hỷ cho tôi ^_^



3. Học Đại Học

Mùa hè năm 1998 cực kỳ sôi động với Giải Bóng Đá Thế Giới tại Pháp vừa qua đi, tôi đã trở thành tân sinh viên của trường ĐHBK TPHCM, bước vào giai đoạn Đại Cương học chung với toàn bộ sinh viên ở tất cả các Khoa của trường.

Sau giai đoạn Đại Cương gồm 3 học kỳ - 1.5 năm, tôi bước vào học giai đoạn Chuyên Ngành ở Khoa IT. Giai đoạn Chuyên Ngành gồm 6 học kỳ học theo mô hình **Credit** (Tín Chỉ).

Các sinh viên được quyền chọn các môn học cho mỗi học kỳ sao cho số Credit trong mỗi học kỳ chỉ dao động từ 14 đến 21 Credit. Nếu sinh viên không tự chọn các môn học thì sẽ học theo sự bố trí sẵn danh sách môn học cho từng học kỳ của Khoa IT.

Lúc đó tôi thật sự không có thông tin gì về ngành IT, chỉ có chút kiến thức từ hồi học nghề IT và cấp 3 (cũng đã lạc hậu hết rồi), không biết IT có những mảng nào, và cũng chưa rõ học xong ra trường có thể làm được những loại công việc gì luôn.

Vì thế tôi đã học theo sự sắp đặt của Khoa IT. Đây cũng là lựa chọn của số đông sinh viên khóa tôi. Chỉ một số nhỏ sinh viên nào đã có định hướng nghề nghiệp rõ ràng, có thể từ người thân đi trước, mới có khả năng tự lựa chọn được các môn học phù hợp cho họ.

Đây là danh sách các môn tôi đã học trong 6 học kỳ chuyên ngành theo sắp xếp của Khoa IT:

Học kỳ 4 • Anh Văn kỹ thuật • Thiết kế hệ thống số • Lịch sử Đảng • Cơ ứng dụng • Mạch điện 1 • Toán tin học • Lý thuyết thông tin • Quản lý doanh nghiệp	Học kỳ 6 • Thực tập điện/điện tử • Xử lý tín hiệu số • Cấu trúc dữ liệu • Automata • Vi xử lý • Kỹ thuật truyền số liệu • Đo điện • Thực tập đo điện	Học kỳ 8 • OOP • Trí tuệ nhân tạo (AI) • Phân tích thiết kế hệ thống DFD • Xử lý song song • Đồ họa máy tính • Thực tập tốt nghiệp
Học kỳ 5 • Cấu trúc máy tính • Ngôn ngữ lập trình • Thực hành NNLT • Mạch điện tử 1 • Logic Toán • Phương pháp tính • CNXH Khoa học	Học kỳ 7 • Chương trình dịch • Cơ sở dữ liệu • Hệ điều hành • Công nghệ phần mềm • Mạng máy tính • Phân tích TK giải thuật • Đồ án tin học	Học kỳ 9 • Vẽ kỹ thuật • Cơ sở dữ liệu phân bố • Lập trình mạng • LUẬN VĂN TỐT NGHIỆP

Hình 91: Bachelor Courses

Ngay thời điểm đăng ký thi IT hồi năm lớp 12, tôi đã hình dung ra viễn cảnh khi vô học sẽ được học lắp ráp Computer, rồi cài đặt Operating System, cài đặt các Software và lập trình trên đó các kiểu sẽ rất là thú vị.

Nhưng thực tế thì không có phần lắp ráp Computer, và tôi đã học hết học kỳ thứ 4 vẫn chưa được đụng đến Computer. Tôi đã tự học một khóa lắp ráp Computer ở Nhà Văn Hóa Thanh Niên TPHCM để hiểu thêm về các Hardware và Software thông dụng.

Đến học kỳ thứ 5 có phần thực hành Programming Language thì tôi mới được đụng đến Computer để lập trình 2 môn là **Prolog** (ngôn ngữ thuộc nhóm lập trình logic) & **Small Talk** (ngôn ngữ thuộc nhóm OOP).

Tư duy lập trình của Prolog theo hướng **Declarative Programming** (Lập Trình Đặc Tả) hơi khác so với các ngôn ngữ thông thường theo hướng **Imperative Programming** (Lập Trình Mệnh Lệnh): chúng ta đặc tả bài toán (WHAT) theo logic -> máy sẽ tự tìm giải pháp (HOW) để giải.

Sau đó tôi học các môn màu **cam** & **màu xanh đậm** thì thực hành nhiều trên Computer hơn.

Những môn màu **cam** là những môn cốt lõi của ngành IT mà bất cứ một dân IT dù theo bất cứ nghề IT nào cũng bắt buộc phải nắm vững phần nền tảng để làm công việc của mình hiệu quả.

1. **Cấu trúc máy vi tính (Computer Structure)**
2. **Ngôn ngữ lập trình (Programming Language)**
3. **Lập trình OOP**
4. **Cấu trúc dữ liệu (Data Structure)**
5. **Cơ sở dữ liệu (Database)**
6. **Hệ điều hành (Operating System)**
7. **Mạng máy tính (Computer Network)**
8. **Công nghệ phần mềm (Software Engineering)**
9. **Trí tuệ nhân tạo (Artificial Intelligence)**

Các môn này liên hệ mật thiết với nhau giúp cho việc xây dựng IT System hoàn chỉnh và đã được chia sẻ ở phần trên.

Vì đây là những môn nền tảng nên học khá nhiều về lý thuyết hơi bị khô khan và ngán.

Ví dụ điển hình là môn “Ngôn Ngữ Lập Trình” học chung chung với rất nhiều khái niệm trừu tượng, lại không được thực hành trên bất cứ Programming Language cụ thể nào hết (Môn “Thực Hành Ngôn Ngữ Lập Trình” là môn khác học sau môn này) nên hơi bị khó hình dung, toàn tự chạy chương trình thầy cho viết bằng **Pseudo-code** (Mã Giả) trên giấy để suy luận xem chương trình chạy như thế nào, kết quả sẽ ra sao 😊.

Thời gian đầu tôi học những môn này phải nói là rất vất vả và hơi nhũn nã. Nhưng nhờ sự đam mê IT cộng với tính kiên trì của một vận động viên cờ tướng, tôi đã cố gắng học hết sức thì từ từ tôi cũng hiểu ra sự liên hệ giữa các nhánh của ngành IT. Từ đây việc học các môn IT khác bớt khó khăn hơn.

Những môn màu **xanh đậm** là những môn cần thiết để học nhằm tích lũy thêm kiến thức.

Những môn màu *xanh nghiêng* (*Quản Lý Doanh Nghiệp, Phương Pháp Tính, CNXH Khoa Học, Vi Xử Lý, Kỹ Thuật Truyền Số Liệu, Phân Tích Thiết Kế Hệ Thống DFD, Xử Lý Song Song, Cơ Sở Dữ Liệu Phân Bố*) thực sự chưa cần thiết để học trong giai đoạn Chuyên ngành, sau khi tốt nghiệp đi làm ai có nhu cầu thì học sẽ phù hợp hơn.

Những môn màu *xanh nghiêng còn lại* (*Cơ Ứng Dụng, Mạch Điện 1, Mạch Điện Tử 1, Thực Tập Điện/Điện Tử, Đo Điện, Thực Tập Đo Điện, Vẽ Kỹ Thuật*) quả thực là những môn không cần thiết để học cho chuyên ngành IT. Có thể sau này ai đi làm đụng đến những mảng này thì học riêng sẽ thuận tiện hơn.

Tôi và các bạn trong khóa thực sự rất cực nhọc để học những môn vô thưởng vô phạt này. Không những vậy mà cũng có khá nhiều bạn vì những môn này mà kẹt tốt nghiệp các học kỳ sau đó dẫn đến không nhận được bằng đại học để đi tìm việc cho thuận lợi.

Tôi may mắn vừa đủ điểm qua những môn này nên không bị kẹt vụ tốt nghiệp, nhưng điểm làng nhàng của những môn này của tôi cũng đã ảnh hưởng phần nào đến điểm tốt nghiệp (GPA) khi tôi nộp đơn xin du học ở các trường nước ngoài sau đó.

Trong thời gian học chuyên ngành, tôi quan sát thấy cũng nhiều bạn trong khóa **học theo nhóm** rất là hiệu quả, vừa giúp họ tiết kiệm thời gian vừa đạt điểm số tốt hơn so với họ tự học.

Rất tiếc lúc đó tôi không có tham gia nhóm nào để học chung mà cứ lủi thủi học một mình. Môn nào có bài tập làm chung với ai đó thì tôi mới cùng học & làm với bạn đó. Học một mình vừa mất nhiều thời gian và chịu nhiều áp lực do số môn học khá nhiều trong mỗi học kỳ.

Giờ ngồi nghĩ lại tôi thấy mình có một tí sai sai khi bỏ thời gian học các môn đều như nhau với kỳ vọng sẽ có điểm tốt nghiệp tốt hơn. Nhưng bất cứ ai học dàn trải như vậy thì không thể nào học tốt tất cả các môn được ngoại trừ nhân tài. Tôi thì không thuộc dạng này rồi!

Cách học phù hợp hơn nên là:

- Tập trung toàn lực học các môn màu **cam** & **xanh đậm** trước, chắc mất khoảng 3 học kỳ
- Tìm công ty phù hợp đi thực tập liền (trong khi vẫn còn nhớ các môn màu **cam**) để tích lũy kinh nghiệm thực tế (thời đó việc này có lẽ khó khăn hơn rất nhiều so với thời nay nhưng là việc rất rất ư là phải làm).
- Học các môn màu **xanh nghiêng** cho qua, không cần thiết phải bỏ nhiều thời gian với kỳ vọng lấy điểm tốt ở những môn này.
- Quan trọng nữa là nên **học theo nhóm!!!**

II. HỌC IT Ở CANADA

1. Nhận học bổng và lo hồ sơ nhập học

Trong thời gian cấp 3 thì khá nhiều bạn trong lớp tôi đã lo xa đến chuyện sẽ du học nước này nước nọ, còn tôi thì cứ vô tư vừa học vừa luyện cờ tướng đi thi đấu kiếm thu nhập trả tiền học và tiêu dùng, hạn chế xin tiền Ba Má.

Vừa lên đại học, một số bạn cấp 3 của tôi đã đi du học liền.

Đến giai đoạn đại học thì bạn bè tôi cũng tìm kiếm thông tin này nọ về du học. Một số đứa học xong năm 1,2 đại học cũng đã lấy được học bổng đi học ở Singapore, USA, Russia... còn tôi thì dính với đội tuyển cờ tướng TPHCM lo đi thi đấu khắp nơi 2-3 giải/năm, mỗi giải từ 10 đến 15 ngày. Mỗi lần tôi đi thi đấu về thì bài vở chồng chất, chạy bỏ hơi tai mới kịp nên mấy vụ du học này nọ tôi chả có đầu óc đâu mà quan tâm.

Sau khi tôi tốt nghiệp đại học thì đợt sóng du học lần thứ 3 trong bạn bè tôi đã nổ ra. Đến thời điểm đó thì bạn bè lớp cấp 3 của tôi đã ra nước ngoài cũng 2/3 lớp. Điều này làm tôi thật sự thắc mắc là ở nước ngoài có cái gì hay ho mà bạn bè tôi đi du học nhiều thế nhỉ??? Tự hỏi vậy chứ tôi cũng chưa quan tâm đến vụ du học.

Khi tôi vừa vô làm ở công ty SaigonTel năm 2003, đứa bạn cấp 3 của tôi tên Tân đang học Master ở Canada hỏi tôi có muốn đi du học Master ở Canada không, nếu muốn thì hắn sẽ hỗ trợ tôi xin học bổng. Hắn nói trước cho tôi biết sẽ học bằng French và ở Canada thì thời tiết rất lạnh vào mùa đông. Tôi nghe đến vụ trời rất lạnh thì thấy ngán rồi vì tôi rất ngại lạnh, mà lại thêm French nữa thì rất ư là thử thách. Vả lại tôi mới học xong 4.5 năm đại học nên ngán học muốn chết luôn, mới khó khăn tìm được việc làm thì đi làm trải nghiệm thực tế tích lũy kinh nghiệm và kiếm chút tiền đi chơi cái đã, chuyện khác từ từ tính.

Thời gian sau đó phong trào đi du học khá nở rộ, nghe mấy người du học ở các nước phát triển xong đi làm ở bên đó nói môi trường làm việc rất tốt, thu nhập cao, cơ hội nâng cao trình độ chuyên môn nhiều. Thế là tôi bắt đầu có ý muốn tìm đường đi ra nước ngoài xem sao, đi làm hay đi học gì cũng được.

Vào đầu năm 2005, Tân về Sài Gòn ăn Tết có ghé nhà tôi chơi. Khi này tôi đã chuyển qua công ty TMA. Không biết Tân đã nói chuyện với Má tôi thế nào mà sau đó Má tôi nói với tôi phải nộp đơn xin học bổng Tân giới thiệu ngay và luôn, nếu may mắn xin được học bổng mà vẫn còn thiếu chi phí thì Má sẽ cho thêm, bằng mọi giá tôi phải đi du học!

Nghe Má tôi nói thế thì tôi cũng hơi ngạc nhiên và thắc mắc tại sao phải đi du học bằng mọi giá. Tôi có hỏi Má thì Má chỉ nói là tôi phải đi để nâng cấp tư duy, tiếp cận tiến bộ, văn minh của thế giới, nếu không thành công thì cũng sẽ thành nhân (trưởng thành hơn).

Khi ấy công việc IT của tôi bắt đầu làng nhàng, còn cờ tướng thì tôi đi làm toàn thời gian không có nhiều thời gian đầu tư như thời sinh viên nên thi đấu các giải cũng không có gì đột phá, do đó ý muốn ra nước ngoài đương đầu thử thách mới đã quay lại trong đầu tôi. Thế là tôi đã nói chuyện kỹ hơn với Tân về học bổng mà Tân dự định hỗ trợ tôi xin ở Canada.

Trước tiên chúng ta tìm hiểu sơ qua về đất nước Canada:

- **Canada** là nước nằm ở châu Mỹ, phía Bắc nước Mỹ với diện tích 9.9 triệu km², dân số ~40 triệu người. Xã hội Canada đa sắc tộc, đa văn hóa với nguồn gốc từ người Aborigines, người Anh, Pháp, châu Âu, châu Á, châu Phi...
- Hiện nay, Canada có 10 **Province** (Tỉnh) và 3 vùng **Territory** (Lãnh Thổ).
- **Thủ đô** (Capital) của Canada là thành phố **Ottawa** thuộc tỉnh Ontario, tỉnh lớn nhất Canada.
- **Thành phố lớn nhất** của Canada là thành phố **Toronto** (cũng thuộc tỉnh Ontario).
- **Tỉnh Quebec** (dân Canada gọi là xứ **Québécois**) là tỉnh tự trị trực thuộc Liên bang Canada có hệ thống luật và thuế riêng. Người nước ngoài muốn vô được tỉnh Quebec thì ngoài việc xin visa Canada phải xin thêm **CAQ (Certificat d'Acceptation du Québec)**.
- Tỉnh Quebec là tỉnh duy nhất của Canada nói French. Vì thế dân Quebec phần lớn nói French là chính trong cuộc sống (dù họ cũng có thể nói English tốt).
- Thủ đô của tỉnh Quebec là **thành phố Quebec** (trùng tên với tỉnh nên nhiều sinh viên Việt Nam lúc mới qua hay bị nhầm lẫn). Dân trong thành phố Quebec hầu như chỉ nói French.
- **Thành phố Montreal** là thành phố lớn nhất của tỉnh Quebec, và lớn thứ hai của Canada. Dân Montreal nói cả French và Anh trong đời sống hàng ngày.
- Theo luật pháp Canada thì English và French là 2 ngôn ngữ chính thức của quốc gia Canada, bất cứ công dân Canada nào cũng có thể yêu cầu giấy tờ hành chính English hoặc French.

Các công ty công nghệ lớn ở Canada luôn có quỹ về **Research and Development – R&D** (Nghiên Cứu Và Phát Triển) cấp tiền cho các dự án R&D đủ loại từ nhỏ đến lớn.

Một số dự án R&D được thực hiện bởi các nhóm R&D trong các công ty.

Một số dự án R&D khác sẽ được đẩy vào các trường đại học ở Canada kèm theo các học bổng R&D cho các giáo sư để họ tuyển các sinh viên học **Master** (Thạc Sĩ) hoặc học **Ph.D** (Tiến Sĩ) tham gia vừa học vừa làm dự án. Dự án làm xong sẽ được bảo vệ thành luận văn tốt nghiệp để lãnh bằng luôn. Người nước ngoài xin các học bổng R&D của Canada thông qua các giáo sư là chủ yếu. Giá trị của học bổng R&D thay đổi tùy ngành, tùy trường, tùy quỹ của mỗi giáo sư. Đối với dân thuộc các nước kém phát triển hơn Canada, họ lấy được học bổng loại này thì cũng là điều quá tốt cho họ rồi.

Về phía Canada thì đây là chính sách khá hay để thu hút chất xám của thế giới với giá rẻ:

- Canada không mất chi phí đào tạo 12 năm phổ thông và 5 năm đại học để có được một sinh viên Master.
- Để thuê một kỹ sư Canada trình độ từ Fresher hay Mid Level làm dự án R&D này thì họ phải trả lương dao động gấp 2-3 lần so với học bổng R&D.
- Kỹ sư bản xứ thì chỉ làm trong tuần, còn thứ 7 và chủ nhật được nghỉ và họ không có ràng buộc phải xong dự án, họ có thể nhảy việc; trong khi sinh viên Master thì học và làm suốt cả tuần, bằng mọi giá phải xong trong 2 năm để tránh hết tiền và phải lấy được cái bằng.

Tuy nhiên, cái gì cũng có 2 mặt của nó, nếu một sinh viên Master không chịu học nghiêm túc hoặc học rồi mà làm không nổi dự án thì cũng tiêu một cái học bổng R&D. Nhưng thực tế thì hầu như sinh viên Master nào tham gia dự án R&D sớm hay muộn đều xong cả, nên số lượng sinh viên bỏ cuộc hầu như chỉ rơi vào sinh viên Ph.D, điều này cũng không bất ngờ vì nghiên cứu để phát minh một cái gì đó mà chưa hề tồn tại trên thế giới đâu có đơn giản !

Một ứng viên sau khi được giáo sư Canada xét duyệt hồ sơ và cấp học bổng R&D thì sẽ nộp hồ sơ vào trường nơi giáo sư đó đang dạy để xin **Study Admission** (Cho Phép Học).

Các trường sẽ xét điểm ở đại học và trình độ ngoại ngữ của ứng viên. Các ứng viên nhận được học bổng R&D thì việc xét điểm số sẽ không còn quan trọng so với ứng viên bình thường bởi đã được vị giáo sư xét rồi, chỉ còn xét ngoại ngữ mà thôi.

Tân có người nhà quen với một giáo sư đang dạy ở trường **ÉTS** (École de Technologie Supérieure) ở Montreal. Ông này đang có một dự án IT tên là “Managing emotions in Human-Computer Dialog Systems” được đẩy qua từ Công ty Bell Canada (công ty điện thoại/viễn thông lớn nhất Canada) hợp tác với Viện nghiên cứu máy tính của Montreal (Computer Research Institute of Montreal – CRIM) và một số học bổng Master và Ph.D để tuyển 6 sinh viên Master và 10 nghiên cứu sinh Ph.D vào tham gia dự án.

Tân thấy dự án IT này có vẻ hợp với tôi và quan trọng là trường ÉTS dạy bằng French nhưng không yêu cầu phải có bằng French đầu vào (mà yêu cầu đầu ra phải có bằng French mới được tốt nghiệp) nên nói tôi nộp hồ sơ cho ông giáo sư để xin học bổng R&D Master.

Trong bảng điểm đại học của tôi, điểm mấy môn IT thì rất ổn, mà mấy môn màu *xanh nghiêng* có điểm lại hơi lằng xằng nên điểm trung bình GPA chỉ xém giỏi (7.7/10) chứ không thuộc loại giỏi (8.0+/10). Vì thế tôi cũng hơi bị không tự tin với bảng điểm của mình. Và rồi tôi cũng đã nộp đơn cho ông giáo sư để xin học bổng Master vào học trường ÉTS.

Ông giáo sư xem điểm mấy môn IT của tôi nhận xét cũng ok và nhờ có người nhà của Tân đã đứng ra đảm bảo về lực học của tôi (còn quảng cáo thêm vụ tôi thi đấu cờ tướng các giải quốc gia cho đội tuyển TPHCM nữa - nhiều khi có tí tài lẻ cũng hay chứ nhỉ hihi) nên ông giáo sư đã đồng ý cấp học bổng cho tôi qua học Master ở trường ÉTS và tham gia dự án của ông.

Sau khi tôi nhận được giấy cấp học bổng R&D từ ông giáo sư. Tôi xin tiếp Study Admission của trường ÉTS.

Các trường đại học dạy bằng French ở Montreal không yêu cầu sinh viên có bằng French trước khi nhập học. Tuy nhiên để tốt nghiệp Master thì sinh viên phải thi một chứng chỉ French theo yêu cầu hoặc vượt qua được một số môn French của trường.

Tiếp đó tôi xin **Study Permit** (Giấy Phép Học) của Canada dành cho sinh viên quốc tế. Xong rồi tôi xin tiếp CAQ của tỉnh Quebec. Khi vừa nhận được CAQ, tôi nộp hồ sơ xin visa Canada.

Ngày lên đường cũng đã đến. Tâm trạng tôi khi ấy háo hức thì ít mà hồi hộp thì nhiều vì tôi sẽ trải qua rất nhiều cái mới, những điều xa lạ, và nhiều thử thách cam go mà lần đầu tiên trong đời mới gặp.

Lần đầu tiên tôi bước lên máy bay khá to (hồi xưa khi đi thi đấu cờ tướng ở Gia Lai tôi cũng đã từng được đi máy bay nhưng là máy bay cánh quạt bé tí, đã vậy lúc hạ cánh do thời tiết xấu chỉ còn một cánh quạt quay được, hên là cả đoàn học sinh TPHCM không sao 😊).

Lúc máy bay cất cánh tôi thấy như bị lên máu não, tim đập thành thịch, cảm giác lâng lâng khó chịu. Rồi khi máy bay hạ cánh xuống sân bay Philippines thì 2 lỗ tai tôi đau nhói, tôi ra khỏi sân bay rồi mà 2 lỗ tai vẫn ù ù suốt 10', nghe người khác nói nhỏ xíu.

Trước khi lên đường thì chị của Tân đã nhiệt tình dẫn Má con tôi ra chợ trung tâm Sài Gòn mua cho tôi quần áo ấm mặc mùa đông, trong đó có cái áo lạnh to tổ bố. Trong khi chờ chuyển tiếp ở sân bay Philippines, tôi đi nhà vệ sinh đã để quên cái áo trên cửa nhà tắm. Sau đó vài phút tôi chợt nhớ quay lại thì cái áo đã bốc hơi 😞

Khoảng 21 giờ ngày 06/12/2005, sau một chuyến bay khá dài với 2 lần quá cảnh (Sài Gòn – Manila – Vancouver – Montreal), tôi đã đáp xuống sân bay Trudeau, Montreal. Tân và Biên, em ruột Tân, đã chờ đón tôi ở ngoài sân bay.

Vừa bước ra sân bay đến cái cổng chuẩn bị ra ngoài thành phố thì tôi bắt đầu thấy hơi lạnh lạnh. Tôi ngẩng nhìn những bông tuyết rơi lã tã bên ngoài khung cửa kính, cảnh tượng êm đềm thật nên thơ. Tôi rất thích thú muốn chạy ngay ra ngoài để rờ vào tuyết xem như thế nào thì chợt nhớ là cái áo lạnh duy nhất mình đã làm mất rồi.

Thế là tôi đành lấy cái áo khoác mùa hè mà tôi gọi là “cái áo nghĩa tình” để mặc tạm. Cái áo này tôi được SCS Team (công ty TMA) tặng vào bữa tiệc chia tay ở một quán canh bún. Bữa đó trời mưa to kinh khủng nhưng 2 đồng nghiệp SCS đã không ngại mặc áo mưa phóng xe máy đi mua áo làm quà cho tôi. Khi về lại quán, 2 đồng chí ấy bị ướt như chuột lột, chị trưởng nhóm thay mặt SCS Team tặng áo cho tôi làm tôi quá xúc động!

Ngoài trời tuyết vẫn đang rơi nhẹ nhẹ trắng xóa dưới ánh đèn đường. Một chiếc taxi được Tân gọi đến. Trên đường bước ra taxi tôi không quên vớt một nắm tuyết trên không trung để “cảm” xem ra sao...à, thì ra tuyết mới rơi nó cũng giống như bông tuyết trong học nước đá của tủ lạnh nhưng nó mềm và xốp hơn. Nhiệt độ ngoài trời cỡ từ -10oC đến -15oC, tôi có một cảm giác lâng lâng rất khó tả, y như tôi đang bay bổng...

Dù cái áo nghĩa tình không đủ ấm nhưng tôi cũng không cảm thấy lạnh lắm, chắc là hơi nóng từ Sài Gòn đem sang vẫn còn chẳng? Tôi nhớ lại hồi ra Hà Nội mới có 9oC, tôi mặc 4 lớp áo mà lạnh teo, run rẩy. Chắc là độ ẩm bên Montreal thấp nên thời tiết khô dễ chịu hơn so với Hà Nội có độ ẩm quá cao dẫn đến rét buốt.

Chiếc taxi đưa chúng tôi về nhà số 1467 đường Wolfe, nơi Nghĩa, một du học sinh Việt Nam đang ở. Tôi sẽ ở chung căn nhà 2 phòng này với Nghĩa.

Thật là cảm ơn Tân & Biên đã sắp xếp trước mà tôi có được chỗ ở ngay khi đến Montreal, chứ nếu không thì thật là đăm đui khi phải đi kiếm nhà trọ trong mùa đông. Cũng vì chuyện này mà về sau nhà tôi thuê ở số 2235 đường Prud'homme trở thành “trạm trung chuyển” cho khá nhiều du học sinh Việt Nam đến Montreal ở tạm một thời gian cho đến khi tìm được chỗ ở.

Chuyến bay 6 tiếng từ Vancouver qua Montreal không có phục vụ ăn uống (mấy chuyến bay nội địa của Canada thường không có dịch vụ này), tôi chỉ ăn được tạm vài miếng bánh ngọt Kinh Đô đem từ nhà sang nên bụng đói meo. Vừa ổn định chỗ, tôi khui hành lý lấy nguyên thùng mì gói ra và làm một gói mì thiệt là quá ngon 🥰

Chúng tôi ngồi chém gió một lúc thì anh em Tân ra về.

Tôi mệt lã chìm vào giấc ngủ ngon lành.

Một hành trình mới đầy cam go thử thách với một cuộc sống hoàn toàn mới đang chờ đợi tôi vào ngày hôm sau.....

2. Học Master (Thạc Sĩ)

Học kỳ mùa đông 2006

Đúng ngày 05/01/2006, tôi bước vào học kỳ đầu tiên ngay trong mùa đông đầu tiên đầy háo hức và rất rất là hồi hộp...

Tôi học trường ÉTS được tách ra từ trường UQAM (Université du Québec à Montréal – ngôi trường rất lớn và lâu đời ở Montreal).



Trường ÉTS chỉ chuyên dạy về công nghệ, sách tham khảo phần lớn là English nên tôi cũng bớt được tí lo lắng cho trình độ French vỡ lòng của mình.

Bữa đầu tôi đi gặp ông giáo sư để chọn chương trình Master và lên kế hoạch cho dự án nằm trong đề tài Master. Tôi hơi bị khộp khi thấy ông thầy đáng cao to, tóc thầy bạc trắng giống y như mấy tiền bối luyện công lâu năm trong phim chưởng Kim Dung vậy á.

Sau phần chào hỏi sơ bộ, ông chọn cho tôi một chương trình Master với 7 Course (21 Credit) và 1 cái **Thesis** (24 Credit). Tổng cộng là 45 Credit.

Có 2 loại chương trình Master ở Canada:

1. **Course-based Master (45 Credit):** học hết các môn học qui định thì tốt nghiệp.
2. **Research-based Master (45 Credit):** học một vài môn học cung cấp kiến thức làm dự án R&D. Chương trình Master của tôi thuộc dạng này.

Nhìn lướt qua 7 môn học thì tôi vã mồ hôi, chả biết gì ráo vì vừa đặc tả bằng French vừa dính rất ít đến mảng Software Engineering mà tôi biết.

Thầy nói học kỳ đầu tôi chưa quen trường mới nên ông chỉ đăng ký cho tôi học 2 môn (3 Credit/ môn) và điền thêm 1 môn của dự án R&D (3 Credit), để đủ ít nhất 9 Credit cho **Fulltime Student** (Sinh Viên Toàn Thời Gian):

1. Sécurité de l'internet (**Internet Security** – Bảo Mật Internet) - (3 Credit): học ở trường ÉTS

2. Reconnaissance de la parole (3 Credit): học ở trường ÉTS, sau đó chuyển qua Course “**Automatic Speech Recognition** – ASR” (3 Credit) học ở trường McGill.

3. **Research Project** (3 Credit) (ÉTS): chỉ mới đọc tài liệu sơ sơ, chưa bắt đầu.

Môn đầu tiên thì tôi đoán được là về bảo mật trên mạng Internet, môn thứ 2 thì tôi mù tịt.

Do ÉTS không mở môn thứ 2 trong học kỳ này nên thầy đã đăng ký cho tôi qua học bên trường đại học McGill (dạy bằng English) môn tương đương có tên là “Automatic Speech Recognition”, có nghĩa là nhận dạng giọng nói.

Với môn **Internet Security**, cái tên môn học là bảo mật trên mạng Internet liên quan đến một vài lĩnh vực bảo mật, tuy nhiên giảng viên lại dạy đến 10 lĩnh vực bảo mật giống như đang luyện cái bằng CISSP (Certified Information Systems Security Professional) của Microsoft nên nội dung rất nhiều.

- Chapter 1: Access Control
- Chapter 2: Telecommunications and Network Security
- Chapter 3: Information Security Governance & Risk Management
- Chapter 4: Software Development Security
- Chapter 5: Cryptography
- Chapter 6: Security Architecture & Design
- Chapter 7: Operations Security
- Chapter 8: Business Continuity & Disaster Recovery Planning
- Chapter 9: Legal, Regulations, Investigations, and Compliance
- Chapter 10: Physical (Environmental) Security

Học kỳ mùa hè 2006

Đến học kỳ hè, sư phụ đánh giá thấy tôi chưa có kiến thức về Speech, Speech Recognition nên đã chuyển tôi qua mảng dự án khác có vẻ phù hợp với tôi hơn là Data Mining, đồng thời cũng đổi chương trình Master của tôi luôn. Chương trình Master mới của tôi gồm:

- 5 Course (15 Credit), bao gồm môn Internet Security (3 Credit) tôi đã xong học kỳ trước.
- Dự án R&D (30 Credit), bao gồm 6 Credit tôi đã đăng ký học kỳ trước dù tôi chỉ mới đọc tài liệu dự án sơ qua, chưa làm gì nhiều trên đó cả.

Như vậy so với chương trình Master cũ, chương trình Master mới đã giảm số Course xuống và tăng số tín chỉ dự án R&D lên để tôi có nhiều thời gian làm dự án hơn.

Trong học kỳ này, ÉTS chỉ mở 2 trong 4 môn học còn lại của tôi, đó là “Compléments Mathématiques” (**Advanced Mathematics** – Toán cao cấp) (3 Credit) và “Conception de logiciel” (**Software Design** – Thiết kế phần mềm) (3 Credit).

Vì vậy, tôi đăng ký 2 môn này cho học kỳ hè và 3 Credit của dự án R&D cho đủ 9 Credit để thành Fulltime Student. Học kỳ này tôi bắt đầu gặp sự phụ nhiễu để thảo luận về dự án.

Nhằm tăng cường khả năng nghe nói French, tôi đăng ký thêm một môn French giao tiếp của ÉTS. Ngoài ra, tôi đăng ký ngoại khóa một môn “English giao tiếp” bên trường Concordia (trường đại học dạy bằng English) để củng cố nền tảng English và khả năng giao tiếp bằng English của tôi luôn.

Môn **Advanced Mathematics** được dạy bởi một Tiến sĩ người Quebec, giảng bài & slides đều bằng French. Tuy nhiên, môn này tôi không gặp vấn đề với rào cản ngôn ngữ vì toán học thì nặng về ký hiệu và Logic nên tôi nghe thầy giảng bằng French chưa hiểu thì đọc slides cũng hiểu được phần nào, cuối giờ hỏi thêm thầy. Khi thực hành với lập trình Matlab chạy ra kết quả thì sẽ hiểu tường tận hơn.

Sách tham khảo của môn Advanced Mathematics lại là một cuốn sách viết bằng English của Mỹ. Thú thật là hồi học Toán ở Việt Nam tôi rất ít khi đọc sách Toán, chủ yếu nghe thầy giảng trên lớp rồi bài tập về nhà thầy cho các bài nào thì giở sách ra đúng các bài đó mà làm cho xong rồi thôi. Sách Toán của Việt Nam viết khô khan với quá nhiều công thức, nặng phần tính toán, ít diễn giải, cũng khá nhiều bài Toán đánh đố mà không mang tính ứng dụng thực tiễn. Ngược lại, cuốn sách tham khảo này viết cực kỳ hay. Bắt đầu mỗi phần là một tình huống thực tế, diễn giải một hồi mới ra vấn đề cần giải quyết, rồi đến phần phân tích để cuối cùng ra bài Toán cần giải, nói chung là rất cuốn hút người đọc. Vì thế lần đầu tiên trong đời tôi đọc hết một cuốn sách Toán từ đầu đến cuối!

Qua việc này tôi nhận thấy một điều: sách của tụi Tây dạy người học cách đặt vấn đề (WHAT) và giải quyết vấn đề bằng máy, còn sách của Việt Nam dạy người học làm thế nào (HOW) để giải một bài toán mang tính chất thuộc lòng và tính toán bằng tay nhiều.

Môn **Software Design** được dạy bởi một Tiến sĩ Mỹ, chuyển qua định cư ở Montreal, làm giáo sư mảng phần mềm ở trường ÉTS. Ông này từng làm ở bộ phận phát triển phần mềm trong NASA (National Aeronautics and Space Administration), cơ quan hàng không vũ trụ của Mỹ.

Theo qui định của trường thì ông phải giảng và soạn slides bằng French. Tuy nhiên, sách tham khảo thì vẫn bằng English, và cũng khá dài, không kém cuốn sách môn Internet Security là bao.

Ông thầy này nói French như gió, không khác gì English là tiếng mẹ đẻ của ông. Do nội dung môn Software Design liên quan đến mảng phần mềm tôi đã làm nhiều và trình độ French của tôi lúc này cũng đã khá lên tí nên tôi có thể hiểu được 50%-60% những gì thầy giảng. Và cũng giống như môn Internet Security, Advanced Mathematics, cuối mỗi buổi tôi đều ở lại hỏi thêm thầy cỡ 30' bằng English.

Môn Software Design cũng có rất nhiều bài tập nhỏ, 1 bài tập thực hành nhóm và 1 bài thi cuối kỳ, không có thi giữa kỳ.

Một học kỳ có 15 tuần, tôi học 2 môn Advanced Mathematics & Software Design có 14 tuần liên tiếp mỗi tuần phải hoàn thành 1 bài tập (vừa tìm hiểu vừa làm, chứ không có bài nào biết làm liền cả) làm tôi chạy trốn chết luôn. Thường thì đến cuối tuần tôi mới làm xong 1 bài để nộp (hạn chót là tối Chủ nhật hàng tuần) nên nhiều lúc hồi hộp thiệt luôn.

Các trường đại học ở Canada chú trọng vào việc thực hành nên bài tập hơi bị nhiều. Vì thế tôi thấy sinh viên đại học bên đây thường chỉ học 4 môn/học kỳ (ở Việt Nam học từ 6-8 môn/học kỳ), và sinh viên Master học 3 môn/1 học kỳ. Nếu học nhiều môn hơn thì nguy cơ làm bài tập không kịp rất là cao.

Trước kỳ thi cuối kỳ môn Software Design, giảng viên cho biết là đề thi sẽ là thi viết, không cho đem tài liệu vào phòng thi. Tuy nhiên, mỗi sinh viên được quyền đem 1 tờ giấy A4 vào phòng thi để tham khảo, trên đó sinh viên muốn viết gì thì viết.

Khi nghe đề thi viết, tôi lập tức xin giảng viên cho tôi viết bằng English thì ông ok luôn, còn việc dịch đề thi từ French sang English thì tôi nghĩ không cần thiết vì nội dung mảng phần mềm thì French dùng rất nhiều từ tương tự English mà tôi có thể đoán được, và thầy cũng cho phép được hỏi thầy bằng English nếu có chỗ nào đó tôi chưa hiểu rõ.

Trong quá trình học môn Software Design, tôi cũng đã tổng hợp kiến thức từ những gì thầy giảng và cuốn sách tham khảo vào 1 file Word. Từ file Word này, tôi lại rút trích những ý quan trọng nhất và cố gắng nén hết cỡ vào tờ giấy A4 càng nhiều chữ càng tốt để đem vào phòng thi tham khảo khi cần.

Thật kỳ diệu thay, sau khi tôi hoàn tất tờ giấy A4 với chữ nhỏ xíu, chỉ chít chằng chịt, hầu như không còn khoảng trống, rồi đọc qua đọc lại vài lần thì tôi thuộc luôn nội dung trong tờ A4 này.

Đến khi làm bài thi, nội dung trong tờ giấy A4 tự nhiên hiện trong đầu tôi mà tôi không cần tham khảo tờ giấy A4 nữa. Nhờ vậy mà tôi làm bài thi kịp thời gian.

Tôi phải công nhận ông thầy cho đề thi khá là hay, ai mà tham khảo tờ giấy A4 thì hầu như làm bài thi không kịp, chỉ có đọc đề xong là cầm đầu viết như cái máy thì mới kịp giờ.

Học kỳ mùa thu 2006

Bước vào học kỳ mùa thu năm 2006, tôi đã đăng ký các môn học sau:

- **Pattern Recognition** - PR (3 Credit) ở Concordia vì ÉTS không mở môn tương ứng ở học kỳ này.
- **Dự án R&D** (6 Credit)

Môn Pattern Recognition này học về nhận dạng **Pattern** (Mẫu) và **Pattern Classification** (Phân Loại Mẫu), thuộc nhánh AI. Khi nào học xong môn này tôi có thể làm phần Data Mining trong dự án R&D của tôi.

Tôi học môn Pattern Recognition ở trường Concordia dạy bằng English nên đỡ vất vả cho tôi hơn là học môn tương tự bằng French ở ÉTS. May là ÉTS không có mở ở học kỳ này, chứ ÉTS mà mở thì chắc chắn sư phụ của tôi sẽ kêu tôi đăng ký ngay vì các môn bên ÉTS ưu tiên hơn.

Nhờ học được 6 tháng trong môi trường dạy bằng English ở McGill nên tôi không bị bối ngỡ với môi trường dạy English ở Concordia, cộng với môn ngoại khóa “English giao tiếp” tôi đã học ở học kỳ mùa hè tại Concordia này, tôi hiểu được giảng viên giảng khoảng từ 80% - 90%.

Tôi vẫn tiếp tục giữ thói quen tốt là cuối giờ ở lại hỏi thầy những gì chưa hiểu trong slides & sách. Nói chung giảng viên nào gặp tôi thì hơi bị mệt vì cuối giờ thường phải ở lại để tôi hỏi tí. Điều đặc biệt là tất cả giảng viên tôi đã học đều rất nhiệt tình giải thích những thắc mắc của tôi cuối mỗi buổi học.

Trường Concordia được dân châu Á vào học khá là đông như Việt Nam, Trung Quốc, Thái Lan, Nhật, Hàn... Trong lớp môn Pattern Recognition này cũng có rất nhiều sinh viên châu Á, nhưng không có một sinh viên Việt Nam nào ngoài tôi.

Giảng viên cho chọn nhóm ngẫu nhiên trong phần thực hành nhóm hay sao đó mà tôi chung nhóm với 1 em Trung Quốc là sinh viên chính quy của trường Concordia, chứ không phải đi học từ trường khác như tôi.

Qua hỏi thăm sơ bộ thì em này cũng học Master of IT, nói English ok, lấy được TOEFL 600 điểm ở Trung Quốc, chà, tôi nghĩ đây không phải là dạng vừa đâu. Tuy nhiên, cái email đầu tiên ẻm gửi tôi thảo luận về bài tập làm tôi rất bất ngờ: ẻm viết sai chính tả, văn phạm khá nhiều.

Làm bài tập chung một thời gian thì tôi nghĩ chắc ẻm lấy TOEFL 600 nhờ luyện chiêu gì đó chứ trình độ English của ẻm cũng bình thường. Có lần tôi hỏi chơi chơi ẻm đã luyện TOEFL ra sao thì ẻm mới nói là bên Trung Quốc có những lò luyện bao đậu TOEFL 600 để nộp hồ sơ du học. Ố ồ, thì ra cái tôi nghĩ là đúng thạc.

Trở lại với bài tập nhóm, trong lớp hầu như các nhóm châu Á đều chọn đề tài cũ để làm cho dễ, trong đó có nhóm tôi với ẻm cũng thống nhất chọn đề tài cũ. Có 1 nhóm Tây chọn 1 đề tài chưa ai làm để tha hồ mà sáng tạo!

Kết quả là hầu hết các nhóm châu Á chọn đề tài cũ đều làm xong, thuyết trình ok nhưng chỉ được 8 điểm. Nhóm Tây kia phân tích, thiết kế ok hết rồi, tiếc là chỉ mới hiện thực được một phần, không đủ thời hoàn thành, lên thuyết trình xong thì lại được...9 điểm!!!

Rõ ràng là bên Canada này đánh giá cao sự sáng tạo dù chưa làm xong hơn là làm ngon lành ra kết quả tốt nhưng làm lại cái người khác đã làm rồi.

Về dự án R&D, tôi đã đọc nhiều tài liệu sư phụ gửi cũng như trao đổi trực tiếp với sư phụ và bắt đầu nghiên cứu bài toán Data Mining trên tập dữ liệu là các cuộc gọi điện thoại trong **Human-Computer Dialog System** (Hệ Thống Người Nói Chuyện Với Máy).

Học kỳ mùa đông 2007

Bước vào học kỳ mùa đông năm 2007, tôi đăng ký các môn sau:

- Planification d'un projet de recherche en ingénierie (**Project Planning** – Lập Kế Hoạch Dự Án) (3 Credit) (ÉTS)
- **Dự án R&D** (3 Credit)
- Français écrit (**French Writing** – Viết Tiếng Pháp) (3 Credit) (ÉTS): môn bắt buộc qua để bổ sung điều kiện French cho cái Study Admission thì mới được ra trường

Môn **Project Planning** hướng dẫn sinh viên cách lập kế hoạch cho các dự án nghiên cứu trong ngành kỹ thuật. Đây là môn bắt buộc phải học trong trường ÉTS mà không được phép đăng ký học ở các trường khác. Môn này thiên về lý thuyết khá nhiều nên sách chữ không là chữ, mà lại bằng French nên tôi khá là ngán 😊

Đến thời điểm này thì tôi có thể giao tiếp bằng French tạm được với các sinh viên khác trong lớp. Vì thế việc thảo luận lập nhóm làm bài tập lớn không còn khó khăn như các học kỳ trước. Phần đuối nhất vẫn là soạn slides bằng French, rồi lên thuyết trình bằng French.

Đúng là có công mài sắt có ngày nên kim, nên lần này tôi làm slides & thuyết trình có vẻ tốt hơn những môn trước.

Vậy là tôi đã chấm dứt phần học 5 môn để lấy kiến thức làm dự án R&D. Sau 5 môn học đã qua, tôi ngồi nghĩ lại thì thấy là nếu vượt qua được rào cản ngôn ngữ và chịu khó cày cuốc thì học ngành kỹ thuật ở Canada có vẻ dễ hơn ở Việt Nam, bởi các yếu tố:

1. Các môn học ở Canada có tính thực tiễn cao hơn và có thực hành nhiều hơn ở Việt Nam hầu như phần lớn lý thuyết.
2. Các môn học ở Canada yêu cầu thuyết trình nhiều cũng làm cho sinh viên học được nhiều lần: học từ nghe giảng, đọc sách, soạn slides, thuyết trình, thầy sửa ngay lúc thuyết trình... trong khi ở Việt Nam việc thuyết trình của môn học hầu như rất rất ít.
3. Các môn học ở Canada được chia ra thành nhiều bài tập nhỏ, lớn nên thi giữa kỳ và cuối kỳ không còn cực kỳ quan trọng để quyết định việc đậu/rớt môn học như ở Việt Nam.
4. Ở Canada, các giảng viên dùng phương pháp “**Mentor**”: giảng viên giúp sinh viên xác định hướng đi, sinh viên chủ động nghiên cứu học & hành theo hướng đó, khi gặp khó khăn thì giảng viên hỗ trợ, sửa sai. Việc sửa sai cực kỳ quan trọng vì nó giúp sinh viên rút ra nhiều kinh nghiệm. Ở Việt Nam, giảng viên nhồi nhét nhiều kiến thức cho sinh viên.

Ngoài việc hoàn tất 45 Credit (gồm 5 môn học & dự án R&D) trong chương trình Master, để tốt nghiệp ra trường tôi cần phải qua được môn “**Français écrit**” do trường ÉTS mở. Môn này dạy thuần về văn phạm French và viết tập làm văn bằng French.

Ui dào, học chuyên sâu ngữ pháp French mới thấy ngữ pháp English đơn giản hơn rất là nhiều. Bài viết nào trong lớp của sinh viên, kể cả đám bản xứ Quebecois nói French là tiếng mẹ đẻ, cũng bị cô giáo sửa nát bét, nhìn đỏ lét tờ giấy. Cái này làm tôi nhớ hồi ở ĐHBK TPHCM có môn “Tiếng Việt thực hành”, anh em cũng từng lên bờ xuống ruộng với nó 😅

Cày cuốc cần cù, chịu khó hỏi cô giáo, chăm chỉ làm lại những sai chỗ cô đã sửa, cuối cùng tôi cũng qua được môn này.

Tóm lại, danh sách các môn và các trường ở Montreal mà tôi đã học qua:

Các môn học ở Master, Canada

Học kỳ 1/2006

- Sécurité de l'internet (ÉTS)
- Automatic Speech Recognition (McGill)
- R&D Project (3 credits)
- Français pratique (UDM)

Học kỳ hè/2006

- Compléments Mathématiques (ÉTS)
- Conception de logiciel (ÉTS)
- R&D Project (3 credits)
- Étude pratique du français oral (ÉTS)
- Communication English (Concordia)

Học kỳ 2/2006

- Pattern Recognition (Concordia)
- R&D Project (6 credits)

Học kỳ 1/2007

- Planification d'un projet de recherche en ingénierie (ÉTS)
- R&D Project (3 credits)
- Français écrit (ÉTS)

Học kỳ hè/2007

- R&D Project (9 credits)

Học kỳ 2/2007

- R&D Project (6 credits)

-
- ÉTS: École de Technologie Supérieure, Montreal, Canada
 - McGill: McGill University, Montreal, Canada
 - UDM: Université de Montréal, Montreal, Canada
 - Concordia: Concordia University, Montreal, Canada

Hình 92: Master Courses

Tổng kết lại các yếu tố đã giúp tôi học ổn được các môn đến thời điểm này:

- Tận dụng môi trường dùng French & English, tự tìm cách xây dựng nền tảng & nâng cao French & English để giảm bớt rào cản ngôn ngữ
- Chịu khó hỏi giảng viên vào cuối mỗi buổi học
- Chịu khó cày cuốc
- Kiên trì theo đuổi mục tiêu
- Lời động viên quý giá & luồng năng lượng tích cực của cậu của Tân
- Một tí may mắn

III. THAM GIA PHÁT TRIỂN CÁC TECH PRODUCT

1. Dự án XiangqiClub (Vietnam)

Hồi năm 2002, tôi bước vào năm cuối và chọn đề tài cho luận văn tốt nghiệp.

Khoa IT đưa ra một danh sách các đề tài tham khảo để sinh viên chọn hoặc sinh viên tự nghĩ ra đề tài rồi tìm giảng viên nào đồng ý hướng dẫn thì làm. Tôi đã xem qua danh sách đó nhưng không thấy đề tài nào mà tôi biết và có cảm hứng để làm.

Thế là tôi đã tự nghĩ ra một đề tài xây dựng 1 Tech Product có thể sử dụng trong thực tế.

Khi ấy tôi đang là Kỳ thủ của đội tuyển Cờ tướng TPHCM. Tôi thấy ở Việt Nam chưa có Câu Lạc Bộ (CLB) Cờ tướng nào hoạt động trực tuyến giúp các Kỳ thủ cả nước tham gia giao lưu học hỏi lẫn nhau cũng như chưa có Software Cờ tướng sử dụng AI nào hay để Kỳ thủ tự luyện tập (lúc đó có chương trình Cờ tướng “cch” của Trung Quốc với các nhân vật vui như Bát Giới, Sa Tăng... chạy trên nền MS-DOS nhưng chương trình đó đánh cũng không quá hay).

Vì vậy tôi đã trình bày ý tưởng làm dự án XiangqiClub (CLB Cờ Tướng Online) với thầy Hồ Văn Quân thì thật là vui khi thầy, một người cũng thích môn Cờ tướng, đã đồng ý cho tôi & bạn Vinh cùng Khoa IT làm luận văn tốt nghiệp. Nếu dự án được đánh giá tốt thì sẽ tiếp tục phát triển tiếp đưa vào sử dụng thực tế tại Việt Nam.

Vào thời điểm đó, số lượng Programming Language trên thị trường IT phục vụ xây dựng Web App không được nhiều cho lắm. Có một số công nghệ phổ biến như **CGI (Common Gateway Interface)**, **ASP (Active Server Pages)**, **JSP (Java Server Pages)**...

Với CGI thì khá cũ và chậm, còn ASP là hàng của Microsoft thì tôi từng có trải nghiệm khá đăm đối với cái thư viện MFC (Microsoft Foundation Class) của nó nên cũng hơi ngán, do đó, tôi quyết định thử món JSP dựa trên nền tảng Java.

Thế là tôi bắt đầu làm quen với Java: học cơ bản với **J2SE 1.4 (Java 2nd Standard Edition)** và phần Web App là **Servlets/JSPs** trên **J2EE 1.3 (Java 2nd Enterprise Edition)**.

Từ ý tưởng ban đầu là một CLB Cờ Tướng Online, tôi đã phải vừa phân tích vừa tham khảo ý kiến của các Kỳ thủ bạn tôi để đưa ra các yêu cầu chi tiết hơn.

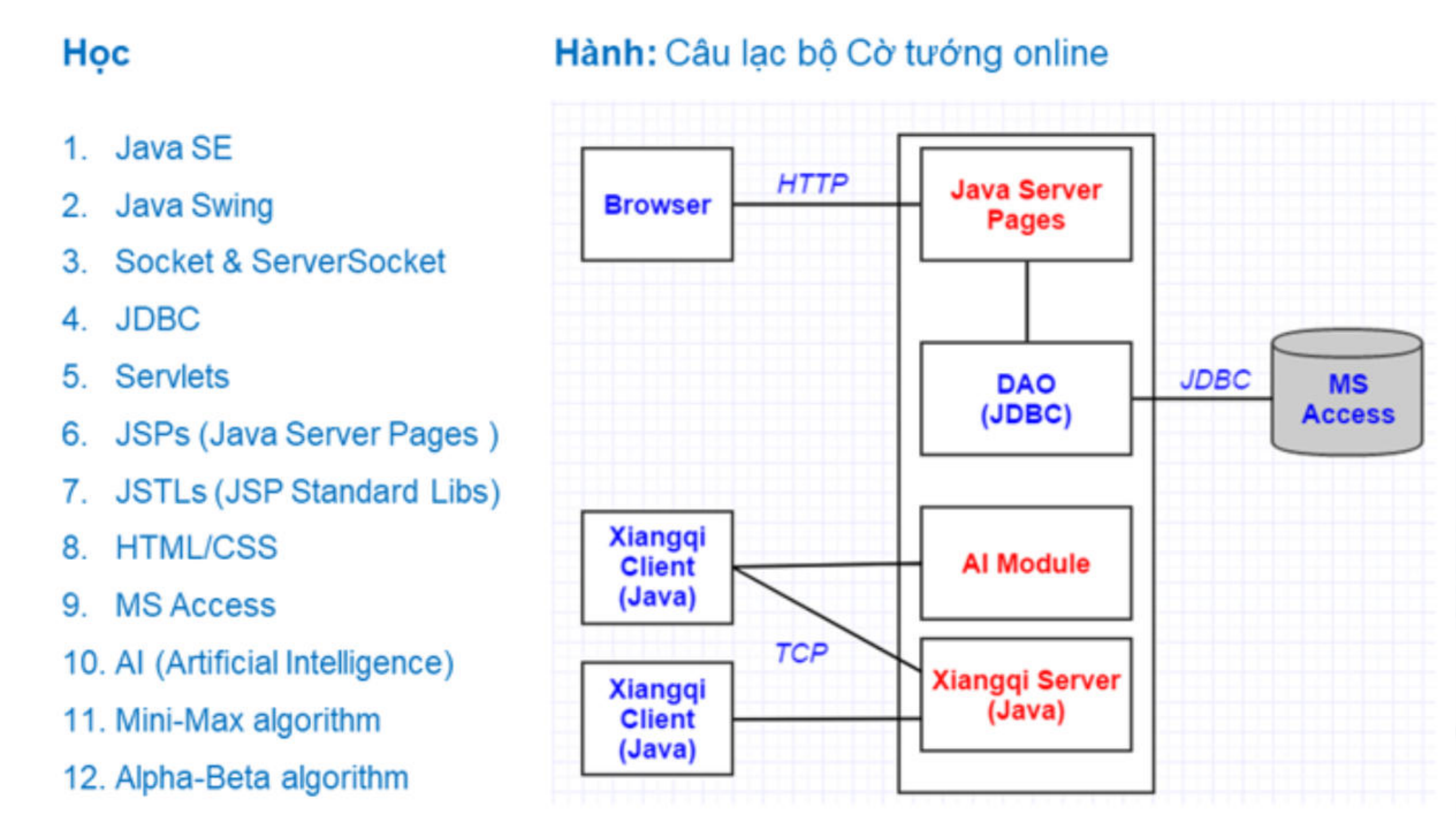
Các tính năng cơ bản của Software XiangqiClub bao gồm:

- 1.Một Website cho phép khách tham quan đăng ký thành viên để trở thành Kỳ thủ của XiangqiClub, tổ chức các giải Cờ tướng Online cho các Kỳ thủ.
- 2.Một chương trình client-server cho phép các Kỳ thủ đấu Cờ tướng với nhau qua mạng, cho phép khách tham quan theo dõi các ván đấu Online.
- 3.Một chương trình Cờ tướng máy đánh với người sử dụng AI để các Kỳ thủ tự luyện cờ Offline, có gợi ý nước đi cho Kỳ thủ.

Các thử thách mà tôi đã gặp phải trong quá trình xây dựng XiangqiClub:

- 1.Lĩnh vực làm Câu Lạc Bộ Cờ tướng trên website khá mới, tôi chưa có kiến thức gì.
- 2.Công nghệ mới do tôi chỉ mới là sinh viên sắp ra trường chưa từng đụng đến.
- 3.Tôi chưa có kỹ năng lập trình nhiều ngoài lập trình cơ bản với Pascal.
- 4.Giải thuật Minimax/Alpha-Beta để tìm nước đi bên AI khá phức tạp.

Mô hình phát triển của dự án XiangqiClub được đặc tả trong hình dưới đây:



Hình 93: Xiangqi Club

Sau khi cùng nhau thảo luận để phân tích, thiết kế xong Software XiangqiClub, tôi và Vinh phân chia công việc để cùng thực thi:

- Tôi và Vinh chia nhau xây dựng các Dynamic Web Page dùng JSPs.
- Tôi đảm nhận viết phần “client” trong chương trình Cờ tướng 2 kỳ thủ đấu qua mạng và phần máy đánh với người (AI Module trong hình). Vinh làm phần “server”.

- Phần web, client-server thì tôi và Vinh tự học tự mò dưới sự hướng dẫn của thầy Quân cuối cùng cũng ok.
- Phần máy đánh với người sử dụng giải thuật Alpha-Beta khá phức tạp, tôi lập trình xong phiên bản đầu tiên thì máy đánh...dốt vãi 😅

Tôi đã dành ra vài tuần tham khảo thêm tài liệu này nọ để cải tiến thì máy đánh lên trình độ tí nhưng vẫn rất dở 😅. May mắn thay tôi gặp được một huynh khóa trên từng làm cái này. Anh đã chia sẻ cho tôi một số kỹ thuật cải thiện khả năng tìm kiếm nước đi của máy cũng như nâng cao hàm lượng giá thế cờ. Kết quả là máy đánh khá lên cũng đáng kể!

Ở thời điểm đó máy của tôi quá ít RAM nên tôi chỉ chỉnh được độ sâu tìm kiếm là 6 tương đương máy suy nghĩ chỉ được 3 nước cờ và phần kiến thức Cờ tướng cho máy đánh giá thế cờ tôi cũng không có thời gian bỏ vô nhiều nên trình độ của máy chỉ ở mức trung bình, không đủ khả năng cho Kỳ thủ tự luyện. Vậy mà máy đánh thắng được thầy Sách, thầy phản biện tôi về giải thuật Alpha-Beta, chắc nhờ vậy mà thầy cho tôi đậu chương trình AI này :D just kidding.

Dự án XiangqiClub bọn tôi làm xong bảo vệ được A+. Tuy nhiên, XiangqiClub chỉ mới gọi là chạy được chứ đem vào sử dụng thực tế thì cần đầu tư thêm rất rất nhiều thời gian và công sức.

Vì vậy tôi đã tạm cất dự án vô kho, đợi khi nào có dịp đem ra làm tiếp để đưa vào sử dụng thực tế trong cộng đồng Cờ tướng ở Việt Nam.

Những kiến thức/kinh nghiệm tôi đã học/hành được trong dự án này:

- 1.Java SE: ngôn ngữ lập trình Java phần nền móng
- 2.Java Swing: thư viện Java giúp xây dựng giao diện ứng dụng Windows
- 3.Socket & ServerSocket: thư viện giúp lập trình kết nối các chương trình máy tính với nhau theo mô hình client-server
- 4.JDBC: thư viện kết nối Database
- 5.Servlets: thư viện làm Dynamic Web Page
- 6.JSPs: mở rộng của Servlets giúp cho việc xây dựng Dynamic Web Page thuận tiện hơn
- 7.JSTLs (JSP Standard Tag Libs): tiện ích bổ sung cho JSPs
- 8.HTML/CSS: ngôn ngữ giúp xây dựng Web Page chạy trên Web Browser
- 9.MS Access: Relational Database của Microsoft
- 10.AI (Artificial Intelligence): Trí tuệ nhân tạo
- 11.Minimax Algorithm: giải thuật tìm nước đi trong trò chơi đối kháng
- 12.Alpha-Beta Algorithm: cải tiến của Minimax Algorithm giúp tìm nước đi nhanh hơn bằng cách cắt bỏ các nhánh đã biết trước sẽ không tìm được nước tốt hơn nước hiện

2. Dự án DMA (Vietnam)

Sau khi bảo vệ luận văn tốt nghiệp với dự án XiangqiClub thì tôi tốt nghiệp đại học và bắt đầu hành trình tìm việc làm. Với kiến thức học & làm dự án XiangqiClub trên nền tảng Java, tôi nhắm tìm việc liên quan đến xây dựng Software sử dụng Java.

Trong giai đoạn năm 2003 thì công cuộc tìm việc làm rất là khó khăn vì các phương tiện để tiếp cận nguồn việc làm rất hạn chế, chủ yếu là qua vài tờ báo hàng tuần có các khung tuyển dụng nho nhỏ.

Tôi là sinh viên mới tốt nghiệp lại không có kinh nghiệm làm việc thực tế (do thời gian học đại học tôi còn đi thi đấu cờ tướng kiếm thêm thu nhập từ lương cờ tướng, còn bạn bè tôi thì đi thực tập/kiếm dự án làm thêm) nên lại càng khó khăn hơn. Tôi đã phỏng vấn vài chỗ thì đều rút do không đưa ra được các ví dụ thực tế sau khi trả lời lý thuyết khá trôi chảy (tôi luôn chuẩn bị lý thuyết kỹ lưỡng trước mỗi cuộc phỏng vấn).

Trong đợt ứng tuyển tiếp theo, tôi tình cờ ứng tuyển chung với 2 đứa bạn cùng Khoa IT vào một công ty gia công phần mềm, tôi đã trả lời phỏng vấn thấy cũng tạm ổn. Kết quả là 2 đứa đó đậu do có kinh nghiệm thực tập thực tế, còn tôi thì lại tiếp tục rút 😞.

Rõ ràng là tụi nó có sự chuẩn bị tốt hơn tôi từ khi còn ngồi trên ghế giảng đường đại học.

Khoảng thời gian đó phải nói là cực kỳ thất vọng đối với tôi. Dù gì tôi cũng đã cày cuốc suốt 4.5 năm đại học, cày luận văn tốt nghiệp được A+, xếp hạng tốt nghiệp đại học cũng thuộc loại khá giỏi (7.7/10). Thế mà giờ đây đi tìm việc làm thật là vất vả quá!

Rồi tôi đã chợt nhận ra lý do khiến tôi học lớp 10 ở trường Lê Hồng Phong khá vất vả và đi tìm việc làm quá khó khăn sau đại học là bởi vì tôi đã bỏ qua một việc cực kỳ quan trọng, đó là HƯỚNG NGHIỆP. Phải có định hướng mới đưa ra được bản kế hoạch chuẩn bị để thực hiện các bước đi theo hướng đã xác định thì kết quả mới tốt được.

Vào một ngày nắng đẹp, tôi bỏ qua vụ án tìm việc làm ra khỏi não mà chạy xe đạp lòng vòng thành phố Sài Gòn dạo chơi thư giãn. Khi đang chạy trên đường Tú Xương, Quận 3, thì xe tôi bị vấn đề gì đó, tôi dừng xe lại xem sao.

Sau khi xử lý xong vấn đề với chiếc xe, tôi leo lên xe chuẩn bị đi tiếp thì tình cờ thấy cái bảng tuyển dụng nhân sự IT ở cửa sổ của một công ty gần đó. Tiện thể tôi ghé vào hỏi thăm thử xem sao luôn. Bác bảo vệ nói tôi muốn ứng tuyển thì về đem hồ sơ đến nộp. Bác đưa tôi danh sách các giấy tờ cần thiết. Haiz, hồi đó mỗi lần chuẩn bị hồ sơ nộp tìm việc làm thì thật là oải 😓

Từ nhỏ đến lúc ấy thì số tôi hơi có duyên với những cái gì TÌNH CỜ nên tôi đã chịu khó chạy về nhà chuẩn bị hồ sơ rồi chạy lên nộp vô ứng tuyển ngay và luôn. Chà hên quá, vừa nộp xong thì ngay ngày hôm sau tôi được gọi phỏng vấn.

Bữa đi phỏng vấn buổi sáng tôi gặp anh Khải hỏi về kỹ thuật & chiều gặp anh Việt Kiều Mỹ tên Chris, Phó giám đốc kiêm Trưởng phòng. Hai anh này chỉ hỏi tôi kiến thức nền tảng IT và một số câu hỏi về Softskill.

Kết quả là tôi đã được nhận vô Software Department làm Software Engineer của công ty SaigonTel ở TPHCM.

Dù tôi tìm việc liên quan đến nền tảng Java nhưng lại được nhận ở một công ty đang sử dụng công nghệ của Microsoft .Net. Thế là tôi đã chuyển từ lập trình Java sang lập trình C# luôn.

Do tôi đã từng học môn Programming Language trong ĐHBK (từng lên bờ xuống ruộng với thầy Phùng 😓), có kiến thức nền tảng về lập trình tương đối ổn và nhờ anh em trong công ty hỗ trợ rất nhiệt tình nên tôi đã nắm bắt cũng khá tốt nền tảng công nghệ .Net mà công ty đã tự xây dựng hỗ trợ phát triển Software trên đó.

Sản phẩm phần mềm tôi tham gia phát triển là xây dựng **Document Management Application – DMA** (Ứng Dụng Quản Lý Công Văn) được sử dụng bởi hàng ngàn đơn vị hành chính trên lãnh thổ Việt Nam. Đây là một Module nhỏ trong System khá lớn có thể xem như ERP cho các doanh nghiệp nhà nước/tư nhân.

Tôi đã đảm nhận phát triển DMA một mình, các anh chị khác làm các Module khác lớn hơn. Tất cả Module đều dựa trên một kiến trúc phần mềm tổng thể do nhóm kiến trúc sư phần mềm của công ty xây dựng ra.

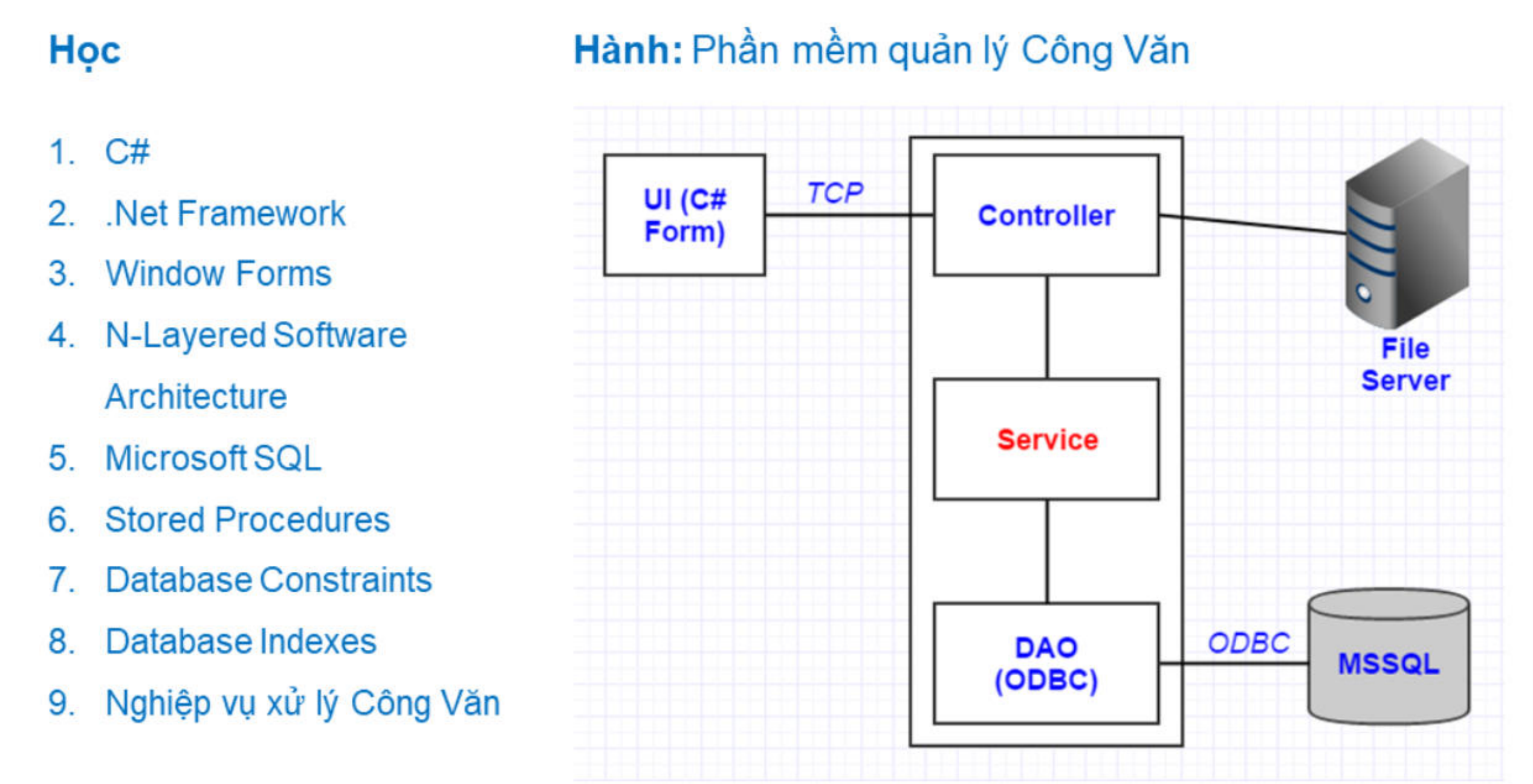
Những tính năng cơ bản của DMA là giúp cho các đơn vị hành chính, doanh nghiệp xử lý công văn đến/đi, duyệt công văn theo dây chuyền một cách hiệu quả.

Các thử thách mà tôi đã gặp phải khi xây dựng DMA:

- 1.Lĩnh vực công văn khá mới với tôi nên tôi chưa có Domain Knowledge, vì vậy gặp nhiều trở ngại khi phân tích nghiệp vụ xử lý công văn.
- 2.Ngôn ngữ lập trình C# và .Net là công nghệ mới với tôi khi tôi chưa có kiến thức/ kinh nghiệm trên đó.
- 3.Tôi làm một mình mà lại chưa có kinh nghiệm nhiều về phát triển phần mềm dù được anh trưởng nhóm hỗ trợ khi cần.

Mô hình phát triển của dự án DMA được đặc tả trong hình dưới đây:

DMA



Hình 94: DMA

Tôi xây dựng UI bằng C# Forms, dựa trên các thành phần thông dụng (table, create form, update form, upload form...) mà công ty đã xây dựng sẵn.

Bên server-side thì công ty đã xây dựng **N-Layered Software Architecture** (Kiến Trúc Phần Mềm Đa Lớp), tôi lập trình C# để xử lý nghiệp vụ là chủ yếu. Khi có vấn đề gì về kỹ thuật thì có các anh chị hỗ trợ, vấn đề về nghiệp vụ thì anh trưởng nhóm hỗ trợ.

Nhờ dự án này mà tôi biết được N-layered Software Architecture thực tế được xây dựng ra sao chứ đọc lý thuyết thời sinh viên khá là mơ hồ. Phần Database công ty dùng Microsoft SQL. Đây cũng là công nghệ mới giúp tôi học thêm được một số tính năng thực tế khi thiết kế và làm việc trên Database như Stored Procedure, Constraint, Index...

Tôi làm dự án DMA cỡ 1 năm thì đã hiểu rõ hơn nhiều về việc phát triển phần mềm trong thực tế như thế nào.

Trong thời gian làm việc ở công ty SaigonTel, tôi đã có dịp tiếp xúc với các đồng nghiệp mới.

- Anh Lý, Graphic Designer, là người lớn tuổi nhất trong phòng phần mềm, vui tính, hát hay.
- Vinh, Graphic Designer, bằng tuổi tôi, là người hòa đồng, chủ xị cáp kèo đá banh cho anh em với các công ty khác mỗi cuối tuần.
- Anh Thuận, Software Architect, cũng là dân IT ĐHBK TPHCM hơn tôi 3 khóa, ít nói, đọc sách IT rất nhiều. Anh tải được rất nhiều sách IT miễn phí dạng ebook và có chia sẻ cho tôi vài quyển khá hay. Tôi đã học được khá nhiều kiến thức thực tế về kiến trúc phần mềm từ anh Mentor này.
- Anh Thảo, Database Designer, là người vui tính.
- Anh Phước, Software Engineer, cũng là dân IT ĐHBK TPHCM hơn tôi 1 khóa, hài hước, đá banh rất hay.
- Anh Đức, Software Engineer, là dân IT Đại học KHTN TPHCM hơn tôi 1 khóa, ít nói, không biết phải code nhiều quá hay không mà ồm tong ồm teo.
- Trúc, Software Engineer, dân IT Đại học Cần Thơ bằng tuổi tôi, khá vui tính và có tinh thần hỗ trợ mọi người tốt.
- Dũng, Quality Control – QC, vui tính.
- Bé Phương, Secretary của trưởng phòng, là Hoa khôi của phòng có duy nhất 1 nữ 😊, thân thiện, hòa đồng.

3. Dự án SER5500 (Canada)

Rời công ty SaigonTel vào năm 2004, tôi đã vào làm cho công ty TMA Solutions ở TPHCM, một công ty gia công phần mềm cho nước ngoài với thị trường chính là Canada.

Sản phẩm phần mềm tôi tham gia phát triển là **SER5500 (Service Edge Router)**, tên dự án lúc ấy là **Shasta**, một sản phẩm khá đồ sộ của công ty Nortel Networks, Canada.

Thời đó công ty Nortel với sản phẩm này là đối thủ cạnh tranh sừng sỏ của Cisco Systems, một công ty cực to của Mỹ từ xưa đến giờ, làm trùm bên mảng viễn thông & mạng máy tính. Sau đó thì Nortel đã tụt lại phía sau và đã phá sản vào năm 2009.

SER5500 là một thiết bị phần cứng to cỡ cái bàn vi tính nhỏ. Trên đó có nhiều khe để cắm rất nhiều bản mạch cũng to như cái mặt bàn, nặng cỡ 5-10kg. Khá nhiều các em gái QC nhỏ nhỏ xinh xinh vác không nổi mấy bản mạch này bằng để nhét vào thiết bị SER5500, nên thường nhờ các anh DEV “cứu nét” khi thay bản mạch.

Mỗi bản mạch hỗ trợ một loại mạng, ví dụ như Ethernet, Frame Relay, ATM...

Mỗi con SER5500 giúp quản lý hàng triệu ISP trên toàn thế giới. Mỗi ISP lại có hàng triệu **Subscriber** (Thuê Bao). Khi triển khai vật lý thì có hàng trăm thiết bị SER5500 rải khắp châu Âu, châu Mỹ, và châu Á.

Phần cứng của SER5500 được điều khiển bởi OS mà Nortel phát triển riêng gọi là iSOS (iService Operating System).

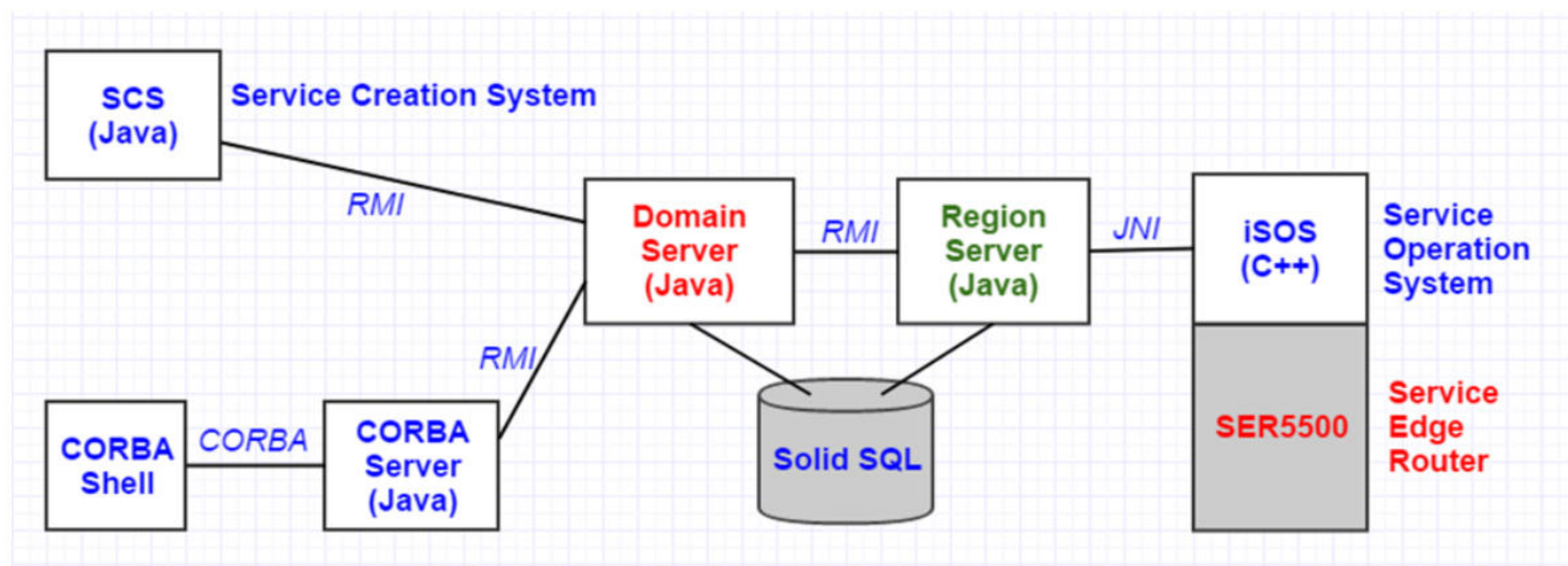
Tầng ứng dụng thì Nortel phát triển hệ thống các phần mềm khá phức tạp có tên SCS (Service Creation System) dựa trên công nghệ **Java RMI** (Remote Method Invocation) và **CORBA** (Common Object Request Broker Architecture).

Logical Diagram (Sơ Đồ Luận Lý) của SER5500 được hiển thị qua sơ đồ sau:

SER5500 – Canada

Học: Java RMI, CORBA, Multiple Client-Server Architecture

Hành: Hệ thống định tuyến và dịch vụ mạng máy tính



Hình 95: SER5500

Network Admin (Nhà Quản Trị Mạng) có thể điều khiển SER5500 bằng Command Line với CORBA Shell hoặc bằng UI với phần mềm SCS được hiện thực bằng Windows Forms sử dụng Java Swing. Lệnh đi qua Domain Server, rồi đi đến Region Server tương ứng với thiết bị SER5500 mà Network Admin đang tương tác. Mỗi thiết bị có một Region Server trên đó.

Dự án này ban đầu được xây dựng ở Nortel, sau khi sản phẩm chạy ổn định các Version đầu thì được đưa qua các Outsourcing Company ở Trung Quốc, Ấn Độ, Việt Nam để bảo trì và phát triển tiếp. Các công ty gia công phần mềm tham gia dự án này phải đầu tư khá nhiều về phòng ốc, thiết bị, đội ngũ nhân lực...

Mỗi công ty tham gia gia công cho dự án Shasta sẽ thành lập 3 nhóm: SCS, iSOS & QC. Tôi tham gia vào nhóm SCS chịu trách nhiệm bảo trì và phát triển các tính năng trên 3 Module là UI, Domain Server & Region Server, tất cả được hiện thực bằng công nghệ Java. Tôi không đụng vô phần CORBA dù có đọc về nó.

Các thử thách mà tôi đã gặp phải trong dự án SER5500:

1. Kiến thức lĩnh vực mới toanh
2. Công nghệ Java RMI cũng mới nhưng do tôi đã làm việc với Java trong dự án XiangqiClub rồi nên cũng tìm hiểu nhanh được
3. Mô hình multiple client-server cực kỳ khó hiểu
4. Dự án quá khổng nên việc cài đặt môi trường phát triển mất khá nhiều thời gian, và cũng rất khó khăn để tìm lỗi ở 3 Module trên
5. Do đưa đi gia công qua nhiều nước với hàng ngàn người làm việc trên đó nhưng Coding Convention không rõ ràng nên Code rất rối rắm. Chưa kể phần đặt tên biến trên Module SCS rất mơ hồ, ví dụ: jLabel1, jPanel2, jButton3... Nghe đồn là hình như SCS được 1 công ty nhỏ phát triển, sau đó Nortel mua lại nên đám Developer trước khi đi đã thực hiện **Code Refactoring** (Tái Cấu Trúc Mã Lệnh) làm cho các **Variable Name** (Tên Biến) có nghĩa thành tên mặc định để người đọc chả hiểu gì.

Vào dự án tháng đầu tiên tôi cài đặt **Local Development Environment** (Môi Trường Phát Triển Cục Bộ) trên máy tính của tôi khá là cực vì quá nhiều thứ linh tinh và tìm hiểu dự án cũng đăm đầu: kiến thức lĩnh vực Computer Networking và Source Code.

Bắt đầu tháng thứ 2 tôi mới có đóng góp cho dự án bằng việc sửa Bug nhỏ để làm quen System, Team và Software Development Process trong dự án.

Tất cả các phần Code của Developer trên toàn thế giới đưa vào dự án phải được xem xét bởi Engineering Manager phía Nortel qua Web Application do thời đó chưa có các Version Control System mạnh như SVN, Git...

Sau khi sửa được một số lỗi trên một số tính năng cơ bản của sản phẩm, tôi đã quen được Software Development Process thì tôi muốn tìm hiểu sâu hơn về các tính năng sản phẩm & kiến trúc các phần mềm trong sản phẩm.

Tuy nhiên do còn non và xanh, tôi mất 2,3 tháng tìm hiểu mà tôi vẫn như đang bơi trong đại dương 😊. Các anh/chị lớn trong nhóm cũng có kinh nghiệm nhiều nhưng chưa ai đủ khả năng nắm hết từng ngóc ngách của sản phẩm khổng lồ này.

Tiếp đó, công ty tuyển được chị Loan vào nhóm SCS, Tiến sĩ về Computer Networking đang làm cho công ty France Telecom của Pháp về nước. Chị này cũng dân IT ĐHBK TPHCM, hơn tôi 5 khóa, tham gia dự án với vai trò Software Architect & Mentor.

Ồ là là, sau khi tìm hiểu dự án 2-3 tháng thì chị ấy nắm khá nhiều về sản phẩm, làm Mentor cho cả nhóm 20+ người, tổ chức lớp Mentor 3 tháng với chuỗi chia sẻ về kiến thức lĩnh vực Computer Networking, mô hình OSI tầng Data Link, IPSec, L2TP Tunnel... rồi ánh xạ vô cách sản phẩm đang hiện thực mấy cái này cũng như Software Architecture đang sử dụng ra sao... Nhờ vậy mà kiến thức lĩnh vực Computer Networking & Software Architecture của tôi tăng lên đáng kể. Đến lúc này tôi mới hiểu Multiple Client-Server Architecture được sử dụng trong SCS.

Nói đến kiến trúc này làm tôi nhớ đến 1 kỉ niệm bi hài 😅.

Tôi phải công nhận ai thiết kế ra kiến trúc này khá hay: phần client-server được phát triển trong cùng một Source Code tương đối gọn, triển khai lên 3 nơi (UI, Domain Server & Region Server) giống nhau. Khi chạy thì:

- UI luôn đóng vai trò là Client
- Domain Server lúc có vai trò là Client (khi kết nối đến Region Server) lúc là Server (khi UI kết nối đến)
- Region Server cũng có lúc là Client (khi kết nối ngược lại Domain Server), lúc là Server (khi Domain Server kết nối đến).

Lúc mới vào dự án thì tôi chưa hiểu được chuyện này. Trong quá trình Debug tôi chạy 3 Process cho UI, Domain Server, và Region Server trong cùng **Integrated Development Environment – IDE** là JBuilder.

Tôi mở 2 files client.java & server.java lên rồi đặt các **Debug Breakpoint** (Điểm Dừng Để Tìm Lỗi) trong 2 files này. Khi tôi thực hiện thao tác trên UI, luồng chạy của chương trình sẽ là: UI -> Domain Server -> Region Server -> SER5500 -> Region Server -> Domain Server -> UI.

Mỗi lần luồng chạy đi qua UI, Domain Server, Region Server thì đều dừng lại ở các Breakpoint trong file client.java, phải nhớ lúc này đang ở UI hay Domain Server hay Region Server để mà xem các thông số trong Activity Table và Call Stack cho đúng. Nguyên tắc là vậy nhưng chỉ cần chạy Debug vài lần là tôi quáng luôn 😊

Sau khi nắm được kiến trúc này thì tôi đã tìm được cách để Debug đỡ quáng hơn:

- Nếu chỉ cần Debug 1 Module (UI or Domain Server or Region Server) thì chỉ chạy Module đó ở chế độ Debug trong IDE, 2 Module còn lại chạy ngoài Command Line.
- Nếu cần Debug 2 Module thì chạy 2 IDE & 1 Command Line.
- Nếu cần Debug 3 Module thì... không thể chạy cùng lúc 3 IDE vì JBuilder ngốn RAM dữ dội lắm, mà RAM cho mỗi máy tính cho Developer lúc đó không đủ làm việc này, phải tìm cách phân tích để chỉ cần Debug 2 Module.

Với việc hiểu rõ hơn System, tôi đã viết vài tài liệu tổng hợp về System giúp cho các bạn mới vào dự án thiết lập Local Development Environment nhanh hơn, Debug dễ hơn, và nắm sản phẩm nhanh hơn so với tôi lúc mới vào dự án.

Đây là việc tôi làm tự nguyện chứ công ty không yêu cầu bởi tôi nghĩ đơn giản là những gì mình đã gặp khó khăn và có người giúp mình thì mình có trách nhiệm giúp lại những người vào sau. Cho dù những việc mình gặp khó khăn đã vượt qua được mà không có ai giúp thì mình cũng nên để lại gì đó giúp những người cần, sau này mình gặp khó khăn khác sẽ có những người khác giúp lại mình. Tôi hoàn toàn tin vào Luật NHÂN QUẢ!

Với việc làm nho nhỏ ở trên của tôi (viết tài liệu tổng hợp về hệ thống & Mentor các bạn mới) mà công ty đã cho tôi vào **Key Resource Group** (Nhóm Nhân Lực Chính) để mỗi tháng ngoài tiền lương tôi được nhận thêm tí **Bonus** (Tiền Thưởng). Phần Bonus này tuy có giá trị vật chất không nhiều nhưng nó cũng đã làm tôi vui lắm vì việc mình làm đã đem lại giá trị cho nhóm cũng như cho công ty.

Điều này cũng đã làm tôi nghiệm ra rằng: khi đi làm mình cứ tập trung làm tốt việc của mình và giúp đỡ đồng nghiệp bất cứ khi nào có thể thì bất cứ công ty nào cũng sẽ nhìn thấy để ghi nhận và có phần thưởng xứng đáng sau đó.

Trong gần 2 năm làm việc ở SCS Team, tôi đã tiếp xúc với một số anh chị em đồng nghiệp:

- Anh Hùng, Engineering Manager của dự án SER5500, quản lý cả 3 Team là SCS, iSOS & QC, là người nói chuyện rất nhẹ nhàng, vui vẻ. Tôi rất có ấn tượng với anh lúc phỏng vấn.
- Chị Sa, Project Manager của SCS, là người bản lĩnh, nói English như gió làm tôi khá ngợp lúc chị ấy phỏng vấn kỹ năng nói English của tôi 😊
- Anh Long Trần, Software Architect, là người trầm tính, có thâm niên nhất trong dự án.
- Anh Dũng, Software Architect, là người lớn tuổi nhất Team, vui vẻ, thích chơi cờ tướng.
- Anh Quyền, Software Architect, là dân IT ĐHKHTN TPHCM hơn tôi 6 khóa, rất vui vẻ, trẻ trung, cùng với anh Long & anh Dũng chuyên trị các Bug khó nhất của hệ thống.
- Chị Loan, Software Architect, Tiến sĩ từ Pháp về, là người vui vẻ, Mentor rất có tâm cho các thành viên mới và cũng hỗ trợ Team rất nhiều về Computer Networking & kỹ thuật.
- Anh Tony, Software Engineer, Việt Kiều Úc, là người khá trầm ngâm, ít khi giỡn.

- Chị Julie, Software Engineer, người Pháp có Ba là người Việt, về VN làm để trải nghiệm nơi Ba chị ấy đã sinh ra và lớn lên, là người rất vui tính, kỷ lưỡng trong công việc. Năm 2017, vợ chồng tôi lúc đi phượt 5 nước châu Âu trong 21 ngày có ghé nhà chị Julie ở Pháp chơi. Chị lấy chồng là anh Thạch, người đã phỏng vấn chị ấy vô công ty TMA Solutions.
- Anh Quân, Software Engineer, Việt kiều Mỹ, dân IT ĐHBK TPHCM hơn tôi 4 khóa, là người trầm tính nhưng cũng hay giỡn.
- Anh Hậu, Software Engineer, là dân IT ĐHBK TPHCM hơn tôi 2 khóa, là người hoạt bát, vui tính.
- Thuận Trương, Software Engineer, cùng khóa IT với tôi, là người ít nói, chăm chỉ. Cứ vào trưa thứ 6 hàng tuần, tôi, Thuận Trương, anh Quân & anh Hậu đi quánh bida độ, đội nào thua mua trái cây bao cả Team khá là vui.
- Chị Linh & anh Long Bùi, Software Engineer, là 2 người tính tình dễ thương, vui vẻ, đã phỏng vấn tôi phần kỹ thuật.
- Nam, Software Engineer, là dân IT ĐHBK TPHCM nhỏ hơn tôi 1 khóa, là người ít nói, rất giỏi English, đã đạt được TOEFL 580 điểm thời đó.
- Chị Mai, Project Manager thay cho chị Sa, du học ở Úc về, là người vui tính, hoạt bát.
- Anh Vinh Võ, Software Engineer, học từ Pháp về, là người ít nói.
- Ngoài ra còn có chị Yến (Project Manager), chị Thủy (Software Developer), anh Vinh Trần (Software Developer), anh Tín (Software Developer), và anh Hưng (Software Developer) ngồi ở phòng bên kia tôi ít dịp tiếp xúc.

4. Dự án USPS (Canada)

Dự án “Managing emotions in **Human-Computer Dialog Systems – HCDS**” là một ứng dụng thực tiễn về Data Mining của đại học ÉTS & viện CRIM cộng tác với công ty Bell Canada.

Trong ngữ cảnh của một **Call Center** (Trung Tâm Điện Thoại), HCDS giúp giảm chi phí vận hành các dịch vụ. Một số ứng dụng phổ biến của hệ thống này là **Air Ticket Booking** (Đặt Vé Máy Bay), **Travel Booking** (Đặt Du Lịch), **Customer Care** (Chăm Sóc Khách Hàng).

Tuy nhiên, thỉnh thoảng các **Dialog** (Cuộc Hội Thoại) cũng có thể bị vấn đề làm cho người gọi đến không hài lòng, ví dụ như họ phải lặp lại nhiều lần các từ mà máy không hiểu, hoặc máy hiểu nhầm các từ khi chúng được phát âm gần giống nhau. Điều này xuất phát từ việc **Speech Recognition System** (Hệ Thống Nhận Dạng Giọng Nói) không cho kết quả hoàn hảo.

Do đó, việc xác định các **Problematic Dialog** (Cuộc Gọi Có Vấn Đề) là thực sự rất cần thiết. Tuy vậy, số lượng Dialog mỗi ngày ở công ty Bell Canada là cực kỳ lớn mà sức người không thể xử lý nổi. Cho nên, cần có một hệ thống xác định các Problematic Dialog một cách tự động. Các Problematic Dialog được xác định sẽ được đưa qua bộ **Emotion Detection** (Phát Hiện Cảm Xúc) để phân tích thêm về cảm xúc của người gọi. Kết quả của 2 phần sẽ được công ty Bell Canada sử dụng để đưa ra các chiến lược cấu hình mới cho hệ thống hoạt động tốt hơn.

Dự án này dự kiến kéo dài trong 15 năm, cần 6 sinh viên Master và 10 nghiên cứu sinh vào tham gia. Tôi là sinh viên Master thứ nhất tham gia dự án cùng với 1 nghiên cứu sinh đến từ Pháp & 1 nghiên cứu sinh đến từ Iran. Hai bạn này nghiên cứu về Emotion Detection.

Tôi đảm nhận nghiên cứu và phát triển 1 phần mềm dự đoán mức độ hài lòng của người dùng (**User Satisfaction Prediction Application – USPA**), trong đây có phần xác định các Problematic Dialog. USPA khi hoàn thành sẽ được đưa vào sử dụng thực tế như sau:

- Khi cuộc gọi đang diễn ra, USPA sẽ dự đoán coi User có đang hài lòng hay không dựa trên những gì họ nói (**User Turn**) & máy nói (**System Turn**).
- Nếu họ hài lòng, cuộc gọi cứ tiếp diễn, ngược lại, cuộc gọi sẽ được chuyển tiếp qua 1 em có giọng ngọt ngào, nhẹ nhàng tiếp chuyện với họ.

USPA thuộc mảng AI, cụ thể hơn là Data Mining/ Machine Learning dùng phương pháp Pattern Classification. Các tính năng chính của USPA gồm:

- 1.Cung cấp một bộ **Parser** xử lý **Raw Data** (Dữ Liệu Thô) về các cuộc gọi, thành **Structured Data** (Dữ Liệu Cấu Trúc) cho phần Data Mining.
- 2.Cung cấp một bộ **Extractor** để xuất ra các **Feature** (Thuộc Tính) ảnh hưởng đến mức độ hài lòng của User từ Structured Data.
- 3.Cung cấp một bộ **Trainer** giúp xây dựng một bộ phận giúp phân loại các cuộc gọi có vấn đề (**Problematic Dialog Identifier – PDI**) dựa trên các **Dataset** (Tập Dữ Liệu) chứa các cuộc gọi trong quá khứ đã được đánh dấu trước là có vấn đề hay không.

Những thử thách mà tôi đã gặp phải:

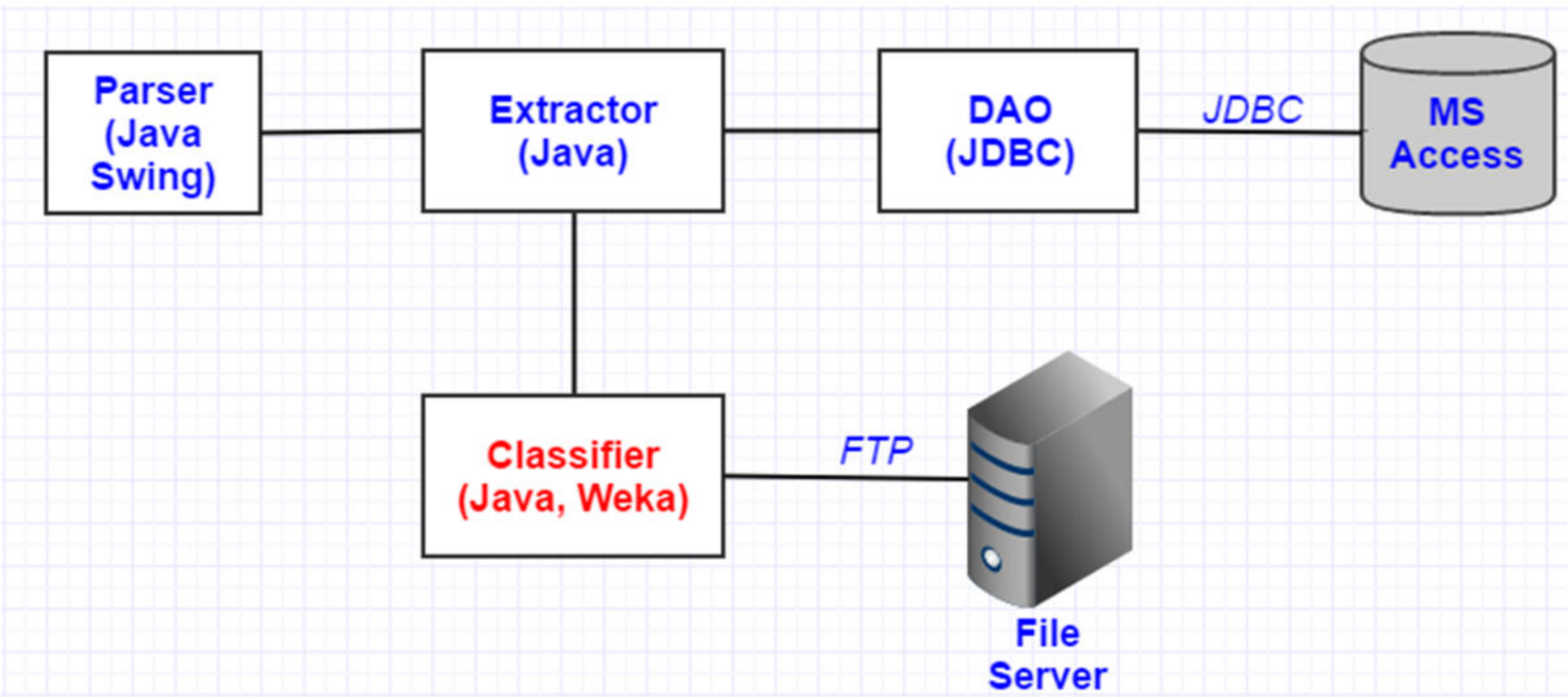
- 1.Kiến thức lĩnh vực bên mảng Call Center.
- 2.Thách thức về việc nghiên cứu khoa học mà tôi chưa có kinh nghiệm gì cả.
- 3.Kiến thức về Data Mining tôi chưa biết.
- 4.Các **Machine Learning Algorithm** (Giải Thuật Máy Học) rất ư là phức tạp.

Mô hình USPA được minh họa bởi hình sau:

USPA – Canada

Học: Domain Knowledge, R&D Skill, Data Mining/Pattern Classification, Weka tool

Hành: Hệ thống dự đoán mức độ hài lòng của người dùng



Hình 96: USPA

Một chị **PostDoc** (Sau Tiến Sĩ), trợ lý của sư phụ, đã Mentor tôi về kiến thức lĩnh vực Call Center cũng như cái mô hình **PARADISE – PARAdigm for Dialog System Evaluation** giúp tôi hiểu hơn bài toán mình cần giải.

Nói thêm ngoài lề một tí là trước khi qua Canada thì tôi có nghe đến chữ “Sau Tiến Sĩ”.

Khi ấy chữ “Tiến Sĩ” đối với tôi đã là người học cao siêu lắm rồi, cho nên “Sau Tiến Sĩ” chắc còn học cao siêu hơn nữa. Thực tế thì không phải vậy. “Tiến Sĩ” là học vị cao nhất rồi, “Sau Tiến Sĩ” dùng để chỉ người đã hoàn thành học vị Tiến Sĩ, vô trường đại học phụ cho các giáo sư trong việc nghiên cứu khoa học, **Teaching Assistant** (Trợ Giảng), làm **Mentor** cho các sinh viên Master/nghiên cứu sinh của các giáo sư.

Mức cao nhất mà một người “Sau Tiến Sĩ” phấn đấu lên chính là học hàm giáo sư của một trường đại học nào đó. Quá trình này nói chung là khá dài, có thể lên đến 10+ năm.

Bài toán tôi cần nghiên cứu là làm sao phân loại/dự đoán các cuộc gọi mà User không hài lòng. Theo mô hình PARADISE, sự hài lòng của User chịu ảnh hưởng bởi 2 yếu tố:

1. Task Success Measure (Đo Lường Thành Công Tác Vụ): thông số này có giá trị luận lý là Successful (Thành Công)/ Failed (Thất Bại), ví dụ: đặt được 1 vé máy bay hay không, đặt được 1 chuyến du lịch hay không. Thông thường User có tác vụ thành công thì phần lớn sẽ hài lòng. Tuy nhiên, vẫn tồn tại 2 trường hợp sau:

- User có tác vụ thành công nhưng không hài lòng vì phải lập lại nhiều lần các từ máy chưa hiểu.
- User không có tác vụ thành công nhưng vẫn hài lòng vì máy giao tiếp tốt, chỉ trục trặc phía hãng máy bay mà không đặt được vé mà thôi.

2. Dialog Cost (Chi Phí Cuộc Gọi) – được xác định bởi 2 nhóm thông số **Efficiency Measures** (Đo Lường Về Hiệu Quả) & **Qualitative Measures** (Đo Lường Về Chất Lượng). Về nhóm Qualitative Measures, trong bài toán của tôi không có Data để nghiên cứu. Về nhóm Efficiency Measures, các thông số được nghiên cứu gồm:

- a. **TimeOnTask**: Tổng thời gian của cuộc gọi
- b. **TurnsOnTask**: Tổng số lần nói của người gọi & máy
- c. **NumOverlaps**: Số lần người và máy cùng nói 1 lúc
- d. **MeanUserTurnDuration**: Trung bình thời gian người gọi nói
- e. **MeanSystemTurnDuration**: Trung bình thời gian máy nói
- f. **MeanWordsPerUserTurn**: Trung bình số từ người gọi nói
- g. **MeanWordsPerSystemTurn**: Trung bình số từ máy nói
- h. **Phonetype**: Loại điện thoại người gọi dùng

Như vậy, 1 Problematic Dialog chịu ảnh hưởng bởi 9 thuộc tính (**TaskSuccess**, **TimeOnTask**, **TurnsOnTask**, **NumOverlaps**, **MeanUserTurnDuration**, **MeanSystemTurnDuration**, **MeanWordsPerUserTurn**, **MeanWordsPerSystemTurn**, và **Phonetype**)

Thế thì với 1 cuộc gọi bất kỳ, làm thế nào để dự đoán nó có vấn đề hay không?

Tôi biết là mảng Data Mining/Machine Learning sẽ giúp trả lời câu hỏi này, vì vậy tiếp đến tôi tìm hiểu về chúng. Theo định nghĩa lý thuyết thì:

- **Data Mining** là việc trích xuất ra thông tin hữu ích từ dữ liệu trong quá khứ dựa trên các Pattern tiềm ẩn nếu có. Tiến trình này có thể hoàn toàn tự động (fully automatic) hoặc bán tự động (semi-automatic).
- **Machine Learning** cung cấp nền tảng kỹ thuật cho Data Mining. Machine Learning liên quan đến việc thiết kế và phát triển các giải thuật, kỹ thuật cho phép máy tính “học”, từ đó giúp đưa ra các dự đoán cho con người. Machine Learning có rất nhiều ứng dụng như **Natural Language Processing** (Xử Lý Ngôn Ngữ Tự Nhiên), **Speech Recognition** (Nhận Dạng Giọng Nói)...

Để dễ hình dung hơn về Data Mining, tôi đã xem qua 1 ví dụ thực tế mang tính kinh điển cho mảng Data Mining:

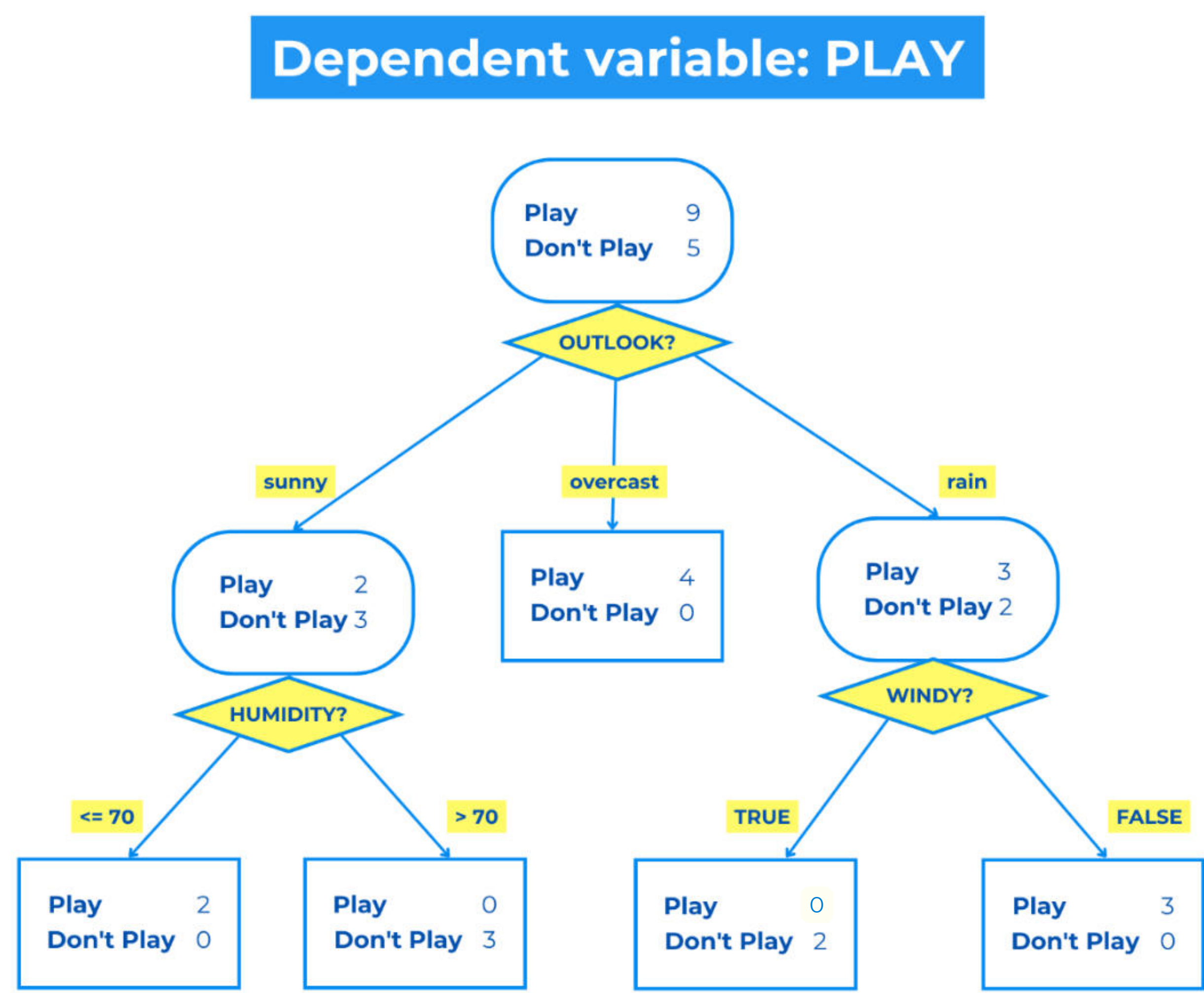
“Một ông chủ của 1 khu du lịch muốn dự đoán xem mỗi ngày khách có đến chơi golf (Golf) hay không, phụ thuộc vào thời tiết, để mà thuê thêm/giảm bớt nhân viên phục vụ.”

Ông quan sát và đo lường 4 thông số sau trong 2 tuần:

1. **Outlook** (Trạng Thái Thời Tiết): nắng (sunny), quang đãng (forecast), hay mưa (rain)
2. **Temperature** (Nhiệt Độ): bao nhiêu độ oC
3. **Humidity** (Độ Ẩm): bao nhiêu %
4. **Windy** (Có Gió): có (TRUE) hay không (FALSE)?

Tương ứng với giá trị của các thông số mỗi ngày, ông lưu lại kết quả là có khách đến chơi hay không, giá trị của biến số PLAY: “Play” / “Don’t Play”.

Tiếp theo, ông xây dựng 1 **Decision Tree** (Cây Quyết Định) như sau:



Hình 97: Decision Tree

Dựa vào Decision Tree trên, nhìn vào 5 **Leaf Node** (Nút Lá), ông ấy có thể kết luận:

- Khách hàng đã chơi ở ngày nắng và độ ẩm ≤ 70 (nút lá số 1: 2 ngày) hoặc ngày quang đãng (nút lá số 3: 4 ngày) hoặc ngày mưa và không gió (nút lá số 5: 3 ngày).
- Khách hàng đã không chơi ở ngày nắng và độ ẩm > 70 (nút lá số 2: 3 ngày) hoặc ngày mưa và có gió (nút lá số 4: 2 ngày).

Với Dataset này, thuộc tính “nhiệt độ” không ảnh hưởng đến việc khách hàng chơi hay không.

Như vậy, vào 1 ngày bất kỳ trong tương lai, vào buổi sáng sớm, ông chỉ cần thu thập Outlook, Humidity & Windy của ngày đó, đem bỏ vào cây quyết định này duyệt thì sẽ dự đoán được là khách hàng có chơi hay không để sắp xếp nhân viên phù hợp.

Decision Tree, là 1 trong những **Machine Learning Model** (Mô Hình Học Máy), khá phổ biến bởi tính trực quan dễ hiểu:

- **Root Node** (Nút Gốc): chứa Dataset đầy đủ ban đầu.
- **Interior Node** (Nút Trung Gian): chứa Dataset là kết quả của việc xét các giá trị của 1 thông số nào đó.
- **Leaf Node** (Nút Lá): chứa Dataset còn lại khi đã xét hết các thông số hoặc tất cả Data đều cùng 1 giá trị của biến đích (ví dụ trên là “Play” / “Don’t Play”).

Khi đã hiểu được Decision Tree, tôi tự lập trình hiện thực thử 1 giải thuật về Decision Tree tên CART (Classification And Regression Tree) dùng ngôn ngữ lập trình Java trong 2 tháng.

Xong rồi tôi áp dụng vô ví dụ kinh điển trên thì giải thuật sinh ra đúng cái Decision Tree ở trên.

Đến đây thì xem như tôi đã học và hành xong phần cơ bản của Data Mining, tiếp theo là nghiên cứu áp dụng giải thuật này vào bài toán PDI.

Trước tiên là phần dữ liệu quá khứ các cuộc gọi sư phụ cung cấp cho tôi là tập Data DARPA (Defense Advanced Research Projects Agency) của Mỹ dưới dạng chữ cùng các thông tin cho biết cuộc gọi nào có/không có vấn đề.

Bước tiếp theo trong quá trình R&D, tôi tìm hiểu xem bài toán PDI trên thế giới những người trước đó làm với PARADISE Framework (hoặc Framework nào khác) trên DARPA Dataset (hoặc các Dataset khác) có kết quả thế nào để từ đó tôi cố gắng tìm cách làm sao để có kết quả bằng hoặc tốt hơn.

Tiếp theo, tôi áp dụng CART của tôi vào DARPA Dataset này để xây dựng 1 Decision Tree. Chạy thử, debug, nâng cấp đã đời thì CART của tôi chạy hết 5'. Kết quả CART của tôi so với các kết quả quá khứ thì còn kém lắm. Đây là điều hiển nhiên vì giải thuật CART tôi viết chỉ là cơ bản, chưa đủ chuyên nghiệp để có thể dùng thực tế.

Tôi nghĩ để phát triển tiếp giải thuật CART của tôi đến mức dùng được sẽ tốn rất nhiều thời gian và công sức, sợ không kịp thời gian tốt nghiệp, chắc phải đi tìm công cụ có sẵn dạng open-source dùng liền cho nhanh.

Sau khi bỏ cả ngày lên mạng dạo vài vòng thì tôi cũng tìm được một thư viện miễn phí mà lại rất tốt có tên là Weka, được phát triển ở 1 trường đại học của New Zealand bởi các giáo sư, tiến sĩ trong thời gian dài.

Tôi tải xuống và cài đặt Weka trên máy để chạy thử giải thuật CART của nó trên DARPA Dataset thì chỉ mất 1' và kết quả tốt hơn giải thuật CART của tôi rất nhiều, cũng tương đương kết quả quá khứ.

Bên cạnh đó, nhóm phát triển Weka cũng xuất bản 1 cuốn sách về Data Mining có tựa "Data Mining: Practical Machine Learning Tools and Techniques" rất thực tế và dễ hiểu so với mấy cuốn sách hàn lâm khác tôi đã đọc khi học môn Pattern Classification trong trường Concordia hơi bị khó hiểu vì dính quá nhiều toán và công thức.

Thế là tôi lao đầu vào đọc hết cuốn sách này để hiểu rõ về Data Mining hơn, cũng như biết được các kỹ thuật để cải thiện chất lượng của việc xây dựng Machine Learning Model như là phương pháp stratify & randomize dataset, kiểm thử dataset với N-fold cross validation...

Ngoài giải thuật CART (Weka J48) thì Weka còn cung cấp rất nhiều giải thuật khác như Bayes, One-rule, Ada Boost, Neuron Networks, Support Vector Machine...

Tôi trải nghiệm thử từng giải thuật trên DARPA Dataset. Mỗi giải thuật lại có nhiều **Configuration Option** (Tùy Chọn Cấu Hình). Tôi lại thay đổi một vài cấu hình trên vài giải thuật để thử tiếp.

Tôi quan sát thấy có giải thuật cho kết quả tốt hơn CART, có giải thuật lại cho kết quả kém hơn CART, cùng 1 giải thuật với cấu hình C1 thì tốt hơn CART, cấu hình C2 thì kém hơn CART. Kết quả này phù hợp với cuốn sách trên có nói là không có một Machine Learning Algorithm nào là tốt nhất cả. Ứng với một Dataset cho trước, sẽ có một giải thuật với cấu hình nào đó phù hợp sẽ cho hiệu năng tốt nhất.

Như vậy thì với 1 Dataset cho trước là DARPA Dataset, tôi phải thử rất nhiều giải thuật trên Weka GUI (Graphic User Interface) để tìm ra giải thuật tốt nhất -> số lượng giải thuật & số lượng cấu hình của mỗi giải thuật đem tổ hợp ra sẽ ra rất rất nhiều trường hợp để thử, nếu làm bằng tay bấm chuột trên GUI chắc...liệt tay & liệt chuột luôn quá 😊

Cũng may cho tôi là thư viện Weka thuộc dạng open-source, nên tôi quyết định lấy phần core của Weka đem về lập trình thêm để chạy tìm giải thuật tốt nhất một cách tự động.

Phần code tôi làm để tìm giải thuật tốt nhất này không nằm trong yêu cầu của dự án. Ở mức độ Master, chỉ cần tôi tạo ra 1 phần mềm PDI chạy được, sử dụng 1 giải thuật chọn trước như Weka CART chạy ra kết quả tương đương với quá khứ là ok rồi. Tuy nhiên, tôi vẫn muốn kết quả tốt hơn nữa nên chịu khó lập trình thêm từ Weka core.

Tiếp theo, kết nối những phần tôi đã làm được lại với nhau, tôi bắt tay vào xây dựng USPA.

Nhờ có kiến thức & kinh nghiệm từ dự án siêu dự SER5500 và kiến trúc USPA cũng đơn giản nên tôi phát triển USPA khá nhanh.

Sau khi hoàn thành USPA, tôi ứng dụng vào DARPA Dataset với Business Flow như sau:

1. Sử dụng bộ Parser xử lý các Dialog bằng text.
2. Sử dụng bộ Feature Extractor để trích ra Dataset trong đó mỗi Dialog được thể hiện qua các thuộc tính của PARADISE Framework và được đánh dấu (**Annotation**) với "Label" tương ứng là "Good" hay "Bad".
3. Sử dụng bộ Trainer cho chạy tự động trên Dataset này để tìm ra giải thuật tốt nhất.
4. Dùng giải thuật tốt nhất chạy trên Dataset để lấy Learning Model, đây là mô hình tốt nhất dùng để dự đoán các Dialog tương lai.

Trong quá trình suy nghĩ và phân tích các thuộc tính của PARADISE Framework, tôi tự tìm ra 2 thuộc tính mới có khả năng giúp cải thiện hiệu năng PDI:

1. **NumNegativeACKwords** : Số từ người gọi trả lời với máy mang tính tiêu cực, ví dụ: no, nop, wrong, incorrect, false, sh*t, f*ck... Về mặt trực quan, khi người gọi nói những từ này thì khả năng cao họ không hài lòng, tuy nhiên không phải lúc nào cũng vậy, tùy thuộc vào cảm xúc của họ nữa, vì thế mới có phần nghiên cứu về Emotion Detection để hiểu thêm về cảm xúc của người gọi.
2. **NumRepetitions** : Số lần máy lặp lại cùng 1 câu. Về mặt logic thì máy lặp lại cùng 1 câu có nghĩa là nó không hiểu người gọi nói gì nên lặp lại để cho người gọi xác nhận.

Tôi đã thử USPA cho trường hợp không có 2 thuộc tính mới này và trường hợp có 2 thuộc tính này thì kết quả cho thấy trường hợp sau có hiệu năng tốt hơn. Như vậy, cả về cảm nhận và thực nghiệm đều cho thấy 2 thuộc tính mới này đem lại hiệu quả cho phần PDI.

Với việc tìm ra 2 thuộc tính mới này, chị PostDoc & tôi đã viết 1 bài báo + demo tham dự 1 hội thảo khoa học dạng conference. Như vậy tôi có được 1 bài báo dạng conference lặn lưng cho vui. Ghi chú một chút là Conference Paper có giá trị thấp hơn Journal Paper, thường thì sinh viên Master không cần phải có Conference Paper hoặc Journal Paper để được bảo vệ luận văn tốt nghiệp, trong khi nghiên cứu sinh Ph.D thì bắt buộc phải có 1-3 Journal Paper (tùy lĩnh vực, tùy trường, tùy hội đồng giám khảo) & N Conference Paper.

Để làm ra bài báo được duyệt là Journal Paper rất là chua, phải là cái mới nhất thế giới chưa ai làm và phải có ý nghĩa lớn lao, nói chung phải mất rất nhiều thời gian nghiên cứu và công sức xây dựng ứng dụng thực tiễn, bởi vậy học Ph.D siêu khó.

Cuối cùng, bước đăm đuối nhất của tôi là phân đi thu thập các cuộc gọi thực tế từ công ty Bell Canada, xử lý Data để chạy trên USPA rồi đo đạc, vẽ đồ thị, so sánh... rồi lặp lại qui trình này n++ lần cho đến khi tìm được PDI Tốt Nhất sử dụng trong các Call Center của công ty để dự đoán mức độ hài lòng của khách hàng. Theo thời gian, khi có thêm những cuộc gọi bổ sung vào Dataset, qui trình trên được chạy lại để tìm PDI Tốt Nhất mới.

Phù...mọi khó khăn đã qua, phần công việc cuối cùng của tôi là bắt đầu viết báo cáo.

Lúc đó dù trình độ French của tôi có tiến bộ đáng kể nhưng tôi vẫn không đủ tự tin để viết bằng French, nên đã xin sư phụ cho tôi viết bằng English. Hiển nhiên sư phụ ok, nhưng tôi phải sử dụng mẫu luận văn của trường ÉTS với cái sườn được viết bằng French chứ không được sửa sang English, chỉ có nội dung thì được phép viết bằng English.

Thế là cái luận văn tốt nghiệp của tôi có ngôn ngữ thập cẩm khi pha trộn giữa French & English:
<https://docs.google.com/document/d/16Kgk5CH7wF4ziQ51HHnA7IH2UCJapTg/edit?usp=sharing&ouid=110129149865834540754&rtpof=true&sd=true>

Viết xong luận văn, tôi nộp cho sư phụ duyệt rồi sau đó sư phụ sẽ đem qua cho các bác giáo sư phản biện 'chém' tới tấp 😂

Cuối cùng tôi cũng đã hoàn tất sản phẩm USPA cho công ty Bell Canada dùng và luận văn tốt nghiệp Thạc Sĩ của tôi được chấm A+, A+ & A từ 3 thành viên trong ban giám khảo -> tổng hợp lại là A+ .

Hành trình vượt đèo Master của tôi với bao nhiêu khó khăn, gian khổ suốt 2 năm trời trên đất Montreal, Canada đã thành công mỹ mãn, vượt ngoài mong đợi làm tôi vô cùng hạnh phúc.

Hành trình này không những cung cấp cho tôi khá nhiều kiến thức IT mới, nâng cao tư duy mà còn giúp tôi trưởng thành hơn về mọi mặt trong cuộc sống.

Đầu năm 2008, sau khi xong chương trình Thạc Sĩ, tôi đã mời sư phụ đi ăn tối mừng tôi tốt nghiệp và cảm ơn sư phụ. Trong bữa ăn, hai thầy trò ôn lại những chuyện quá khứ, nói những chuyện về khoa học, về môi trường làm việc ở Canada.

Cuối buổi, sư phụ hỏi tôi có muốn học lên Tiến Sĩ và tiếp tục tham gia dự án qua phần khác với học bổng mới không?

Tôi suy nghĩ một lúc rồi cảm ơn sư phụ đã ngỏ ý chứ tôi không muốn học tiếp Tiến Sĩ. Trong thời gian học Master, tôi có quan sát và tìm hiểu thì học Tiến Sĩ rất cực khổ, hi sinh thời gian/ tuổi trẻ (từ 5-7 năm ở Canada/Mỹ, châu Á/châu Âu có thể sớm hơn), tỉ lệ thành công rất thấp, mà học được xong thì cũng cực kỳ khó đi tìm việc làm, chỉ có vô các trường đại học giảng dạy và nghiên cứu khoa học, nhưng đây không phải là định hướng sự nghiệp của tôi. Cái tôi muốn làm là phát triển các Tech Product giải quyết những bài toán của doanh nghiệp và xã hội.

5. Dự án LMA (Canada)

Sau khi tốt nghiệp Master thì tôi bắt đầu hành trình trải nghiệm làm việc ở Canada với việc gia nhập công ty Canada đầu tiên tên là Bibliomondo. Sản phẩm phần mềm của công ty này là **Library Management Application – LMA** (Ứng Dụng Quản Lý Thư Viện).

LMA được sử dụng bởi hàng trăm thư viện ở Canada và Pháp để quản lý việc xuất bản sách, cho mượn sách và **Book Binding** (Đóng Gộp Sách) các tạp chí để bán/cho mượn.

Khi tôi tham dự án thì sản phẩm đã phát triển được mấy năm rồi, sử dụng những công nghệ như Java Swing, EJB, SOAP và đã ra được nhiều Version. Vì lý do này mà vòng 2 khi phỏng vấn kỹ thuật, anh quản lý dự án tên là Alexandre đã không hỏi tôi những Framework mới nhất như Spring/Hibernate. Công việc của tôi là bảo trì và phát triển tiếp một số tính năng mới cho LMA.

Những thử thách mà tôi đã gặp phải:

- Kiến thức lĩnh vực mảng thư viện mới với tôi.
- Công nghệ **SOAP – Simple Object Access Protocol**, và **EJB – Enterprise Java Beans** mới với tôi.
- Dù không phức tạp bằng dự án SER5500 nhưng môi trường phát triển cũng quá nhiều thứ.
- Ác mộng với núi XML khi dùng với SOAP và EJB.
- Môi trường làm việc bằng French: tài liệu yêu cầu, tài liệu thiết kế, nhân viên đa sắc tộc nói nhiều giọng French khác nhau, nói French suốt ngày... là một thách thức lớn với tôi.

Tôi khá may mắn là gặp được anh Technical Lead tên là Christophe, người Pháp qua Canada làm việc, có tính tình rất tốt, thân thiện, nhiệt tình Mentor cho tôi, nhất là anh kiên nhẫn nghe tôi nói French gặp tình huống thiếu từ để diễn đạt (lần đầu tiên trong đời tôi phải nói French cả ngày mỗi miệng lẩm luôn) và sửa sai cho tôi, khi nào khó quá tôi lái qua English thì anh cũng chuyển sang English luôn.

Nhờ anh Mentor về Domain Knowledge, Software Architecture cũng như các công nghệ trong LMA mà tôi đã thích ứng tốt với dự án và vượt qua được 2 tháng thử việc. Phù...tôi khá là vui vì lúc mới vô công ty rất lo lắng về nhiều thứ mình còn yếu.

Từ đây tôi rút ra được 1 điều là chúng ta muốn học nhanh bất cứ cái gì thì phải nỗ lực thực hành hàng ngày và tốt hơn nữa là có Mentor theo sát hướng dẫn cũng như sửa sai.

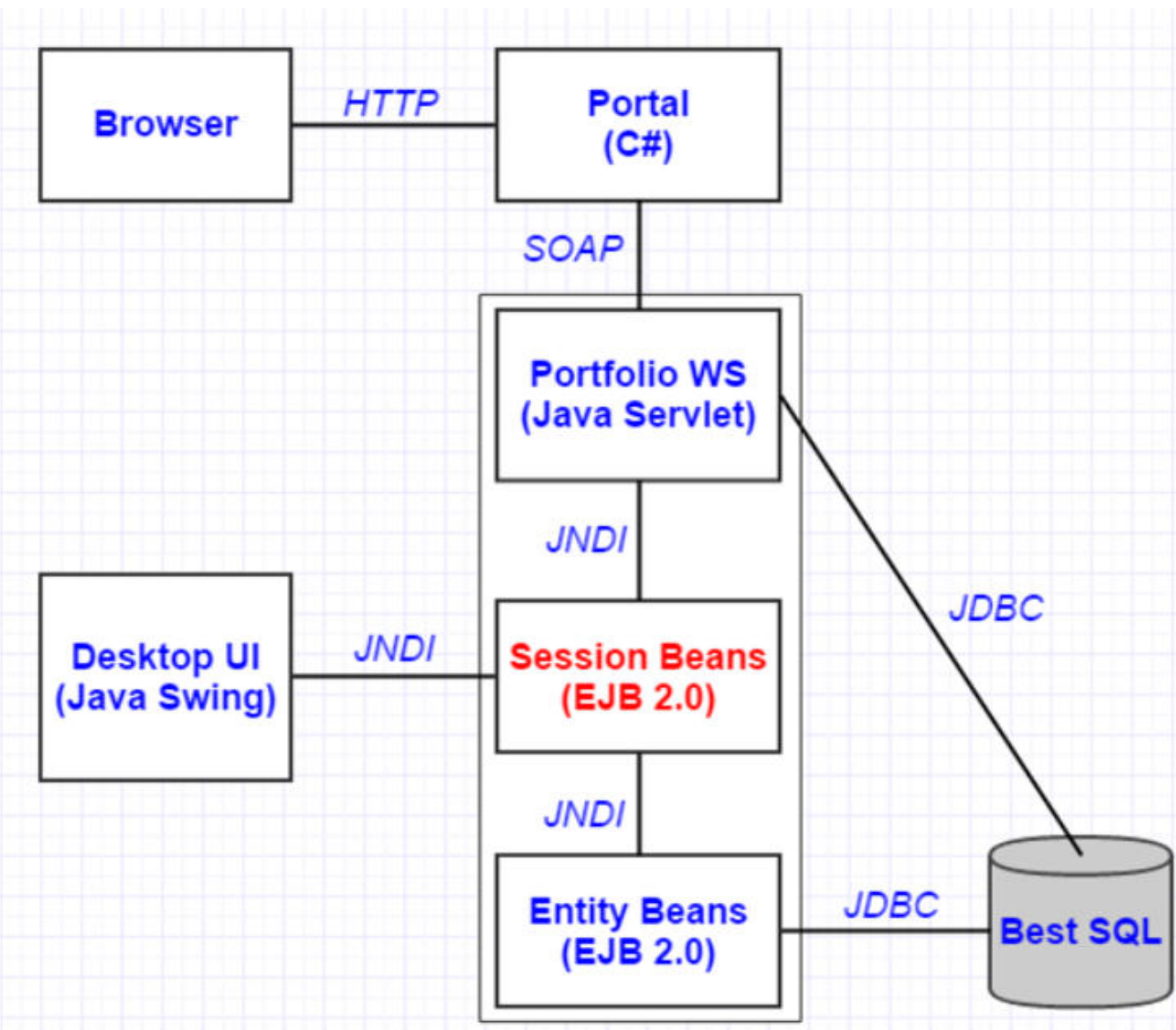
Mô hình phát triển dự án được thể hiện qua hình dưới đây:

LMA – Canada

Học

- 1. Domain Knowledge: Library
- 2. SOAP
- 3. JNDI
- 4. EJB Session Beans
- 5. EJB Entity Beans
- 6. Best SQL

Hành: Phần mềm quản lý Thư viện



Hình 98: LMA

Dự án gồm các phần chính:

- 1. Một cái Web App chính được viết bằng công nghệ J2EE, cụ thể là EJB 2.0.
- 2. Một cái Web Portal được phát triển bằng công nghệ .Net cho User dùng. Web Portal kết nối với Web App thông qua SOAP Web Service.
- 3. Một cái Windows App cho Admin sử dụng được viết bằng Java Swing. Công ty đã bỏ công xây dựng 1 bộ thư viện các UI Components chuyên dụng như table, list, form... giúp cho việc hiện thực các giao diện rất nhanh. Windows App này kết nối qua Web App thông qua **JNDI – Java Naming and Directory Interface** tới EJB Session Bean.
- 4. Phần Database công ty tự viết ra đặt tên là Best SQL. Tôi hỏi chơi anh TL tên này có nghĩa gì thì ổng nói nghĩa là Database tốt nhất thế giới. Lúc đó MySQL, PostgreSQL... chưa ra, còn Oracle SQL thì mắc quá.

Tôi đảm nhận các việc trên phần Windows App, SOAP Web Service & EJB 2.0 (Session Bean & Entity Bean). Phần việc trên Windows App tôi làm với các giao diện rất là thú vị bao nhiêu thì phần việc trên 2 cái còn lại thật sự là ác mộng với cả núi XML.

Có nhiều file XML lên đến mấy ngàn dòng, kéo chuột muốn gãy tay, rồi mở con mắt để tìm ra chỗ bỏ vô đoạn nội dung mới 😊. Cũng vì lý do này mà sau này SOAP-XML đã dần bị thay thế bởi REST-JSON và EJB 2.0 bị Spring Framework với khái niệm POJO vượt mặt để rồi sau đó EJB 3.0 ra đời (bắt chước khá nhiều tính năng của Spring Framework) đập tan ác mộng XML cho Software Developer.

Thật không thể tin được, thông tin quản lý của 1 quyển sách nhiều kinh khủng, lên đến gần 150 **Property** (Thuộc Tính), khi ánh xạ vô cơ sở dữ liệu thành 150 **Column** (Cột).

Trên UI, để chỉnh sửa 1 cuốn sách thì User phải thao tác trên cả chục **Tab** (Phân Trang), mỗi Tab có cả đồng **Field** (Trường).

Phần tính thuế liên quan đến Book Binding thiệt là mỗi não luôn đó. Đoạn Code cũ tính thuế lên đến 14 trang A4 với rất nhiều công thức phức tạp liên quan đến từng loại thuế của Canada. Tôi đọc đoạn Code đó để hiểu cách tính mà muốn quay cuồng đầu óc.

Các đồng nghiệp Developer đến từ nhiều nước khác nhau như Canada, Pháp, Anh (ông này cũng mê bóng đá lắm, nhất là giải ngoại hạng Anh), Trung Quốc, Việt Nam (tôi và 1 anh Việt Kiều). Mọi người thân thiện, vui vẻ, thường cùng bia vào Happy Hour thứ 6 mỗi tuần.

Ngồi cạnh tôi là 1 đồng chí Trung Quốc. Anh này French nói tạm tạm, English thì bập bẹ. Trình độ IT thì cũng bình thường, không có gì nổi bật. Vậy mà anh ấy đem được vợ con qua Canada theo dạng định cư cả gia đình. Tôi phải công nhận xứ Trung Quốc có dịch vụ TOFEL và định cư Canada hay ghê đó.

Qua dự án LMA, tôi đã học thêm được một lĩnh vực mới, công nghệ mới, và kiến trúc phần mềm mới kết hợp cả 2 công nghệ Java & .Net.

Nhân dịp tổng kết cuối năm 2008, công ty Bibliomondo tổ chức tiệc tất niên ở 1 nhà hàng 4 sao của Ý. Lần đầu tiên tôi được dự 1 cái tất niên sang trọng ghê gớm, từ phong cảnh trang trí, đến bàn, ghế, đĩa... đều rất đẹp.

Món súp gì đó cái tô thì to, nhưng súp bên trong thì có tí xíu, quét vài thìa nhỏ là hết. Món cá hồi thì cái đĩa cực to, miếng cá bé xíu nằm ở giữa, trang trí khá đẹp mắt, nhưng ăn thì đúng là lấy hương lấy hoa. Cứ thế mà 5-6 món dọn ra, tôi ăn xong bữa tiệc mà vẫn đói (tối về phải mần thêm gói mì tôm 😂).

Sau phần ăn uống thì công ty tổ chức trò chơi rất vui, tặng quà có ý nghĩa cho những ai trả lời câu hỏi đúng hoặc chiến thắng trong các trò chơi đối kháng tập thể.

À, tôi nghĩ có thể dân Tây khi tham dự tiệc từng bọn nó quan tâm đến ăn lấy hương lấy hoa, vui chơi giải trí hơn là ăn uống no đủ, nhạo nhẹt này nọ theo kiểu dân châu Á.

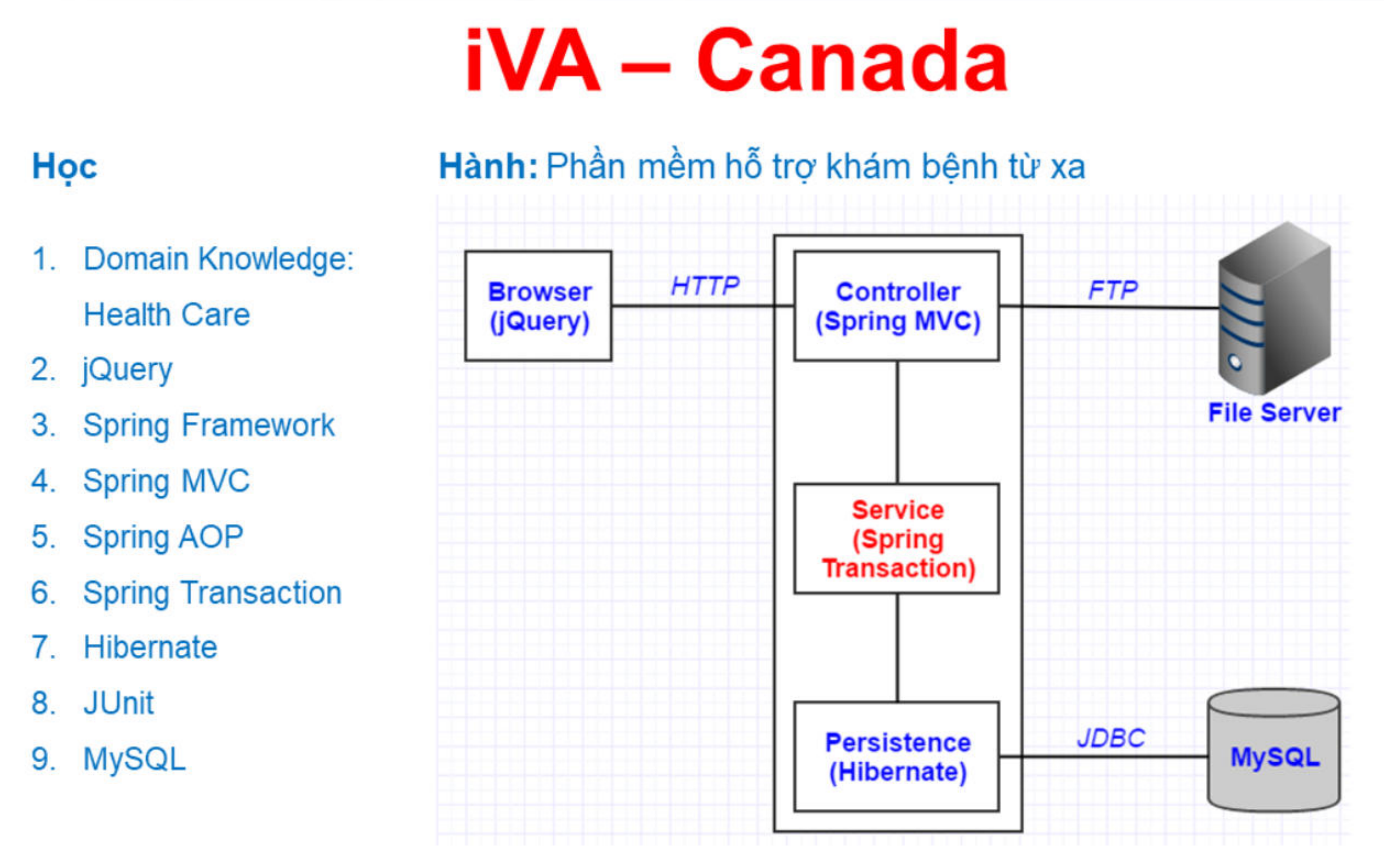
6. Dự án iVA (Canada)

Rời công ty Bibliomondo, tôi chuyển sang công ty Retina được thành lập bởi 1 nhóm các bác sĩ tư nhân. Công ty đang xây dựng 1 sản phẩm công nghệ có tên là **iVA** (iVision Application) bên mảng Health Care được triển khai trên hàng trăm phòng khám khắp Canada giúp cho các bệnh nhân ở vùng sâu/xa có thể khám bệnh ở địa phương họ nhưng vẫn được các bác sĩ ở thành phố lớn chẩn đoán và điều trị mà không cần phải đi xe buýt lên thành phố. Hiển nhiên là họ chỉ bị các bệnh không quá nặng, không cần thiết phải gặp bác sĩ trực tiếp hoặc phải phẫu thuật. Sản phẩm được ứng dụng ở rất nhiều thành thị của Canada.

Các tính năng của sản phẩm gồm có:

- 1.Quản lý các **Clinic** (Phòng Mạch) ở địa phương
- 2.Lập lịch hẹn khám ở địa phương cho **Patient** (Bệnh Nhân)
- 3.Lấy **Image Pattern** (Mẫu Hình Ảnh) của bệnh nhân ở phòng mạch địa phương họ
- 4.**Doctor** (Bác Sĩ) ở phòng mạch chẩn đoán và ra lược đồ điều trị, nếu bệnh nặng quá thì ra yêu cầu cho bệnh nhân lên phòng mạch khám và điều trị trực tiếp
- 5.Quản lý các **Diagnosis** (Chẩn Đoán) và theo dõi qui trình điều trị tương ứng

Sơ đồ dự án được thể hiện qua hình dưới:



Hình 99: iVA

Những thử thách mà tôi đã gặp phải:

1. Kiến thức lĩnh vực chăm sóc sức khỏe mới với tôi.
2. Công nghệ Spring Framework có nhiều khái niệm khá mới với tôi như POJO (Plain Old Java Object), IoC, DI, AOP. Ngoài ra lần đầu tiên tôi đụng đến phần Web với Javascript Framework khá phổ biến là jQuery.
3. Lần đầu tiên tôi đụng bài toán **Object-Relational Mapping – ORM** (Ánh Xạ Quan Hệ Đối Tượng) với Hibernate Framework.
4. Thăng Team Lead (nhỏ hơn tôi 1 tuổi) tên Mark là fan cuồng của Unit Test, nó bắt cả nhóm viết Unit Test một xâu (kể cả Controller layer) làm mất rất nhiều thời gian và công sức.
5. Áp lực deadline rất nặng nề để ra tính năng mới đưa vào phục vụ khám chữa bệnh.

Thời gian đầu tôi tham gia dự án đúng là một nỗi với quá nhiều từ chuyên môn về các bệnh bên y học, thật sự rất khó nhớ. Các quy trình nghiệp vụ cũng phức tạp. Cũng may là môi trường này nói English đỡ hơn công ty trước chơi French suốt ngày khá mới miệng. Nhờ chú TL và các bác sĩ giải thích một cơ sở lần thì tôi cũng nắm được các quy trình nghiệp vụ của hệ thống.

Về công nghệ thì Spring Framework, Hibernate và jQuery tuy mới với tôi nhưng nhảy vô vừa học vừa làm có chú TL làm Mentor thì thấy dễ dàng hơn nhiều so với EJB 2.x.

Mà cũng nhờ chú này mà tôi làm quen với Unit Test cùng JUnit Framework và thấy nó cũng hữu ích khi mình bảo trì cũng như phát triển tính năng mới cho ứng dụng. Tuy nhiên thời gian để lập trình viết Unit Test quá tốn kém, vì vậy mấy công ty startup sau này tôi thấy bỏ qua phần này lúc đầu, khi nào sản phẩm ổn định có đầu tư/doanh thu tốt rồi mới quan tâm đến Unit Test.

Để đối phó lại áp lực Deadline thì tôi cần phải hiểu rõ ứng dụng từ nghiệp vụ cho đến kỹ thuật, các công việc liên quan với nhau thế nào để phân độ ưu tiên giữa các công việc. Khi công việc này bị tạm ngưng vì lý do khách quan thì phải chuyển qua công việc khác liền chứ không thể ngồi chờ cho nó được giải phóng rồi làm tiếp.

Sau khi làm xong dự án này tôi từng có suy nghĩ sau này về Việt Nam xây dựng 1 hệ thống giống như vậy để giúp bà con dưới quê đỡ khổ khi lên các thành phố lớn như Sài Gòn, Hà Nội, Đà Nẵng... khám bệnh.

Nhưng khi hỏi một số người về ý tưởng này thì họ nói không khả thi lắm đâu vì người Việt Nam có tâm lý khi khám bệnh là muốn gặp trực tiếp bác sĩ mới chịu và trình độ của các nhân viên y tế ở dưới quê không cao cùng cơ sở vật chất không đủ hiện đại để lấy mẫu cho chính xác nữa. Thế là ý tưởng của tôi chết từ trong trứng nước 😅

Sản phẩm iVA không lớn lắm, nên xây dựng 1 thời gian ngắn thì nó chuyển sang giai đoạn vận hành và bảo trì. Sau khi tham gia bảo trì vài tháng thì tôi đi tìm việc khác hứng thú hơn...

Phải công nhận tìm việc làm đầu tiên ở Việt Nam và Canada đều rất vất vả, nhưng từ việc làm thứ hai trở đi thì tôi đỡ hơn rất nhiều.

7. Dự án MPA (Canada)

Rời công ty Retina, tôi vào công ty 3CE với sản phẩm nổi bật là **Expert System** (Hệ Chuyên Gia) tên là **3CE** (Code Commodity Classification Engine) giúp phân loại hàng hóa xuất nhập khẩu nhanh chóng và chính xác hơn so với các Keyword Matching System (Hệ Thống Tìm Kiếm Mã Hàng Theo Từ Khóa) hoặc tìm mã hàng bằng phương pháp thủ công của hải quan.

Ứng với 1 **Description** (Đặc Tả) của 1 **Commodity** (Hàng Xuất Nhập Khẩu) thì phần mềm thông thường cho ra rất nhiều kết quả và các nhân viên hải quan (NVHQ) phải dò trên từng kết quả để xác định chính xác loại HS (Harmonized System) Code (xác định sai HS Code sẽ áp thuế sai), việc này tốn rất nhiều thời gian và cũng dễ sai sót, ảnh hưởng rất nhiều đến các doanh nghiệp xuất nhập khẩu trên thế giới.

Sản phẩm 3CE được phát triển dựa trên phần lõi là **Ontology** (Bản Thể Học) được 5-6 tiến sĩ thuộc các lĩnh vực khác nhau như thực phẩm, dầu, hóa học... xây dựng từ đầu.

Nói nôm na ví dụ có cụm từ “cotton red shirt wide” thì ngành Ontology giúp xác định được chủ thể của cụm từ trên là “shirt” chứ không phải “cotton”. Việc xác định đúng chủ thể của 1 cụm từ rất quan trọng trong việc phân loại commodity ra mã HS Code để áp thuế xuất/nhập khẩu.

Việc phân loại commodity được tiến hành chậm hoặc sai sẽ gây thiệt hại rất lớn cho chủ hàng là các **Im/Export Company** (Công Ty Xuất Nhập Khẩu) hoặc các **Agent** (Đại Lý) trung gian.

Sản phẩm 3CE hoạt động theo cơ chế của 1 Expert System:

1. User đưa đặc tả của Commodity vào cho 3CE phân tích sử dụng Ontology
2. Nếu 3CE chưa thể xác định được HS Code của Commodity thì 3CE sẽ tự sinh ra câu hỏi cho User trả lời thêm về hàng hóa
3. User trả lời câu hỏi
4. Bước 2 & 3 được lặp lại vài lần thì 3CE có thể xác định được mã của Commodity với thời gian rất nhanh và độ chính xác khá cao

Được đầu tư hàng triệu USD, 3CE đã giúp các công ty xuất nhập khẩu lớn của Canada & USA tiết kiệm được hàng tỉ USD chi phí hàng năm với vấn đề phân loại Commodity chậm/ sai. Khách hàng của 3CE là các hải quan, doanh nghiệp xuất nhập khẩu của Canada & USA.

Tôi tham gia đội ngũ phát triển phần mềm ứng dụng của công ty có nhiệm vụ phát triển các phần mềm có tích hợp hệ thống 3CE cho khách hàng sử dụng theo nhu cầu của họ.

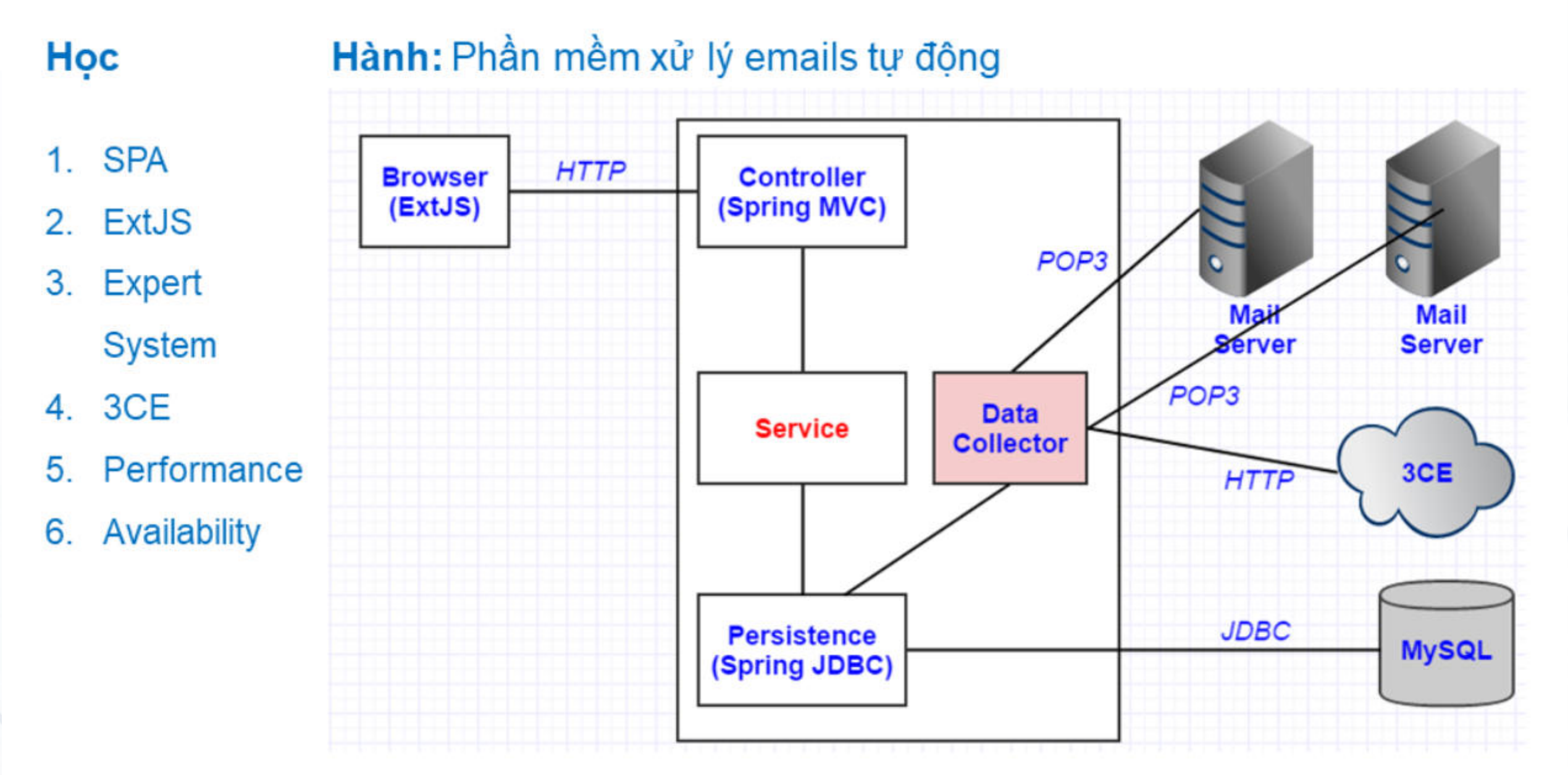
Nhóm phát triển phần mềm ứng dụng này gồm có 1 ông Team Lead người Canada tên Marc, lấy **Raw Requirement** (Yêu Cầu Thô) từ khách hàng về cho tôi & 1 Developer khác mần từ A-Z cho đến khi ra sản phẩm dưới sự Mentor của ông ấy, và 2 QCs.

Ông TL này chuyên môn rất giỏi, nói nhanh như gió cả English & French nên trao đổi với khách hàng để lấy yêu cầu phần mềm hay lắm, nhờ vậy tôi cũng học hỏi được 1 tí, và ông cũng là fan cuồng của Javascript (nhất là jQuery Framework) & Java (Spring Framework).

Sản phẩm **MPA** (Mail Processing Application) tôi xây dựng cho công ty USPS (US Postal Service) của Mỹ giúp họ xử lý tự động một lượng lớn email mỗi ngày có đưng đến phần Commodity Classification phải nhờ đến hệ thống 3CE.

Mô hình phát triển MPA thể hiện trong hình dưới:

MPA – Canada



Hình 100: MPA

Những thử thách tôi đã gặp phải:

1. Lần đầu tiên tôi dựng vô **Single Page Application – SPA** (Ứng Dụng Đơn Trang) với công nghệ Frontend mới là ExtJS, trước kia toàn là **Multiple Page Application – MPA** (Ứng Dụng Đa Trang).
2. Thách thức trong việc xử lý nội dung không đồng nhất của lượng lớn email cũng như trùng lặp giữa các Commodity đã/chưa phân loại dựa vào 3CE.
3. Vấn đề System Performance và System Availability.

Lập trình với ExtJS dạng component-based trên nền Web App rất giống với Java Swing trên nền Windows App mà tôi đã quá quen thuộc nên chuyển sang cũng không gặp vấn đề lớn, chỉ có vấn đề là lập trình web hơi khó debug hơn lập trình windows.

Tôi đã tìm hiểu thêm về cách thức AJAX hoạt động là nắm được phân giao tiếp giữa Frontend & Backend.

Ông TL đã giúp tôi hiểu cách thức 3CE hoạt động để tích hợp với MPA cũng như một số kỹ thuật để tăng hiệu năng hệ thống và cách dùng **Load Balancer** (Bộ Cân Bằng Tải) để giải quyết vấn đề System Availability.

Vào dịp Noel 2010, công ty tổ chức đi ăn tối với các món ẩm thực của Quebecois và Nga. Sau buổi ăn tối thì cả công ty vô quán bar uống vài ly rượu nhẹ và quánh bida lỗ khá là thú vị. Nhân viên của công ty 3CE đến từ khá nhiều nước: Canada, Pháp, Nga, Belarus, Trung Quốc, Guyana (Nam Mỹ), và Việt Nam (1 mình tôi). Mọi người rất là thân thiện và cởi mở.

Tôi có làm việc chung với 1 chị tiến sĩ người Trung Quốc tên là Xiao chịu trách nhiệm 1 phần trong phần Ontology. Chị này tính tình rất là dễ thương, hòa đồng. Sau khi chị học xong ở lại Canada làm mà không muốn về lại Trung Quốc. Chị ấy định hưởng sẽ sang Mỹ làm và định cư lâu dài luôn. Ngoài ra tôi cũng thường nói chuyện với 1 anh QC người Quebecois tên là Patrick. Anh này siêu vui tính nên tôi rất hay đùa với anh, tìm hiểu thêm về cuộc sống, con người & văn hóa của dân Quebec.

Thời gian đầu mọi người hay đi ăn trưa cùng nhau. Một thách thức phổ biến cho hầu như bất cứ nhóm nào khi đi ăn trưa là...ăn cái gì? ở đâu? Thế là tôi đã viết 1 cái phần mềm nhỏ cứ đúng 11.00h mỗi sáng là gửi email cho mọi người 1 cái link. Khi bấm vào cái link thì họ sẽ được đưa vào 1 trang để biểu quyết (vote) hôm nay họ muốn ăn gì, ở đâu. Sau đó 11h50 phần mềm sẽ tự động gửi email cho mỗi người báo kết quả biểu quyết. Nói chung khá là vui!

8. Dự án GPM (Canada)

Bước sang năm 2011, tôi tham gia phát triển sản phẩm **GPM** (Global Product Manager) cho công ty 3CE.

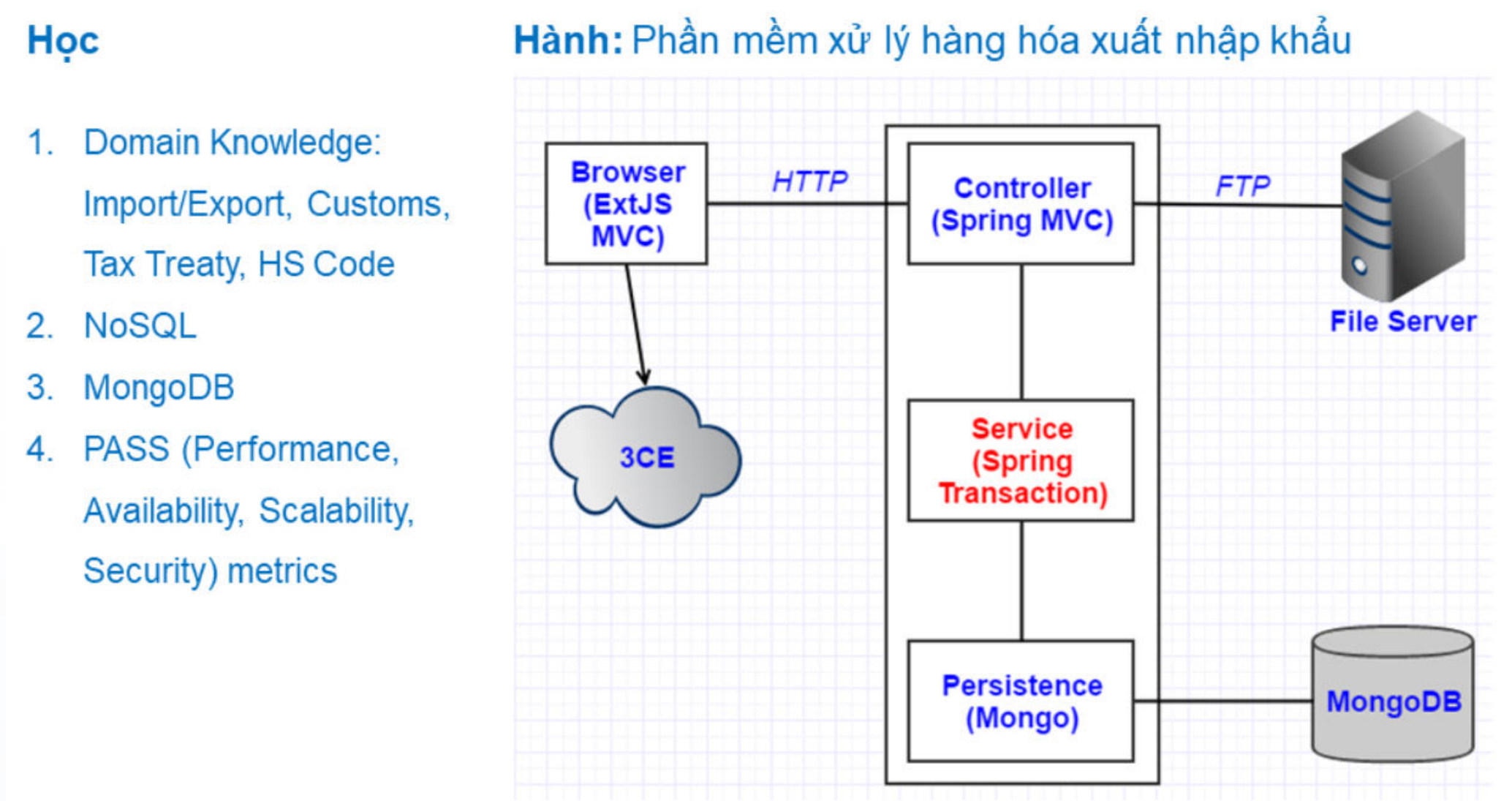
Đây là sản phẩm nội bộ của công ty, tuy nhiên yêu cầu phần mềm được thu thập từ một số khách hàng lớn của 3CE vì GPM giúp các công ty có sử dụng hệ thống 3CE quản lý sản phẩm xuất nhập khẩu của họ.

Các tính năng chính của phần mềm bao gồm:

- 1.Quản lý các **Vendor** (Nhà Bán Hàng)
- 2.Quản lý **Commodity**, có bao gồm **Category** (Danh Mục)
- 3.Phân loại hàng hóa sử dụng hệ chuyên gia 3CE để tìm ra **HS Code**
- 4.Quản lý các **Destination** (Điểm Đến) mà hàng hóa được xuất đi với các **Tax Duty** (Mức Thuế Xuất) tương ứng

Mô hình phát triển GPM được đặc tả trong hình dưới đây:

GPM – Canada



Hình 101: GPM

Những thử thách tôi đã gặp phải:

- 1.Kiến thức lĩnh vực xuất nhập khẩu, thuế, hải quan.
- 2.Kiến trúc phần mềm không có gì thay đổi so với MPA, chỉ có 1 phần công nghệ mới đó là khái niệm NoSQL database và một Database mới ra khi đó là MongoDB.
- 3.Lại là vấn đề xử lý trùng lặp giữa các sản phẩm đã/chưa phân loại dựa vào 3CE, cái này hơi khác tí so với MPA.
- 4.Lần đầu tiên tôi tiếp xúc với các **Non-Functional Requirement – NFR** (Yêu Cầu Phi Chức Năng), đó là PASS (Performance, Availability, Scalability and Security).

Đến lúc này thì tôi đã đi làm được 7-8 năm, trải qua cũng khá nhiều dự án phần mềm thì kiến thức/ kinh nghiệm phát triển phần mềm của tôi cũng đã tiến bộ đáng kể, cộng thêm sự Mentor nhiệt tình của ông TL rất giỏi thì những khó khăn trên không là vấn đề gì với tôi cho lắm.

Sau 3 tháng tôi đã xây dựng xong **POC – Proof of Concept** cho sản phẩm phần mềm GPM từ A-Z cho công ty 3CE đem cho khách hàng ở Canada & USA dùng thử thì họ đã phản hồi khá tốt vì GPM đã giúp họ tiết kiệm thời gian và chi phí rất nhiều.

Tiếp đó họ đã yêu cầu rất nhiều tính năng mới cho GPM. Ông TL đã thu thập yêu cầu thô trong một chồng giấy A4 rồi đưa tôi xem thì tôi rất ư là...ngộp lun 😅

Sau khi phát triển thêm một vài Version cho GPM thì tôi quyết định về lại Sài Gòn vào đầu năm 2012 để làm gì đó mới mẻ và sôi động hơn sau 6-7 năm trải nghiệm học hành và làm việc đầy thách thức và thú vị trên đất nước Canada xinh đẹp ☺

Trước khi về làng Sài Gòn, tôi cũng đã tranh thủ đi du lịch khám phá nhiều chỗ khác nhau trên đất nước Canada cùng các bạn du học sinh Việt Nam như tham quan các tỉnh khác của Canada là New Brunswick, Nova Scotia và Prince Edward Island (được mệnh danh là Vương Quốc Tôm Hùm do tôm hùm ở đây rất nhiều và rất ngon). Ngoài ra bọn tôi còn ghé thăm đất nước anh em Cuba.

9. Dự án YCMS (Vietnam)

Đầu năm 2012, tôi về lại Sài Gòn vào đúng dịp Tết Ta để tận hưởng một cái Tết nguyên đán vui tươi, đầm ấm bên gia đình sau hơn 6-7 năm không được ăn Tết Ta ở nhà.

Khi còn đang làm ở Canada năm 2011, tôi đã cùng với vài bạn học thời trước mở một công ty dạng SME chuyên phát triển các Tech Product cho thị trường Việt Nam cũng như gia công phần mềm cho các công ty Việt Nam & nước ngoài.

Tech Product đầu tiên chúng tôi phát triển có tên là YCMS, viết tắt của YGS **CMS (Content Management System – CMS)**, là một hệ quản trị nội dung giúp cho các quản trị viên của các Website có thể thay đổi **Static Content** (Nội Dung Tĩnh) trên các Website nhanh chóng mà không cần nhờ đến Developer lập trình để sửa rồi triển khai trở lại lên máy chủ.

YCMS cung cấp các tính năng phổ biến của một CMS cùng với khả năng mở rộng các tính năng khá dễ dàng:

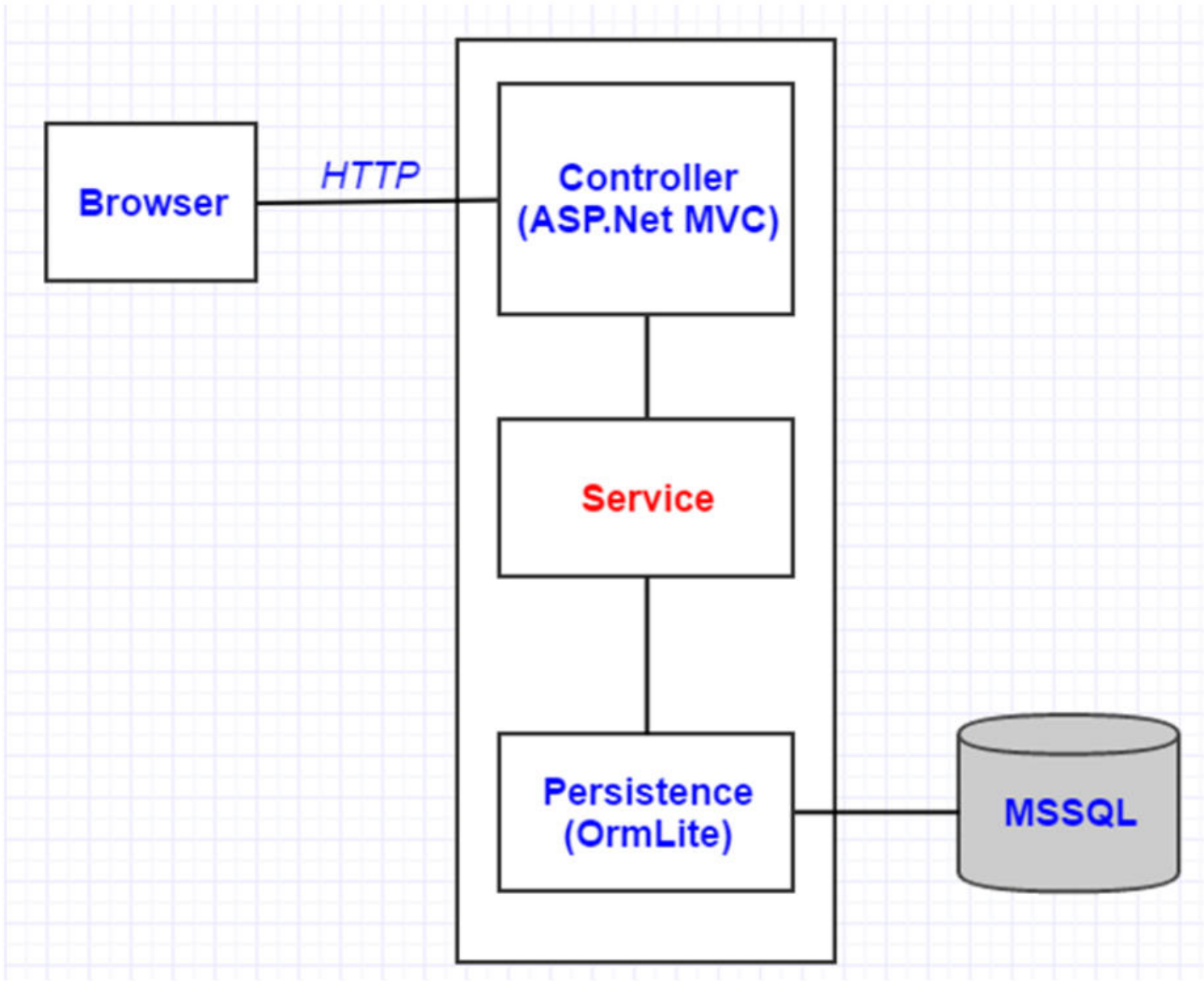
- 1.Quản lý các Website
- 2.Quản lý các Design Template (Mẫu Thiết Kế)
- 3.Quản lý các Page (Trang)
- 4.Quản lý các Section (Khu Vực)
- 5.Quản lý các thành phần được xây dựng sẵn (Component) như HTML, Menu, Language, Photo Gallery, Video Gallery...

Thời điểm đó các phần mềm CMS trên thị trường chưa có nhiều và mạnh, cho nên chúng tôi xác định xây dựng sản phẩm YCMS để giúp cho việc phát triển các dự án phần mềm gia công của công ty chúng tôi nhanh hơn và cũng cho thuê bao YCMS cho những ai có nhu cầu.

Công ty YGS là SME nên khi xây dựng sản phẩm phần mềm thì mọi thành viên công ty đều tham gia tất cả các công đoạn phát triển phần mềm.

Mô hình phát triển YCMS được đặc tả trong hình sau:

YCMS – Vietnam



Hình 102: YCMS

Công nghệ .Net được sử dụng trong sản phẩm này vì bạn tôi trước đó đã xây dựng được 1 bộ khung phát triển sản phẩm công nghệ trên nền .Net rồi.

Loại hình ứng dụng chúng tôi đã chọn là Multiple Page Application – MPA.

10. Dự án YCRM (Vietnam)

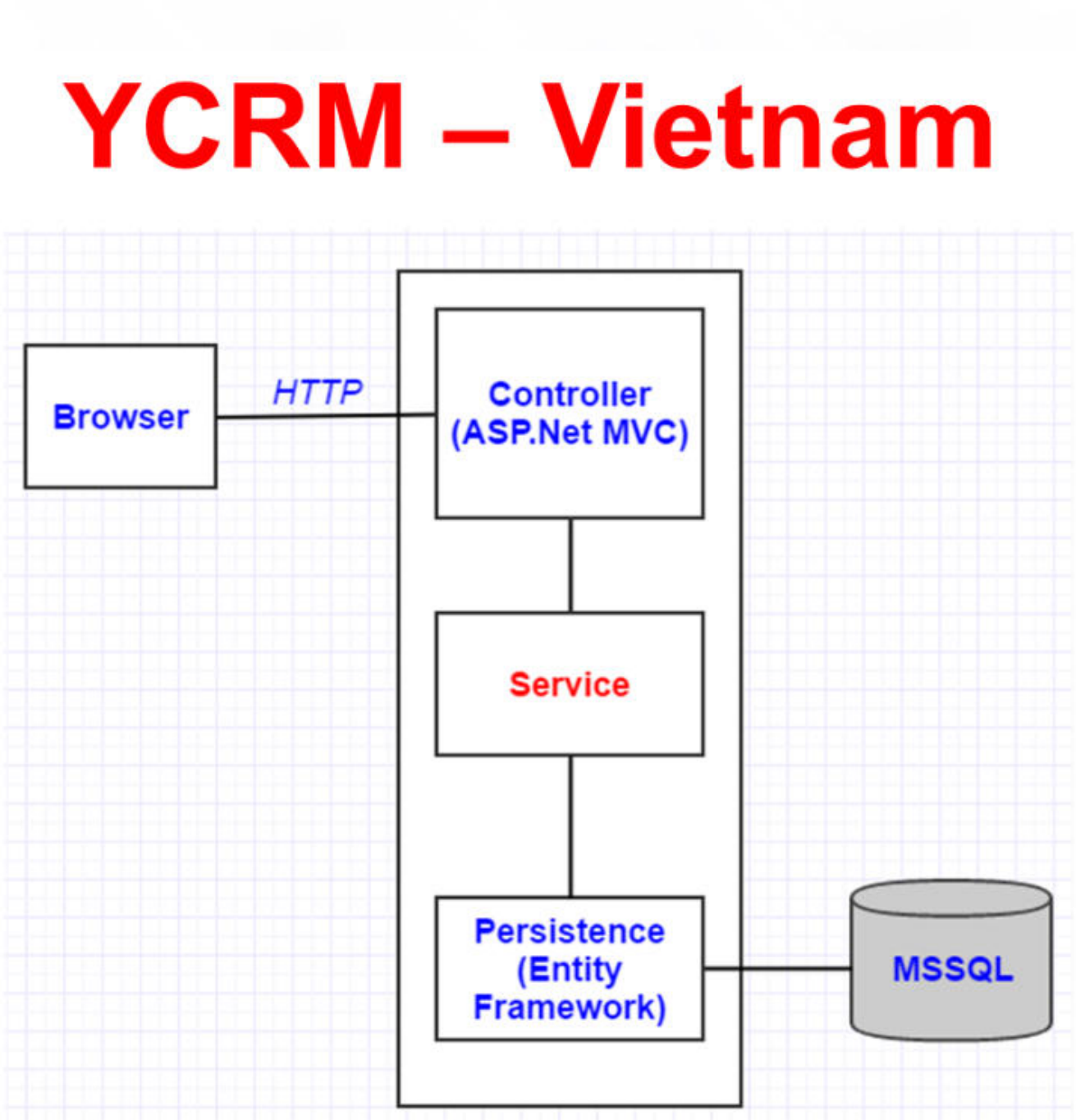
Tech Product thứ 2 mà YGS phát triển có tên là YCRM, viết tắt của YGS **CRM** (**C**ustomer **R**elationship **M**anagement – **CRM**), là một hệ quản lý quan hệ khách hàng của doanh nghiệp.

YCRM cung cấp các tính năng thiết yếu để giúp các doanh nghiệp tìm kiếm, khai thác và nuôi dưỡng các khách hàng của họ hiệu quả nhất:

- 1.Quản lý **Lead** (Đầu Mối)
- 2.Quản lý **Opportunity** (Cơ Hội)
- 3.Quản lý **Case** (Trường Hợp)
- 4.Quản lý **Project** (Dự Án)
- 5.Quản lý **Task** (Tác Vụ)
- 6.Quản lý **Account** (Tài Khoản)
- 7.Quản lý **Contact** (Liên Hệ)
- 8.Quản lý **Customer** (Khách Hàng)

Chúng tôi không xây dựng sản phẩm YCRM từ đầu, mà tham khảo sản phẩm SugarCRM, của công ty SugarCRM của Mỹ, khá phổ biến trên thị trường khi ấy.

Mô hình phát triển YCRM tương tự YCMS, chỉ thay đổi 1 tí ở Persistence Layer:



Hình 103: YCRM

11. Dự án SmartPush (Australia)

Sau khi rời công ty YGS, tôi và vài người bạn mở một công ty Tech Startup ở bên Úc vào năm 2013, trong đó tôi & bạn ở Vietnam tên Hùng lo phần phát triển sản phẩm công nghệ và bạn ở Úc lo phần vận hành & phát triển kinh doanh. Tech Product của chúng tôi đã phát triển có tên là **SmartPush**.

Tình hình khi ấy thì eCommerce đang bùng nổ ở thị trường Úc với số lượng cửa hàng bán hàng Online - eShop mọc lên rất nhiều. Vì thế mảng Data Science, cụ thể hơn là Data Analytics cho các eShop có nhu cầu khá cao.

Lúc đó chỉ có mỗi gã khổng lồ Google có sản phẩm Google Analytics (GA) cung cấp giải pháp Data Analytics cho eCommerce là tương đối mạnh và miễn phí. Tuy nhiên, GA chỉ mới cung cấp một số Metric & Report cơ bản cho eCommerce Website.

Do đó, SmartPush được định hướng xây dựng vừa có các tính năng cơ bản như GA vừa có các tính năng cao cấp hơn nhằm phục vụ tốt hơn nhu cầu phân tích hành vi người dùng của các chủ cửa hàng thương mại điện tử.

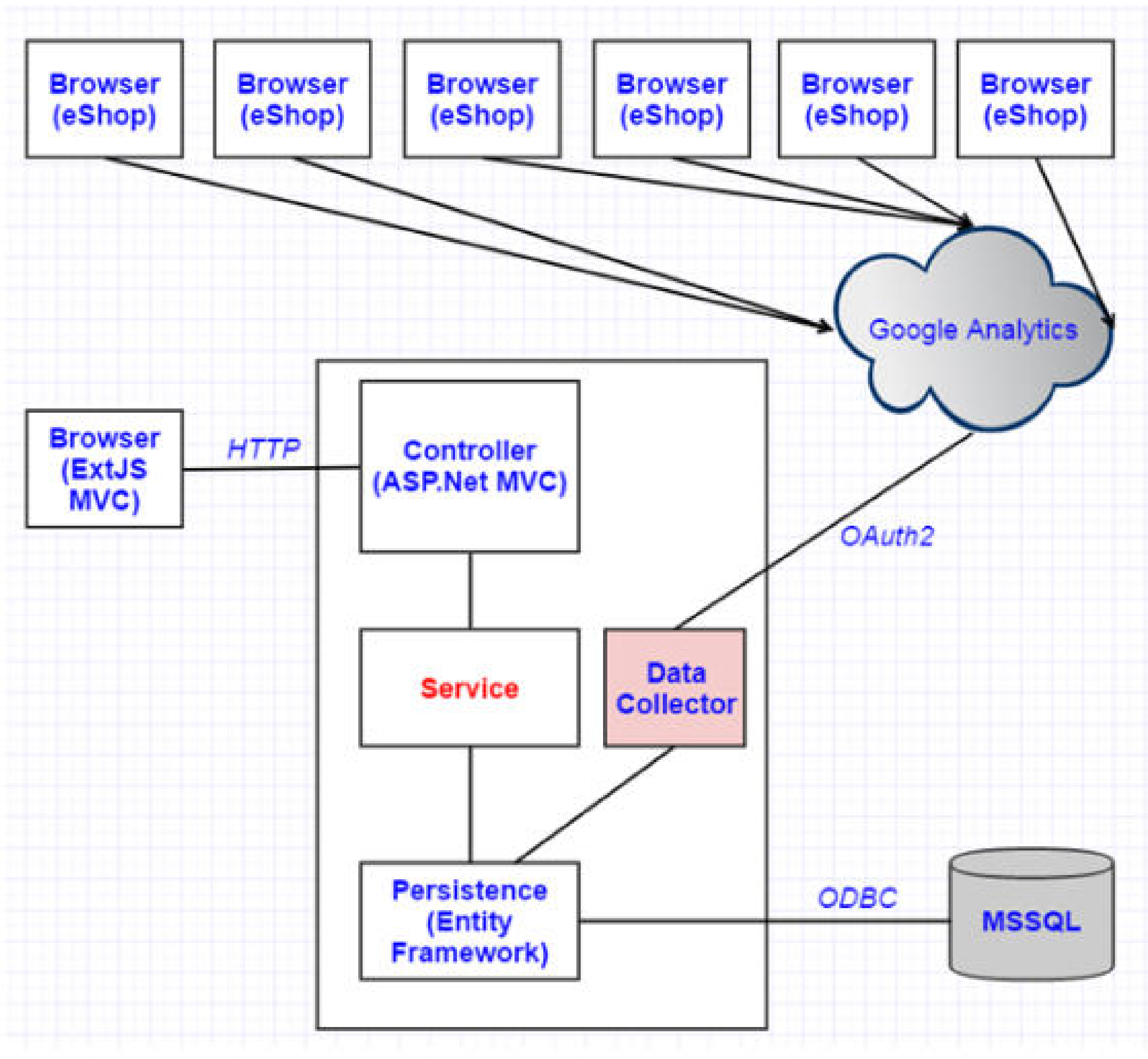
Lúc đầu, chúng tôi nghĩ sẽ xây dựng SmartPush từ con số 0. Sau khi phân tích kỹ thì chúng tôi thấy chi phí thời gian và công sức để làm SmartPush từ đầu khá lớn, rồi khi làm xong thì chi phí để lưu trữ dữ liệu tương tác của người dùng trên các eShop là rất nhiều, phải dùng đến công nghệ Big Data để lưu và xử lý dữ liệu thì cực kỳ tốn kém.

Sau khi tìm hiểu kỹ cách GA hoạt động thì chúng tôi thấy nó cho phép người dùng đẩy **Custom Data** (Dữ Liệu Tùy Biến) lên kho dữ liệu của nó. Từ đây chúng tôi quyết định là nên “đứng trên vai người khổng lồ” để xây dựng giải pháp của mình !

Chúng tôi đã nghĩ ra cách mô hình hóa Custom Data thu thập được từ các eCommerce Website để đẩy lên GA nhờ nó lưu dùm, sau đó chúng tôi định kỳ lấy Custom Data này về xử lý rồi lưu vào Database của SmartPush phục vụ cho việc xây dựng các Advanced/ Custom Metric & Report theo yêu cầu của các chủ cửa hàng thương mại điện tử.

Đây là Development View của SmartPush:

SmartPush – Australia



Hình 104: SmartPush

- Các thử thách mà chúng tôi đã gặp phải trong dự án này:
- 1.Kiến thức lĩnh vực thương mại điện tử & Data Analytics.
 - 2.Công nghệ mới như OAuth2, GA APIs.
 - 3.Môi trường phát triển phần mềm khá phức tạp với nhiều thành phần khác nhau: Website Plugins, Frontend Web for SPA (Single Page App), Backend APIs, Data Collector, Background Processes...
 - 4.Nhân lực Software Developer/ QC

Tôi và Hùng đã chia nhau tìm hiểu kiến thức lĩnh vực, các công nghệ mới và thay nhau hiện thực môi trường phát triển và các thành phần của hệ thống SmartPush.

Vấn đề tìm nhân lực Software Developer/ QC cho Tech Product đúng là nan giải, đã làm mất rất nhiều thời gian và công sức của tôi và Hùng.

Trong giai đoạn phát triển **Minimum Viable Product – MVP** (Sản Phẩm Tối Thiểu Dùng Được), Tech Startup của chúng tôi không có ý định kêu gọi **Angel Investor** (Nhà Đầu Tư Thiên Thần), các **Co-Founder** (Nhà Đồng Sáng Lập) bỏ tài chính và công sức vô sản phẩm. Vì thế chúng tôi không nhắm tuyển Full-Time Senior Developer, chỉ sử dụng một vài Part-Time Senior Developer và tìm vài Full-Time Fresher Developer và Full-Time QC.

Về việc tìm Full-Time Fresher Developer, tôi và Hùng đã mất rất nhiều thời gian để phỏng vấn rất nhiều bạn trẻ IT mới ra trường (mỗi 1 cuộc phỏng vấn mất cỡ 90') nhưng chỉ chọn được một bạn vô làm thử, nhưng làm được 1-2 tháng không chịu nổi thì cũng rút.

Còn Full-Time QC thì tôi kiếm được 1 đồng môn cùng trường Lê Hồng Phong vô làm khá tốt.

Đi cà phê thuê thì mỗi ngày phải bắt buộc có mặt ở công ty 8 tiếng trong 5 ngày/1 tuần, xong việc là xong, cuối tuần nghỉ ngơi thư giãn, còn cà phê cho mình thì không bắt buộc phải có mặt 8 tiếng ở công ty mỗi ngày nhưng thời gian làm là không đếm được, rất nhiều lúc không biết buổi tối hay cuối tuần là gì, cứ cắm đầu làm cho xong việc thì thôi.

Sau khoảng 6 tháng thì chúng tôi cũng hoàn thành MVP của sản phẩm SmartPush với một số Advanced/ Custom Metric & Report hữu dụng cho các chủ cửa hàng thương mại điện tử.

Từ góc nhìn của các chủ cửa hàng thương mại điện tử thì những tính năng này tương đối tốt, giúp ích được cho họ thấy tình hình hoạt động của các eCommerce Website của họ cũng như hành vi của khách hàng tiềm năng/khách hàng hiện tại của họ như thế nào.

Tuy nhiên, cái họ muốn hơn nữa là dựa trên các số liệu này thì họ cần phải làm gì tiếp để tăng doanh thu cho các eCommerce Website của họ?

Đây là bài toán xây dựng **Recommendation System** (Hệ Thống Gợi Ý) bên ngành AI thuộc mảng R&D sẽ tốn rất nhiều thời gian & công sức chứ không đơn thuần là phát triển phần mềm ứng dụng.

12. Dự án SmartIntegrate (Australia)

Thị trường thương mại điện tử tiếp tục phát triển khá mạnh ở thị trường Úc trong năm 2014 với số lượng eCommerce Website lên đến hơn 2 triệu.

Các chủ eShop gặp vấn đề với việc tăng **Traffic** (Lưu Lượng Truy Cập) vào eShop, tăng lượng **Lead** (Khách Hàng Tiềm Năng) cũng như tăng **Order** (Đơn Hàng).

Họ bỏ tiền làm **Search Engine Optimization – SEO** (Tối Ưu Hóa Máy Tìm Kiếm) để tăng Traffic cũng khá tốn kém nhưng Traffic vào cũng không quá nhiều. **Conversion Rate** (Tỉ Lệ Chuyển Đổi), là tỉ lệ số lượng **Visitor** (Khách Viếng Thăm) trở thành **Customer** (Khách Hàng) trên tổng số lượng Visitors, cũng không cao nếu so với những **Sales Channel** (Kênh Bán Hàng) lớn đã có tiếng như Channel Advisor, Magento, eBay...

Vì vậy họ có thêm 1 lựa chọn là đẩy sản phẩm của họ lên các Sales Channel để tăng Traffic, Lead & Order.

Giải quyết tương đối ổn vụ Traffic và bán hàng thì họ đối mặt tiếp với vấn đề thứ 2: Quản lý Order trên các Sales Channel.

- Thử hình dung một chủ eShop đẩy Product lên 10 Sales Channel, cuối tháng họ phải đăng nhập vào 10 Sales Channel này với 10 tài khoản khác nhau để kiểm tra Order.
- Tiếp đó họ phải xuất ra 10 danh sách Order từ 10 Sales Channel cùng với danh sách Order trên eShop của họ rồi gộp chúng lại với nhau để ra 1 danh sách Order tổng hợp của tháng -> Ôi...công việc cơ bắp cuồn cuộn



Vấn đề thứ 2 cũng mệt mỏi đấy nhưng không gay gắt bằng vấn đề thứ 3: Quản lý công việc **Shipment** (Vận Chuyển) đến các **Carrier** (Nhà Vận Chuyển).

Bên Úc thì có các Carrier phổ biến của Úc như eParcel, StarTrack, còn Carrier toàn cầu tại Úc thì có FedEx, DHL...

Tại sao lại gay gắt?

Quy trình chuyển 1 Order sang 1 Shipment để 1 Carrier như eParcel vận chuyển đi khá nhiều công đoạn BẰNG TAY mất rất nhiều công sức & thời gian (cỡ 10'/1 Order nếu làm đúng):

1. Đăng nhập vào website của 1 Carrier (lại phải nhớ tài khoản)
2. Tạo 1 Shipment
3. Nhập 1 nút thông tin của Shipment mới tạo & Order sẽ được vận chuyển
4. Xuất ra 1 Manifest (không biết tiếng Việt gọi là gì trong lĩnh vực này)
5. Cập nhật cái Manifest này lại danh sách Orders để quản lý tiến trình vận chuyển

Tạm tính thử: 10'/1 Order -> 1 tiếng/6 Orders -> 8 tiếng ~ 1 ngày làm việc của 1 người chỉ xử lý được 48 Order mà số lượng Order trong 1 tháng có thể lên hàng trăm thì... chi phí xử lý bằng tay là rất lớn.

Vấn đề gay cấn này cũng xảy ra tương tự khi chủ eShop xuất hoạt động kinh doanh ra các **Accounting Software** (Phần Mềm Kế Toán) như MYOB, QuickBooks...

Các vấn đề mệt mỏi, gay cấn cho chủ eShop là vậy, thế thì có giải pháp nào giúp họ tiết kiệm thời gian và công sức không?

Thật xui cho họ là thị trường Úc khi đó là chưa thấy một giải pháp nào tổng hợp xử lý hết các vấn đề trên, trong khi các Sales Channel thì không cung cấp mấy dịch vụ nhỏ nhặt này.

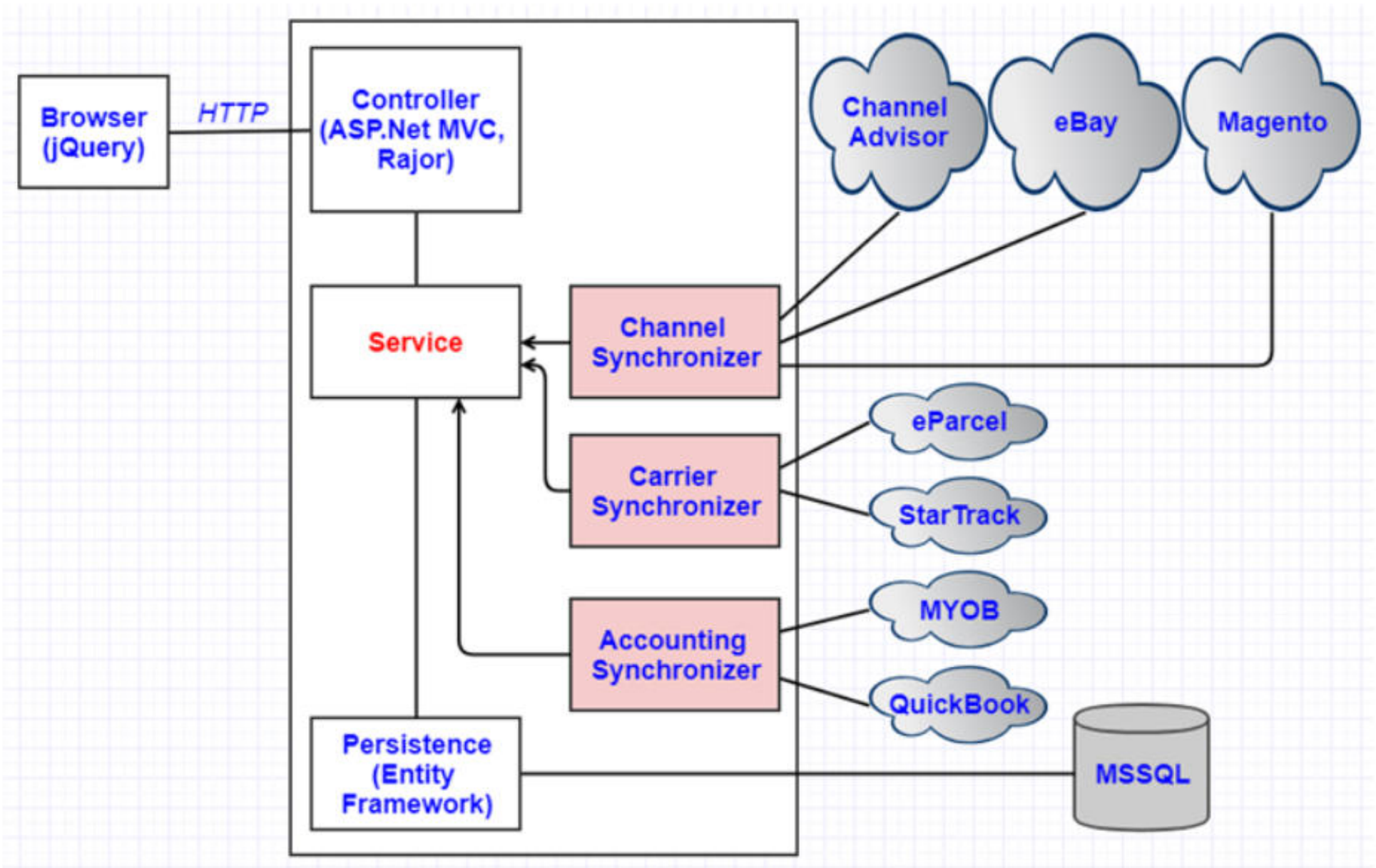
Thế là chúng tôi quyết định xây dựng Tech Product có tên là **SmartIntegrate** nhằm giúp cho chủ eShop thoát khỏi các vấn đề đau đầu bằng giải pháp **one-stop-shopping** (nôm na là chỉ cần đến 1 chỗ là có tất cả những thứ cần):

1. Tích hợp các Sales Channel (Channel Advisor, Magento, eBay, Amazon) về 1 chỗ để cuối tháng chủ eShop chỉ cần đăng nhập vào 1 chỗ với 1 tài khoản phải nhớ là được thấy toàn bộ Order từ các Sales Channel mà giải pháp hỗ trợ.
2. Xuất nhiều Order sang Carrier mà giải pháp hỗ trợ (eParcel, StarTrack) và hoàn toàn tự động với thời gian tính bằng giây/phút.
3. Xuất tình hình kinh doanh sang Accounting Software mà giải pháp hỗ trợ (MYOB, QuickBooks) hoàn toàn tự động và nhanh chóng.

Trong phiên bản MVP, chúng tôi chỉ lên kế hoạch tích hợp các Sales Channel là Channel Advisor, Magento, Carrier là eParcel và Accounting Software là MYOB.

Đây là Development View của SmartIntegrate:

SmartIntegrate – Australia



Hình 105: SmartIntegrate

Những thử thách mà chúng tôi đã gặp phải:

1. Kiến thức lĩnh vực thương mại điện tử và tích hợp hệ thống các Sales Channel. Chúng tôi đã mất rất nhiều thời gian tìm tòi và xây dựng phần ánh xạ Data Model từ nhiều Sales Channel về Domain Model của SmartIntegrate cho đồng nhất.
2. Phần hiện thực **Data Synchronization** (Đồng Bộ Dữ Liệu) giữa các Sales Channel với SmartIntegrate cực kỳ phức tạp. Một ví dụ điển hình là khi SmartIntegrate lấy Order về từ các Sales Channel thì các Order này có thể được chỉnh sửa ở trên Website của Sales Channel và cũng như trên SmartIntegrate, có Order sửa lúc chưa vận chuyển và lúc đã vận chuyển. Còn có trường hợp 1 Order đã được lấy về SmartIntegrate từ 1 Sales Channel, sau đó nó được cắt ra thành nhiều Sub-Order trên Sales Channel đó để vận chuyển nhiều lần vì chưa có đủ Product để mà có thể vận chuyển 1 lần duy nhất... Việc đồng bộ dữ liệu giữa Carrier/ Accounting Software với SmartIntegrate thì đỡ hơn Sales Channel tí mà cũng rất phức tạp.
3. Order có nhiều trạng thái, Shipment cũng vậy và liên quan đến trạng thái của Order nữa, đem tổ hợp 2 phần tử này lại cho ra rất nhiều trạng thái phải xử lý, rất dễ sai sót.
4. Rất là tốn thời gian chạy kiểm tra SmartIntegrate sau khi lập trình xong một tính năng nào đó vì liên quan đến tầm lum hệ thống bên ngoài, phải đăng nhập rất nhiều trang web, coi dữ liệu nhiều nơi, so sánh tầm lum thứ, rất là dễ sai. Phương pháp kiểm tra tự động (Automation Test) trong ngữ cảnh này cũng không là 1 giải pháp tốt.

5. Các Sales Channel, Carrier, Accounting Software chưa hỗ trợ REST, phải dùng SOAP -> lại gặp ác mộng với 1 núi XML. Cũng hên là sau đó một số Sales Channel chuyển dần từ SOAP qua REST. Tuy nhiên cái thên eBay lúc ấy vẫn là SOAP, mà eBay API siêu khó hiểu và rắc rối kinh khủng: muốn tạo 1 Order trên eBay phải chạy...10 API với 1 đống XML trong khi trên Channel Advisor & Magento chỉ cần 2 API.

6. Vấn đề tìm nhân lực Full-Time Fresher Developer.

Sau khi hoàn thành MVP, chúng tôi tiếp tục lên kế hoạch tích hợp các Sales Channel là eBay, Amazon, Carrier là StarTrack và Accounting Software là QuickBooks trong các Version sau.

13. Dự án Awesm (USA)

Đầu năm 2015, tôi vào làm ở công ty NashTech, chi nhánh gia công phần mềm tại Việt Nam của tập đoàn Harvey Nash (chuyên về mảng Head Hunt). Tại đây tôi gặp lại chị Loan (từng làm chung trong dự án SCS ở công ty TMA năm 2004) đang làm CTO của Nash Tech.

Tôi tham gia vào nhóm SWAT (trong phim Mỹ thì SWAT là đội phản ứng nhanh) chuyên hỗ trợ các dự án IT đang gặp vấn đề.

Khi đó Nash Tech có 1 khách hàng rất lớn tên là Unified, một công ty Tech Startup của Mỹ được đầu tư 14 triệu USD vào tháng 06/2012 bên mảng Digital Marketing, đang xây dựng 1 nền tảng dữ liệu lớn (Big Data) cực to tên là Unified Platform (UP) dựa trên vai các người khổng lồ mạng xã hội như Facebook, Twitter, LinkedIn...

Chức năng chính của UP là thu thập dữ liệu từ các mạng xã hội về lưu trữ rồi xuất ra các **Report** (Báo Cáo), **Statistics** (Thống Kê) tổng hợp cho dân Digital Marketer sử dụng. Thật ra các mạng xã hội cũng có làm các báo cáo, thống kê riêng của họ, tuy nhiên các báo cáo, thống kê đó không đáp ứng được những nhu cầu cao cấp hơn của Digital Marketer bằng UP.

Unified mở 1 **Offshore Development Center – ODC** (Trung Tâm Phát Triển Bên Ngoài) tại NashTech, có nghĩa là NashTech sẽ tuyển nhân lực IT theo yêu cầu của Unified. Những người này sẽ ngồi tại văn phòng của NashTech ở Việt Nam làm việc từ xa cho Unified thông qua **Virtual Private Network – VPN** (Mạng Riêng Ảo).

Về mặt luận lý thì những người này là nhân viên của Unified làm chung dự án với các nhân viên khác của Unified ở Mỹ và trên thế giới, nhưng về mặt vật lý thì họ là nhân viên của NashTech do NashTech tự tuyển vào, ký hợp đồng lao động theo Luật pháp Việt Nam.

Tôi được cử vào Unified ODC ở NashTech để hỗ trợ 1 Tech Product có tên là **Awesm** được Unified mua lại từ 1 Tech Startup của Mỹ (cũng tên là Awesm).

Tech Startup Awesm (chơi chữ trên từ English “awesome” có nghĩa là tuyệt vời) được đầu tư 1 triệu USD (**Seeding Round**) vào tháng 12/2010 để xây dựng sản phẩm công nghệ Awesm dựa trên 2 mạng xã hội lớn là Facebook/Twitter cho dân Digital Marketer với ý tưởng chính:

- Từ 1 URL dài của 1 trang web hay ho nào đó, Awesm sẽ tạo ra 1 **Short Link** (Link Ngắn) có dạng “awe.sm/xy126” (domain “awe.sm/” + 1 chuỗi được sinh ra ngẫu nhiên).

- 1 Digital Marketer D nào đó sẽ đem short link “awe.sm/xxy126” này share trên Facebook/Twitter của D
- Bạn của D trên Facebook/Twitter bấm vào short link trên, họ sẽ được đưa qua hệ thống Awesm rồi về URL dài ban đầu, Awesm sẽ đếm đã có 1 click trên short link trên.
- D vô phần quản trị của Awesm sẽ thấy được báo cáo, thống kê của các short links mà D đã tạo ra và chia sẻ trên Facebook/Twitter để biết được những short links nào được nhiều người quan tâm/tương tác.
- Nếu URL dài là 1 trang web thương mại điện tử, Awesm sẽ tính được luôn Conversion Rate cho D, đây là 1 thông số bên mảng Data Analytics.

Với sự thành công ban đầu của sản phẩm Awesm, tháng 12/2011, startup Awesm gọi vốn thành công vòng Early VC (**Venture Capital**) thêm 4.3 triệu USD để phát triển thêm các tính năng đột phá thị trường để rồi đến tháng 5/2014, startup Awesm được Unified mua với giá không được công bố, nhưng theo định giá thì startup Awesm có giá từ 17 – 26 triệu USD.

Unified mua startup Awesm với mục tiêu tích hợp sản phẩm Awesm vào UP của họ. Sau khi mua xong Awesm, họ mời 1 ông Technical Director (TD) của Oracle tên Will về làm TD để vừa làm trên sản phẩm UP vừa nắm sản phẩm Awesm để tích hợp vào UP.

Tuy nhiên đến tháng 2/2015 mà ông này vẫn chưa tích hợp được gì bởi vì các lý do sau:

- Không giữ được bất cứ Software Developer nào cũ của startup Awesm và tài liệu của sản phẩm rất ít lại rời rạc nên không ai hiểu rõ bên trong hệ thống Awesm hoạt động ra sao.
- Vận hành hệ thống Awesm tốn rất nhiều thời gian và công sức do Production Environment không có tài liệu đặc tả cách triển khai.
- Công việc bên Unified quá nhiều, ông Will không có nhiều thời gian cho vụ tích hợp.

Vì vậy ông Will mới đẩy cục Awesm này về Unified ODC ở NashTech và tôi là người hứng nguyên đồng búi nhúi khổng lồ này 😂.

Trong buổi phỏng vấn vô dự án Awesm, ông Will hỏi tôi tùm lum thứ trên trời dưới đất suốt 90’ với cái giọng New York của ông hơi bị khó nghe, mà ông lại vừa nói nhanh vừa nuốt chữ nhiều nữa chỗ, cũng may tôi có vài năm làm việc trong công ty nói English ở Canada nên từ từ cũng ổn, nhưng thiệt là khá đuối với ông luôn.

Phỏng vấn xong là 11h30 sáng, tôi định ra ngoài kiếm gì ăn trưa trong khi ông Will phỏng vấn tiếp 1 bạn Senior Developer của NashTech để vô dự án Awesm làm chung với tôi.

Nhưng tôi vừa định đi thì thấy ông Will đặc tả bài toán trong 2' mà bạn ấy không hiểu nổi do nghe không ra những gì ông ấy nói, thế là tôi phải ở lại vẽ lên bảng bài toán, xong làm thông dịch viên bất đắc dĩ cho họ 😊. Và rồi ông Will cũng nhận bạn đó vô dự án Awesm.

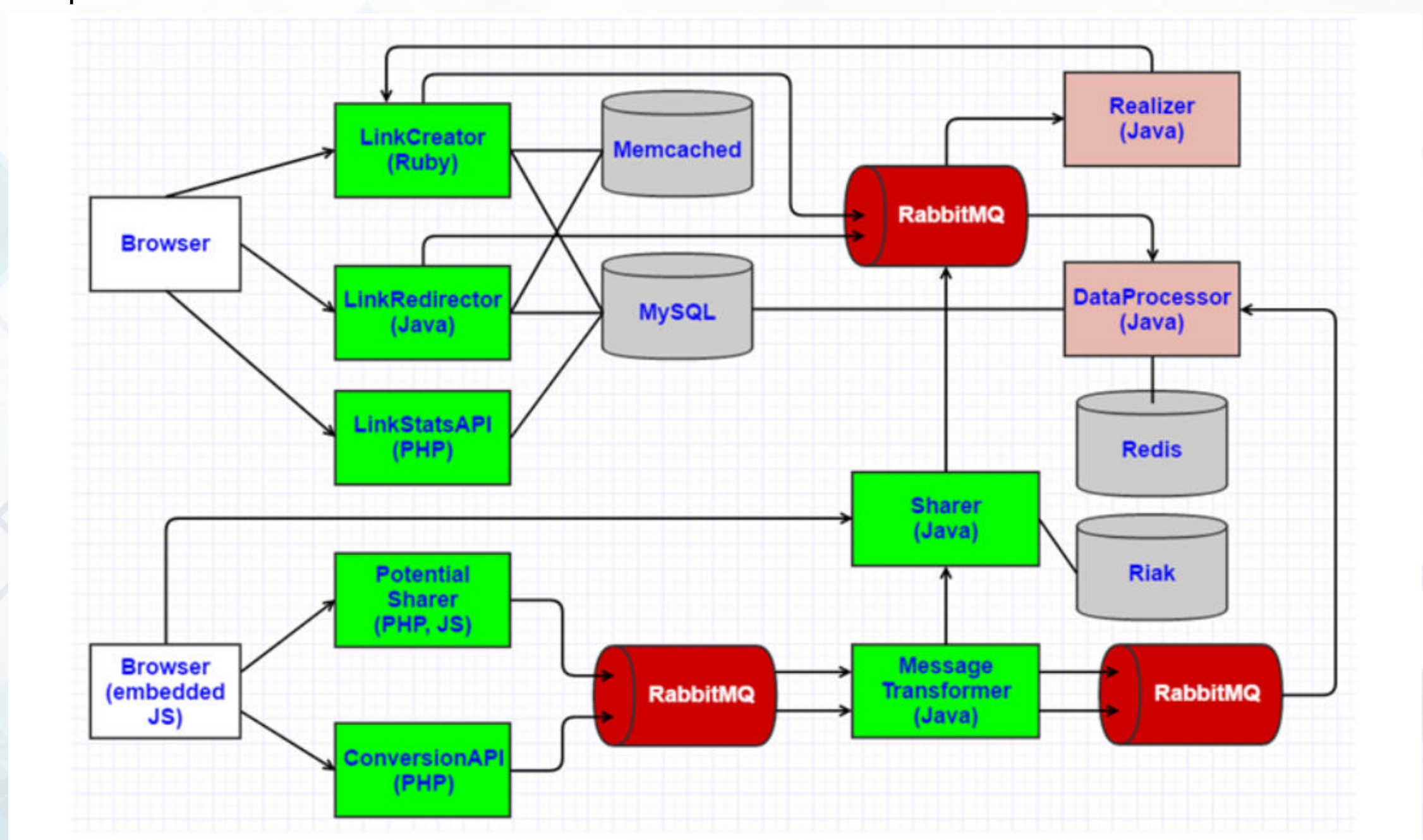
Thế là Awesm Team chính thức được lập trong Unified ODC ở NashTech với 2 thành viên ban đầu: tôi và 1 Senior Developer.

Nhiệm vụ đầu tiên của Awesm Team là thực hiện Reverse Engineering, để hiểu được Awesm System đang hoạt động thế nào.

Những thử thách mà Team tôi đã gặp phải với dự án Awesm:

- Kiến thức lĩnh vực Digital Marketing, Short Link
- Quá nhiều dự án con: **30+ Github Project**. Các Module này được viết trên nhiều Programming Language như **Ruby, PHP, Java, Javascript** với nhiều công nghệ mới như **Dropwizards, Snowflake** (của Twitter), **RabbitMQ, Memcached, Riak, Redis**.
- Tài liệu của dự án quá ít, rời rạc, viết không chi tiết, chủ yếu là viết kiểu ghi chú dạng **Bullet Point** (Gạch Đầu Dòng) để Software Developer cũ đọc chứ không dành cho người ngoài dự án đọc. Đây cũng là điều dễ hiểu bởi tính chất của Tech Startup là phải phát triển nhanh, hầu như không có thời gian nhiều đầu tư cho việc viết tài liệu.
- Quá nhiều Server: các Module được triển khai lên tổng cộng **150 AWS (Amazon Web Services) EC2 Instance**, và cũng không hề có tài liệu đặc tả mô hình triển khai thế nào.

Sau cỡ 3 tháng mò mẫm đọc tài liệu, vọc code, UI, thì chúng tôi vẽ được Development View:



Hình 106: Awesm

Hình trên chỉ thể hiện những Module chính của Awesm, cỡ 12 Github Project.

- Các Module màu **xanh lá cây** là các **Application Programming Interface – API** (Giao Tiếp Lập Trình Ứng Dụng) hoặc các **Message Consumer** (Thành Phần Xử Lý Thông Điệp) từ các **Queue** (Hàng Đợi) màu **đỏ**.
- Các Module màu **hồng nhạt** là các **Background Process** (Tiến Trình Ngầm) để xử lý các luồng nghiệp vụ hoặc dữ liệu.
- Quan hệ và tương tác giữa các Module khá là phức tạp và rắc rối, đã vậy còn theo mô hình **Messaging Integration** (Tích Hợp Bằng Thông Điệp), 1 trong 4 phương pháp **System Integration** (Tích Hợp Hệ Thống), có tính chất **Asynchronous Processing** (Xử Lý Bất Đồng Bộ) nên cực kỳ khó debug.

Bước tiếp theo là ông Will muốn team tôi chuyển 1 Module viết bằng Ruby & 1 Module viết bằng PHP sang Java và bắt đầu vọc trên Production Environment để hiểu cách các Module được triển khai.

Hai Module Ruby & PHP này được phát triển đầu tiên khi startup Awesm bắt đầu. Họ chọn 2 ngôn ngữ này có lẽ họ muốn ra sản phẩm nhanh và Team của họ khi đó có kiến thức/ kinh nghiệm trên 2 ngôn ngữ này.

Những Module khác, sau khi họ gọi được vốn đầu tư, được phát triển trên nền tảng Java, có thể họ mong muốn sự ổn định. Vì thế, việc ông Will muốn chuyển 2 Module trên sang Java cũng là bước đầu trong kế hoạch tạo sự đồng nhất và ổn định cho sản phẩm Awesm trước khi tích hợp vào UP.

Việc vọc trên Production Environment với 150 AWS EC2 Instance có vẻ quá khả năng của Team tôi khi ấy, cho nên SWAT Team của NashTech đã cử 1 bạn SWAT tên Phú chuyên về DevOps vô hỗ trợ Team tôi bán thời gian. Phú có kiến thức DevOps rất tốt và kinh nghiệm khá nhiều, từng quản lý 200+ servers của vnexpress.net, một trong những trang báo điện tử lớn nhất Việt Nam. Đây là một sự bổ sung rất cần thiết và đúng thời điểm cho Team tôi.

Thật là một thử thách không hề nhỏ cho Team tôi, vốn là dân Java và không ai có kiến thức/kinh nghiệm về Ruby & PHP, khi đọc code Ruby & PHP để suy đoán nghiệp vụ, Logic rồi xây dựng lại giống vậy ở 2 Module mới bằng Java. Quá trình chuyển đổi này ngốn Team tôi cũng khá nhiều thời gian và công sức.

Trong thời gian làm việc chuyển đổi nền tảng, tôi vẫn tiếp tục tìm hiểu tiếp các tính năng của toàn bộ sản phẩm Awesm cùng với sự hỗ trợ đặc lực của Phú đi mò các kết nối giữa các AWS EC2 Instance trên Production Environment để lên được danh sách thông tin chi tiết, tình trạng các AWS EC2 Instance cũng như những Module nào đang chạy trên mỗi Instance, chi phí cho toàn bộ Instance (ước tính cỡ 90kUSD/1 năm, cũng khá chất).

Tính năng nổi bật nhất của Awesm mà tôi mất nhiều thời gian và công sức nhất để hiểu được, chắc cũng là tính năng giúp Awesm gọi vốn Early VC thành công, chính là **Dark Social** (có thể tạm dịch là Xã Hội Đen 🕶️).

Mới nghe cái tên thì đã gợi ra cái gì đó tiêu cực, nhưng không, đây là 1 khái niệm rất hay. Ý tưởng như sau:

- 1 người A chia sẻ 1 URL nào đó (ví dụ: <http://adtmag.com>) trên Facebook/Twitter.
- Bạn Facebook/Twitter của A bấm vô URL đó thì Facebook/Twitter đếm được số lượng **Click** (Bấm) của URL này xảy ra trên Facebook/Twitter.
- Nếu bạn của A thấy URL này hay quá, họ gửi URL tiếp cho bạn của họ là C,D,E qua SMS, email hay các kênh khác ngoài Facebook/Twitter.
- Khi C,D,E bấm vô URL nhận được thì số lượng click này sẽ không được Facebook/Twitter tính vì 2 mạng xã hội này không hề biết vụ chia sẻ này.
- Nếu A chuyển URL sang short link của Awesm rồi mới chia sẻ lên Facebook/Twitter (thay vì chia sẻ trực tiếp URL) thì Awesm sẽ đếm được số click của C,D,E xảy ra bên ngoài Facebook/Twitter luôn.

Tính năng Dark Social được hiện thực dựa trên 1 khái niệm rất là nhũn não: **Potential Share**. Hiểu được khái niệm này chính là chìa khóa để hiểu được tính năng Dark Social. Sau khi tốn khá nhiều nơ-ron để hiểu được khái niệm này thì tôi phải thành thật công nhận người đưa ra được khái niệm này rất ư là tuyệt vời. Để hiểu được giá trị của Dark Social đem lại thế nào cho Digital Marketer, xem thử 1 ví dụ có tính định lượng sau:

- 1 Digital Marketer A có 1 URL X chia sẻ trên Facebook được 100 click sau 1 ngày.
- Trong các bạn A đã click trên X, họ gửi X qua email cho bạn của họ thì X đã được click thêm 900 lần.
- Như vậy thực tế là URL X đã được click **1000** lần. Tuy nhiên A chỉ thấy X được **100** click trong phần Facebook Admin, 900 click kia A không hề biết.
- Giả sử sếp của A giao **Key Performance Indicator – KPI** (Chỉ Số Năng Lực Chính) cho A là phải đạt được 1000 click cho X và giá của 1 click mà A phải trả cho Facebook là 0.1\$/click -> A được cấp ngân sách là 100\$.

- Quay lại tình huống trên, để được 1000 click thì A có thể chờ thêm 9 ngày nữa hoặc chia sẻ thêm các bài khác có chứa X -> cuối cùng để được 1000 click thì A sẽ tốn hết 100\$ và thời gian là hơn một ngày.
- Nếu A sử dụng tính năng Dark Social của Awesm thì chỉ mới 1 ngày thì A đã đạt được ngay 1000 click và chỉ tốn cho Facebook 10\$ (cho 100 click trên Facebook), còn lại 90\$ hiển nhiên A sẽ trích ra 1 ít trả cho dịch vụ của Awesm -> A sẽ không bị mất hết 100\$ và đạt được KPI chỉ sau 1 ngày.

Nếu mà nghe ai nói ý tưởng Dark Social từ đầu thì sẽ thấy đơn giản, dễ hiểu, nhưng đi ngược lại từ đuôi (phần hiện thực), cực kỳ phức tạp liên quan đến cả đồng Module & RabbitMQ Queue, để lần mò ra hiểu được ý tưởng thì không hề đơn giản tí nào.

Khi vừa bắt đầu chạy thử 2 Module mới xây xong để đánh giá, kiểm tra xem thế nào thì 1 tin khá sốc xảy đến với Team tôi: Unified mời 1 ông mới tên là Shawn về làm CTO, quản lý cả 2 sản phẩm UP, Awesm thay cho bác Will.

Ông Shawn vừa vào Unified là cho Team tôi ngưng ngay việc chuyển đổi 2 Module kia sang nền tảng Java, tuyển thêm khá nhiều người vào Team tôi đảm nhiệm đủ vai trò của 1 Scrum Team: BA, FE Developer, BE Developer, QC, DBA, còn bên Unified ở New York thì tuyển 1 **Product Manager** mới tên là Evan vào quản lý sản phẩm Awesm, bắt đầu việc tích hợp sản phẩm Awesm vào UP ngay và luôn (tính năng Dark Social đóng vai trò chính yếu), đồng thời tìm giải pháp cắt giảm chi phí cho AWS EC2 Instance (tắt các Instance không còn dùng hoặc không chứa Module quan trọng cho việc tích hợp...) của Awesm.

Chà, thay đổi thật sự hơi đột ngột nhưng được cái theo hướng có vẻ tích cực hơn!

Ngay khi vừa ổn định nhân lực mới cho Team tôi cũng như 1 phần nhân lực từ sản phẩm UP trong Unified ODC hỗ trợ cho việc tích hợp, ông Shawn cùng với 1 bà COO & 1 Engineering Manager sang thăm Unified ODC.

Project Manager của Unified ODC tại NashTech đã tiếp đón các sếp Unified rất nồng hậu, vui vẻ với 1 buổi tiệc hoành tráng khởi động dự án tích hợp Awesm vào UP.

Tới thời điểm này thì tôi gần như hoàn tất việc Reverse Engineering: vẽ được Logical View, Development View, Process View, phân chia được Features/User Stories, chỉ còn cái Physical View (chưa vẽ nổi nhưng Phú đã liệt kê xong danh sách chi tiết các Instance).

Cho nên sẵn dịp các sếp Unified có mặt ở Nash Tech, tôi đã thuyết trình chia sẻ toàn bộ kiến thức về sản phẩm Awesm cũng như demo tương ứng, ông Shawn cực kỳ hứng thú với tính năng Dark Social & khái niệm Potential Share vì ông Shawn mới vào Unified, khi nhận nhiệm vụ quản lý sản phẩm Awesm chắc ông Shawn chỉ mới biết qua ý tưởng tổng quan này mà chưa biết được nó được hiện thực cụ thể ra sao và chạy trong thực tế thế nào.

Khi các sếp Unified trở về lại New York thì Team tôi cũng lao vào thực hiện việc tích hợp Awesm vào UP cho đến cỡ tháng 4/2016 thì xong được giai đoạn 1, đã tích hợp xong những phần cốt lõi. Tôi cũng hoàn thành việc vẽ cái Physical View của Awesm, phức tạp khủng khiếp với toàn bộ 150 AWS EC2 Instance, ngón của tôi hết 1 tháng trời.

Tháng 4/2016 cũng là lúc Unified tổ chức sự kiện gọi là “Strategic Planning Meeting” diễn ra 2 lần/năm để cho tất cả các nhóm chủ chốt của Unified thảo luận lên kế hoạch chiến lược phát triển các sản phẩm của startup (tổng cộng 4 sản phẩm, trong đó có phần tích hợp Awesm & UP). Thế là Unified mời tôi và Tín (TL dự án UP của Unified ODC ở NashTech) qua văn phòng của Unified ở New York trong 2 tuần vừa công tác vừa tham gia sự kiện luôn.

Được làm việc ở văn phòng và tham gia “Strategic Planning Meeting” của startup triệu đô là một trải nghiệm tuyệt vời của tôi. Môi trường làm việc của Unified thân thiện, rất là thoải mái, trang trí khá đẹp, có War Room cho mọi người vô đó tranh luận nảy lửa, có hình vẽ về thiết kế nằm rải rác khắp nơi trên kiếng, trên tường... đúng tinh thần của startup.

Có 1 điểm đặc biệt là cứ đến chiều cỡ 5PM là nhiều nhân viên Unified mở tủ lạnh (chứa mấy chục loại bia khác nhau sản xuất ở Mỹ) lấy mỗi người 1 chai bia ra phòng thư giãn vừa cùng chai vừa bàn luận công việc thay vì ngồi trên máy tính hay trong phòng họp.

Tôi cũng đã uống thử vài chai bia địa phương của Mỹ, có vài loại vị hơi kỳ, rất ư là khó uống, một số loại thì uống cũng ngon.

Khi vào giai đoạn “Strategic Planning Meeting” diễn ra suốt 1 tuần từ thứ 2 đến thứ 6, mỗi ngày Unifieders họp từ 9h sáng đến 6-7h tối theo nhiều chủ đề khác nhau, các người thuyết trình nói ào ào như gió (kỹ năng thuyết trình tuyệt đỉnh), dân chúng ở dưới nghe xong ai cũng giơ tay phát biểu, trao đổi, tranh luận nảy lửa nhưng mang tính tích cực, trên tinh thần xây dựng.

Riêng tôi và Tín là khách mời, tham gia 1 phần nhỏ trong các sản phẩm của Unified nên cũng chỉ phát biểu một vài câu chứ không nói nhiều như những người còn lại.

Phần thuyết trình ấn tượng nhất với tôi là của ông Shawn CTO. Theo tôi xem trên LinkedIn về Profile của ông Shawn thì ông này đã kiêm qua rất nhiều vai trò từ kỹ thuật (CTO) cho đến giám đốc quản lý (Managing Director), đã đầu tư (Funding), xây dựng (Building), mua & bán (Buying & Selling) các công ty công nghệ khá lớn trên thế giới.

Những kế hoạch ông Shawn trình bày đều dựa trên những dữ liệu thực tế từ **Market Research** (Nghiên Cứu Thị Trường), từ team Marketing/Sales, từ team Data Science, các công thức tính toán chi tiết khá phức tạp gắn liền với các mục tiêu tăng trưởng của công ty & **Return On Investment – ROI** (Chỉ Số Lợi Nhuận Trên Đầu Tư). Lúc này tôi đoán ra một phần nguyên nhân mà ông Will bị Unified cho nghỉ việc có lẽ là chưa quan tâm đúng mức đến cái ROI: kế hoạch của Will là bỏ chi phí làm cho sản phẩm Awesm đồng nhất qua nền tảng Java (thuần túy kỹ thuật mà chưa đem lại lợi ích kinh doanh) trước khi tích hợp vô UP dẫn đến thực tế là Unified đầu tư mua Awesm gần cả năm mà chưa thu lại được gì. Vì thế ông Shawn vừa về là cho làm phân tích hợp ngay để dự định bán dịch vụ của UP đi kèm với tính năng cộng thêm (Addon) Dark Social của Awesm.

Cũng trong chuyến đi này, tôi đã có dịp chia sẻ trực tiếp với một số nhân viên Unified về toàn bộ nghiệp vụ, các lược đồ, Features/User Stories, các chi tiết thiết kế, kỹ thuật, tài liệu của Awesm mà Team tôi đã tìm hiểu và làm được. Họ rất cảm kích và đánh giá cao những nỗ lực của Team tôi đã bỏ vào sản phẩm Awesm của họ.

Sau chuyến công tác này về, Team tôi tiếp tục thực hiện thêm một số tính năng mới cho phần tích hợp cỡ vài tháng nữa thì đóng gói cho Unified tung ra chào hàng cho khách hàng của họ.

14. Dự án UP (USA)

Phân tích hợp Awesm vào UP kết thúc cũng là lúc Team tôi chuyển qua làm dự án UP chung với team của Tín và hỗ trợ bảo trì phân tích hợp khi cần.

Sản phẩm UP được xây dựng trên kiến trúc micro-services, một công nghệ mới nổi khi ấy, và công nghệ Big Data với Framework nổi tiếng là Hadoop.

Ý tưởng của UP là đi gom dữ liệu của hầu như tất cả mạng xã hội lớn (Facebook, Twitter, LinkedIn...) để xử lý rồi cung cấp các thông tin, những gợi ý có giá trị cho Digital Marketer giúp họ đưa ra những quyết định về đầu tư cho việc quảng cáo trên các mạng xã hội.

Phần phức tạp nhất của UP chính là **Data Collector** (Bộ Thu Thập Dữ Liệu) từ các mạng xã hội (và hệ thống Awesm khi phân tích hợp được thực hiện), sau đó dữ liệu đi qua 1 **Data Pipeline** (Ống Dữ Liệu) được hiện thực bởi các frameworks Spark và Hadoop trước khi được đổ vào cơ sở dữ liệu **AWS Redshift**.

Các ngôn ngữ lập trình được sử dụng trong UP:

- **Go** (hoặc **Golang**): cung cấp các API hiện thực các luồng nghiệp vụ chính cho phần tương tác với người dùng.
- **Java**: hiện thực phần thu thập và xử lý dữ liệu bên dưới hệ thống.
- **Ruby**: cung cấp API hiện thực phần trích xuất và xử lý các báo cáo, thống kê để hiển thị cho người dùng.

Ngoài ra ở phía Frontend, Framework nổi tiếng **ReactJS** của Facebook được sử dụng trên nền ES5. Kiến trúc **Fluxxor** hỗ trợ việc quản lý trạng thái của UIs và ứng dụng.

Tháng 8/2016, Unified tổ chức sự kiện “Strategic Planning Meeting” lần 2 trong năm 2016, tôi và Tín lại được mời tham dự tiếp. Thế là hai anh em đi công tác 2 tuần nữa, nhưng lần này chúng tôi trải nghiệm nhiều hơn ở New Jersey trong thời gian ngoài công việc.

Cũng giống như lần trước, các cuộc họp lần này cũng diễn ra với mật độ dày đặc và liên tục suốt 1 tuần từ thứ 2 đến thứ 6. Lần này thì các chủ đề về kỹ thuật được thảo luận nhiều hơn.

Trong đó có 1 ngày rất nổi bật là nguyên nhóm kỹ thuật + CTO + COO tham gia thảo luận, tranh luận sôi nổi 1 chủ đề suốt cả ngày, đó là: **Authorization** (Phân Quyền) trên hệ thống UP. Chỉ có ngày này là chúng tôi tham gia thảo luận được nhiều nhất so với các ngày khác.

Kết thúc chuyến công tác lần 2 này, tôi trở lại NashTech làm với Team tôi được vài tháng thì tôi rút khỏi Unified ODC, quay về lại với SWAT team. Việc tham gia 2 dự án Awesm & UP giúp tôi học hỏi được rất nhiều kiến thức cũng như tích lũy thêm nhiều kinh nghiệm quý giá trong việc phát triển các Tech Product lớn.

Tiếp đó, tôi đã hỗ trợ thêm 2 dự án khác ở Mỹ bên mảng Digital Marketing (nền tảng cho người xem quảng cáo Online được trả tiền) & Social Service (hệ thống quản lý các tiệm làm móng tay ở Mỹ) trước khi rời NashTech để tìm kiếm thử thách khác.

15. Dự án P4SVVN (Vietnam)

Trước khi tôi tham gia hỗ trợ dự án Awesm thì SWAT Team có phân công tôi hỗ trợ 1 dự án nhỏ khác cho khách hàng là công ty Struts của Úc có liên quan đến công nghệ Java.

Dự án đó cần thêm Java Developer. Công ty NashTech giới thiệu 5 Java Developer từ 1-2 năm kinh nghiệm đang ngồi trong hàng đợi của nhánh Java trong công ty (tức là những bạn này đang rảnh, chưa có tham gia dự án nào) cho tôi phỏng vấn thử trước, nếu bạn nào vượt qua được thì sẽ giới thiệu cho khách hàng phỏng vấn. Kết quả là các bạn chỉ đạt được 1/5 & 2/5 điểm phỏng vấn nên không ai được giới thiệu cho khách hàng phỏng vấn tiếp.

Chị Loan nhận được kết quả này thì chị ấy đã trách tôi là phỏng vấn khó quá thì sẽ không có người để tham gia dự án cho Team tôi đang hỗ trợ đâu. Tôi cũng chia sẻ thật lòng với chị ấy là tôi không hề khó khăn trong phỏng vấn. Tôi chỉ hỏi những kiến thức nền tảng chứ không có gì cao siêu, và hỏi bằng tiếng Việt chậm rãi mà các bạn còn trả lời không ổn thì khi phỏng vấn với khách hàng họ hỏi chuyên sâu hơn và nói bằng English nữa thì các bạn sao trả lời nổi.

Đến khi tôi tham gia dự án Awesm được 2-3 tháng (tức là cách thời điểm tôi phỏng vấn 5 bạn đó cỡ 4-5 tháng) thì họ vẫn chưa được dự án nào nhận, vẫn tiếp tục ngồi trên hàng chờ...

Rồi 2/5 bạn nghỉ Nash Tech, còn lại 3 bạn ngồi chờ tiếp...

Thấy cũng tội nên tôi đã kêu họ lại lập 1 nhóm, đưa cho 1 dự án nhỏ tôi đã làm trước kia để họ làm lại từ A-Z dưới sự Mentor của tôi nhằm mục đích giúp họ củng cố nền tảng IT, tăng kiến thức và kinh nghiệm thực tế làm phần mềm.

Mỗi tuần tôi họp nhóm một lần để chỉnh sửa cho họ. Cứ thế sau 3 tháng thì trình độ họ tăng đáng kể và đã được nhận vô các dự án của các khách hàng ở Châu Âu.

Từ đây tôi nhận ra là các bạn chịu khó suy luận Logic, chịu cày và kiên trì, chỉ cần củng cố nền tảng IT, chỉ hướng đi, chỉ tài liệu phù hợp cho họ, và cho họ thực hành trên dự án thực tế rồi theo chỉnh sửa những sai sót của họ thì họ sẽ đáp ứng yêu cầu của công việc thực tế thôi.

Trong thời gian tham gia dự án Awesm & UP, tôi cũng đã hỗ trợ NashTech phỏng vấn rất nhiều bạn trẻ IT mới ra trường hoặc có 1-2 năm kinh nghiệm xin vào vị trí Fresher/Junior Java Developer. Kết quả là số bạn đạt yêu cầu chỉ đếm trên đầu ngón tay.

Qua trải nghiệm tiếp xúc với các sinh viên IT cũng như phỏng vấn các ứng viên mới ra trường hoặc có tí kinh nghiệm làm việc tính từ lúc về Sài Gòn, tôi thấy hầu hết các bạn đều không có được kiến thức nền tảng IT vững, chỉ lo chạy theo công nghệ nên giống như xây nhà từ nóc.

Khó khăn nhất là các bạn sinh viên IT phần lớn không có được một định hướng rõ ràng cho nghề tương lai nên không biết nên tập trung thời gian và công sức học những gì, theo thứ tự nào để ra trường tìm được việc làm phù hợp nhanh hơn.

Đầu năm 2017, ăn Tết Ta xong tôi đã nghỉ NashTech để vừa xây dựng chương trình Mentor mà tôi đã có bước đầu thành công với các bạn ở NashTech bài bản hơn, vừa đi chia sẻ kiến thức/kinh nghiệm của tôi cho các bạn IT trẻ trong cộng đồng và cũng tìm cơ hội làm Tech Startup tiếp.

Tình cờ tham gia 1 sự kiện về chia sẻ kinh nghiệm du học, tôi gặp 1 anh bạn trẻ tên là Eric (tên tiếng Việt là Ngọc Anh), dân đầu 9X, đã từng du học Úc rồi về Việt Nam khởi nghiệp.

Eric đã thành lập và điều hành 1 công ty ở Việt Nam chuyên hỗ trợ các học sinh Việt Nam lúc mới sang nước ngoài du học có thể hòa nhập tốt nhất vào môi trường mới. Công ty này đã hoạt động được 4-5 năm, mở rộng thị trường ra 6 nước, cũng đi vào ổn định.

Tiếp đến, Eric muốn mở 1 Tech Startup tên là 4SV (For Students of Vietnam, nghĩa là Vì Sinh Viên Việt Nam) với ý tưởng xây dựng 1 nền tảng công nghệ hỗ trợ sinh viên Việt Nam trong nước định hướng nghề nghiệp, nâng cao kiến thức và tích lũy kinh nghiệm thực tiễn để ra trường có thể xin được việc làm phù hợp sớm. Startup 4SV sẽ được đầu tư bởi các Angel Investor.

Thấy tôi có kinh nghiệm làm Tech Startup, Eric mới mời tôi tham gia 4SV với vai trò CTO & Co-Founder (đồng sáng lập). Tôi nhận thấy ý tưởng này rất hay, đem lại giá trị lớn cho sinh viên Việt Nam, và cũng có liên quan đến chương trình Mentor tôi đang xây dựng cho sinh viên IT, vì thế tôi đã đồng ý tham gia 4SV với Eric.

Một lần nữa tôi lại bắt tay vào xây dựng đội ngũ công nghệ cho 1 Tech Startup từ con số 0. Việc này làm tôi tốn rất nhiều thời gian và công sức để tìm các đồng đội phù hợp thông qua hầu như mọi kênh tôi sử dụng được: mạng lưới bạn bè của tôi, LinkedIn, các Facebook Group về sinh viên IT, các trang tuyển dụng IT có uy tín...

Cuối cùng tôi cũng xây dựng được 1 Scrum Team tối thiểu để có thể bắt tay vào phát triển 1 nền tảng công nghệ tên là **P4SVVN**.

Trước khi bước vào giai đoạn cày cuốc bất kể buổi tối hay cuối tuần do tính chất của Tech Startup, tôi đã tranh thủ cùng vợ tôi lên kế hoạch và thực hiện 1 chuyến phượt lịch sử qua 9 thành phố của 5 nước chính ở Châu Âu là France, Netherlands, Spain, Italy, và Germany kéo dài trong suốt 21 ngày.



Nền tảng P4SVVN cung cấp các tính năng chính sau:

- Định hướng nghề nghiệp cho sinh viên
- Cung cấp các việc làm giúp sinh viên tăng kiến thức, phát triển kỹ năng mềm và tích lũy kinh nghiệm thực tế
- Hỗ trợ sinh viên ứng tuyển việc làm
- Tổ chức các sự kiện cho sinh viên tham gia học hỏi thêm
- Kết nối sinh viên với các Mentor

Nếu tôi nhớ chính xác thì từ khi học Master ở Canada, tôi đã bắt đầu một thói quen cũng hơi mất thời gian nhưng khá tốt là thường xuyên tổng hợp những kiến thức & kinh nghiệm mình đã học & làm được để khi cần thì dùng lại, chứ càng thêm tuổi thì sẽ càng quên.

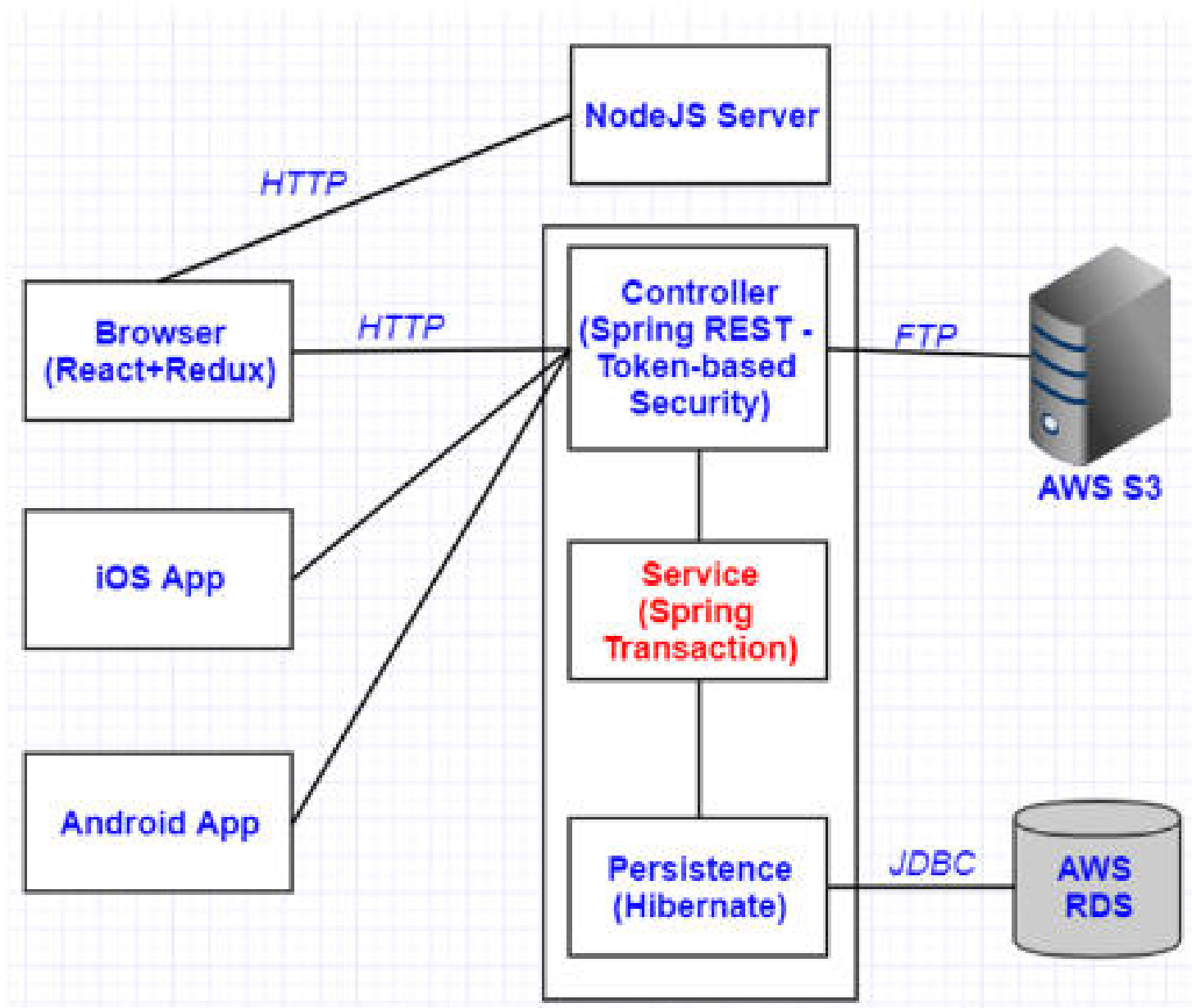
Nhờ vậy từ lúc vọc vô Spring và Hibernate Framework, tôi đã tự xây dựng nên 1 Web Application mẫu dựa trên 2 Framework này, chứa hầu hết những tính năng phổ biến nhất của phần mềm cũng như các giải pháp kỹ thuật thường dùng nhất.

Khi tôi làm 1 dự án mới cho tôi thì tôi đem Web Application mẫu ra dùng, chắc chắn là phải thêm/ thay đổi một số thứ để đáp ứng yêu cầu của dự án, và khi hoàn thành dự án thì tôi lại lấy những cái mới bổ sung trở lại Web Application mẫu.

Hoặc khi tôi tham gia 1 dự án đã có sẵn mà có giải pháp kỹ thuật nào đó hay ho, xong rồi thì tôi cũng hiện thực những phần kỹ thuật hay ho đó vào Web Application mẫu luôn.

Cho nên mô hình của P4SVVN đã dựa trên Web Application mẫu của tôi.

P4SVVN – Vietnam



Hình 107: P4SVVN

Song song với xây dựng nền tảng P4SVVN, tôi cũng mở 1 chương trình Mentor ở 4SV để thu hút 10 bạn sinh viên IT năm 3,4 vào học và thực tập trên dự án P4SVVN luôn. Kết quả cũng tương đối tốt khi tôi nhận được 5 bạn vào Team tôi làm chính thức.

16. Dự án RMS (Vietnam)

Khoảng tháng 5/2018, khi nền tảng P4SVVN đã chạy ổn định, tôi tham gia vào 1 Tech Startup tên là DevNET ở TPHCM của 1 chủ đề tên Quốc tình cờ tôi gặp ở Nha Trang trước đó với vai trò Tech Product Consultant & Software Architect.

Nhiệm vụ của tôi là tư vấn cho DevNET cách xây dựng 1 Scrum Team cũng như tìm nhân lực phù hợp vào Scrum Team để phát triển sản phẩm công nghệ của công ty có tên là RMS (Referral Marketing System).

Ý tưởng của RMS là giúp các SME giảm chi phí Marketing rất nhiều đồng thời tăng doanh thu đáng kể bằng cách xây dựng các mạng lưới **Affiliate** (Cộng Tác Viên), những người đã trải nghiệm và đánh giá cao sản phẩm của các SME, để họ trực tiếp/gián tiếp đưa sản phẩm đến các khách hàng tiềm năng, ngược lại họ sẽ được trả công.

Ngoài ra tôi cũng tham gia trực tiếp vào việc phân tích, thiết kế và hiện thực sản phẩm RMS cùng với các thành viên Scrum Team luôn. Cũng giống như giai đoạn đầu làm startup SBS ở Úc và 4SV, tôi cày cuốc suốt ngày, đôi khi cả đêm và cuối tuần để làm sao ra được phiên bản MVP càng nhanh càng tốt.

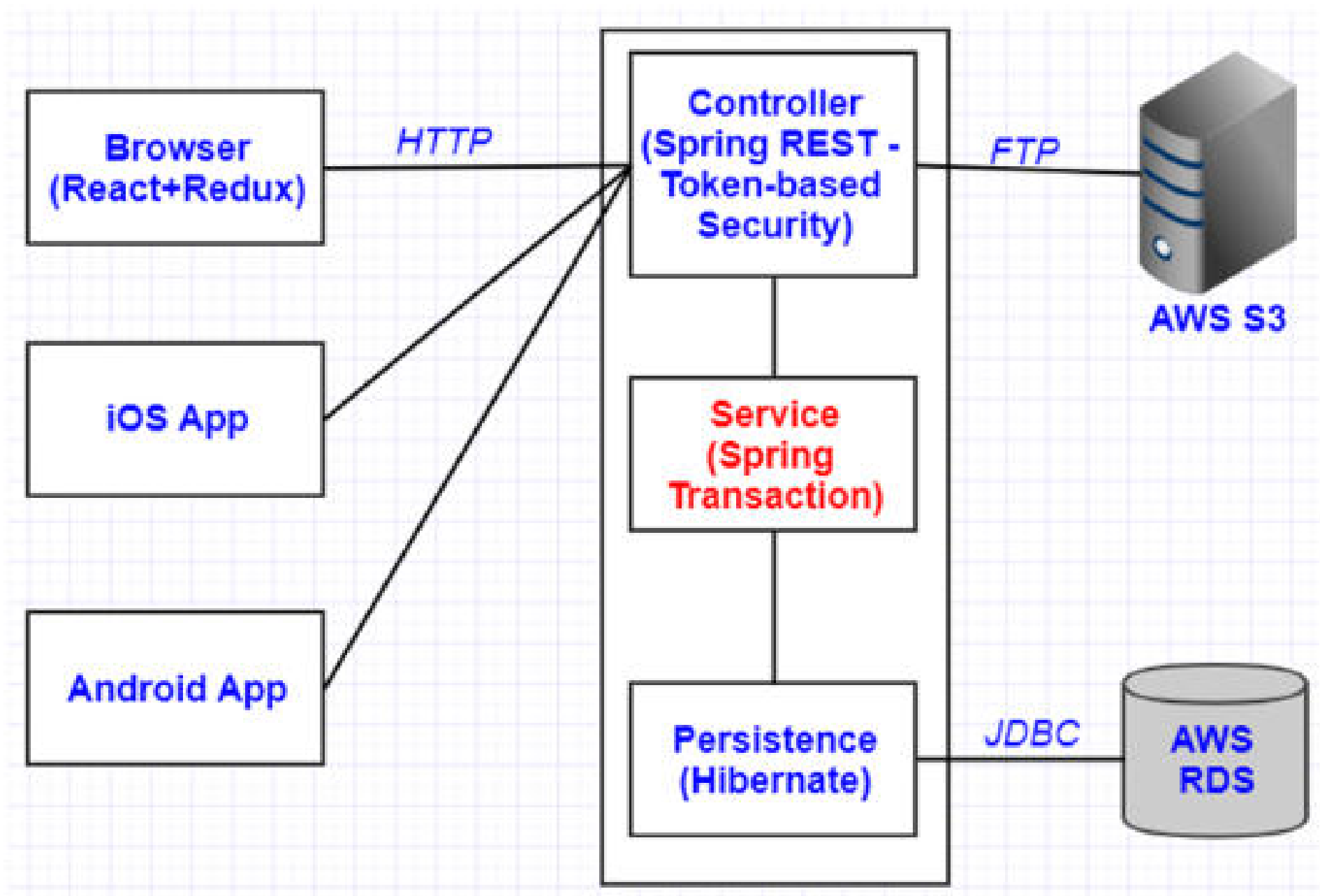
Các tính năng chính của RMS:

- Quản lý các Sales Channel (Kênh Bán Hàng)
- Cấu hình các Sales Channel
- Quản lý Affiliate
- Quản lý các Affiliate Network (Mạng Lưới Cộng Tác Viên)
- Quản lý các Shared Link (Trang Web Được Chia Sẻ)
- Quản lý Customer (Khách Hàng)
- Quản lý Order (Đơn Hàng)
- Quản lý Product (Sản Phẩm)
- Quản lý Commission (Hoa Hồng)
- Cấu hình cho các loại Commission

Phần phức tạp nhất của sản phẩm này là mô hình tính Commission cho các Affiliate trong các mạng lưới. Có 2 mô hình: theo Level hay theo Position.

Mô hình của RMS cũng dựa trên Web Application mẫu của tôi, có chỉnh sửa 1 chút.

RMS – Vietnam



Hình 108: RMS

Chỉ sau 4-5 tháng với 1 Scrum Team tinh gọn, RMS đã hoàn thành được phiên bản MVP.

Sau khi hỗ trợ Scrum Team ra thêm một vài phiên bản nữa thì tôi cũng đã kết thúc vai trò của mình ở DevNET vào cuối năm 2018.

17. Dự án SUPA (Taiwan)

Đầu năm 2019, tôi tham gia vào EZTABLE, 1 Tech Startup ở Taiwan với sản phẩm công nghệ cùng tên với công ty là EZTABLE, chuyên cung cấp dịch vụ đặt bàn ở các nhà hàng, khách sạn khắp châu Á.

CEO của công ty EZTABLE tên là Alex, người Đài Loan, từng du học ở Mỹ về IT, sau đó chuyển sang học ngành tài chính, làm cho các ngân hàng ở phố Wall, New York một thời gian đến năm 2008 trở về Taiwan khởi nghiệp đầu tiên với EZTABLE.

Tính đến đầu năm 2019, EZTABLE đang ở giai đoạn series C với 3 lần kêu gọi đầu tư với tổng số tiền là 15.3 triệu USD.

Tôi đã xây dựng 1 Scrum Team tinh gọn cho EZTABLE ở TPHCM, trong đó có vài thành viên từng làm chung với tôi trong dự án P4SVVN & RMS. Theo kế hoạch mở rộng EZTABLE của ông Alex, nhiệm vụ của Team tôi là nghiên cứu và phát triển các Innovative Tech Product mới ở thị trường Indonesia.

Tech Product đầu tiên Team tôi làm có tên là SUPA.

Cái tên này không phải viết tắt của các từ nào cả, mà là tên do anh bạn trẻ (U30) Product Manager của EZTABLE đặt tựa tựa như chữ “super” của English với ý nghĩa là một cái gì đó tuyệt vời.

Ý tưởng của SUPA là cung cấp 1 nền tảng công nghệ dạng **O2O (Online-to-Offline)**:

- Các **Merchant** (Cửa Hàng/Nhà Bán Buôn/Bán Lẻ) đưa lên đó 1 số lượng lớn các **Voucher** (Phiếu Mua Hàng Giảm Giá) cho khách hàng của họ mua và thanh toán cho SUPA (phần Online).
- Sau đó khi Customer đến cửa hàng của họ (phần Offline) sử dụng sản phẩm thì Customer sẽ **Redemption** (Dùng Dàn) các Voucher đã mua, rồi cuối mỗi tuần hoặc tháng, hệ thống SUPA sẽ **Reimbursement** (Hoàn Trả) phần tiền mà Customer đã dùng sản phẩm của họ.
- Như vậy, Merchant được lợi là bán được thêm nhiều sản phẩm sẽ tăng doanh thu mà không cần tự đi tìm khách hàng, hiển nhiên Merchant cũng phải giảm lợi nhuận 1 tí cho Voucher, còn Customer có lợi là được dùng những sản phẩm có giảm giá tí.

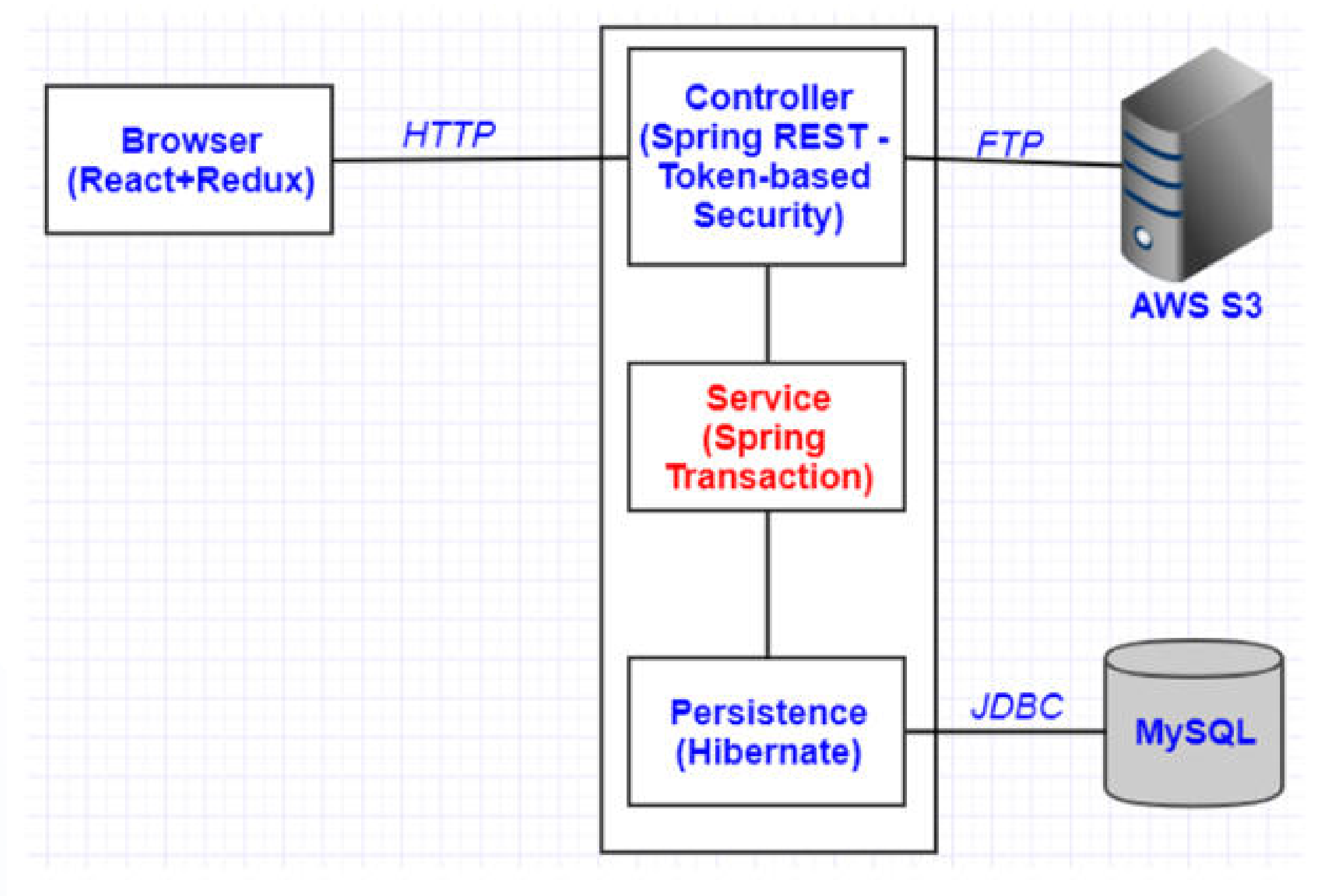
Các tính năng chính của SUPA bao gồm:

- Quản lý Merchant
- Quản lý Voucher
- Thương mại điện tử

- Thanh toán online
- Quản lý Order
- Quản lý Customer
- Quản lý Redemption
- Quản lý Reimbursement
- Tích hợp hệ thống
- Quản lý Feedback (phản hồi)

Mô hình của SUPA tương tự RMS:

SUPA – Taiwan



Hình 109: SUPA

Phần phức tạp nhất trong hệ thống SUPA là mô hình tính toán **Cash Flow** (Dòng Tiền) chảy vào SUPA từ Customer và chảy ra khỏi SUPA đến Merchant.

Vì yếu tố này mà sản phẩm SUPA ban đầu tôi nghĩ chỉ là 1 nền tảng bán Voucher trực tuyến nhưng thực chất đây là 1 sản phẩm bên mảng **Fintech** (Công Nghệ Tài Chính) đang rất thịnh hành những năm gần đây.

Theo kế hoạch của Alex là làm SUPA MVP trong 6 tháng, nhưng Team tôi chỉ tốn 4.5 tháng là xong do đã làm chung nhiều dự án, đến 6 tháng thì đã thêm được khá nhiều tính năng nữa.

18. Dự án CPT (Australia)

Sau dự án SUPA thì tôi tham gia vào một Tech Startup với 1 anh bạn đang ở Úc vào năm 2020 trong lĩnh vực xây dựng.

Tech Product mà Team tôi làm có tên là **CPT** (Construction Project Tracking).

Ý tưởng của CPT là giúp cho các công ty xây dựng có thể theo dõi, kiểm soát và truy vấn lại những dự án xây dựng đã/đang diễn ra. Những dự án này có rất nhiều nhân lực (Kiến Trúc Sư, Kỹ Sư, Công Nhân...) từ rất nhiều công ty khác nhau (Nhà thầu chính, phụ) tham gia vào nên số lượng tài liệu thiết kế lớn, nhiều **Workflow** (Luồng làm việc) và việc tương tác giữa những nhân lực khi dự án diễn ra rất phức tạp mà những công cụ lưu trữ tài liệu trực tuyến thuần túy không giúp được chủ dự án.

Team tôi đã quen với những công nghệ và qui trình phát triển sản phẩm mà chúng tôi đã dùng để xây dựng những Tech Product trước đó nên không có khó khăn gì về mặt kỹ thuật.

Team tôi chỉ gặp khó khăn ở mảng nghiệp vụ khi đây là một mảng mới, có nhiều Business Flow/Rule phức tạp.

Ngoài ra phần **Authorization** (Phân Quyền) cũng khá là thử thách vì nếu làm không chuẩn chỉnh thì sẽ gây ra rủi ro về mặt bảo mật, ví dụ như một kỹ sư ở công ty A có thể truy cập vào dự án X của công ty B mà anh/cô ấy không được mời tham gia vào X.

Theo kế hoạch của anh bạn tôi là làm MVP cho CPT trong 6 tháng, nhưng Team tôi chỉ tốn 5 tháng là hoàn thành đem đi cho khách hàng xem thử.

19. Dự án MPT (Australia)

Song song với dự án SUPA thì Team tôi cũng xây dựng 1 Tech Product có tên là **MPT** (Manufacturing Project Tracking).

Tech Product này bổ trợ thêm cho CPT trong việc giúp cho các công ty sản xuất thiết bị trong mảng xây dựng có thể theo dõi, kiểm soát và truy vấn lại những dự án sản xuất đã/đang diễn ra.

Về mặt công nghệ thì MPT sử dụng Technology Stack tương tự CPT, cái khác của MPT so với CPT là đối tượng khách hàng, kiến thức lĩnh vực và các Business Rules.

20. Dự án DDS (Canada)

Cuối năm 2020, tôi đã chuyển vô làm cho 1 công ty Canada tên là Absolute Security có trụ sở chính ở Vancouver, Canada, và những văn phòng khác ở nhiều nước như Mỹ, Nhật, Châu Âu, Úc và Việt Nam. Tôi làm việc ở văn phòng tại Thành Phố Hồ Chí Minh, Việt Nam.

Tôi tham gia phát triển sản phẩm công nghệ của công ty tên là **DDS** (Data and Device Security) đã được bắt đầu xây dựng từ ~30 năm trước thuộc mảng **Cyber Security** (An Ninh Mạng).

Ý tưởng của DDS là giúp cho các công ty có khả năng theo dõi và kiểm soát các máy vi tính của nhân viên khi họ làm việc bất cứ nơi đâu. Khi một máy nào đó có dấu hiệu bị trộm thì người quản lý máy đó sẽ dùng DDS để thực hiện một số thao tác bảo mật từ xa:

1. Khóa máy lại để không cho ai sử dụng được, muốn mở khóa thì phải có mã
2. Xóa những dữ liệu quan trọng cần bảo mật trên máy
3. Tạo một báo cáo trộm cho máy đó để các cơ quan điều tra truy tìm máy đang ở đâu bằng việc sử dụng tính năng Geograph Location

Phần bảo mật máy móc và dữ liệu là chức năng cốt lõi của DDS.

Ngoài ra, một giải pháp có giá trị rất lớn mà DDS cung cấp cho các công ty chính là **Resillience** (Khả năng phục hồi).

Bạn cứ hình dung thử nghĩ cảnh một công ty lớn có hàng triệu máy bị bọ tin tặc tấn công một lượng lớn máy làm hoạt động của công ty bị ảnh hưởng nặng nề. Thời gian và công sức mà công ty phải bỏ ra để hồi phục lại những máy đó là cực kỳ lớn, gây nhiều thiệt hại cho công ty về mặt hình ảnh, vận hành, doanh thu... => Giải pháp Resillience sẽ giúp công ty giảm cực kỳ đáng kể lượng thời gian và công sức này.

Các thử thách ban đầu của tôi khi tham gia vô công ty Absolute Software và dự án DDS:

- Kiến thức lĩnh vực mới
- Quy trình hoạt động phức tạp (do đây là công ty lớn với 800+ nhân viên toàn cầu) dẫn đến thời gian để ra tính năng mới hoặc bảo trì khá dài so với Tech Startup
- Rất nhiều công nghệ mới và được cập nhật liên tục

PHẦN THAM KHẢO

Bộ tài liệu tra cứu dành cho dân IT chuyên nghiệp

Kiến trúc phần mềm, lập trình, hạ tầng, NFR, công cụ dev – tất cả nằm trong Bản PRO.

👉 [Nâng cấp lên PRO để truy cập ngay]

SELF-EXPERIENCE

Với 3S Method bạn đã học ở trên, đến đây bạn đã hoàn thành chữ S đầu tiên đó là **Study**.

Tiếp theo, bạn sẽ thực hành tiếp chữ S thứ hai đó là **Self-Experience** thông qua 2 Tech Product thực tiễn nhỏ sau đây để đạt được chữ S thứ 3 là **Skill**:

1. Xây dựng 1 Tech Product giúp việc quản lý thu/chi trong gia đình bạn, và triển khai cho các thành viên trong gia đình bạn sử dụng thông qua LAN.
2. Xây dựng 1 Tech Product giúp bạn bán một số mặt hàng của bạn online, và triển khai cho các hộ gia đình trong xóm bạn/ bạn bè của bạn sử dụng thông qua Internet.

LỜI KẾT

Thế giới công nghệ thật là rộng lớn, đầy màu sắc và cực kỳ cuốn hút cho những ai ĐAM MÊ công nghệ. Bộ Cẩm Nang đã dẫn dắt bạn khám phá rất nhiều kiến thức tổng hợp rất là thú vị rồi đó, hi vọng bạn đã thu nhận được nhiều điều bổ ích từ Bộ Cẩm Nang này.

Bây giờ đến lượt bạn cầm dây cương bắt đầu tự khám phá thế giới công nghệ. Sử dụng những kiến thức ở trên để hỗ trợ bạn tốt hơn.

Hãy để Bộ Cẩm Nang này là điểm khởi đầu cho hành trình của bạn với thế giới công nghệ. Luôn khám phá, thử nghiệm và đổi mới trong nghề nghiệp của bạn. Thách thức ranh giới của sự sáng tạo và năng suất. Công nghệ luôn sẵn sàng giúp bạn vượt qua ranh giới của những gì có thể.

Chúc bạn luôn có nhiều niềm vui trong học tập, làm việc và có được một cuộc sống bình an!

Homer Truong Le Hoang



HOA

HOMer Academy

Hiếu đúng - Học đúng - Hành đúng

KHÁM PHÁ THẾ GIỚI CÔNG NGHỆ



Homer Truong Le Hoang

Vietnam Top IT Mentor